



算法设计与分析

Lecture 12: Branch and Bound

曹刘娟

厦门大学信息学院计算机系

caoliujuan@xmu.edu.cn



Limitation of Backtracking 回溯法的缺点

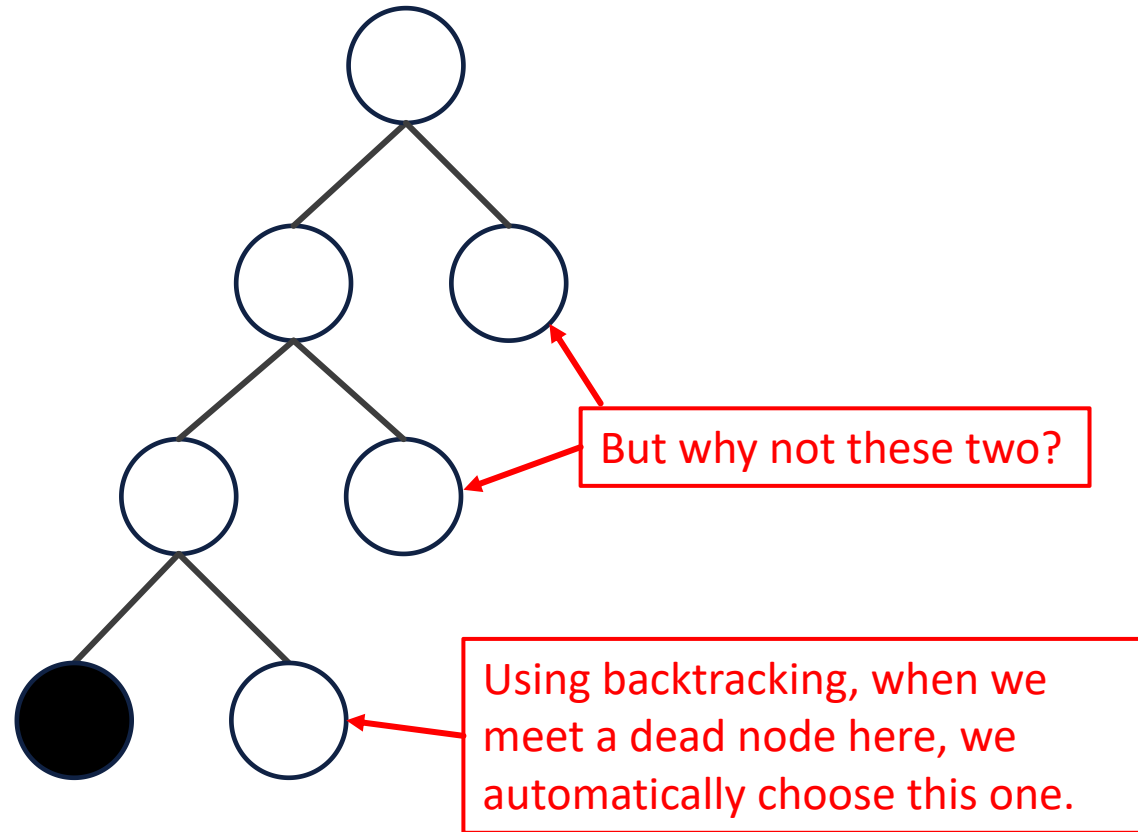
- Backtracking works better if we can improve over the bounding function. 如果能够开发出更加紧致的限界函数，回溯会更加高效
- However, there is still a mechanism that limits backtracking to be more efficient: 但是还是有一个核心部分导致回溯不够高效

DFS

- No matter how you improve the bounding function, the traversal is still based on DFS.
 - Can we based on other methods to explore the solution space?



Limitation of Backtracking





Limitation of Backtracking

- Can we try BFS?
- No, it is very inefficient.
 - No solution is reached until level 1 to level $n - 1$ of the tree is built.
 - No solution means bounding function is useless.
- **Branch-and-bound (分支限界)** is the techniques to improve BFS for solution space tree traversal.
 - FIFO branch-and-bound.
 - Max-profit branch-and-bound.



Branch-and-Bound

- Different from backtracking, the branch-and-bound method
 1. does not limit us to any particular way of traversing the tree;
 2. is used only for optimization problems.
- A branch-and-bound algorithm computes a **upper bound** and **lower bound** at a node.
- For maximization problem:
 - Upper bound is calculated by the bounding function.
 - Lower bound is recorded by the best solution so far.
- We increase the lower bound and decrease the upper bound until they are equal.

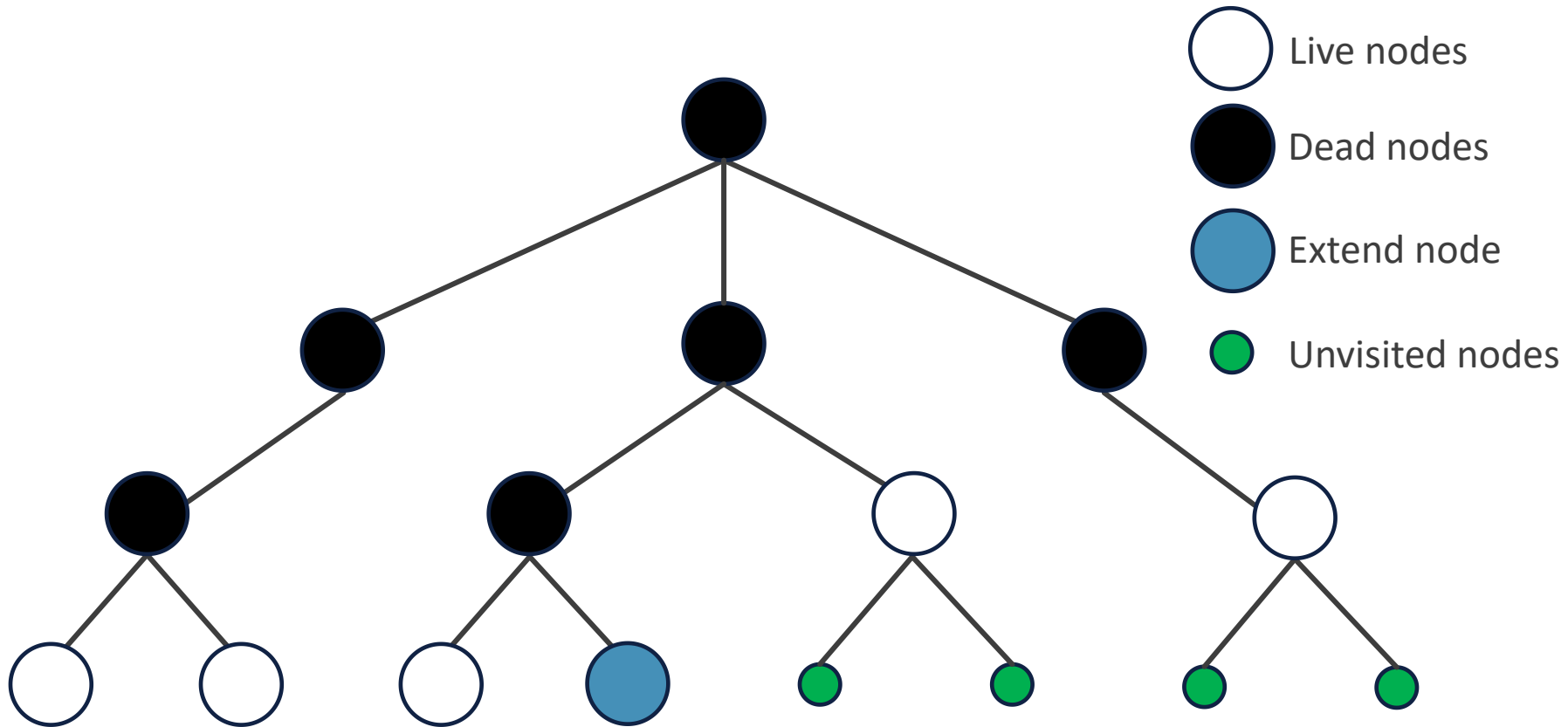


Branch-and-Bound

- Branch-and-bound is based on BFS, but does not exactly follow its FIFO mechanism.
- We select the next node to branch based on some rule. Namely, we branch a node **with the highest hope**.
- All nodes can be separated into:
 - Live nodes: Visited but waiting for branching.
 - Dead nodes: Visited.
 - Extend node: Selected to branch in the next step.
 - Unvisited nodes: Unvisited.



Branch-and-Bound



Extend node is selected among all live nodes based on designed rules.



CONTAINER LOADING PROBLEM



Container Loading Problem

- Given n containers (集装箱), container i has weight w_i . The ship can hold containers of total weight up to W .
- Container Loading problem is to load as many containers as is possible without sinking the ship.
- Assuming that the solutions are represented by vectors $(x_1, x_2, \dots, x_n)$, where $x_i \in \{0,1\}$. 1 denotes taking container i and 0 denotes not taking container i .
- The container loading problem can be formally stated as follows:

$$\max \sum_{i=1}^n w_i x_i \quad s. t. \quad \sum_{i=1}^n w_i x_i \leq W$$



Container Loading Problem

In this example, we go through **three versions** of branch-and-bound.

- FIFO branch-and-bound with only constraint function.
- FIFO branch-and-bound.
- Max-profit branch-and-bound.



Container Loading Problem

- The constraint function is same as backtracking.
- Let $cw(i)$ denote the current weight up to level i , namely

$$cw(i) = \sum_{j=1}^i w_j x_j$$

then the constraint function is

$$C(i) = cw(i - 1) + w_i$$

- The pruning condition is $C(i) > W$, which means there is no capacity to take container i .



FIFO with Only Constraint Function

FIFOMaxLoading(w, W, n)

```
1  $i \leftarrow 1$ 
2 Enqueue( $Q, -1$ )
3  $cw \leftarrow 0; bestw \leftarrow 0$ 
4 while  $Q \neq \emptyset$  do
5   if  $C(i) \leq W$  then
6     SaveQueue( $Q, C(i), bestw, i$ )
7   SaveQueue( $Q, cw, bestw, i$ )
8    $cw \leftarrow \text{Dequeue}(Q)$ 
9   if  $cw = -1$  then
10    if  $Q = \emptyset$  then return  $bestw$ 
11    Enqueue( $Q, -1$ )
12     $cw \leftarrow \text{Dequeue}(Q)$ 
13     $i \leftarrow i + 1$ 
14 return  $bestw$ 
```

We insert -1 in the queue to show the separation between different levels.

Level i is used to check solution.

SaveQueue($Q, wt, bestw, i$)

```
1 if  $i = n$  then
2   if  $wt > bestw$  then
3      $bestw \leftarrow wt$ 
4 else
5   Enqueue( $Q, wt$ )
```

Enqueue left and right child.

The current level is fully explored.

No live node to branch, terminate.

Continue to explore the next level.



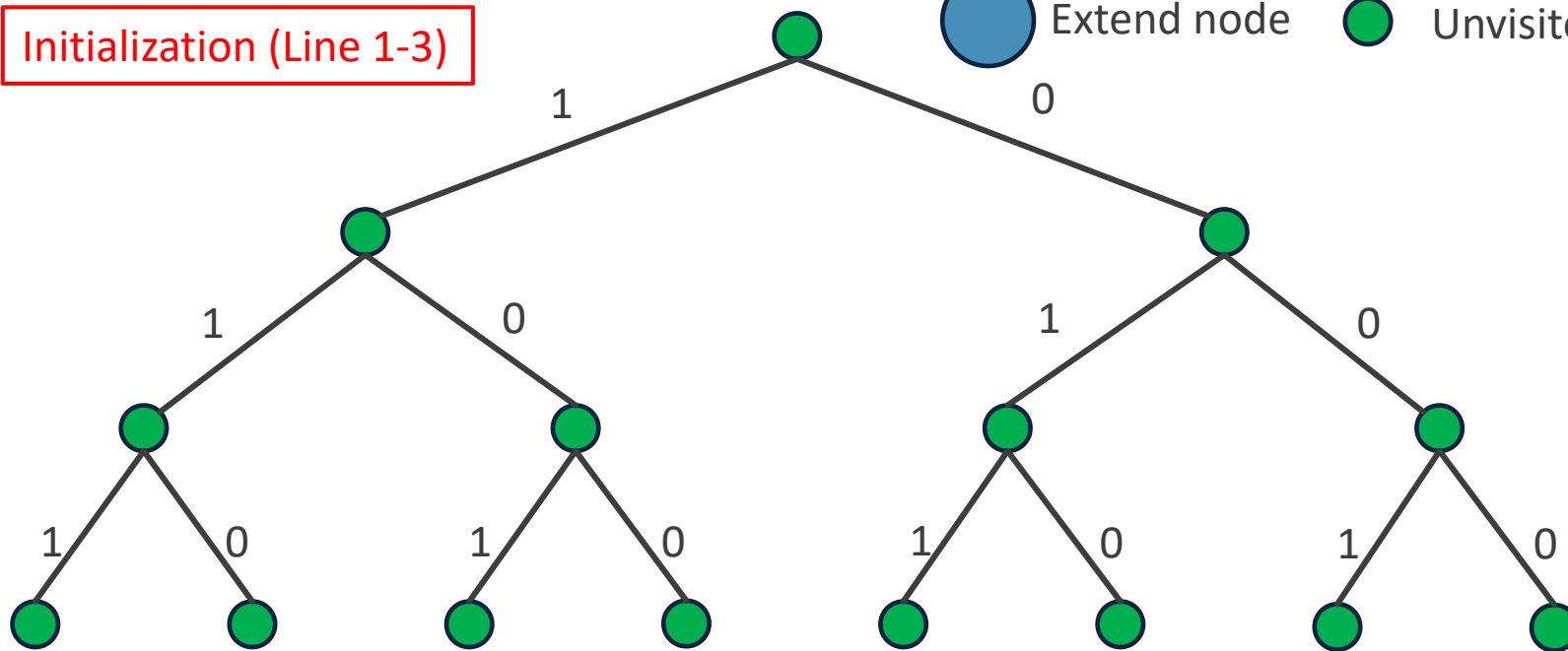
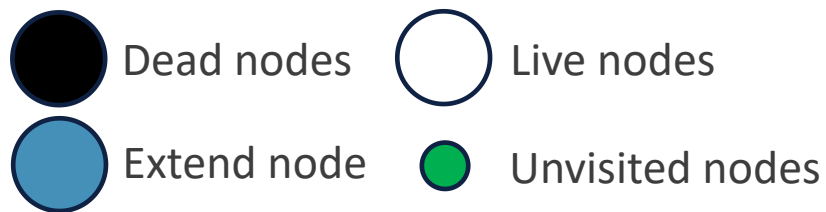
Example

Q :

-1			
----	--	--	--

 $i = 1$

Initialization (Line 1-3)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



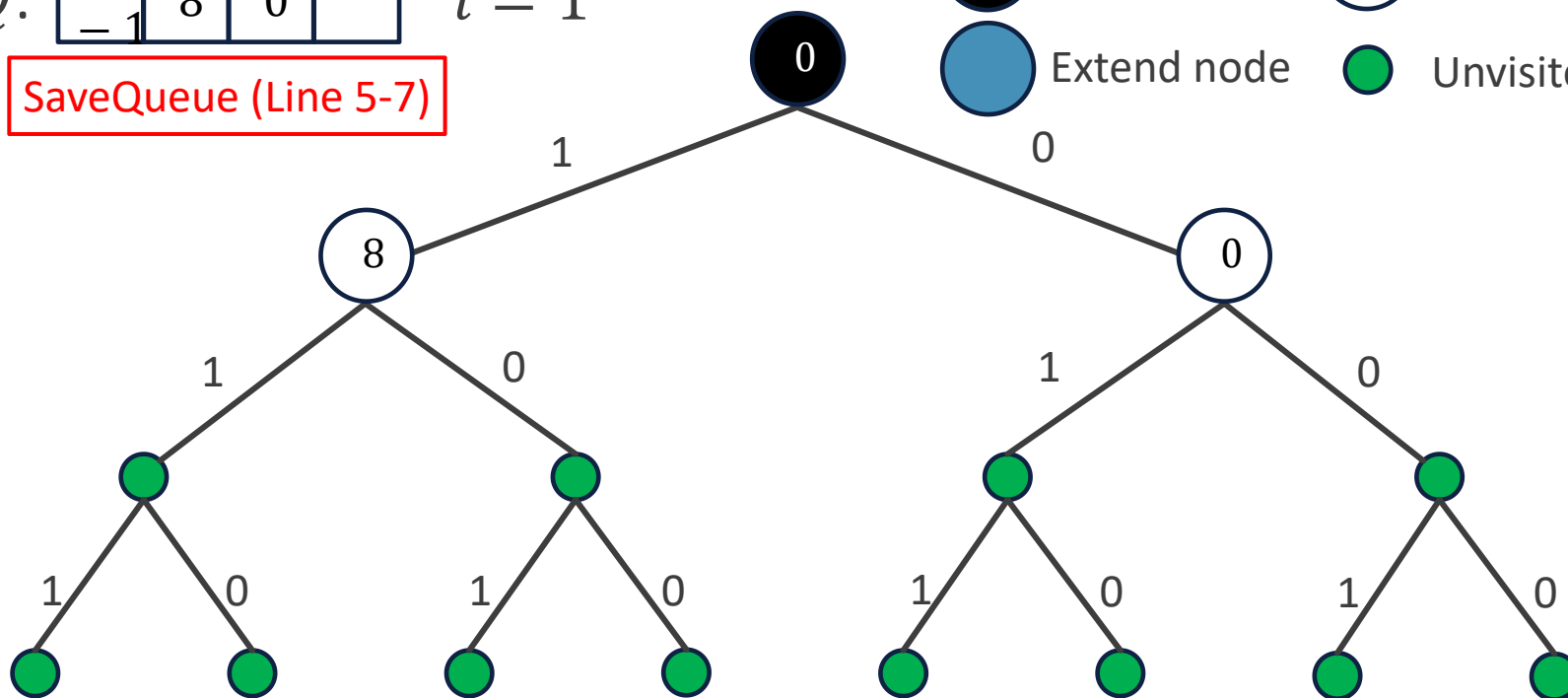
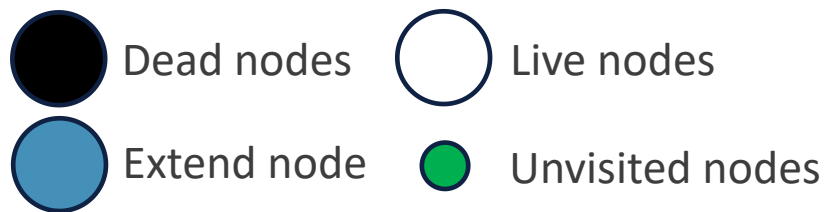
Example

Q :

-	1	8	0	
---	---	---	---	--

 $i = 1$

SaveQueue (Line 5-7)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



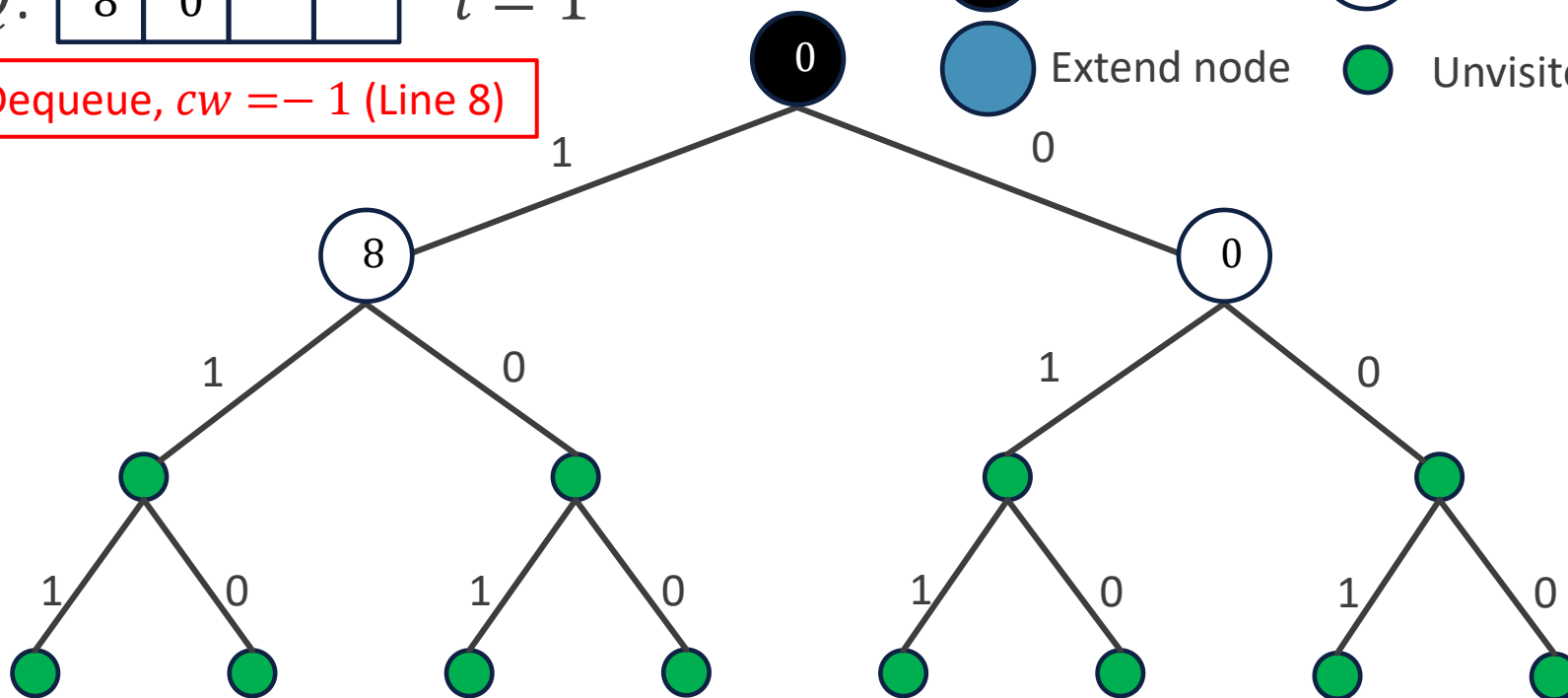
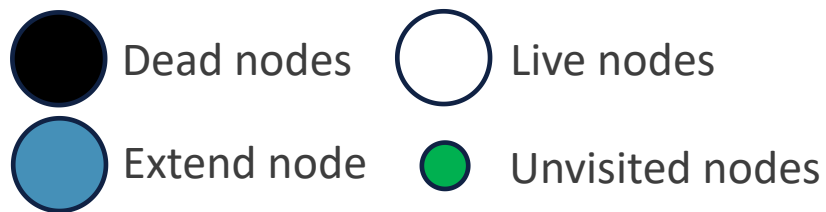
Example

Q :

8	0		
---	---	--	--

 $i = 1$

Dequeue, $cw = -1$ (Line 8)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



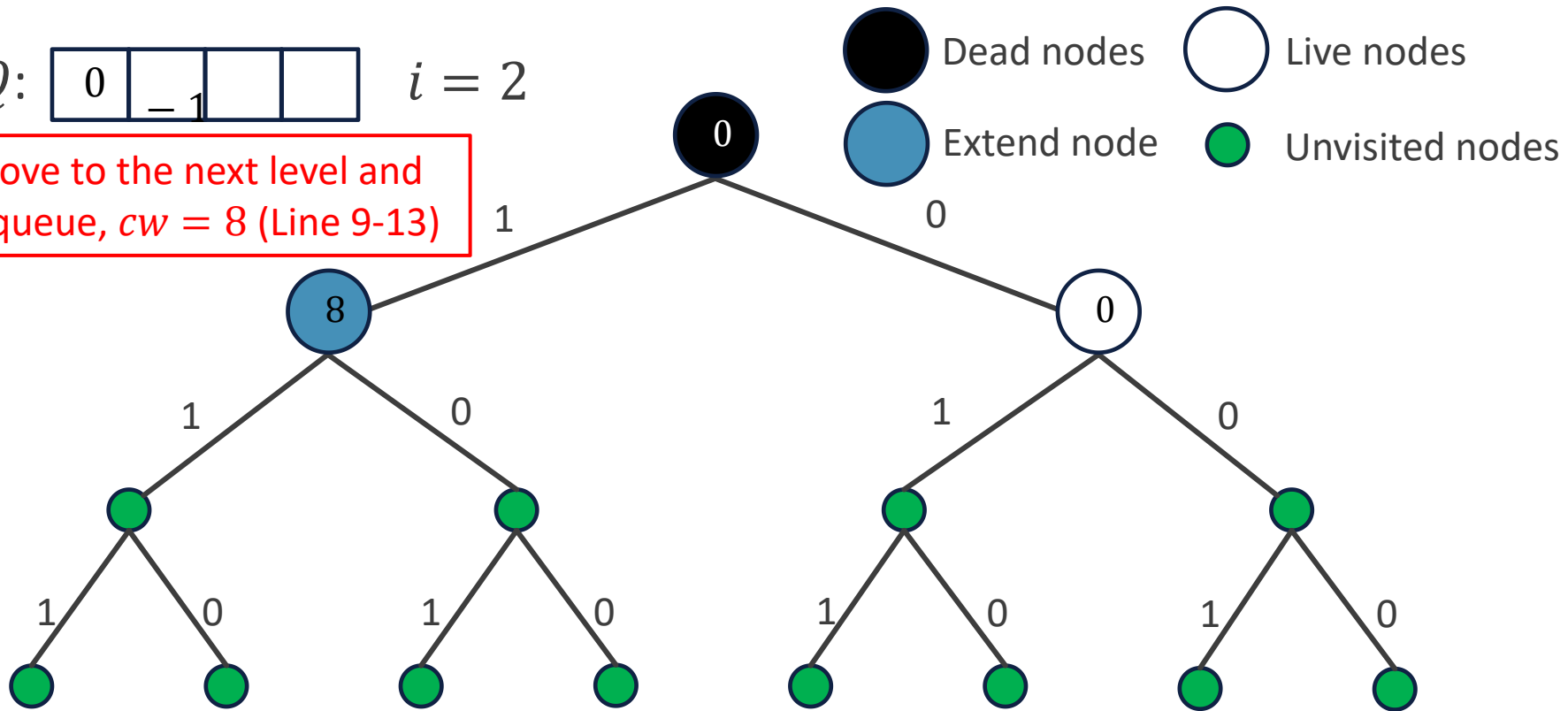
Example

Q :

0	-	1	
---	---	---	--

 $i = 2$

Move to the next level and
dequeue, $cw = 8$ (Line 9-13)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



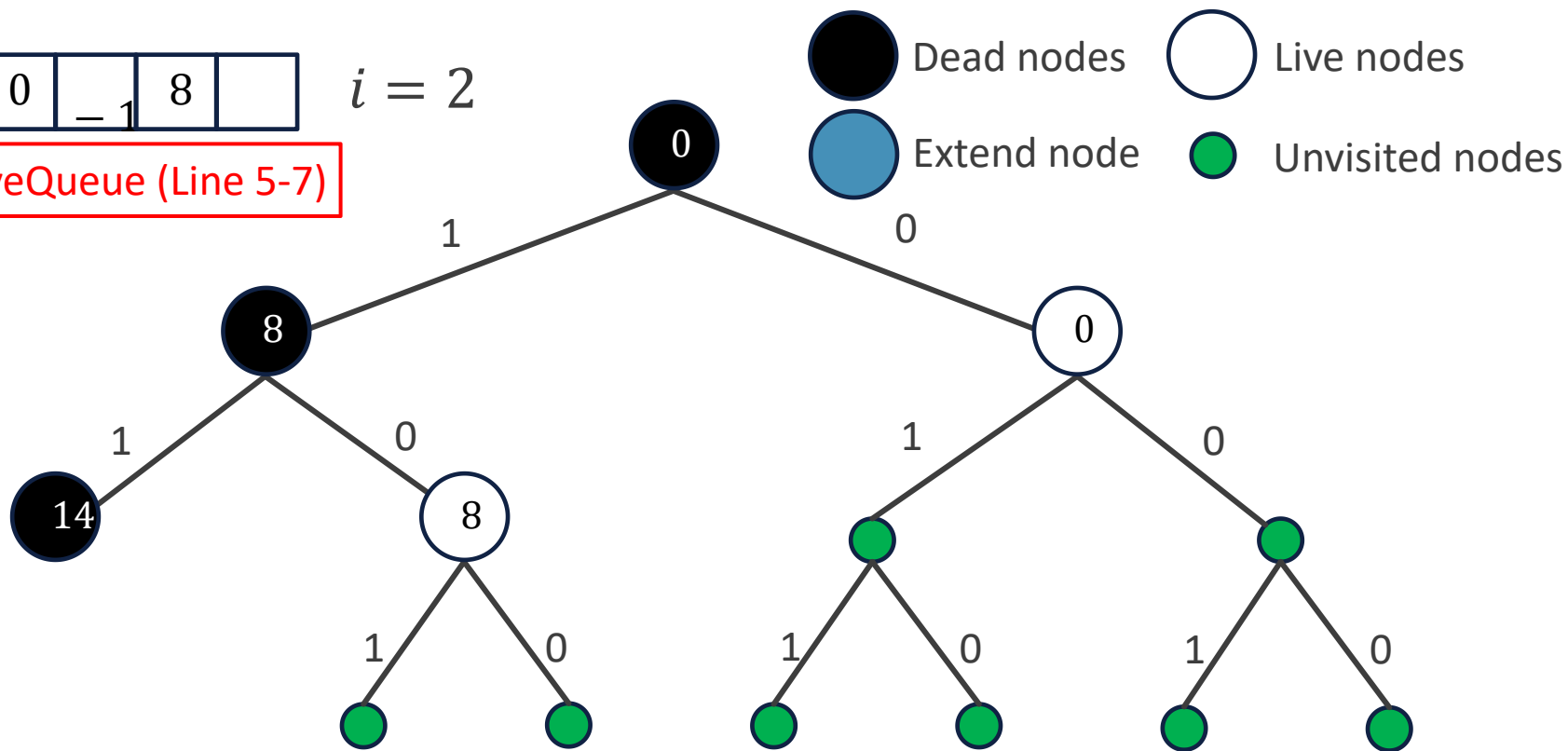
Example

Q:

0	-1	8	
---	----	---	--

 $i = 2$

SaveQueue (Line 5-7)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



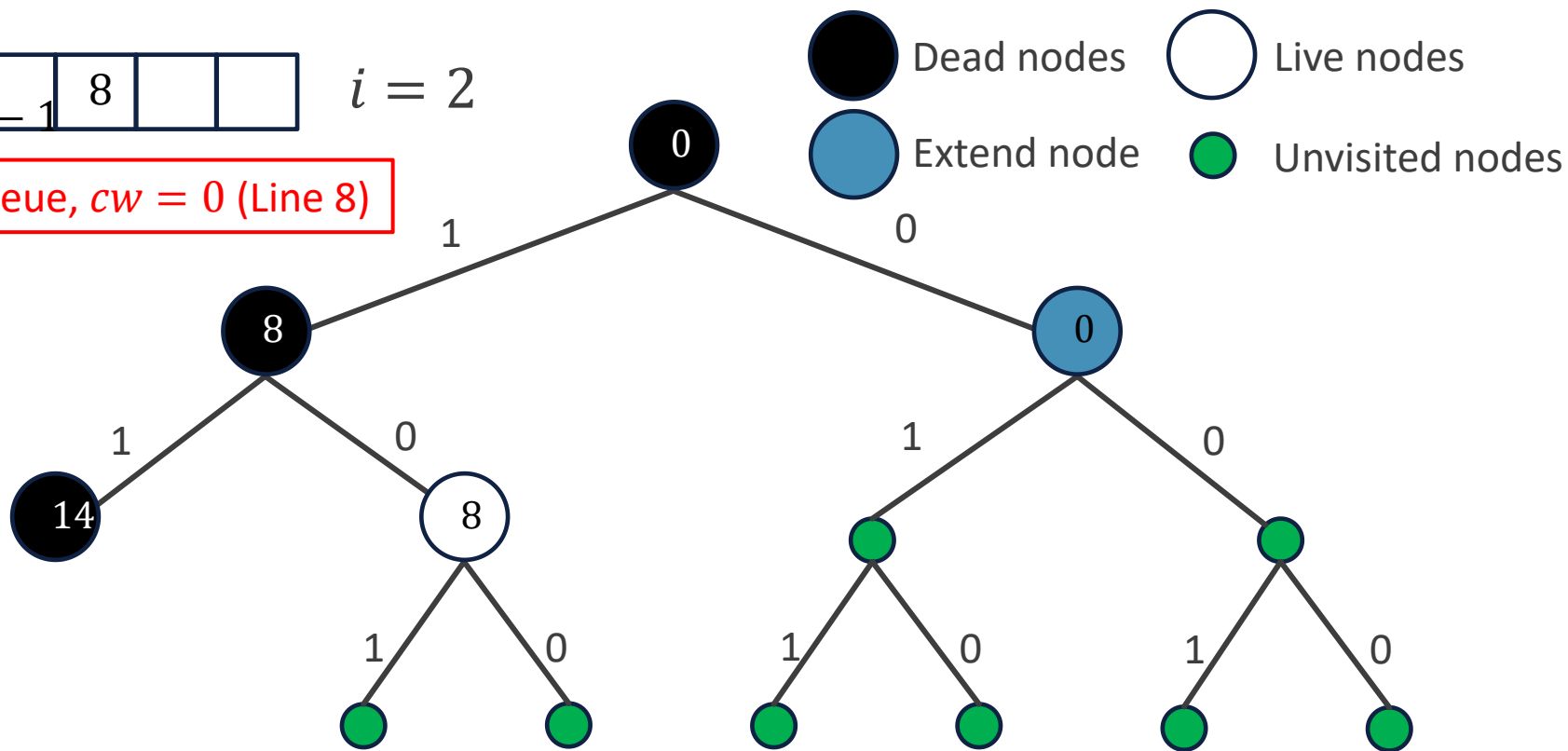
Example

Q :

_	1	8		
---	---	---	--	--

 $i = 2$

Dequeue, $cw = 0$ (Line 8)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



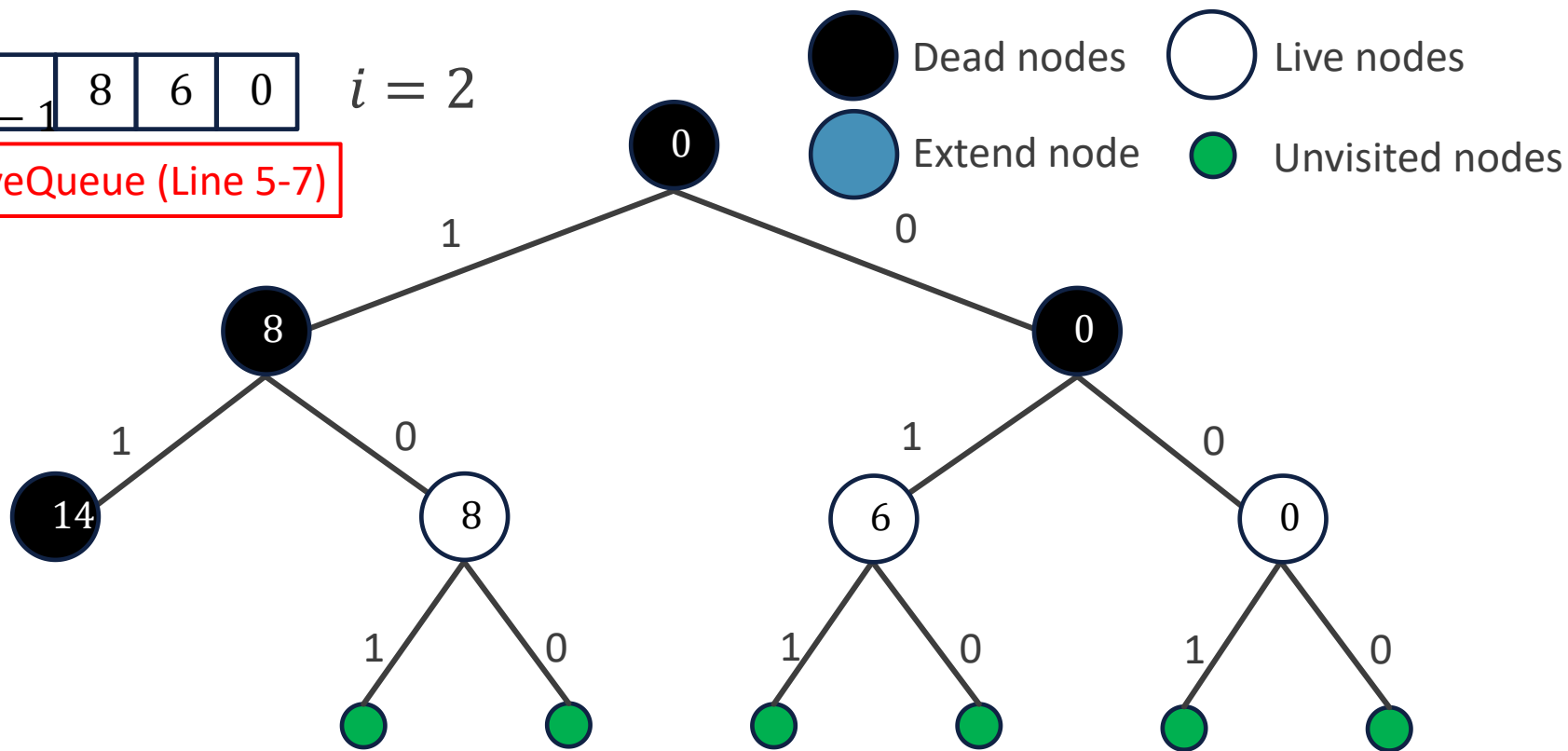
Example

Q:

-	1	8	6	0
---	---	---	---	---

 $i = 2$

SaveQueue (Line 5-7)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



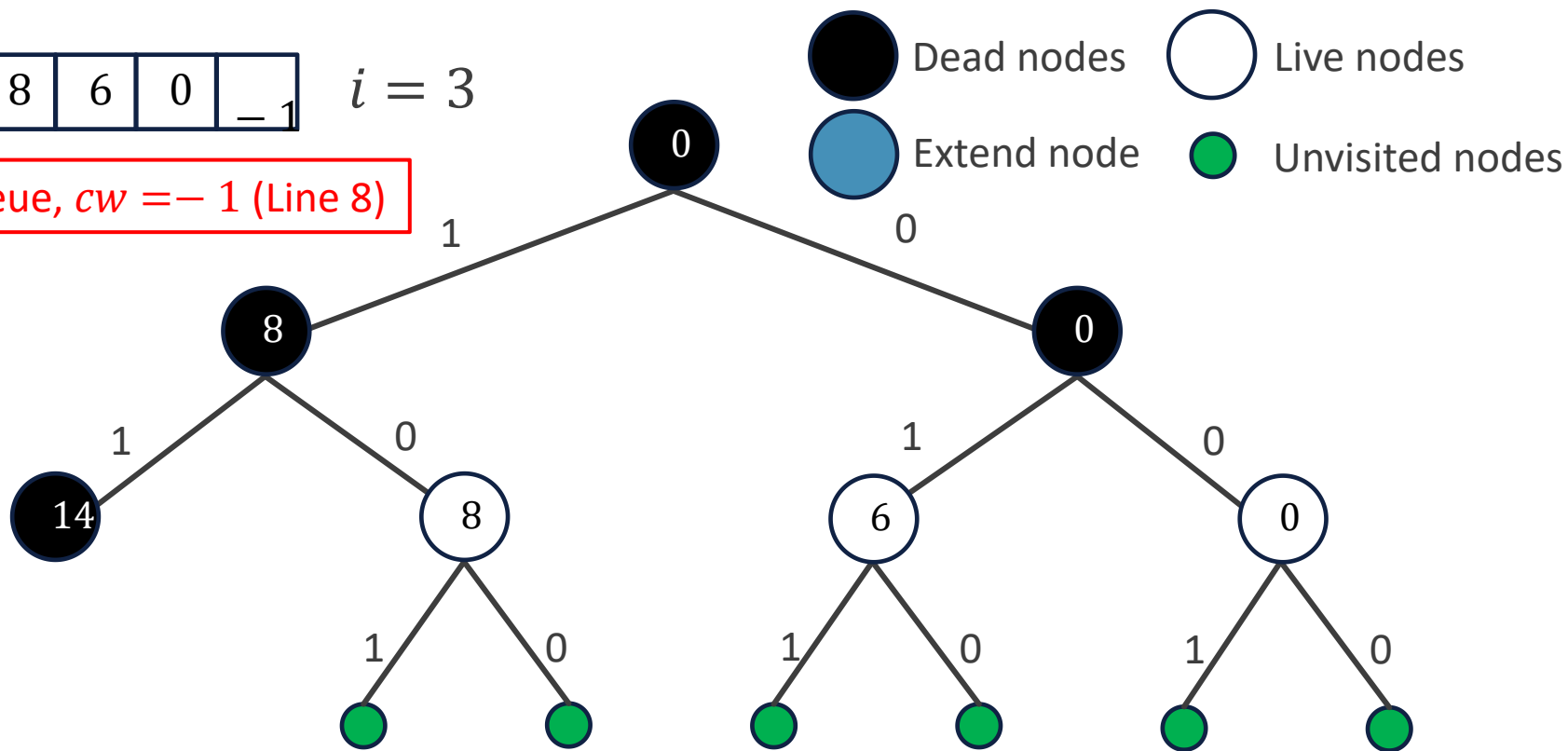
Example

Q :

8	6	0	—
---	---	---	---

 $i = 3$

Dequeue, $cw = -1$ (Line 8)



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



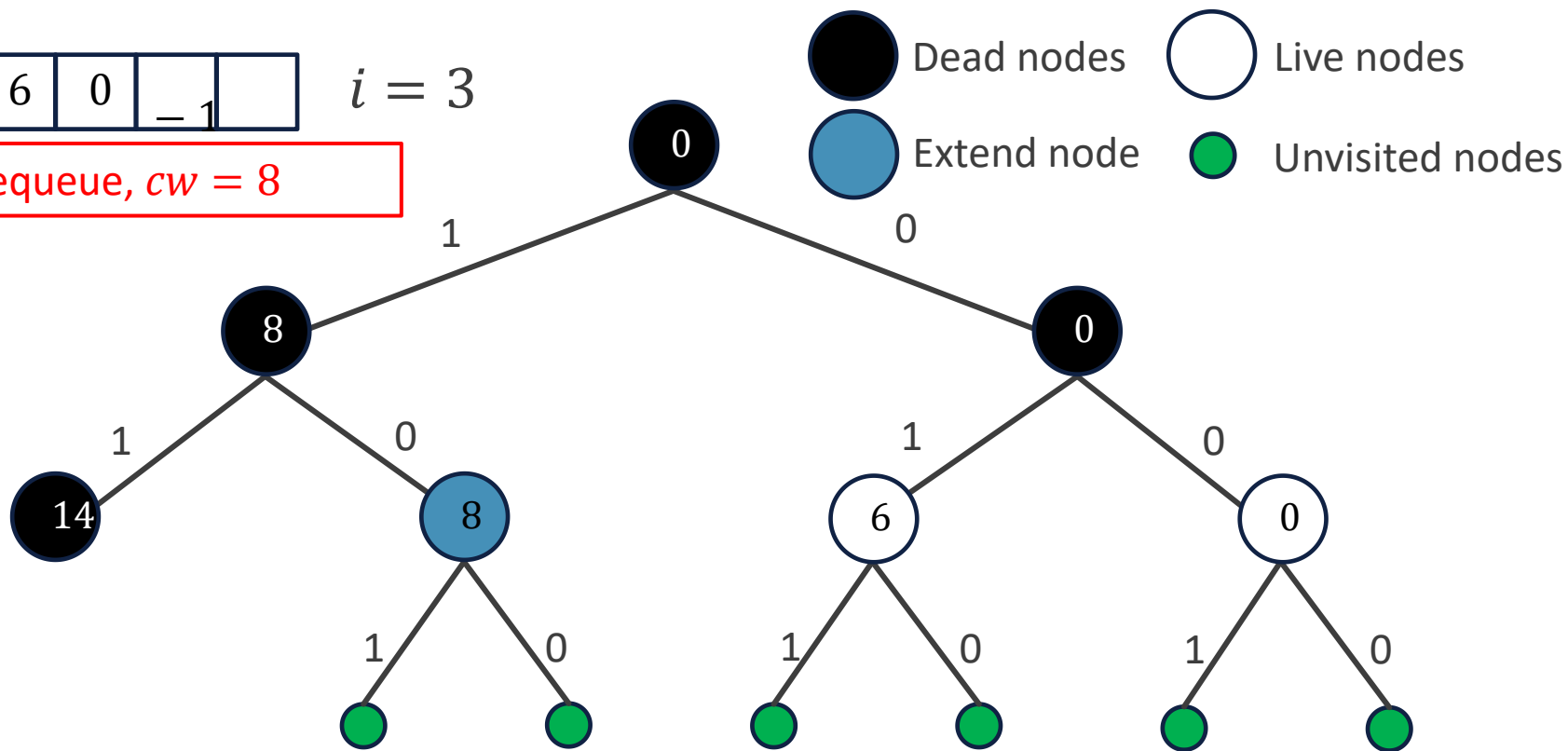
Example

Q:

6	0	-	1
---	---	---	---

 $i = 3$

dequeue, $cw = 8$



FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



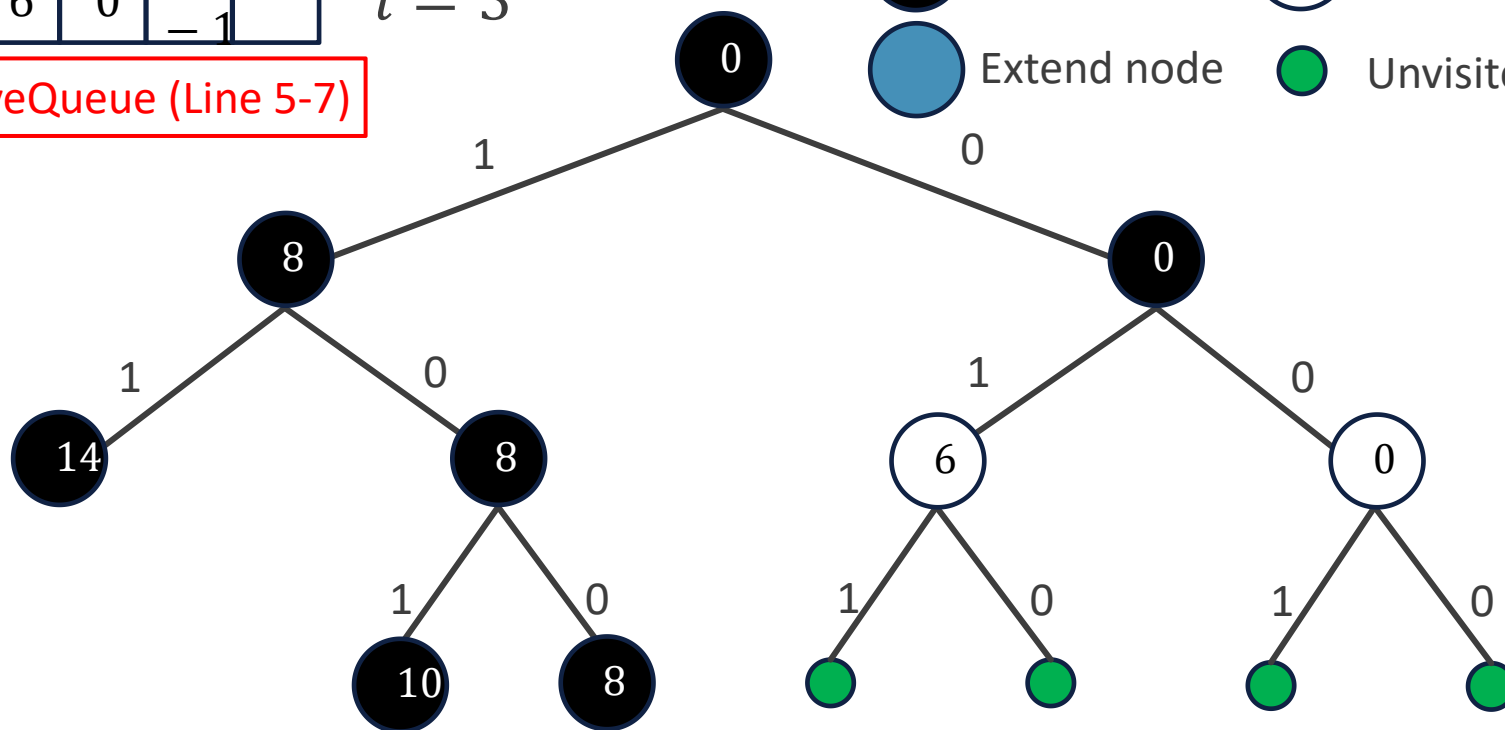
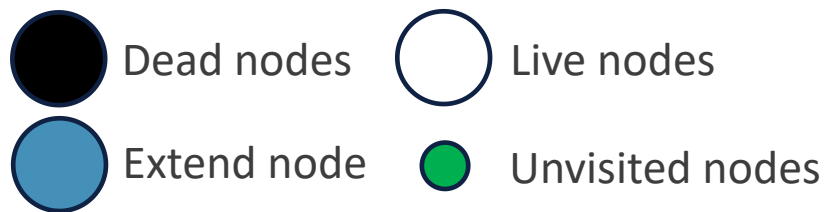
Example

Q:

6	0	-	1
---	---	---	---

 $i = 3$

SaveQueue (Line 5-7)



bestw = 10

FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



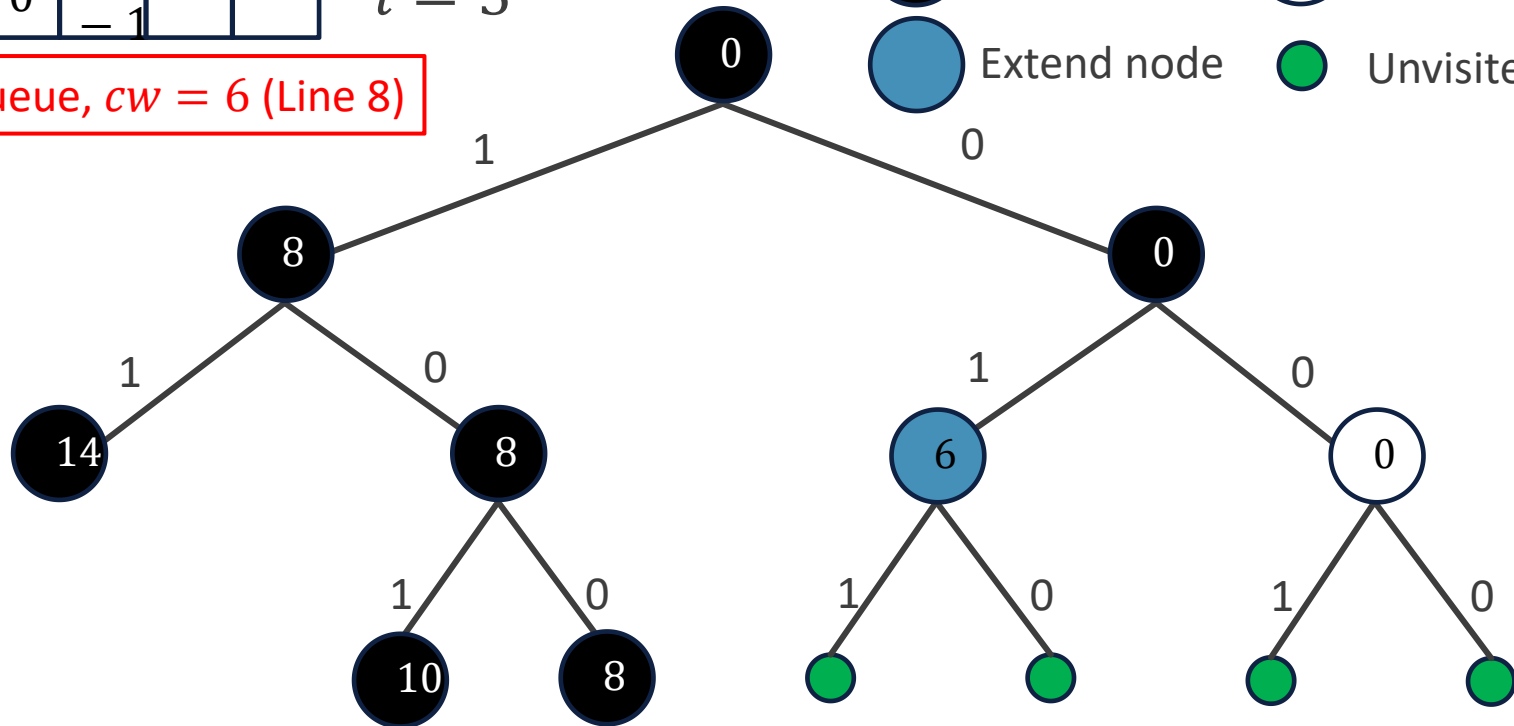
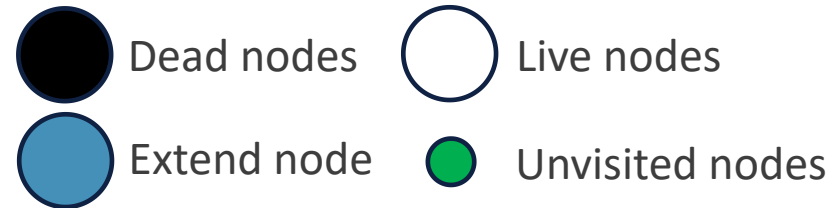
Example

Q:

0	-	1	
---	---	---	--

 $i = 3$

Dequeue, $cw = 6$ (Line 8)



$bestw = 10$

FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



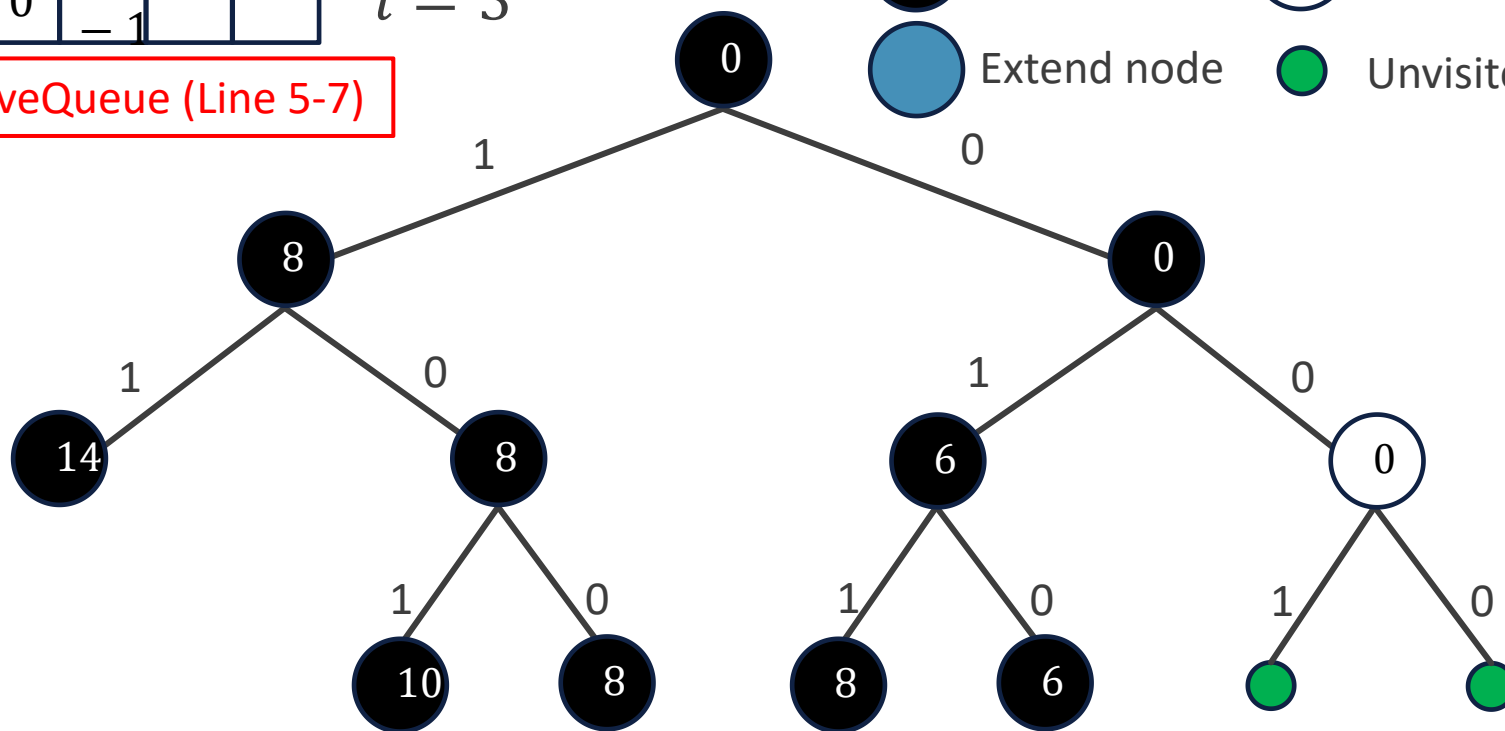
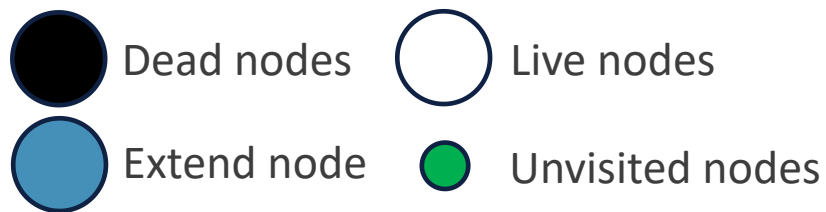
Example

Q:

0	-	1	
---	---	---	--

 $i = 3$

SaveQueue (Line 5-7)



bestw = 10

FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



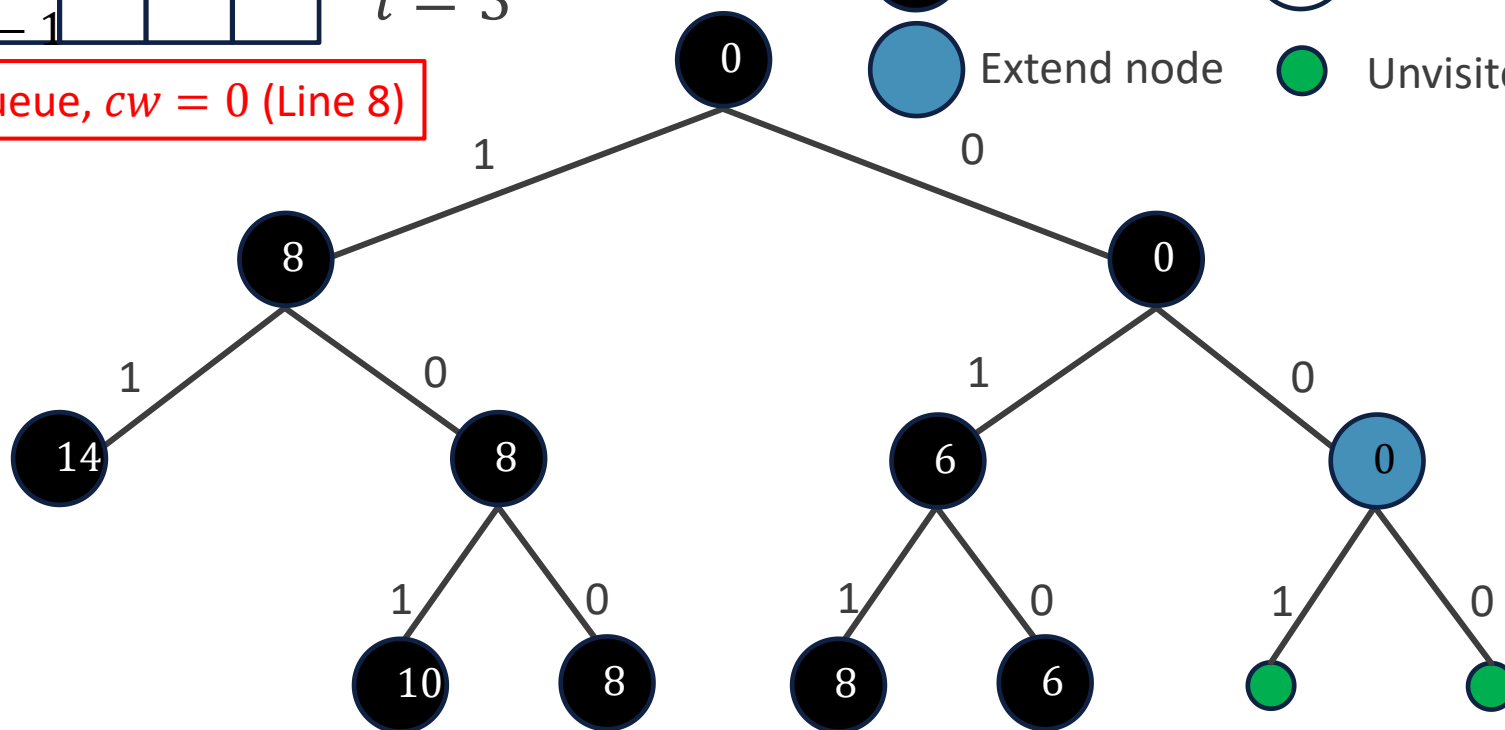
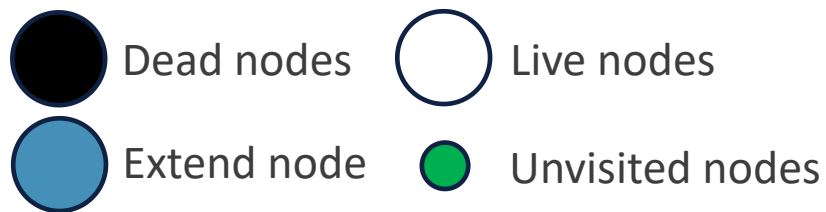
Example

Q:

-	1		
---	---	--	--

 $i = 3$

Dequeue, $cw = 0$ (Line 8)



$bestw = 10$

FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



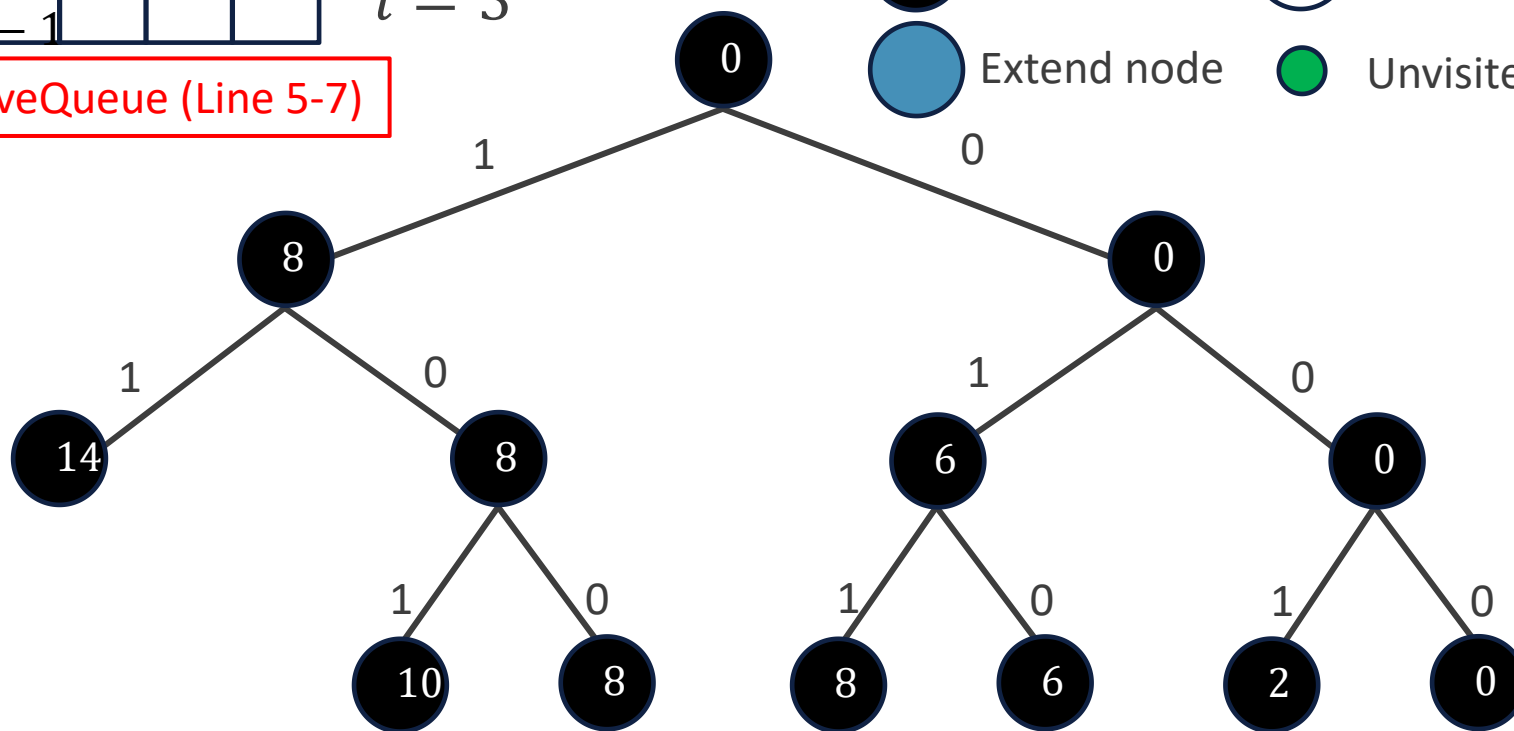
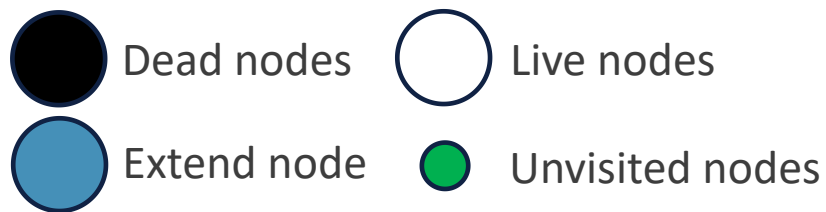
Example

Q:

-	1		
---	---	--	--

 $i = 3$

SaveQueue (Line 5-7)



$bestw = 10$

FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



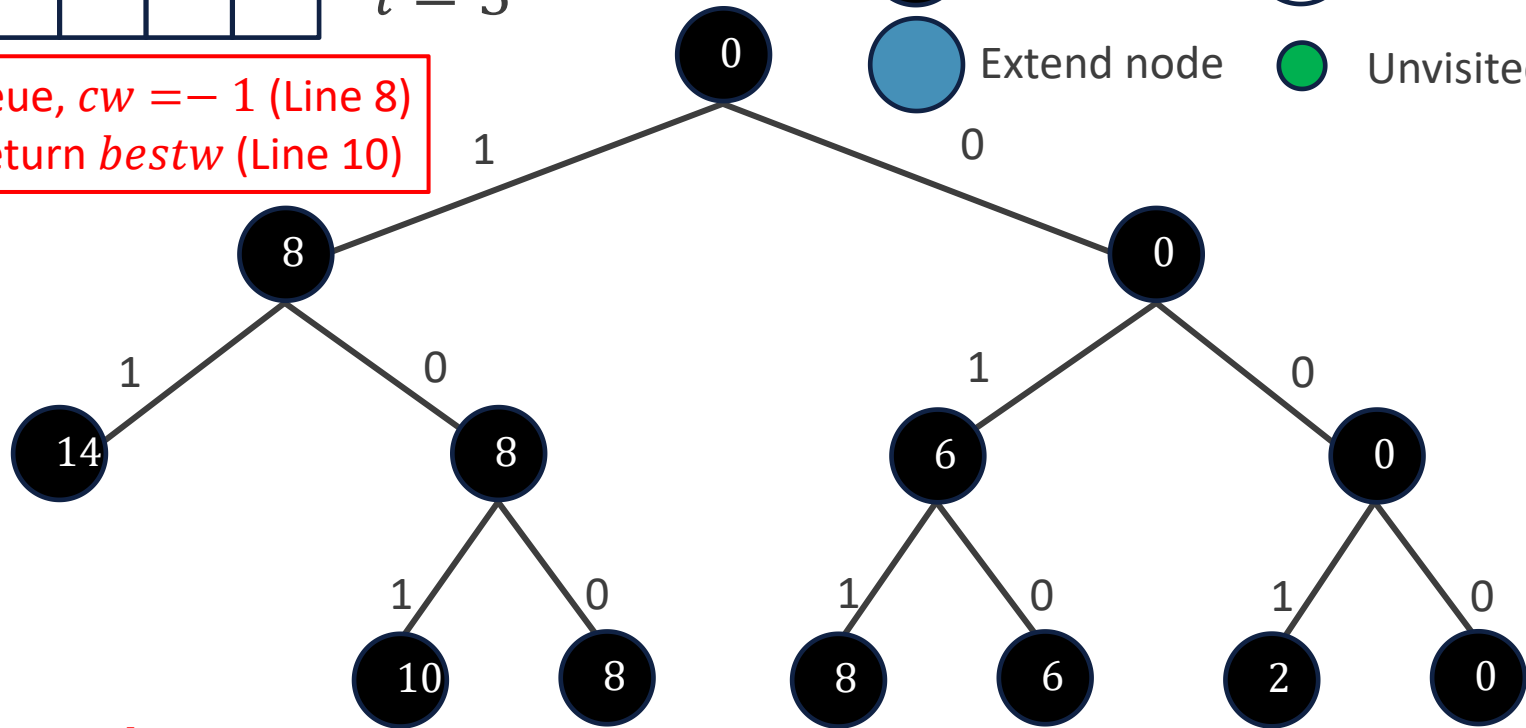
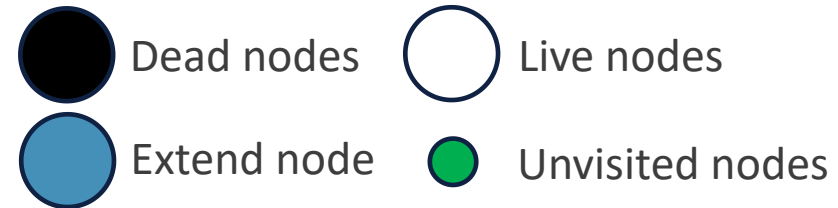
Example

Q:

--	--	--	--

 $i = 3$

Deque, $cw = -1$ (Line 8)
and return $bestw$ (Line 10)



$bestw = 10$

FIFO for $n = 3$, $w = [8, 6, 2]$, $W = 12$



FIFO Branch-and-Bound

- This version is obviously inefficient, because we didn't add the bounding function yet.
- We add the bounding function:

$$B(i) = C(i) + r(i)$$

where, $r(i)$ denotes the weight sum of the remaining containers, namely,

$$r(i) = \sum_{j=i+1}^n w_j$$

- The pruning condition is $B(i) \leq bestw$



FIFO Branch-and-Bound

ImprovedFIFOMaxLoading(w, W, n)

1 $i \leftarrow 1$

2 Enqueue($Q, -1$)

3 $cw \leftarrow 0$; $bestw \leftarrow 0$; $r \leftarrow 0$

4 **for** $j \leftarrow 2$ **to** n **do** $r \leftarrow r + w[j]$

5 **while** $Q \neq \emptyset$ **do**

6 **if** $C(i) \leq W$ **then**

7 **if** $C(i) > bestw$ **then** $bestw \leftarrow C(i)$

8 **if** $i < n$ **then** Enqueue($Q, C(i)$)

9 **if** $B(i) > bestw$ **and** $i < n$ **then** Enqueue(Q, cw)

10 $cw \leftarrow$ Dequeue(Q)

11 **if** $cw = -1$ **then**

12 **if** $Q = \emptyset$ **then return** $bestw$

13 Enqueue($Q, -1$)

14 $cw \leftarrow$ Dequeue(Q)

15 $i \leftarrow i + 1$

16 $r \leftarrow r - w[i]$

17 **return** $bestw$

Upper bound is calculated from the second item, and reduced when level increased

We don't enqueue leaf node

Enqueue live node with bounding condition



FIFO Branch-and-Bound

```
6          if  $C(i) \leq W$  then  
7              if  $C(i) > bestw$  then  $bestw \leftarrow C(i)$   
8              if  $i < n$  then Enqueue( $Q, C(i)$ )  
9          if  $B(i) > bestw$  and  $i < n$  then Enqueue( $Q, cw$ )
```

- Let's take a deep look into this part.
- Why can we update $bestw$ without checking it is a solution or not?
 - In backtracking, the bounding function works only after a feasible solution is obtained. 在回溯里面，只有当获得一个可行解时， $bestw$ 才会被更新
 - However, using FIFO, we can update $bestw$ first to kill more nodes at the same level. 分支限界法中， $bestw$ 可以很早被更新，于是可能剪枝更多
- This is the key factor that makes FIFO branch-and-bound efficient.



Example

Q :

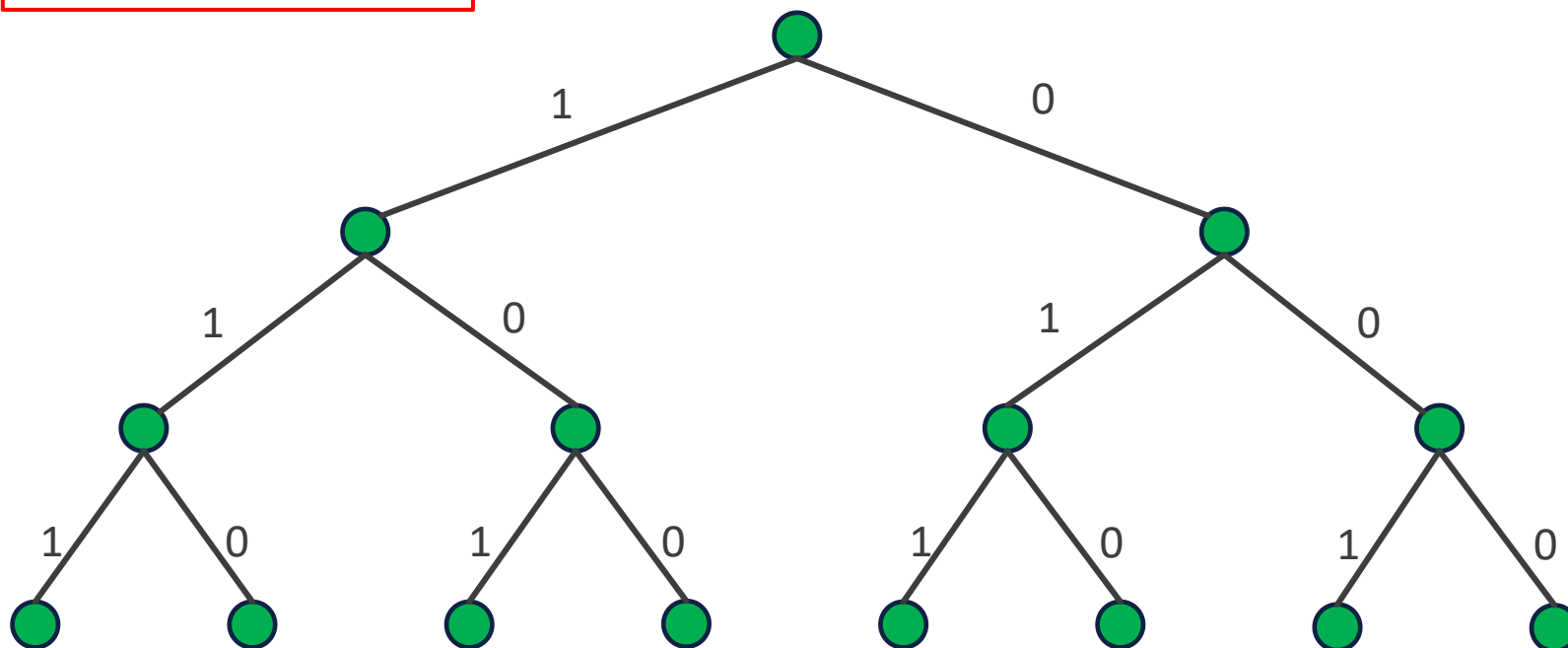
-1			
----	--	--	--

 $i = 1$

Initialization (Line 1-4)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

Q :

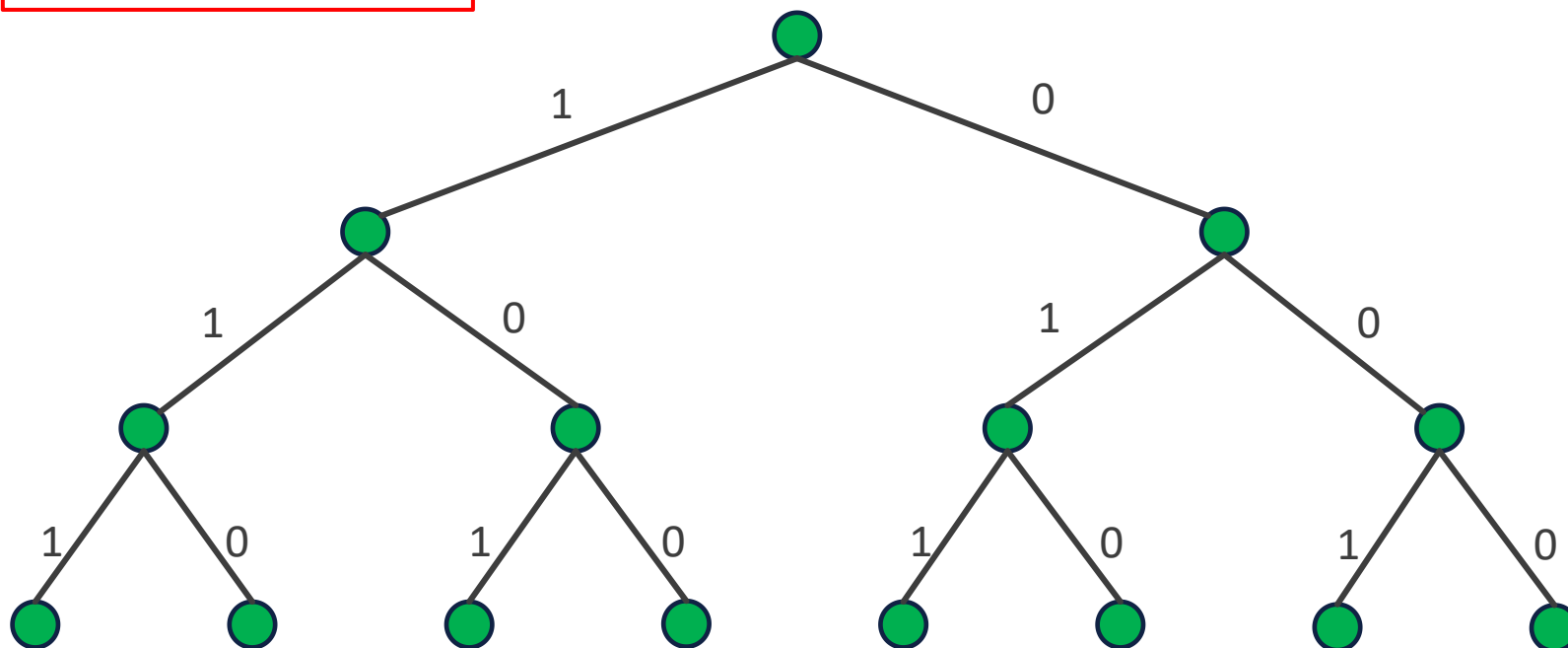
-1			
----	--	--	--

 $i = 1$

Initialization (Line 1-4)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Enqueue (Line 6-9)

A diagram showing a node labeled $0/16$ connected to a node labeled $0/8$. The edge between them is labeled 0 .

Directly kill this node!

FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

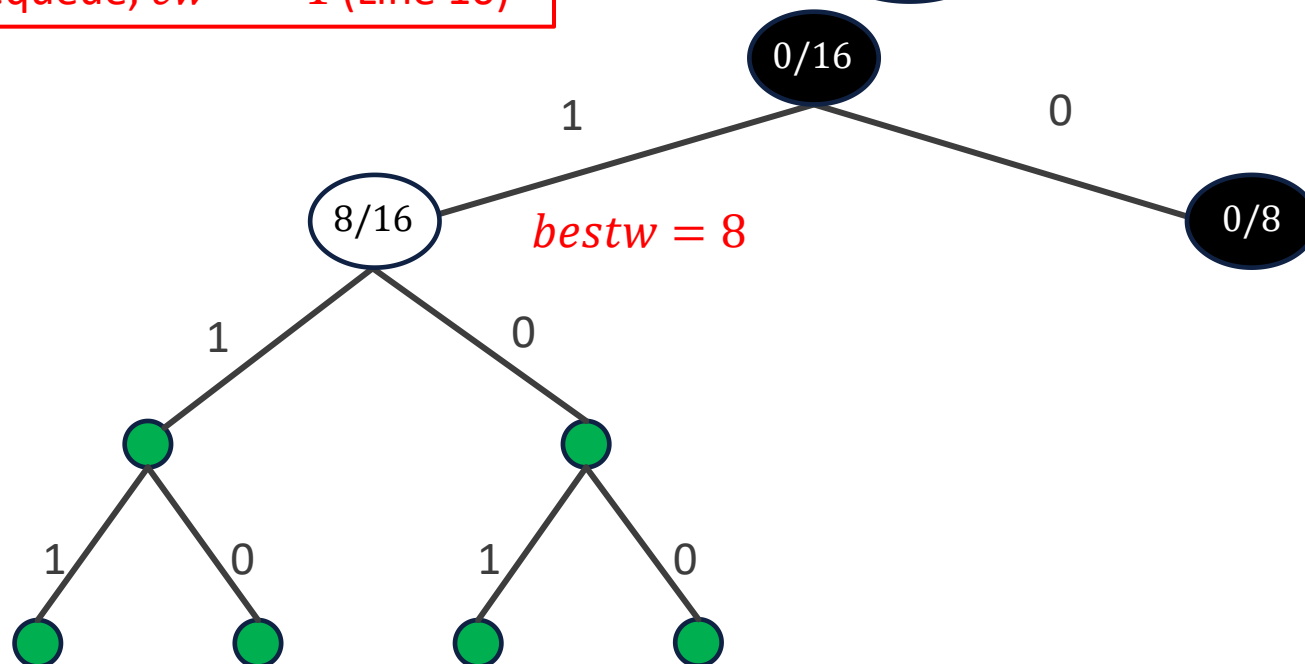
Q:

8			
---	--	--	--

 $i = 1$

Dequeue, $cw = -1$ (Line 10)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

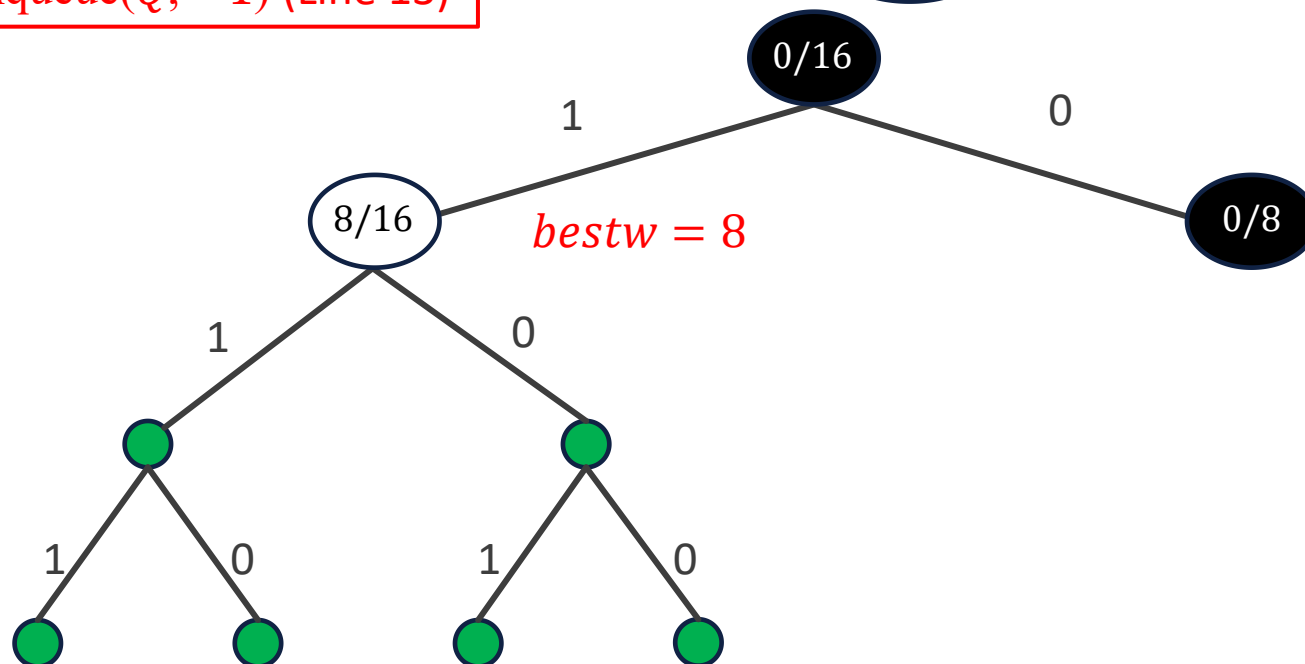
Q :

8	-1		
---	----	--	--

 $i = 1$

Enqueue($Q, -1$) (Line 13)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3, w = [8,6,2], W = 12$



Example

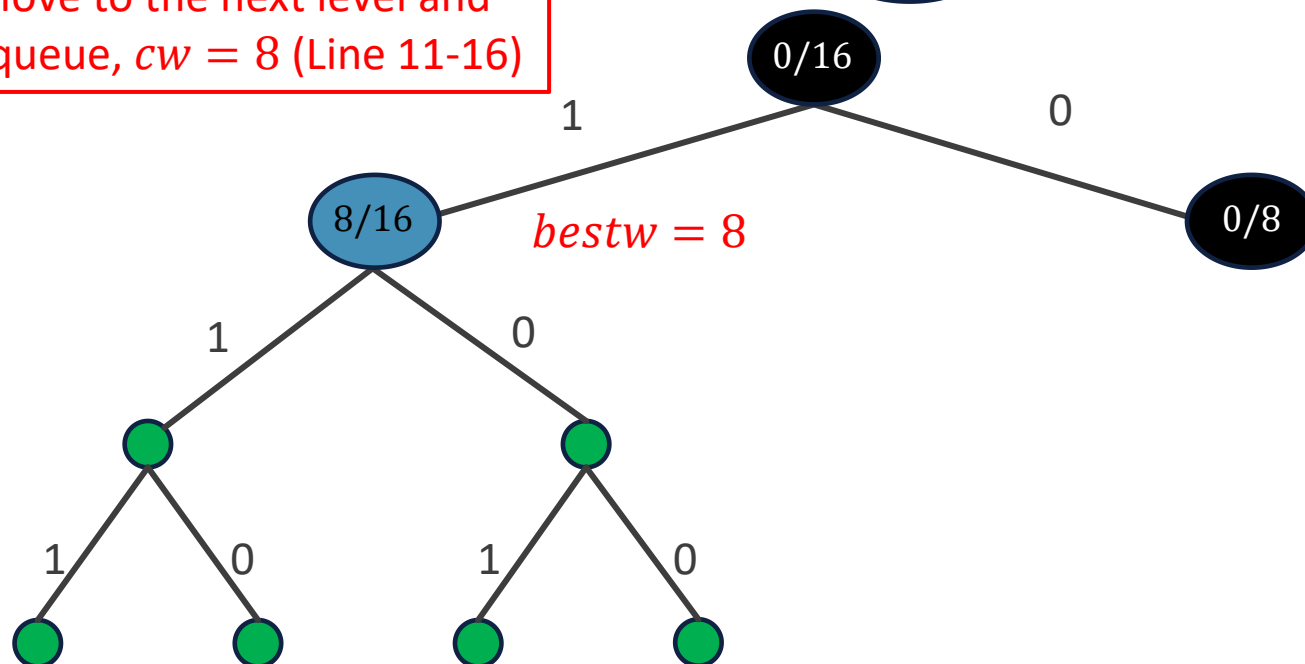
Q :

-1			
----	--	--	--

 $i = 2$

Move to the next level and
dequeue, $cw = 8$ (Line 11-16)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8,6,2]$, $W = 12$



Example

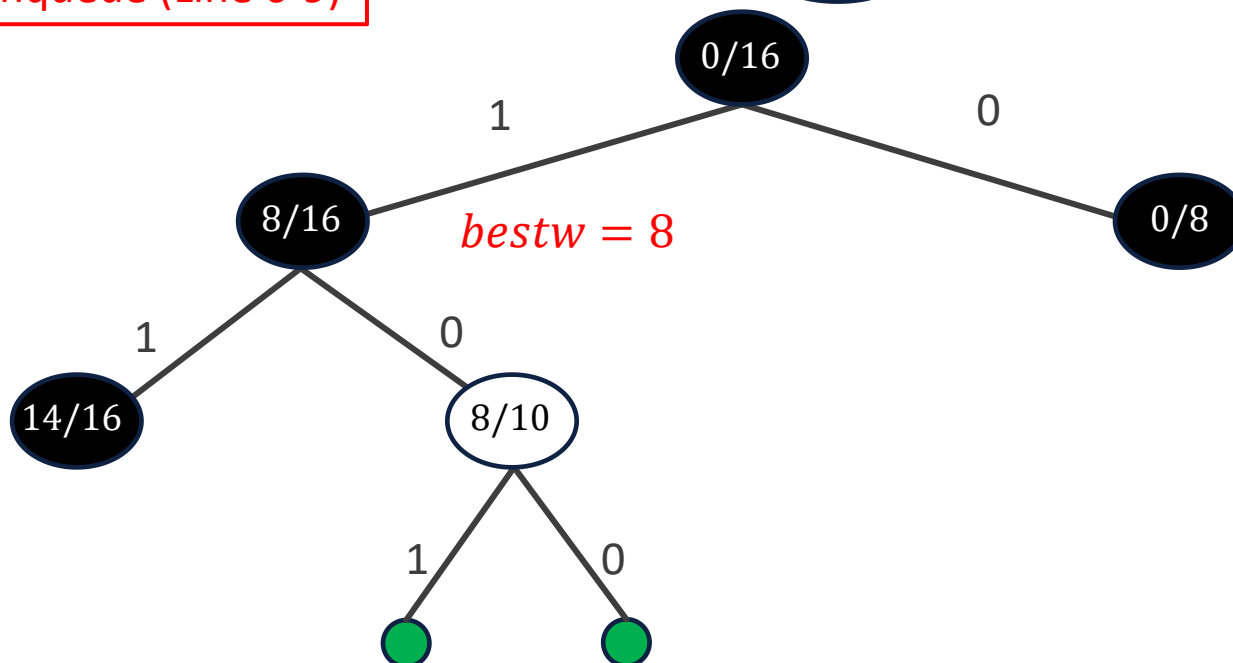
Q:

-1	8		
----	---	--	--

 $i = 2$

Enqueue (Line 6-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

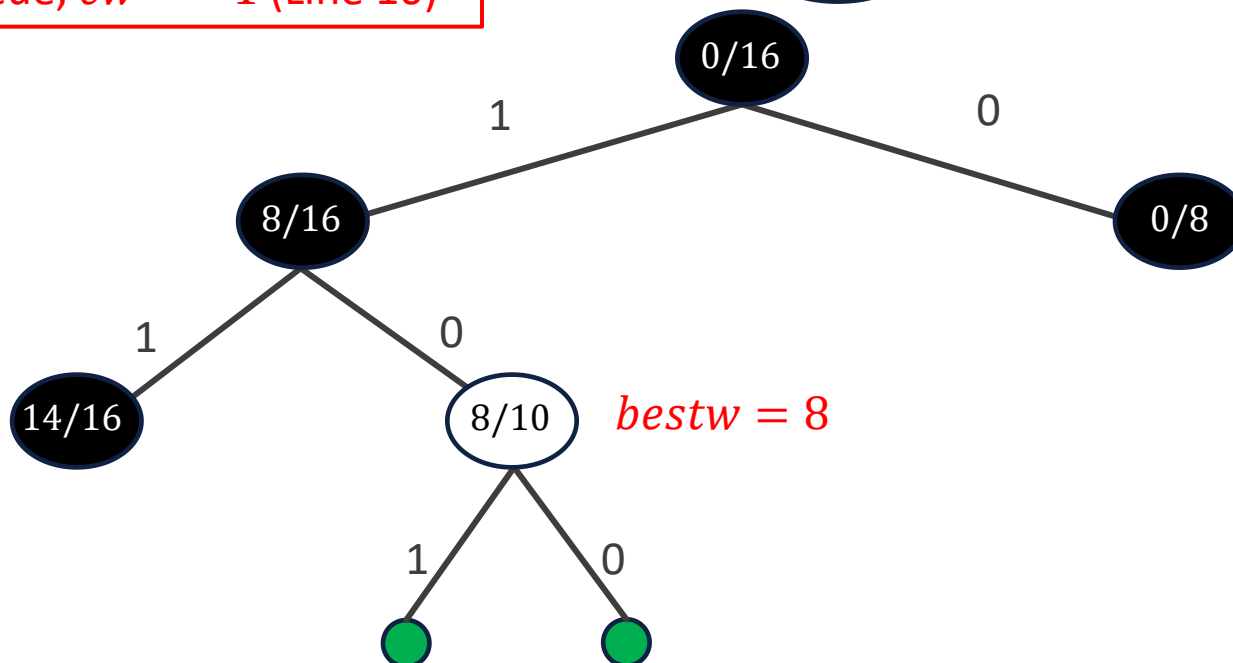
Q:

8			
---	--	--	--

 $i = 2$

Dequeue, $cw = -1$ (Line 10)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

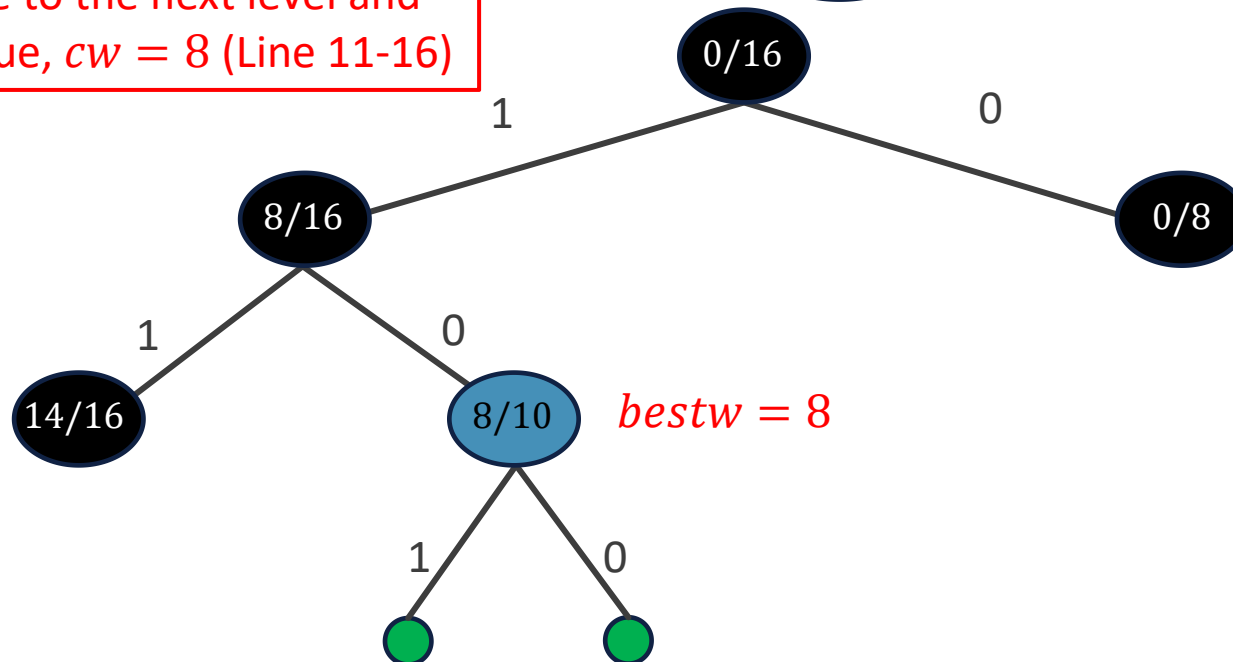
Q :

-1			
----	--	--	--

 $i = 3$

Move to the next level and
dequeue, $cw = 8$ (Line 11-16)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

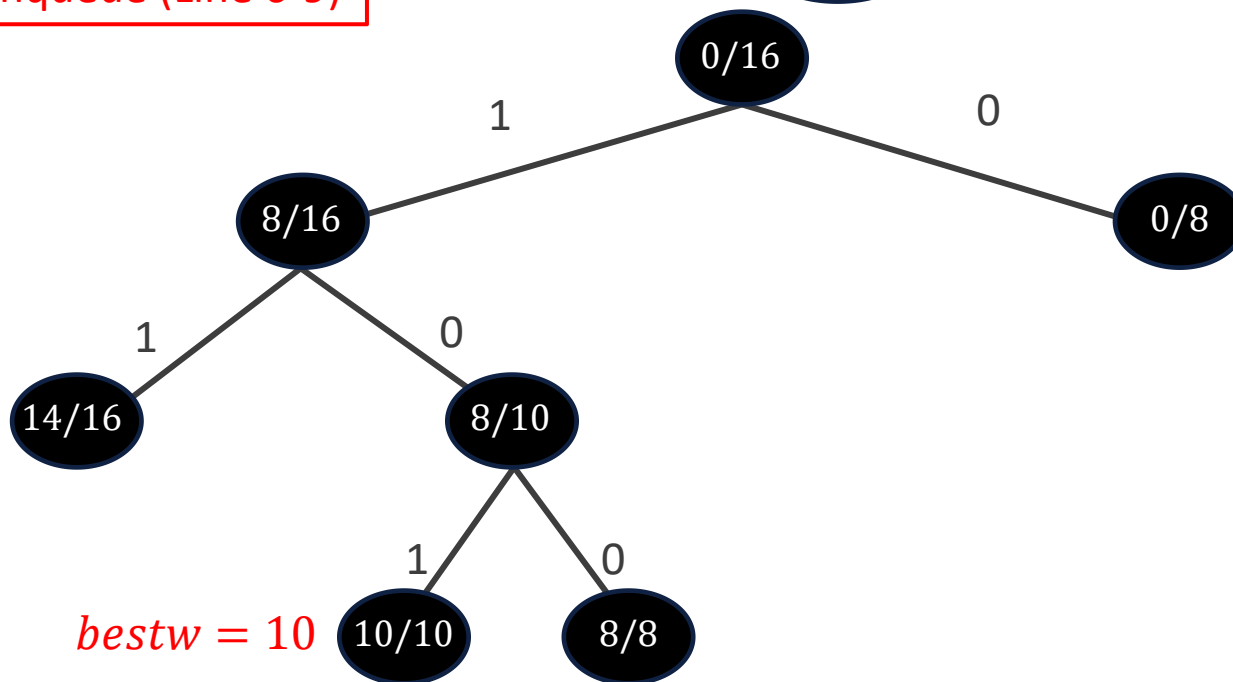
Q:

-1			
----	--	--	--

 $i = 3$

Enqueue (Line 6-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



$bestw = 10$

FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Example

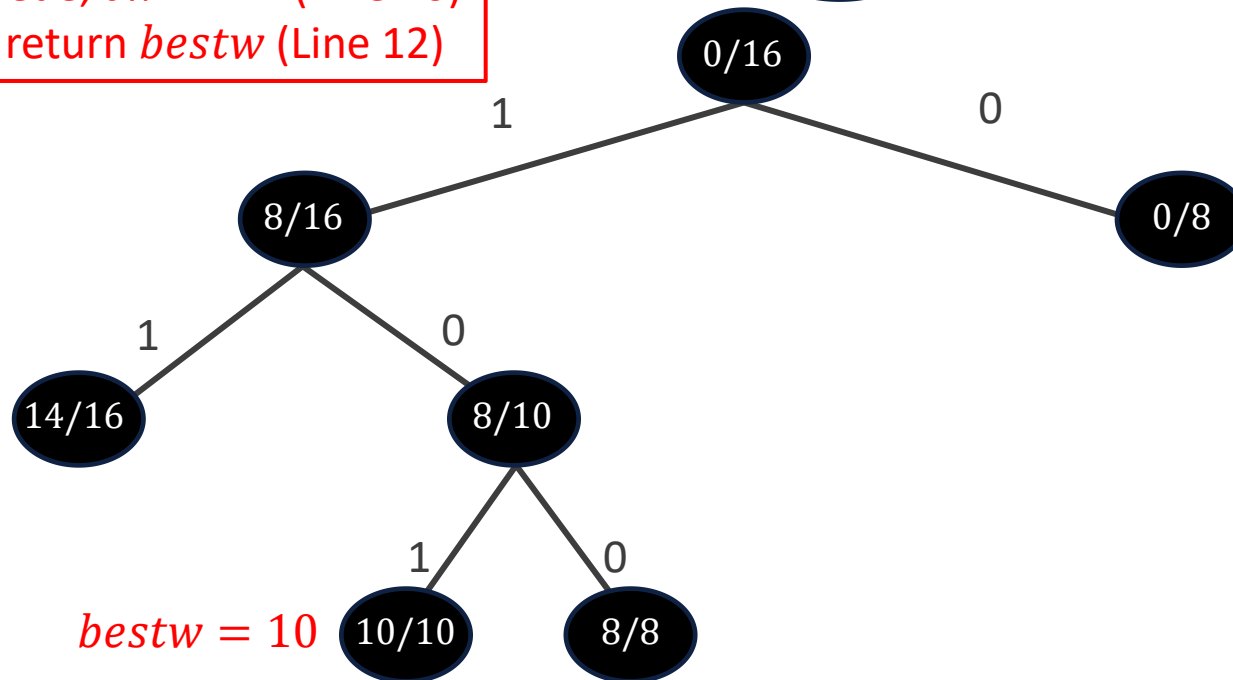
Q:

--	--	--	--

 $i = 3$

Dequeue, $cw = -1$ (Line 10)
and return $bestw$ (Line 12)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



$bestw = 10$

FIFO with bounding for $n = 3$, $w = [8, 6, 2]$, $W = 12$



Record the Solution

SolutionFIFOMaxLoading()

```
1   $i \leftarrow 1$ 
2  Enqueue( $Q$ , -1)
3   $cw \leftarrow 0$ ;  $bestw \leftarrow 0$ ;  $r \leftarrow 0$ 
4  for  $j \leftarrow 2$  to  $n$  do  $r \leftarrow r + w[j]$ 
5  while  $Q \neq \emptyset$  do
6      if  $C(i) \leq W$  then
7          SaveQueue( $Q$ ,  $C(i)$ ,  $i$ ,  $bestw$ ,  $E$ ,  $bestE$ ,  $bestx$ , 1)
8      if  $B(i) > bestw$  then
9          SaveQueue( $Q$ ,  $cw$ ,  $i$ ,  $bestw$ ,  $E$ ,  $bestE$ ,  $bestx$ , 0)
10      $E \leftarrow$  Dequeue( $Q$ )
11     if  $E = -1$  then
12         if  $Q = \emptyset$  then return  $bestw$ 
13         Enqueue( $Q$ , -1);  $E \leftarrow$  Dequeue( $Q$ );  $i \leftarrow i + 1$ ;  $r \leftarrow r - w[i]$ 
14      $cw \leftarrow E.weight$ 
15 for  $j \leftarrow n - 1$  downto 1 do
16      $bestx[j] \leftarrow bestE.Lchild$ 
17      $bestE \leftarrow bestE.parent$ 
18 return  $bestw$ 
```

$bestx[i]$ records the
decision made on step i .

Use data structure:

E.weight: Current weight

E.parent: Parent node

E.Lchild: Decision (0/1)

SaveQueue(Q , wt , i , $bestw$,
 E , $bestE$, $bestx$, ch)

1 if $i = n$ then

2 if $wt > bestw$ then

3 $bestE \leftarrow E$

4 $bestw \leftarrow wt$

5 $bestx[n] \leftarrow ch$

6 else

7 $b.weight \leftarrow wt$

8 $b.parent \leftarrow E$

9 $b.Lchild \leftarrow ch$

10 Enqueue(Q , b)



FIFO Using Constraint Function and Bounding Function

- It seems as good as backtracking. Can we further improve?
- The current version of branch-and-bound uses FIFO, just like backtracking using FILO.
 - It is still limited by FIFO when we are branching.
- Can we choose the node to branch out of the order determined by FIFO or FILO? 可不可以用不同于FIFO或者FILO的方式来选择结点进行展开?



Max-Profit Branch-and-Bound

- Instead, we use max-priority queue. Select node with maximum upper bound!
- Live nodes become E -nodes in decreasing order of $B(i)$.
 - Notice that if x is a node with an upper bound, then no node in its subtree has weight more than this upper bound.
- When do we stop? The node with maximum upper bound is a leaf, which means no remaining live node can lead to a leaf with more weight. 目前某个叶子节点有最大的 $B(i)$, 意味着其他的中间节点的 $B(i)$ 比他小, 由于这些中间节点的叶子节点的 $B(i)$ 时候小于等于中间节点的 $B(i)$, 所以可以判定现在的这个在队列最前面的叶子节点是最优解



Max-Profit Branch-and-Bound

MaxCostLoading()

```
1   $i \leftarrow 1$ 
2   $r[n] \leftarrow 0$ 
3  for  $j \leftarrow n - 1$  downto 1 do  $r[j] \leftarrow r[j + 1] + w[j + 1]$ 
4  while  $i \neq n + 1$  do
5      if  $C(i) \leq W$  then
6          AddLiveNode( $Q, E, C(i) + r[i], 1, i + 1$ )
7          AddLiveNode( $Q, E, cw + r[i], 0, i + 1$ )
8           $N \leftarrow \text{ExtractMax}(Q)$ 
9           $i \leftarrow N.level$ 
10          $E \leftarrow N.ptr$ 
11          $cw \leftarrow N.weight - r[i - 1]$ 
12     for  $j \leftarrow n$  downto 1 do
13          $bestx[j] \leftarrow E.Lchild$ 
14          $E \leftarrow E.parent$ 
15 return  $cw$ 
```

In FIFO, the level is always increasing, so we don't need to store it. Now, we need to store level.

Use data structure in max-priority queue:

Stop if extracted node is a leaf.

N.weight: Node upper bound
N.level: Node level
N.ptr: Pointer to node *E*
E.parent: Parent node
E.Lchild: Decision (0/1)

No bounding condition here.
You can also add it.

```
AddLiveNode( $Q, E, wt, ch, lev$ )
1   $b.parent \leftarrow E$ 
2   $b.Lchild \leftarrow ch$ 
3   $N.weight \leftarrow wt$ 
4   $N.level \leftarrow lev$ 
5   $N.ptr \leftarrow b$ 
6  Insert( $Q, N$ )
```



Max-Profit Branch-and-Bound

We store upper bounds, rather than current weight

```
5  if  $C(i) \leq W$  then
6      AddLiveNode( $Q, E, C(i) + r[i], 1, i + 1$ )
7      AddLiveNode( $Q, E, cw + r[i], 0, i + 1$ )
8       $N \leftarrow \text{ExtractMax}(Q)$ 
9       $i \leftarrow N.\text{level}$ 
10      $E \leftarrow N.\text{ptr}$ 
11      $cw \leftarrow N.\text{weight} - r[i - 1]$ 
```

The current weight is calculated by upper bound – remaining weight.



Example

Since we store level, we don't need -1 any more.

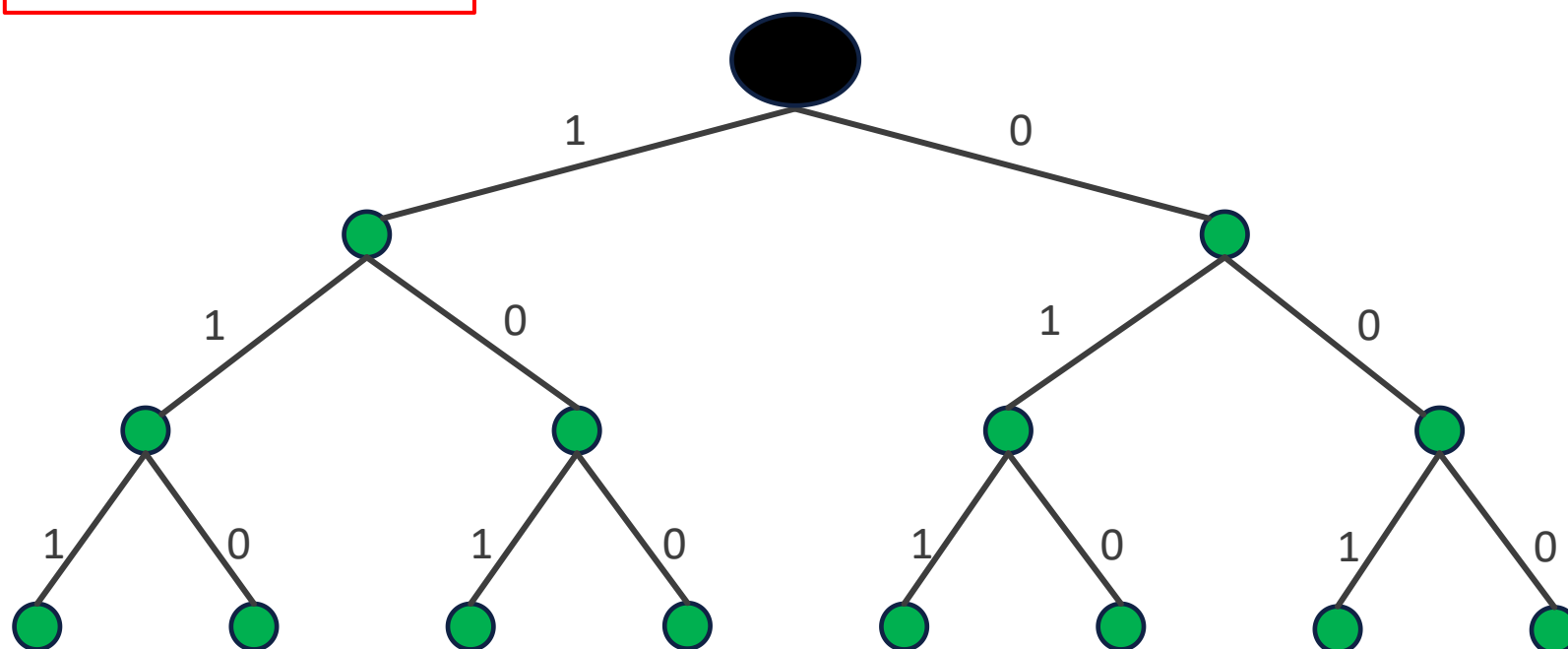
Q:



Initialization (Line 1-3)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node  Unvisited nodes



Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$

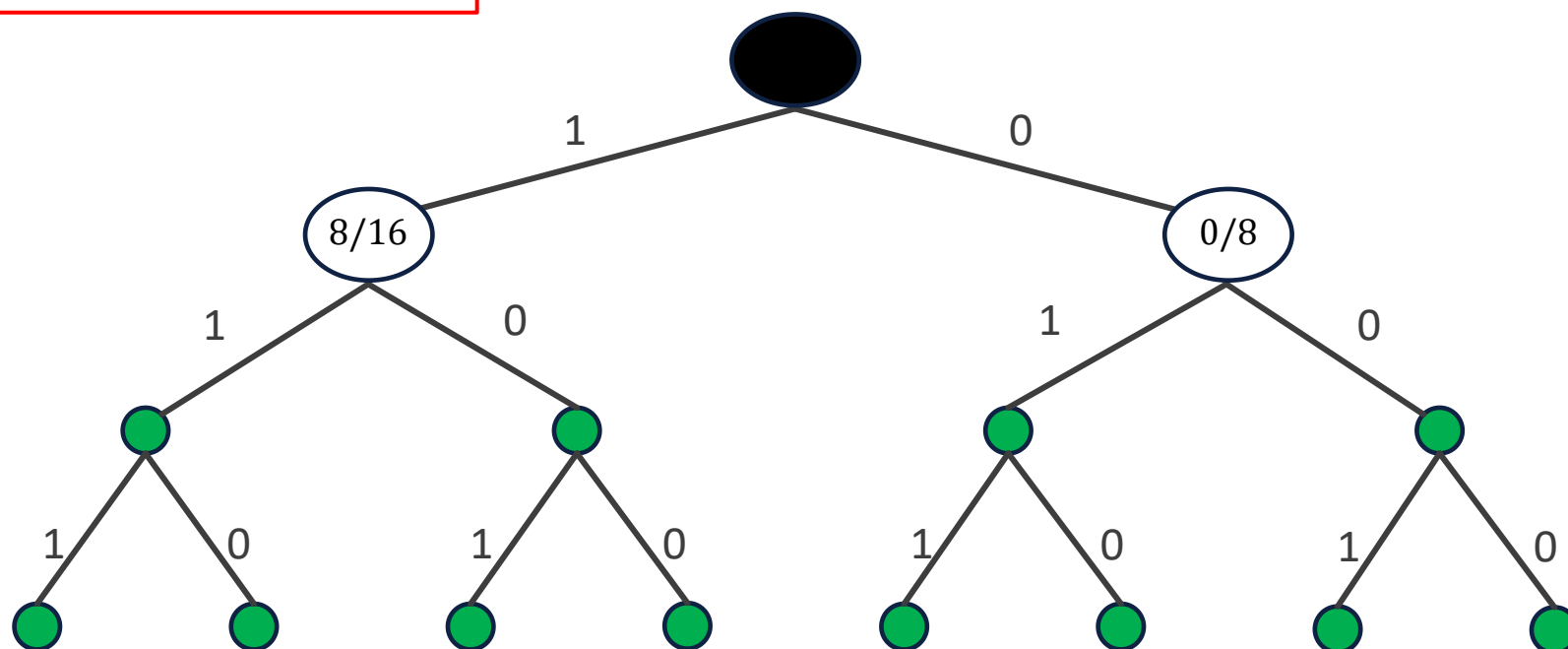
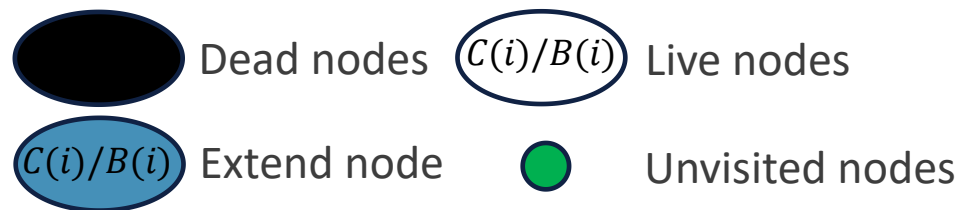


Example

Q:

16	8		
----	---	--	--

AddLiveNode (Line 6-7)



Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$



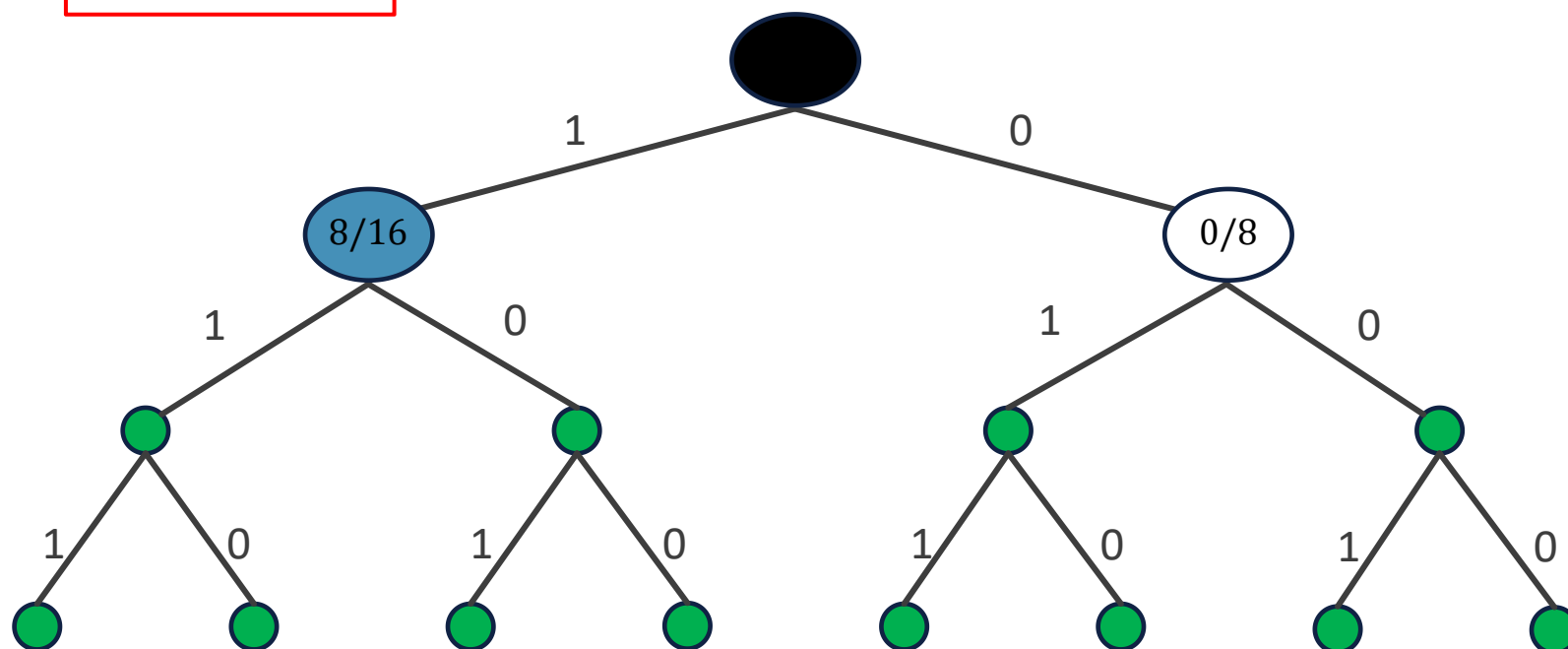
Example

Q:

8			
---	--	--	--

ExtractMax: 16

Dead nodes Live nodes
 Extend node Unvisited nodes



Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$

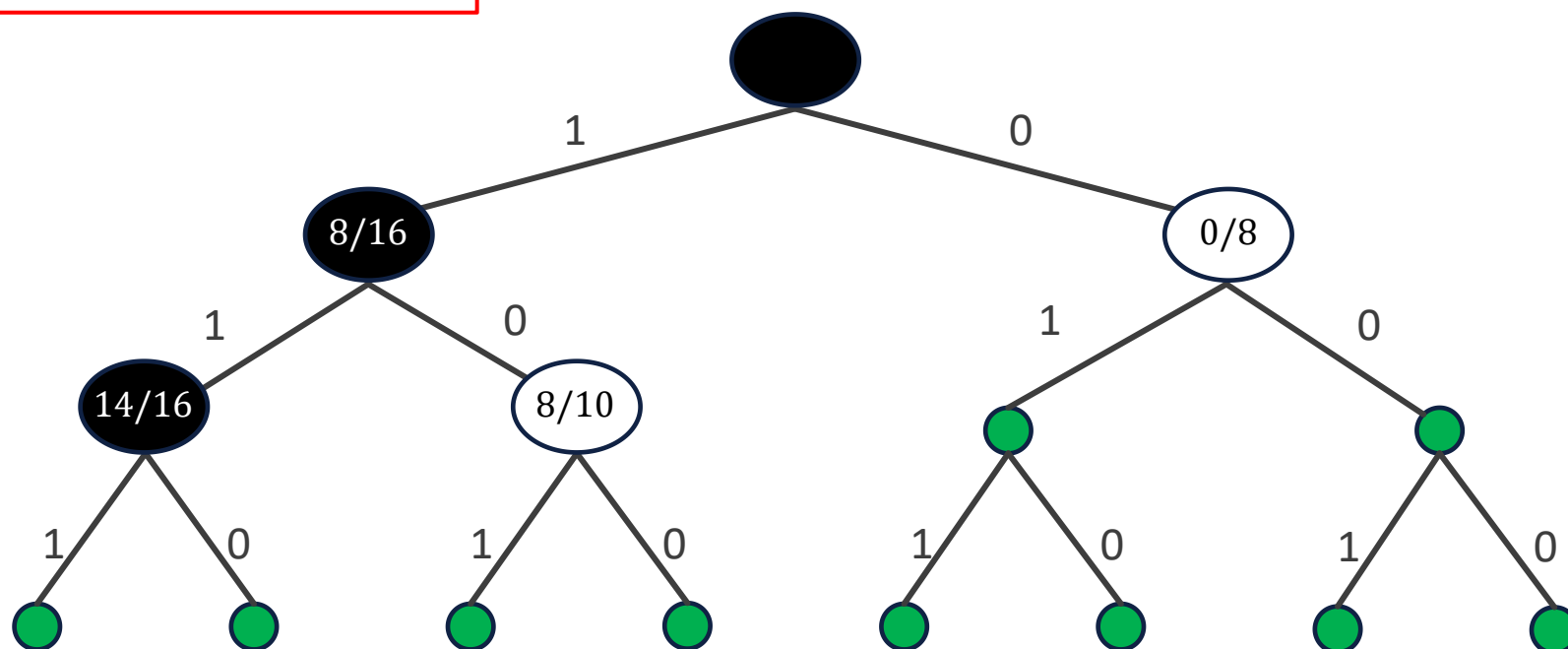
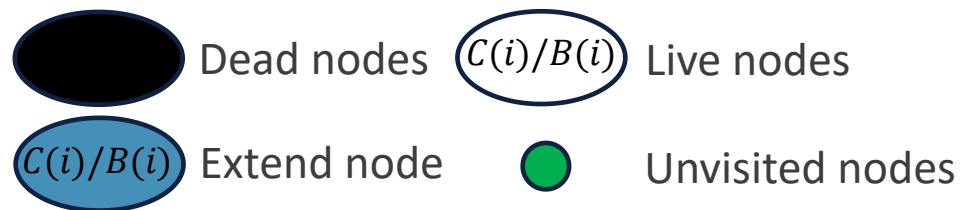


Example

Q:

10	8		
----	---	--	--

AddLiveNode (Line 6-7)



Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$



8

ExtractMax: 10

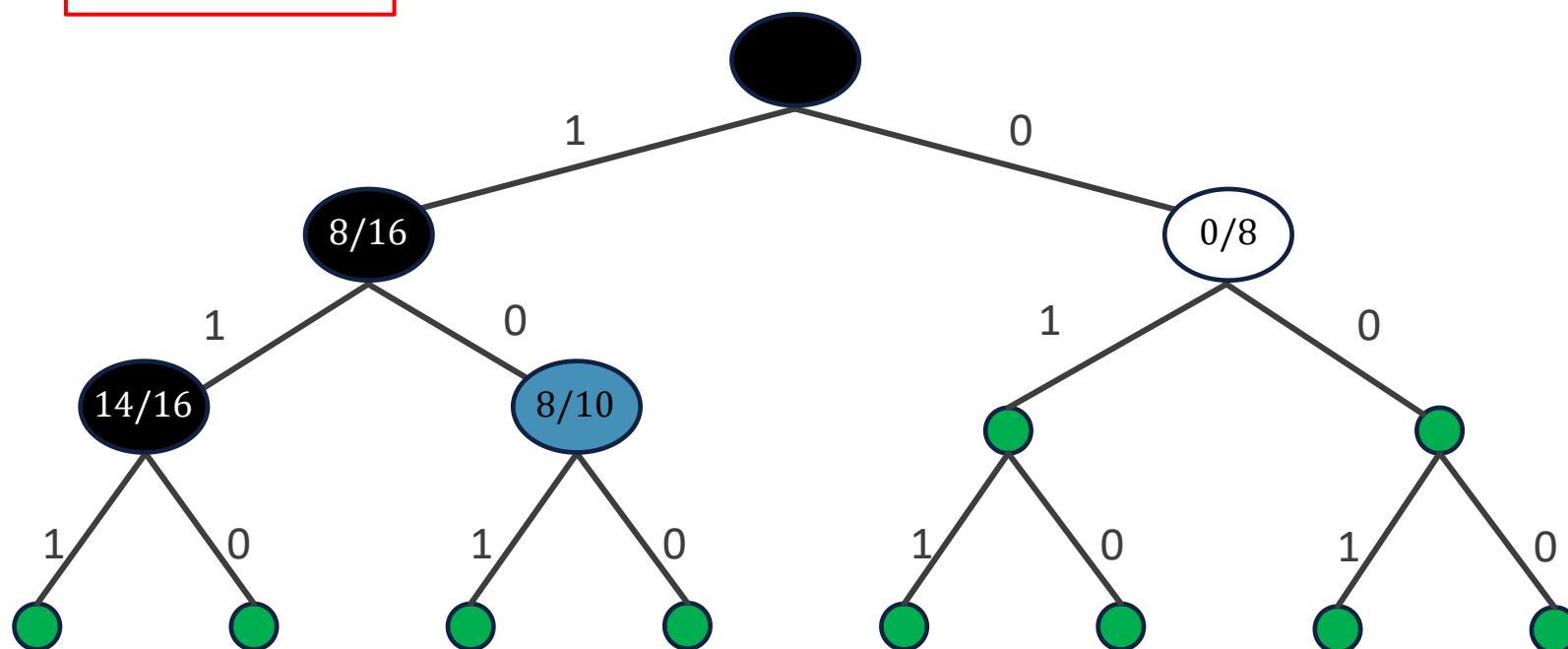
$$C(i)/B(i)$$

Live nodes

$$C(i)/B(i)$$

Extend node

Unvisited nodes



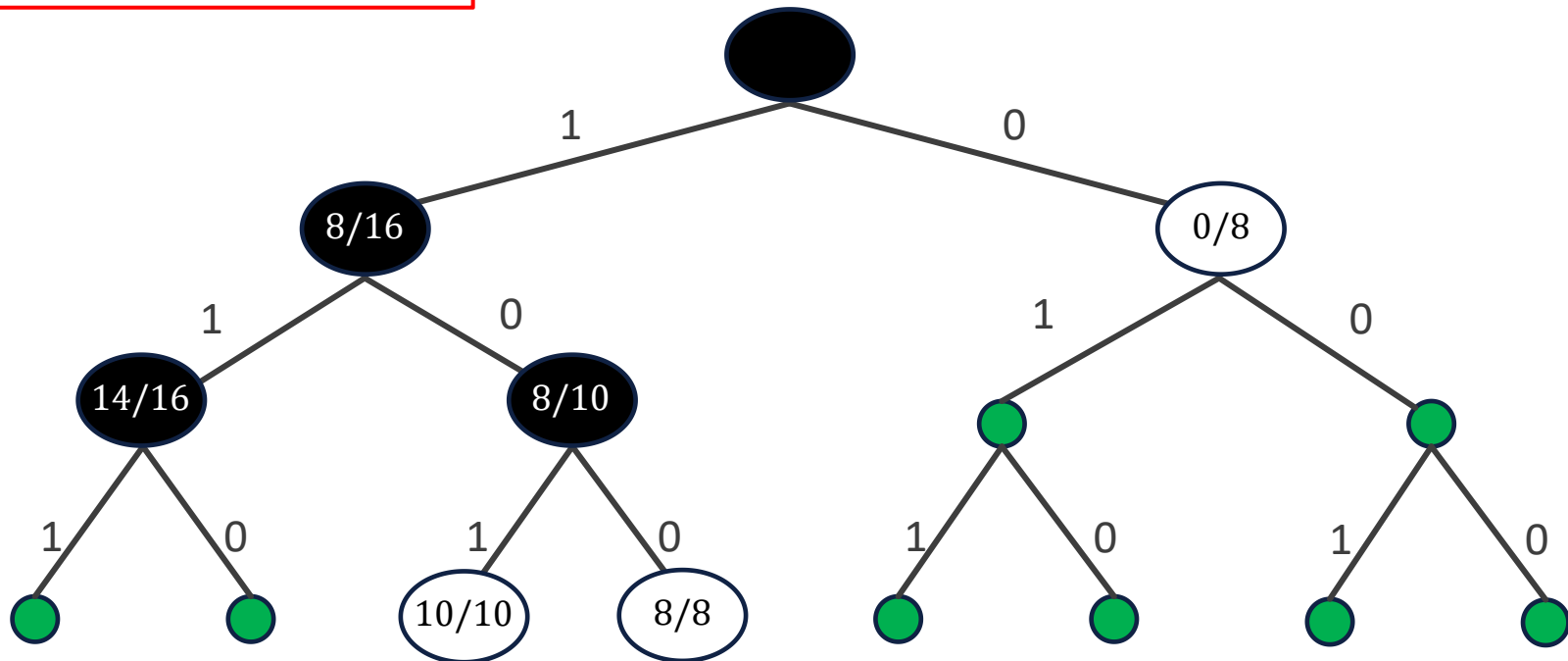
Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$



AddLiveNode (Line 6-7)

 Dead nodes
  Live nodes

 Extend node
  Unvisited nodes



Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$



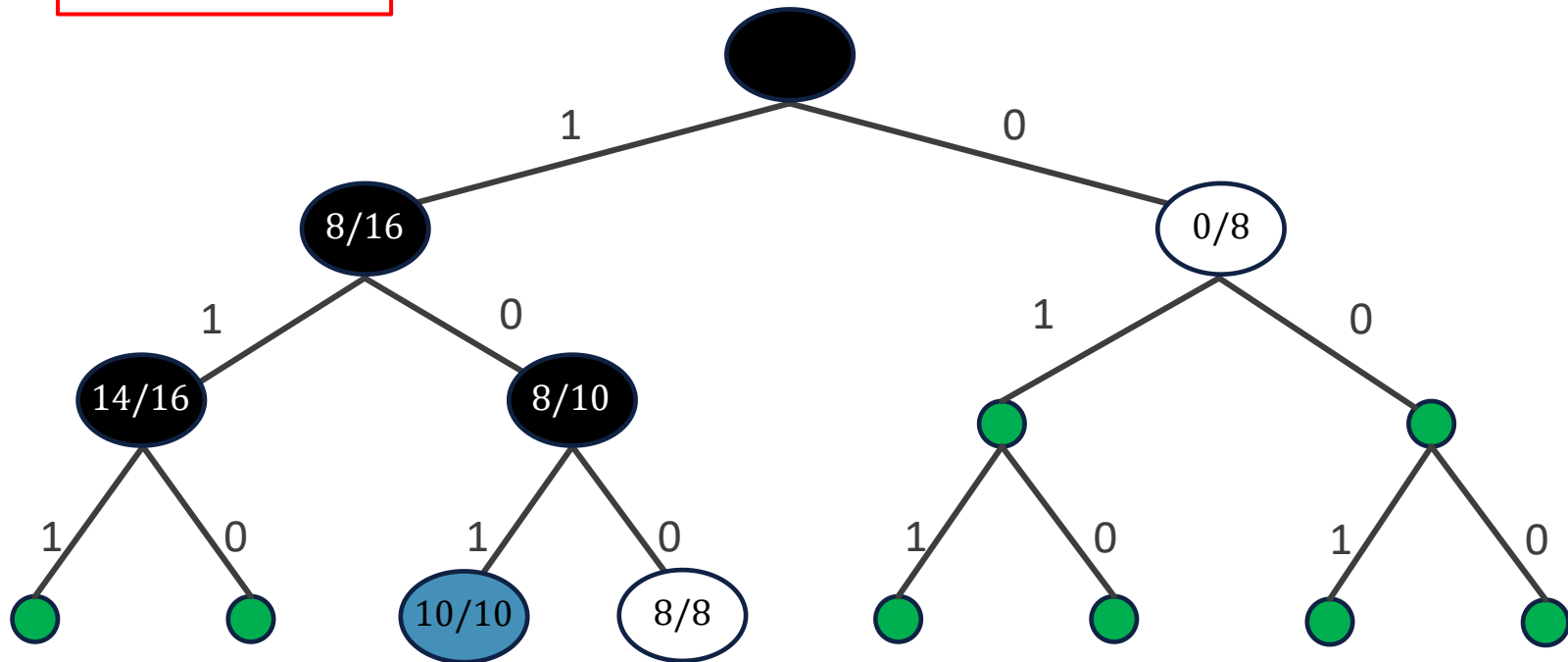
Example

Q:

8	8		
---	---	--	--

ExtractMax: 10

Dead nodes Live nodes
 Extend node Unvisited nodes



Max-profit branch-and-bound for $n = 3$, $w = [8, 6, 2]$, $W = 12$

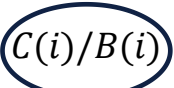
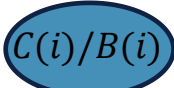


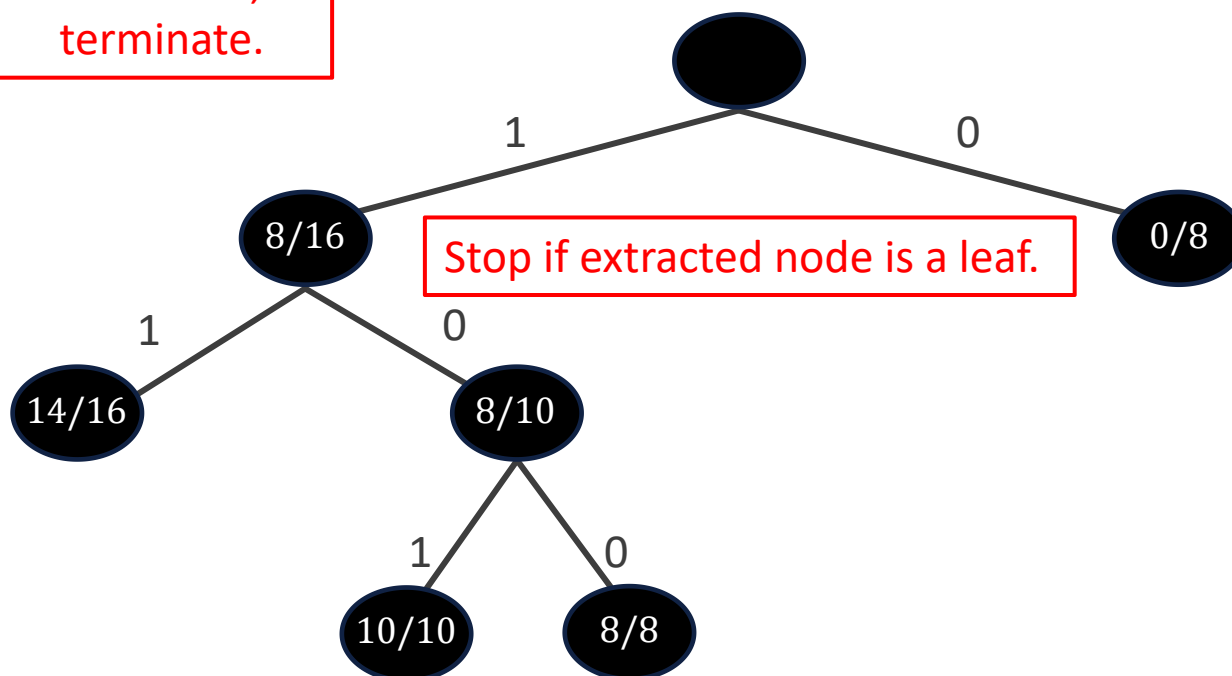
Example

Q:

8	8		
---	---	--	--

$i = n + 1,$
terminate.

 Dead nodes  Live nodes
 Extend node  Unvisited nodes



Max-profit branch-and-bound for $n = 3, w = [8, 6, 2], W = 12$



Branch-and-Bound

- Now, we look back the name of branch-and-bound:
- Branch: We explore all of candidate branches.
 - That's why branch-and-bound is based on BFS.
- Bound: We select a branch based on its bound.
 - Bound represents the degree of hope.



Classroom Exercise

- Draw the pruned solution space tree for the following container loading problem instance by FIFO branch-and-bound and max-profit branch-and-bound.

$$n = 4, w = [4, 7, 5, 3], W = 15$$

- Compare these two results with the one solved by backtracking.

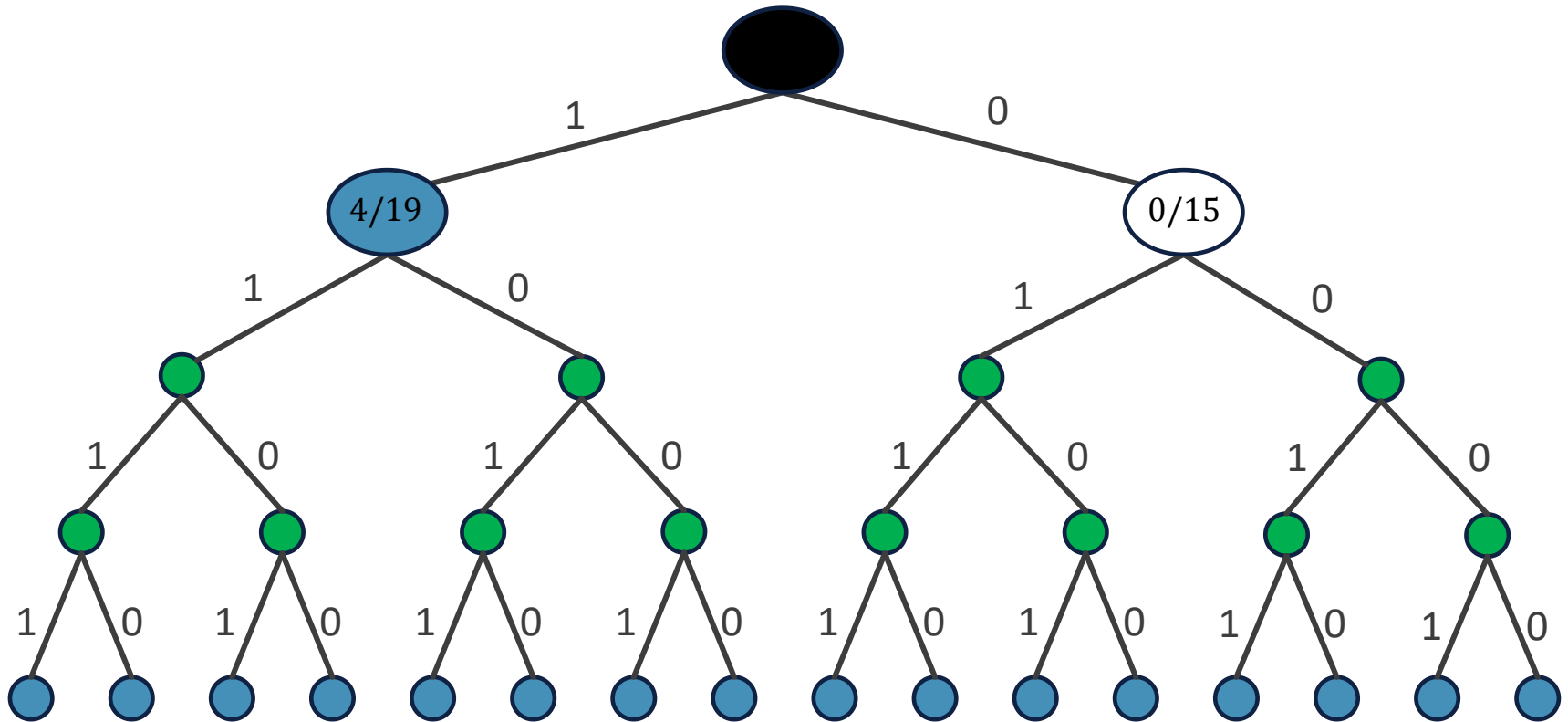


Q:

19	15		
----	----	--	--

Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$

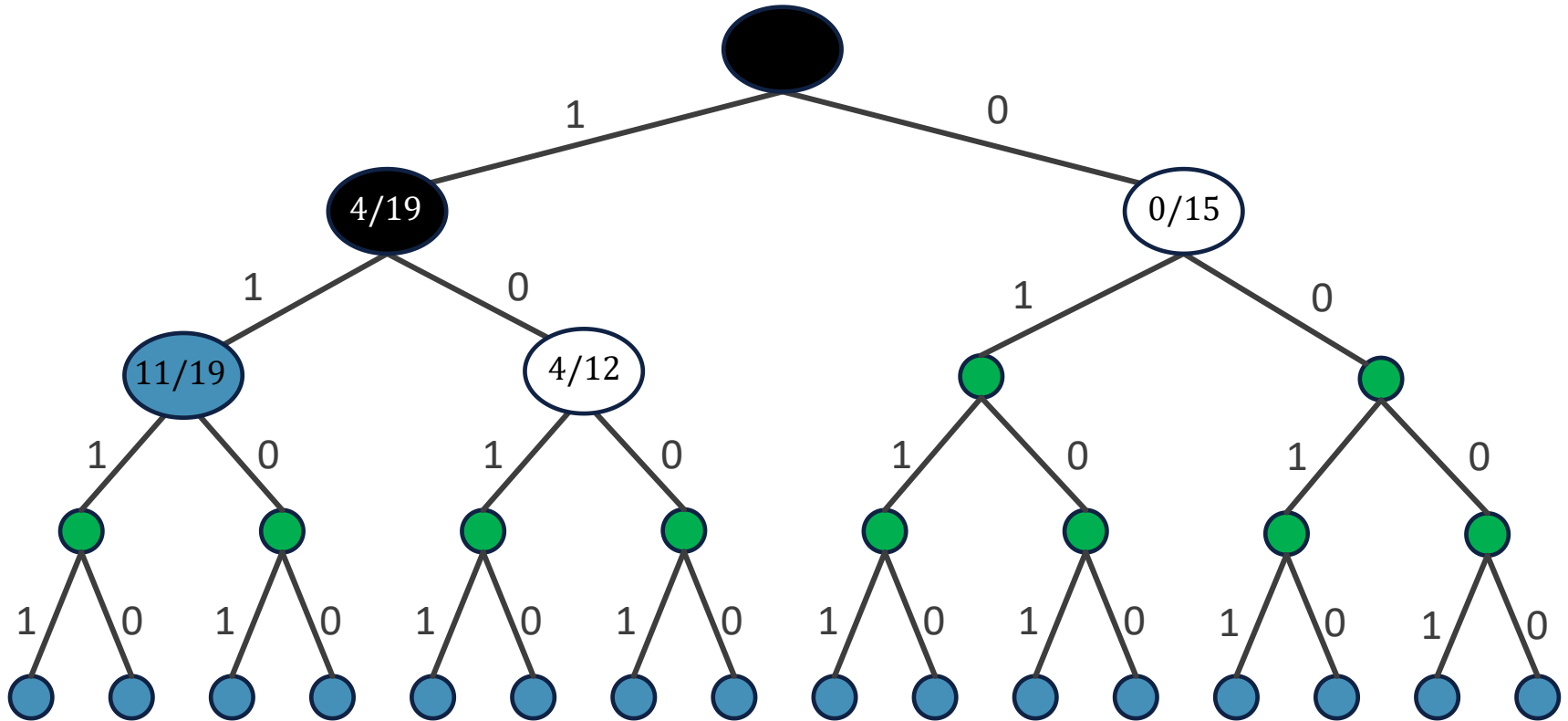


Q:

19	15	12	
----	----	----	--

Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$

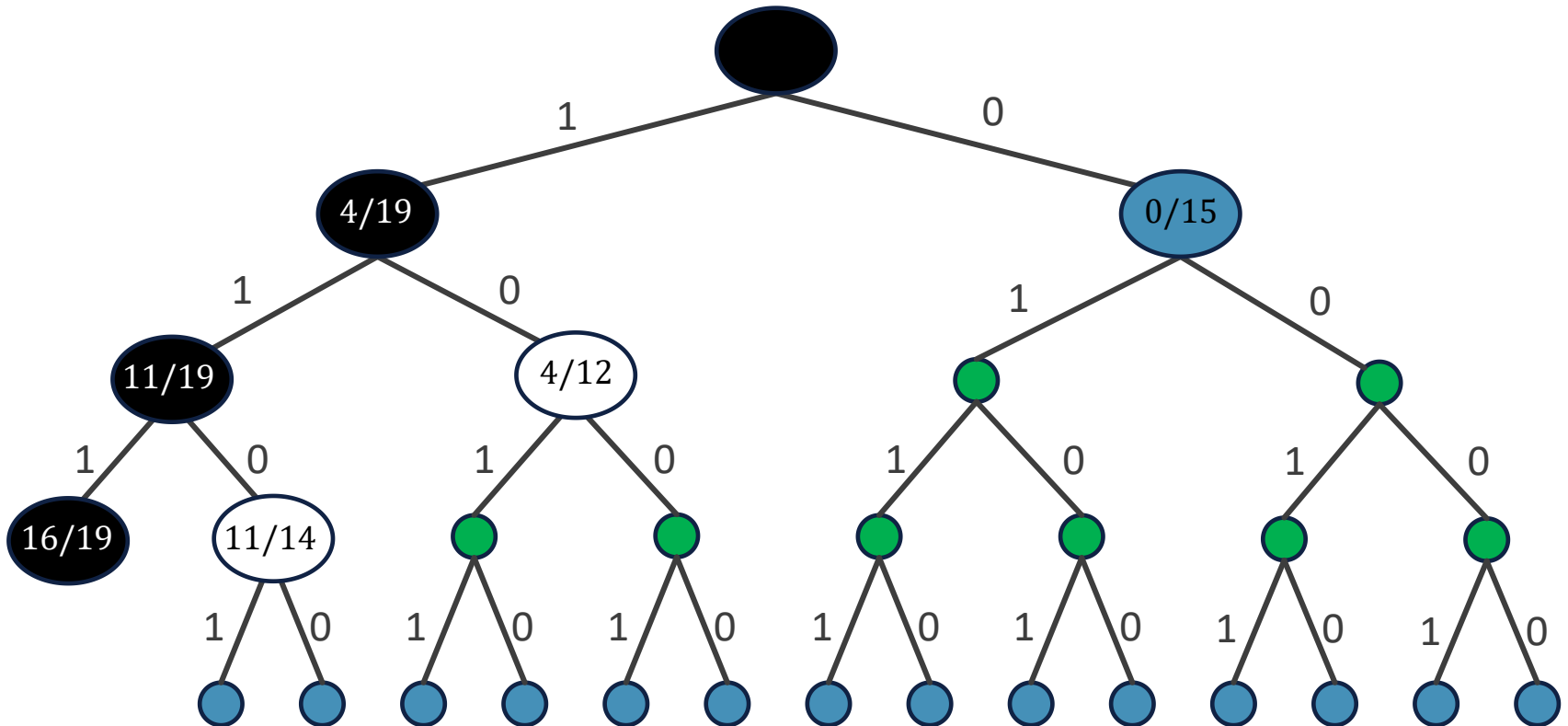


Q:

15	14	12	
----	----	----	--

Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$

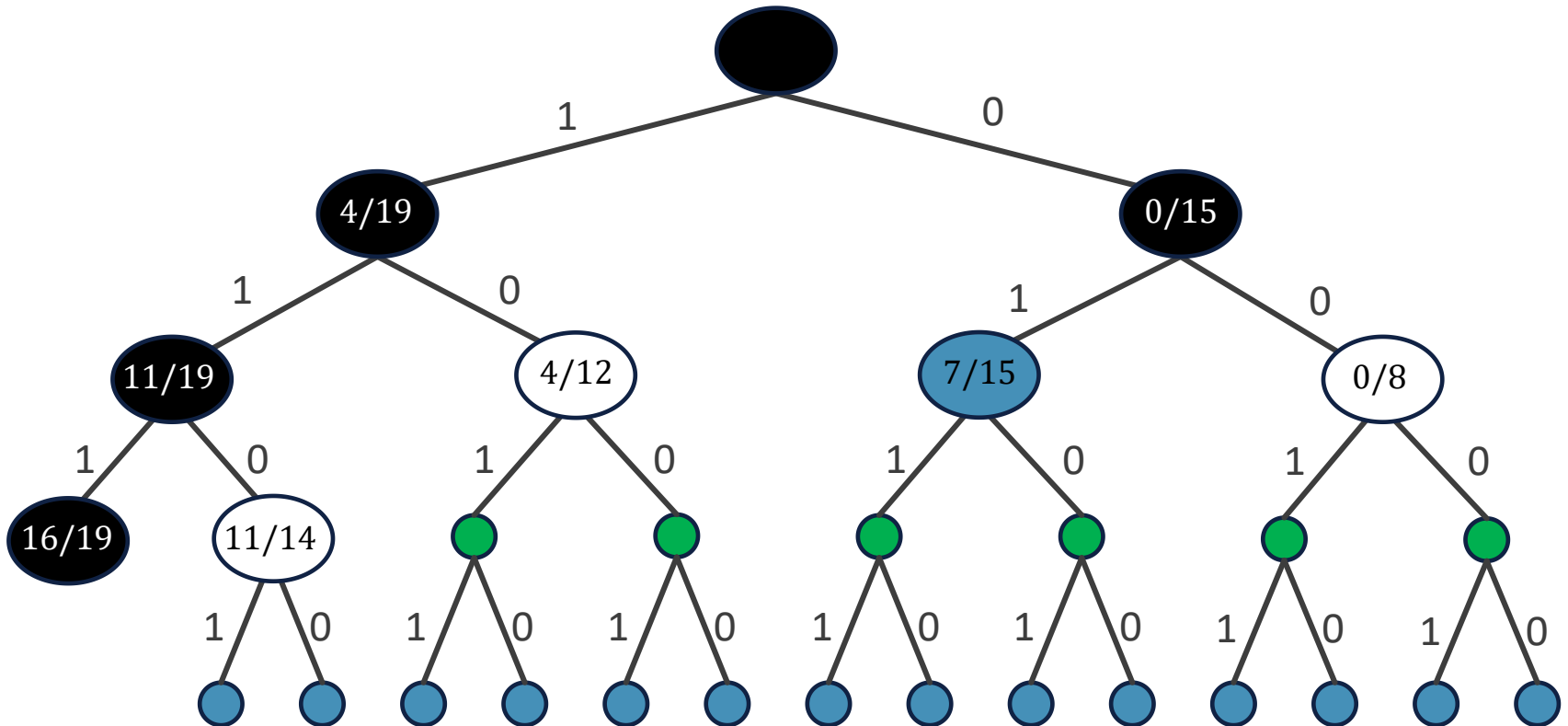


Q:

15	14	12	8
----	----	----	---

Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$

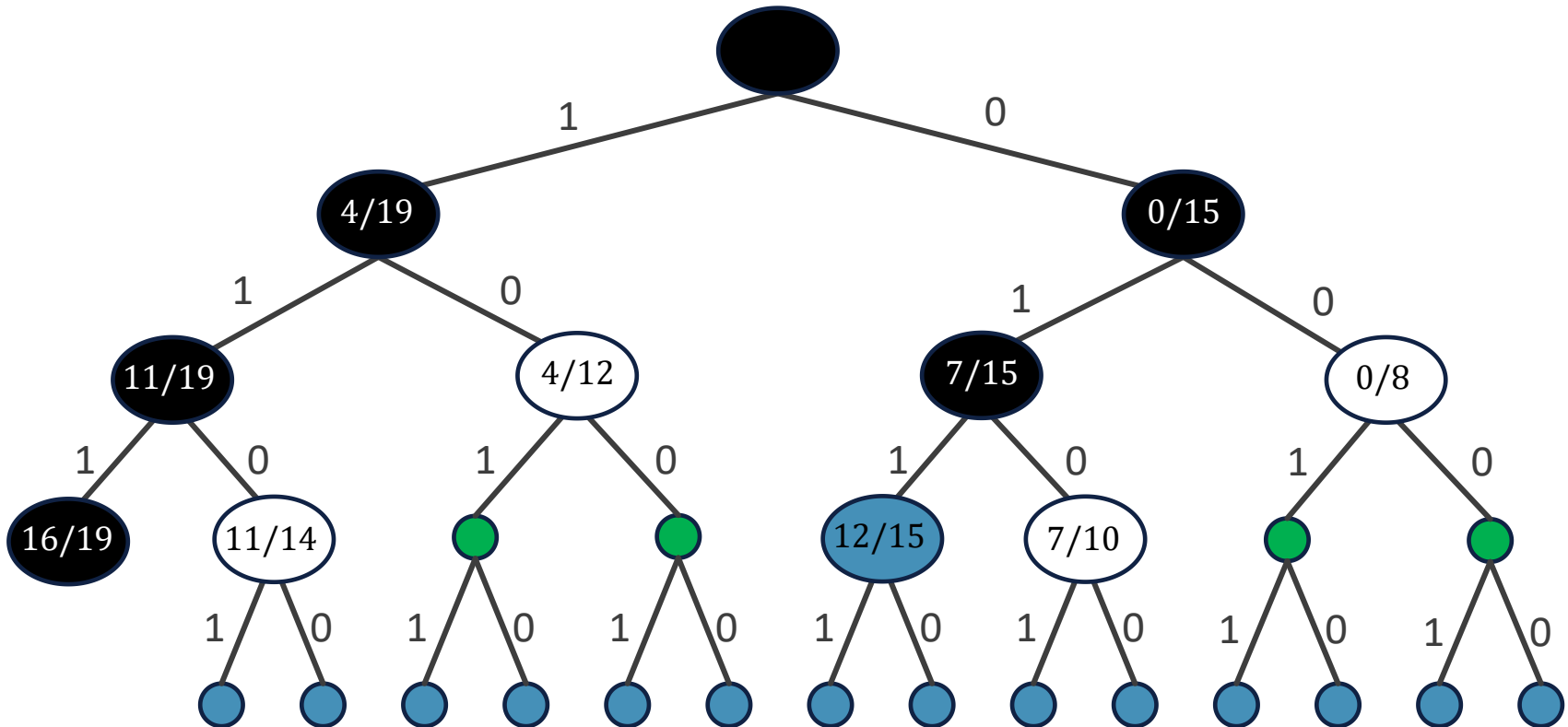


Q:

15	14	12	10	8
----	----	----	----	---

Dead nodes $C(i)/B(i)$ Live nodes

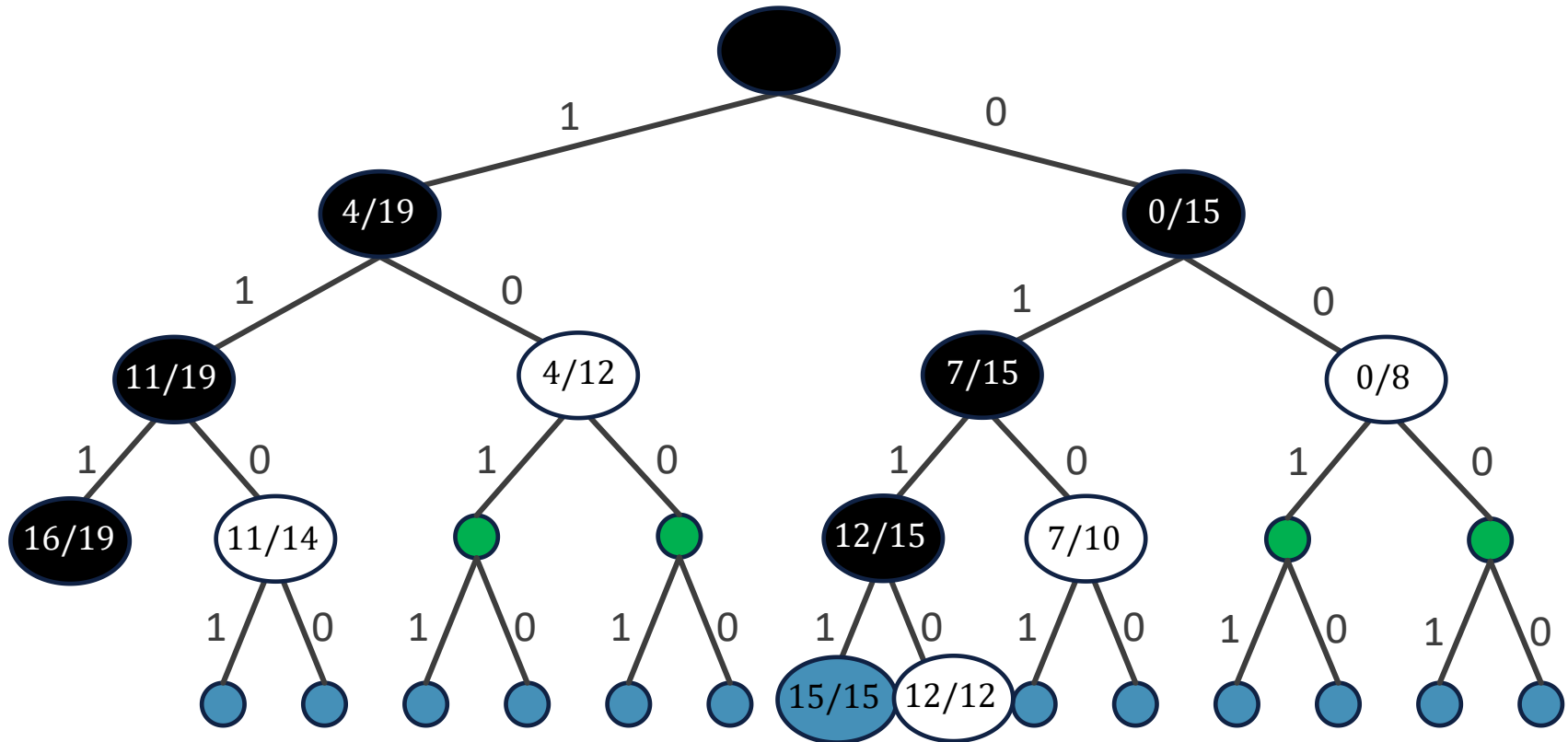
$C(i)/B(i)$ Extend node ● Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$



$C(i)/B(i)$ Extend node ● Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$

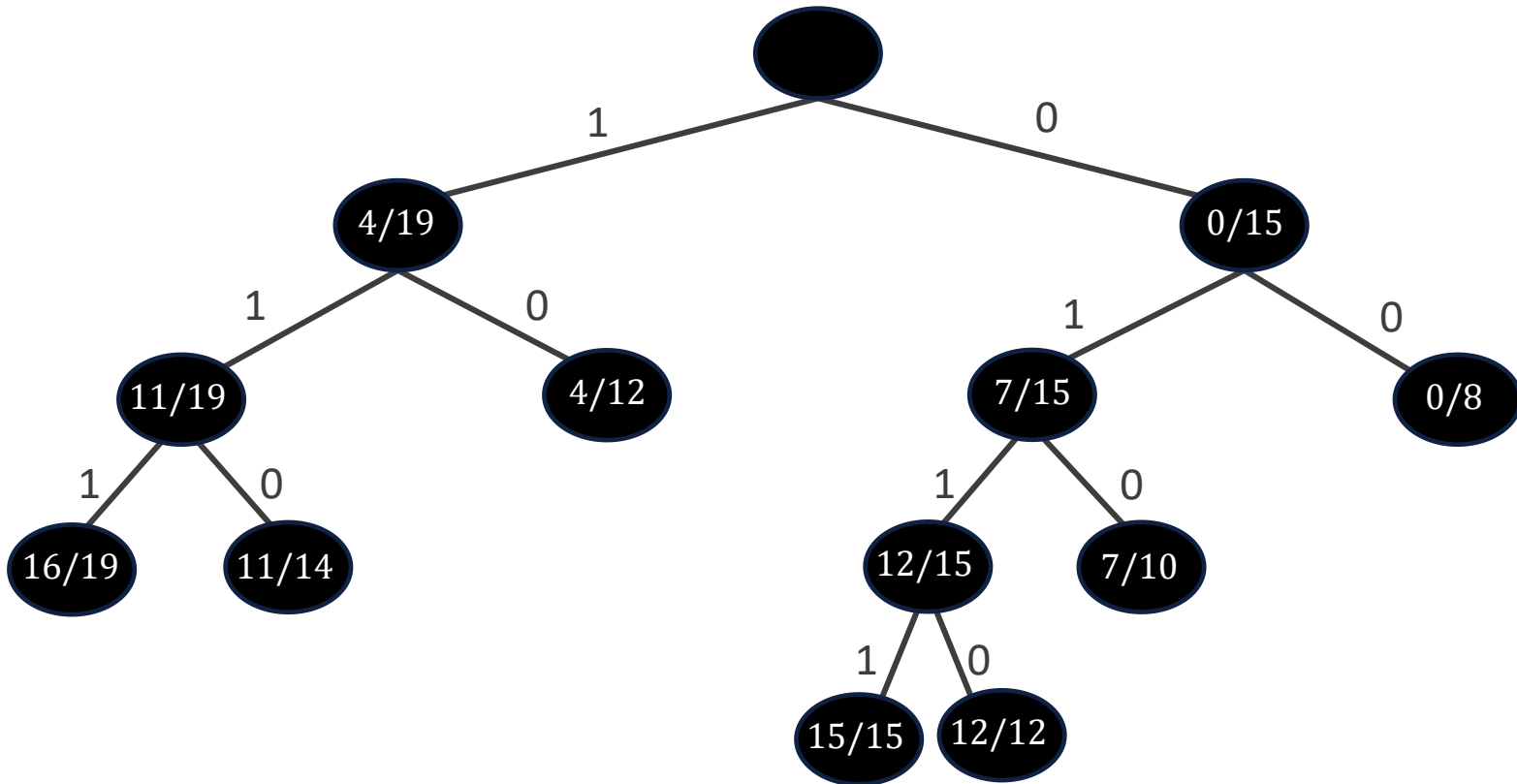


Q:

14	12	12	10	8
----	----	----	----	---

Dead nodes $C(i)/B(i)$ Live nodes

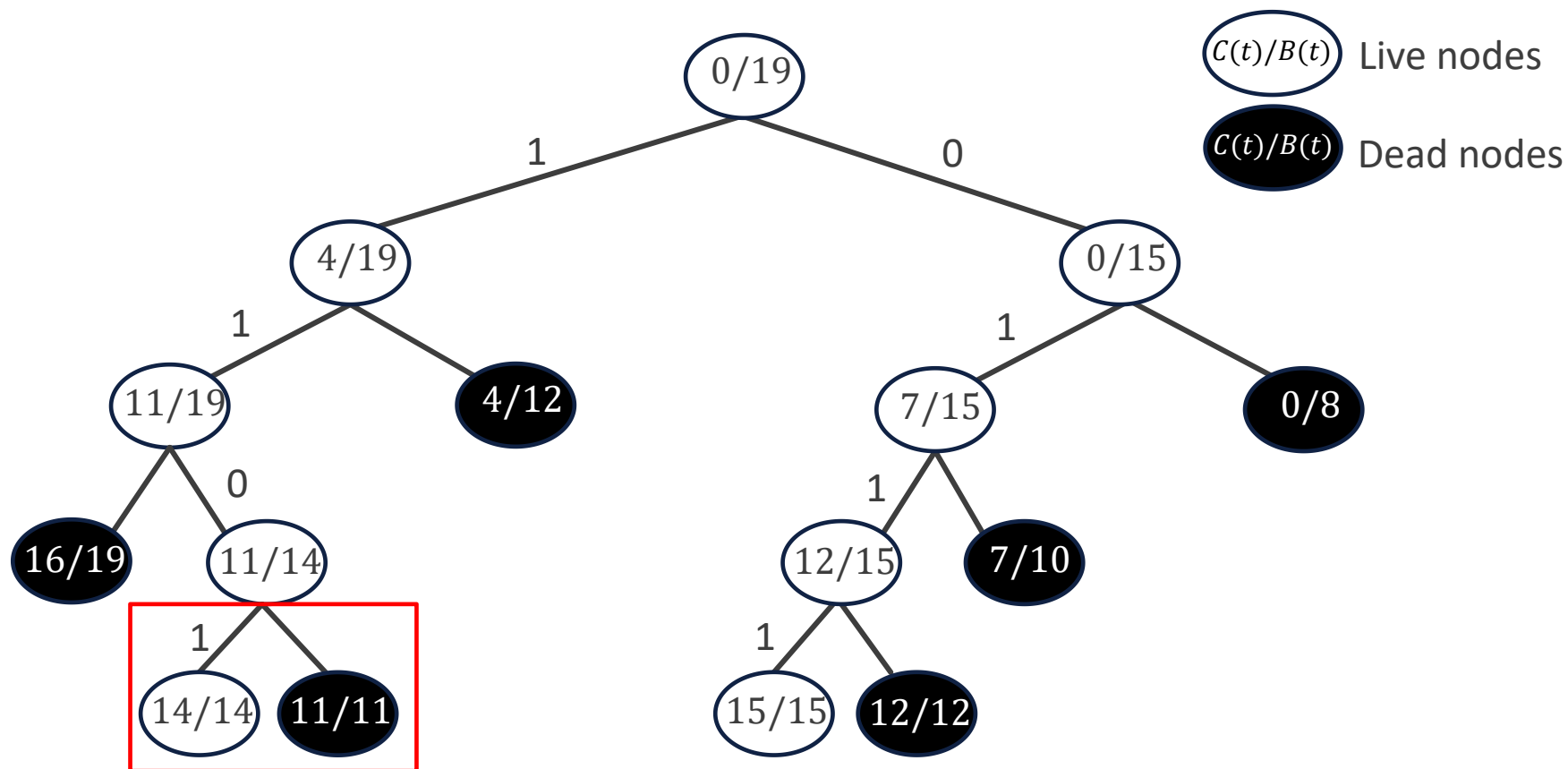
$C(i)/B(i)$ Extend node  Unvisited nodes



Max-profit branch-and-bound for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$



Classroom Exercise



Backtracking for $n = 4$, $w = [4, 7, 5, 3]$, $W = 15$

少了一点搜索空间



0/1 KNAPSACK PROBLEM



0/1 Knapsack Problem

- Constraint function and bounding function are same as the ones used in backtracking.
- Now, we use **max-profit branch-and-bound** to solve.



Pseudocode

```
MaxProfitKnapsack()
1   $i \leftarrow 1$ 
2   $uv \leftarrow B(1)$ ;  $bestv \leftarrow 0$ 
3  while  $i \neq n + 1$  do
4      if  $C(i) \leq W$  then
5          if  $cv + v[i] > bestv$  then  $bestv \leftarrow cv + v[i]$ 
6              AddLiveNode( $uv$ ,  $cv + v[i]$ ,  $C(i)$ , 1,  $i + 1$ )
7          if  $B(i) \geq bestv$  then
8              AddLiveNode( $B(i)$ ,  $cv$ ,  $cw$ , 0,  $i + 1$ )
9           $N \leftarrow \text{ExtractMax}(Q)$ ;  $E \leftarrow N.ptr$ ;  $cw \leftarrow N.weight$ 
10          $cv \leftarrow N.value$   $i \leftarrow N.level$ 
11     for  $j \leftarrow n$  to 1 do
12          $bestx[j] \leftarrow E.LChild$ ;  $E \leftarrow E.parent$ 
13     return  $bestv$ 
```

uv : upper value calculated by $B(i)$

Add
bounding
condition.
It is also ok
it we don't
add it.



Example

Q:

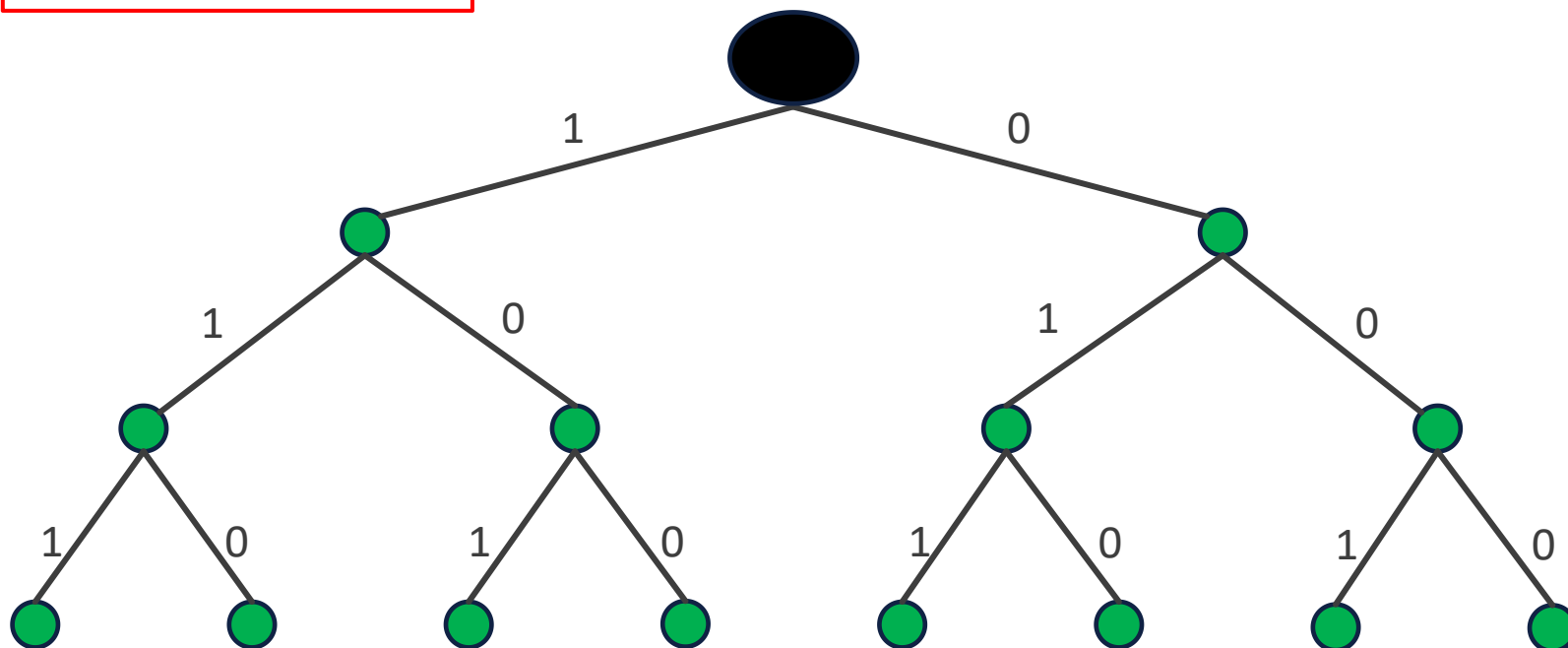
--	--	--	--

Initialization (Line 1-2)

$bestv = 0$

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], W = 30$



Example

Q:

56.6	50		
------	----	--	--

$bestv = 40$

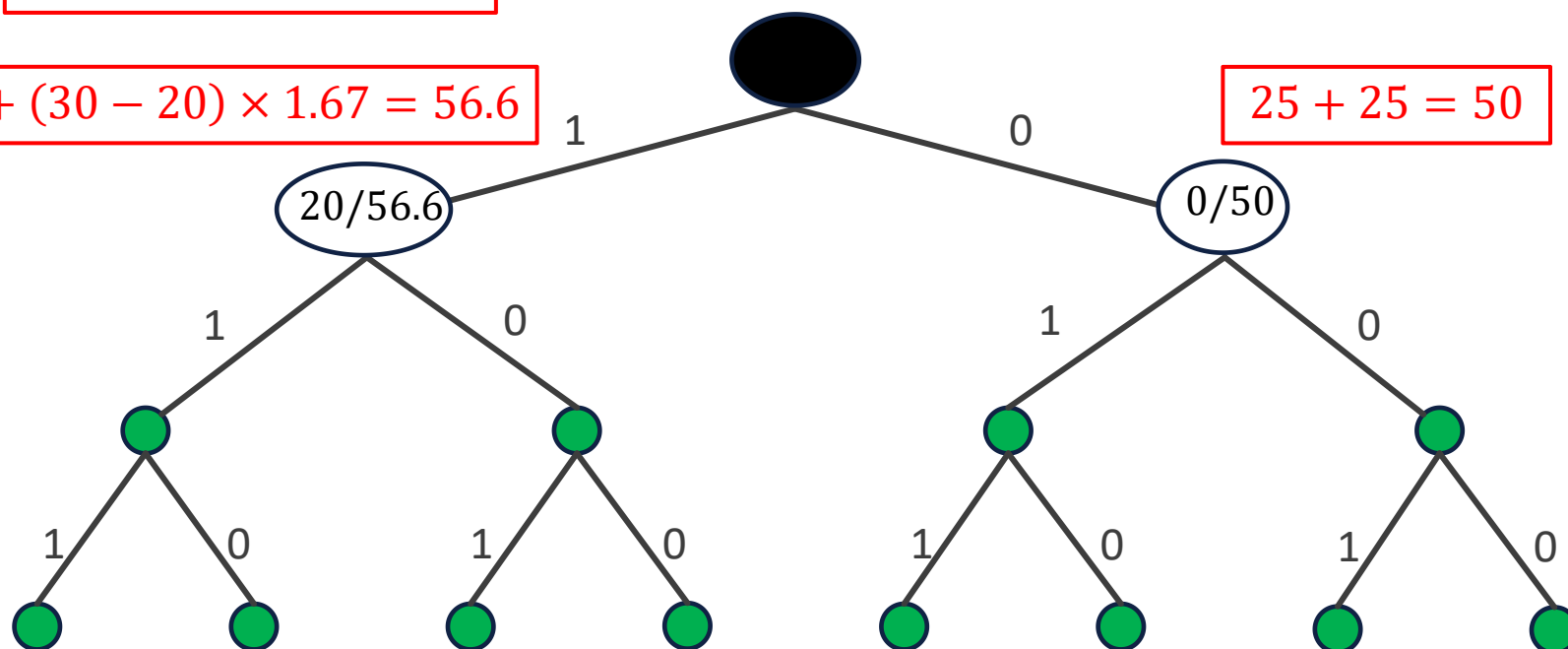
$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes

AddLiveNode (Line 4-9)

$$40 + (30 - 20) \times 1.67 = 56.6$$

$$25 + 25 = 50$$



$$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$$



50

ExtractMax: 56.6

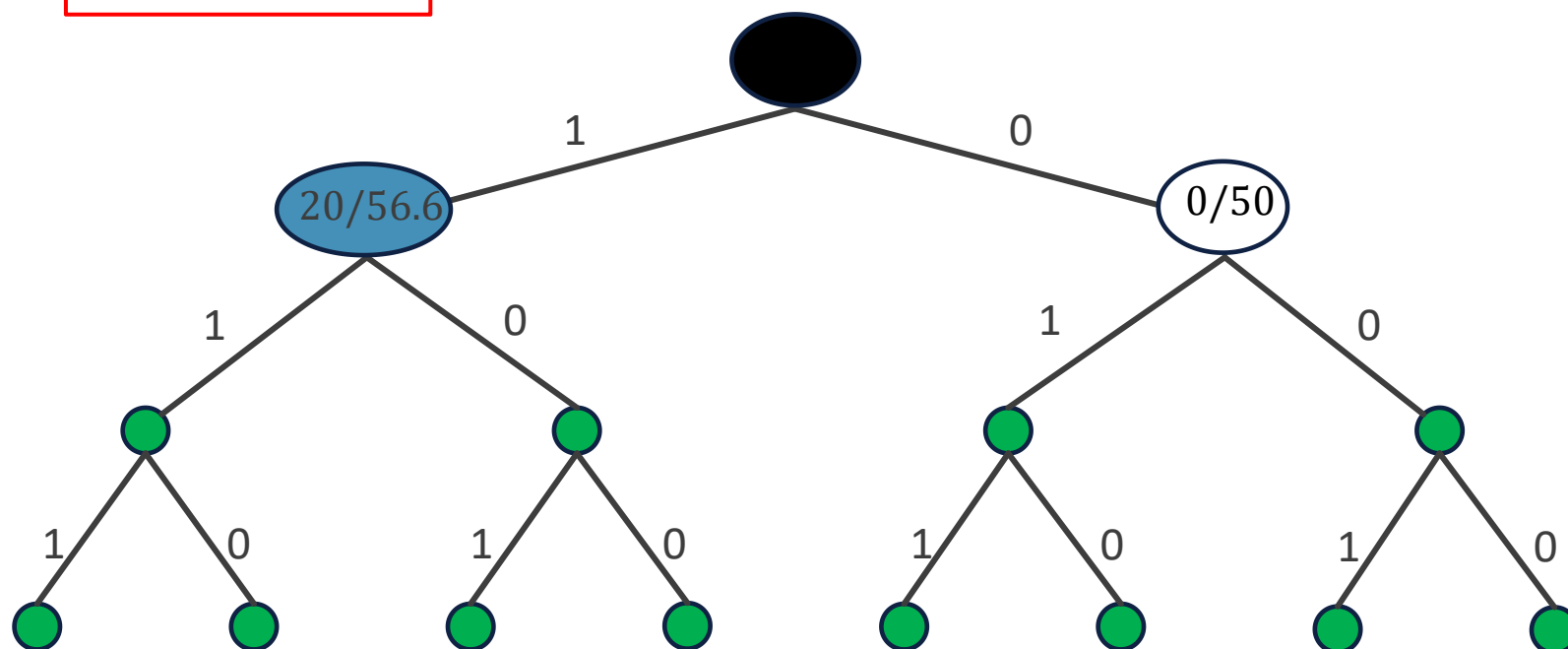
$bestv = 40$

Dead nodes

Live nodes

Extend node

Unvisited nodes



$$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$$



Example

Q:

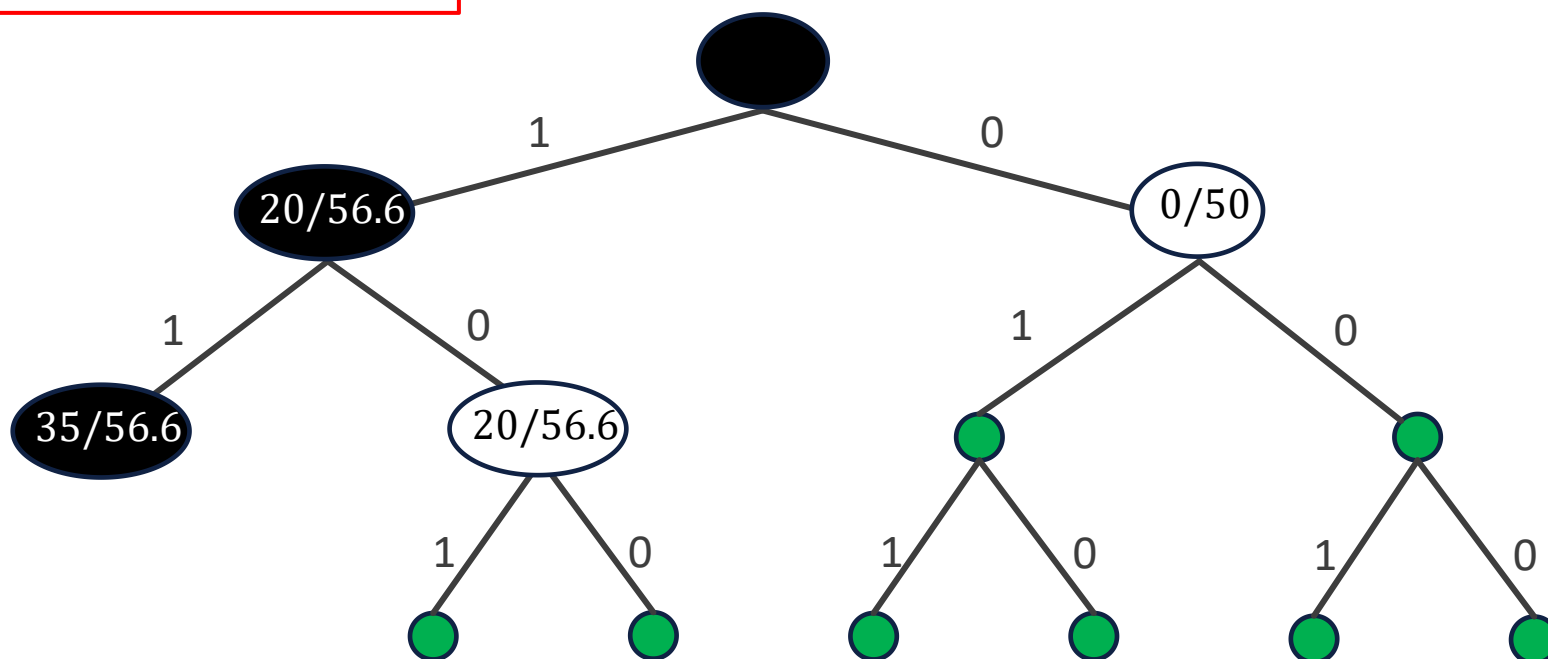
56	650		
----	-----	--	--

bestv = 40

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



ExtractMax: 56.6

$C(i)/B(i)$ Extend node ● Unvisited nodes

$$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$$



Example

Q:

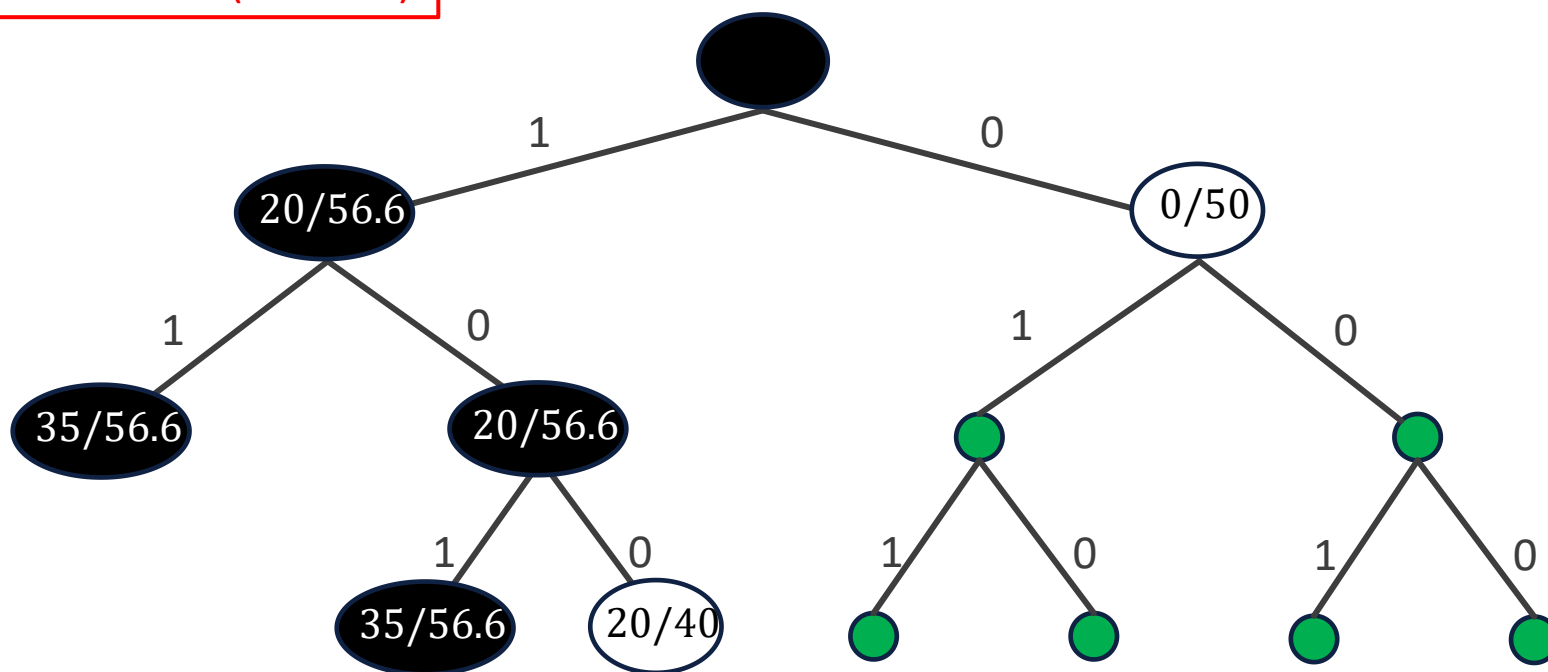
50	40		
----	----	--	--

bestv = 40

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Example

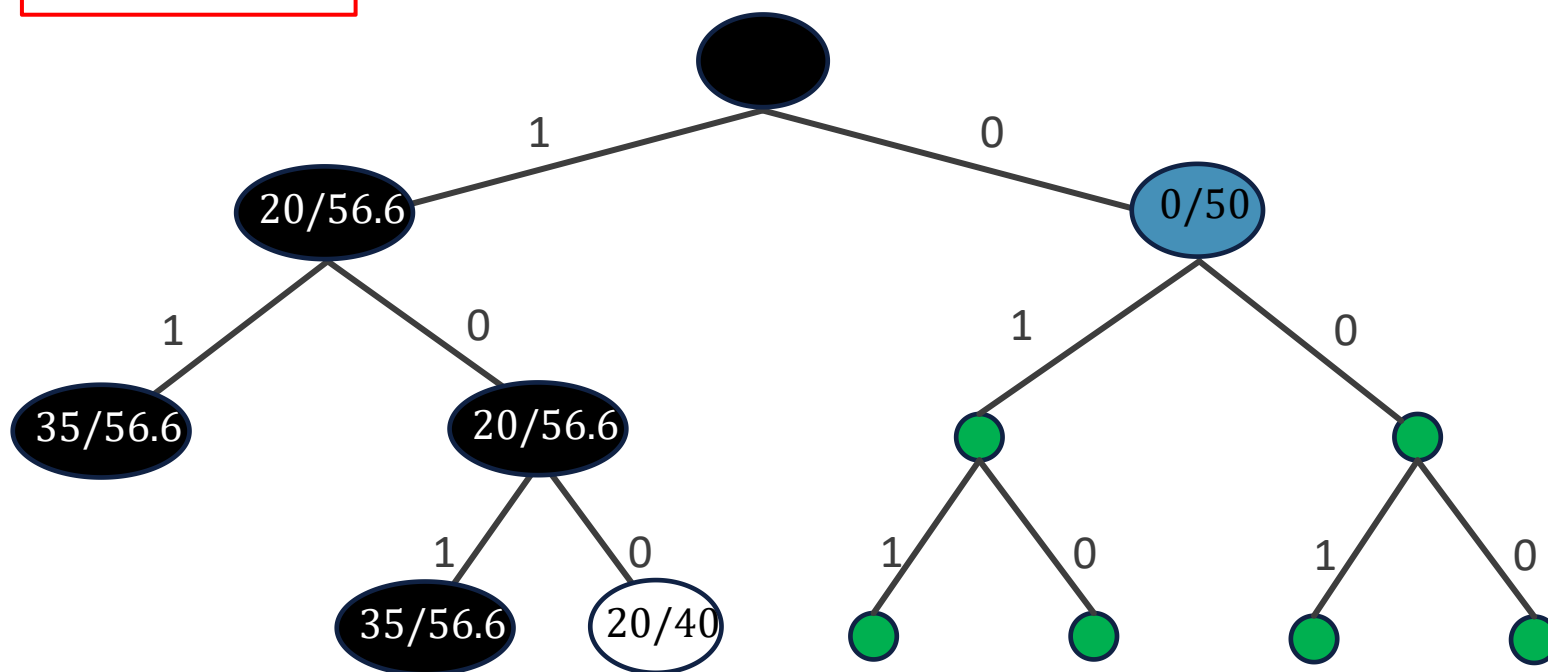
Q:

40			
----	--	--	--

ExtractMax: 50

bestv = 40

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Example

Q:

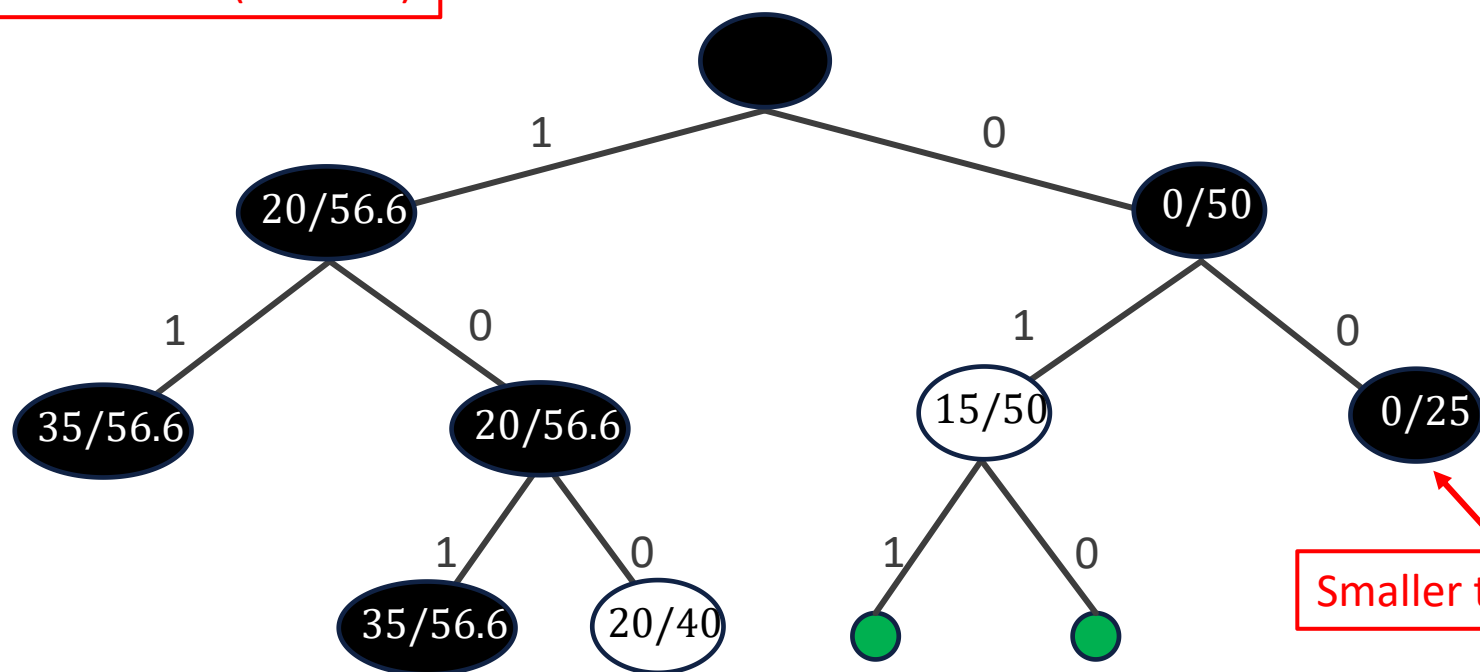
50	40		
----	----	--	--

$bestv = 40$

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



Smaller than $bestv$

$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Example

Q:

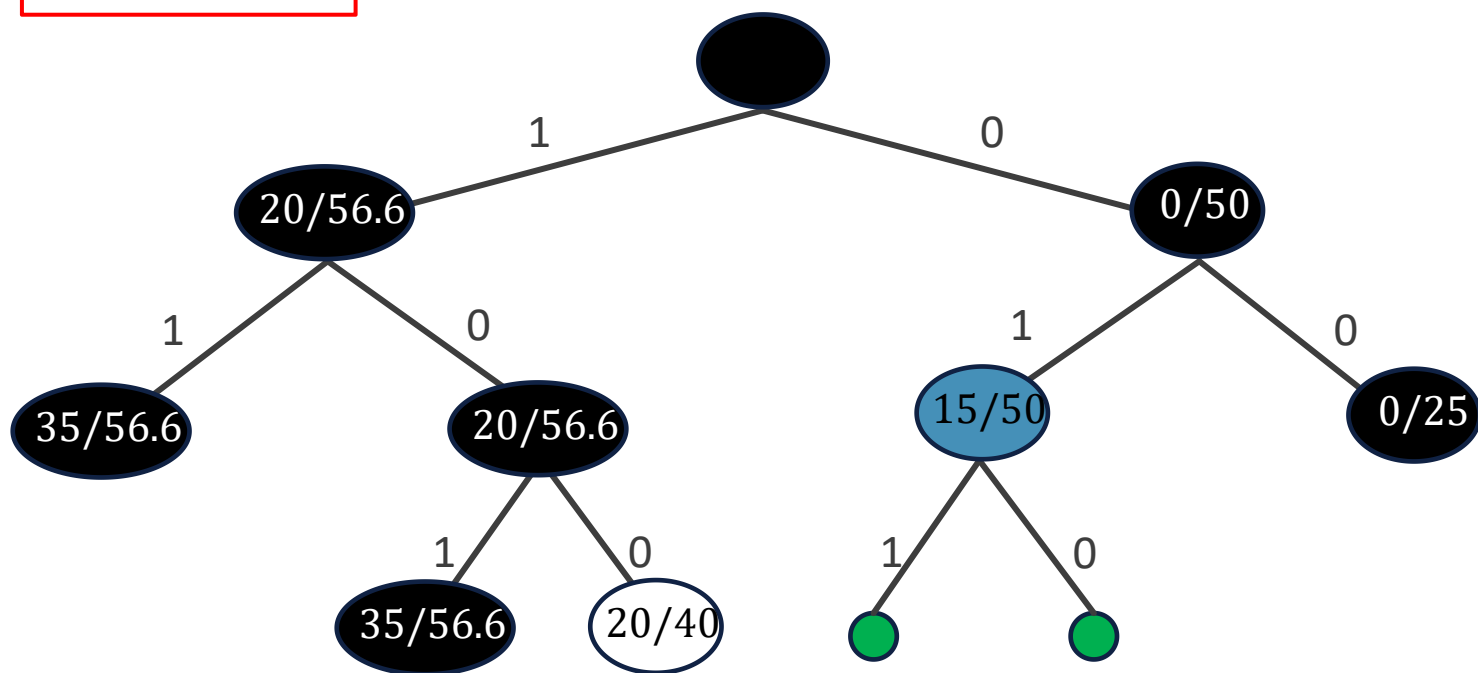
40

ExtractMax: 50

bestv = 40

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node  Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Example

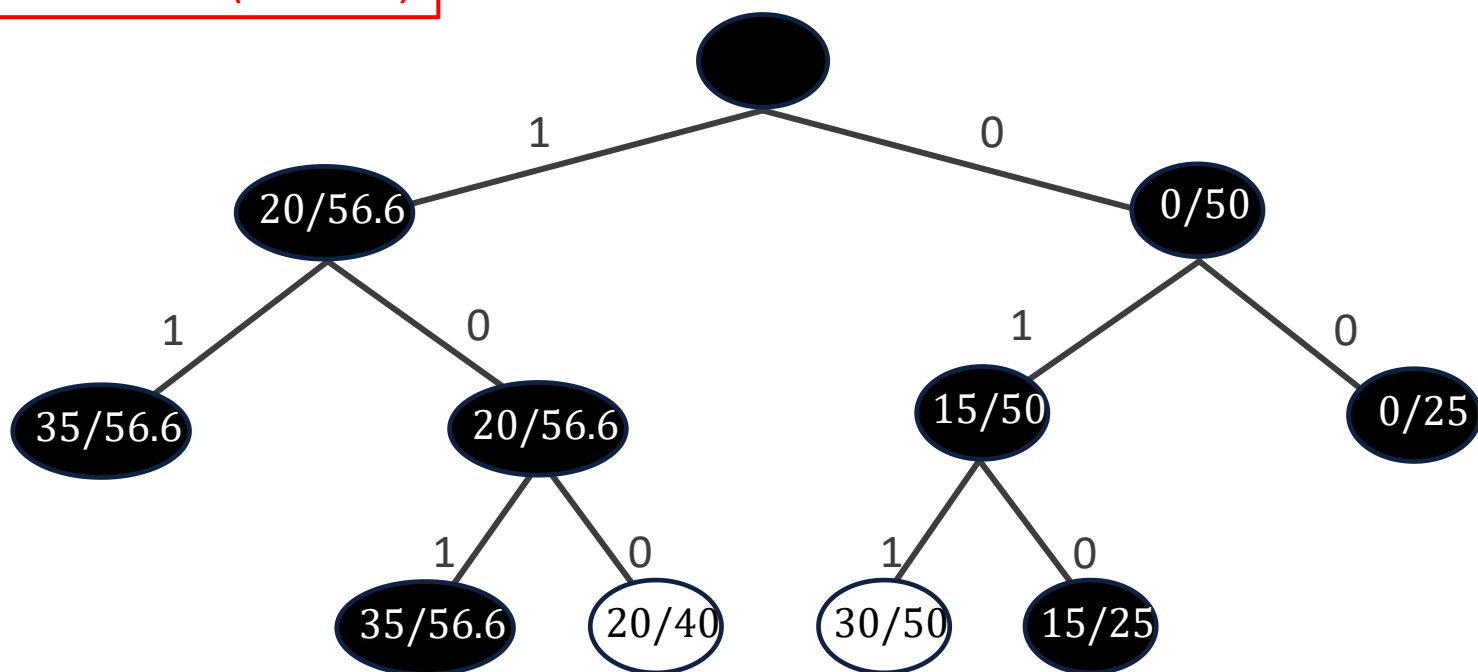
Q:

50	40		
----	----	--	--

bestv = 50

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node  Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Example

Q:

40

ExtractMax: 50

bestv = 50

$C(i)/B(i)$

Dead nodes

$C(i)/B(i)$

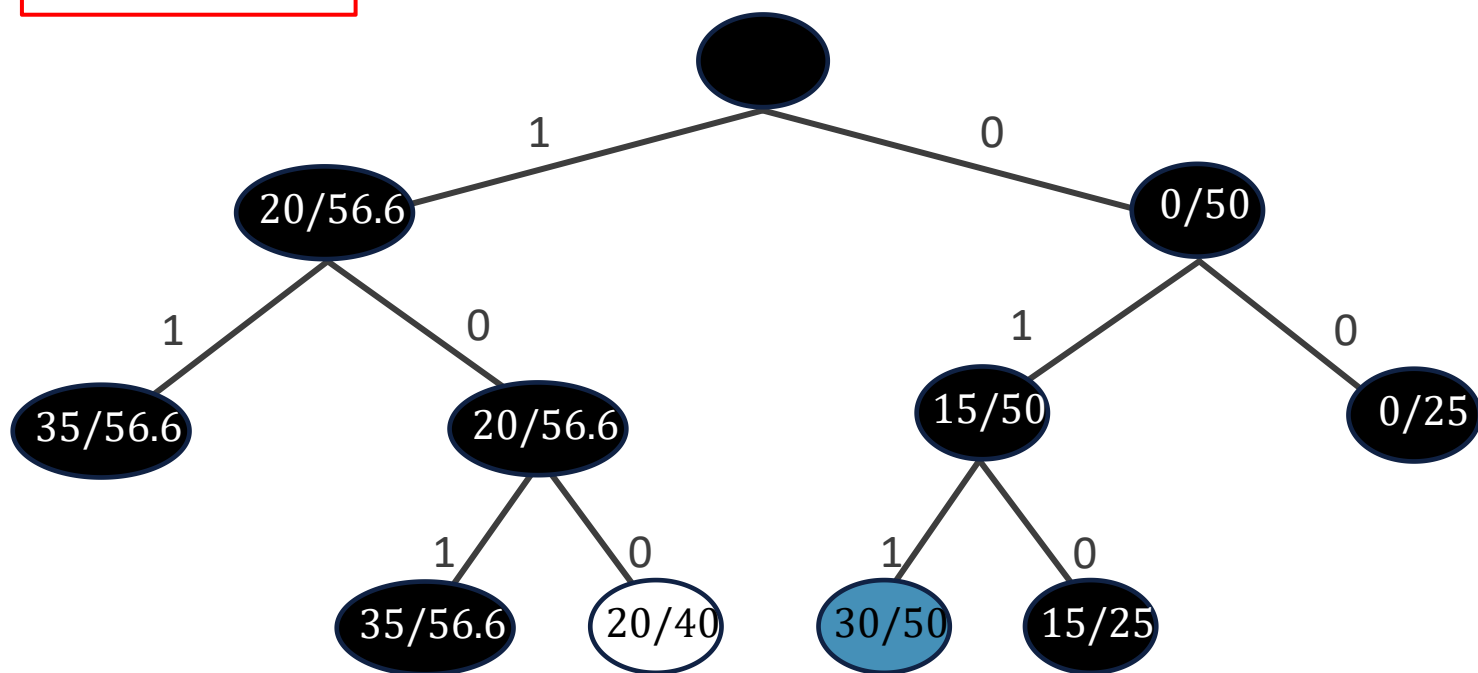
Live nodes

$C(i)/B(i)$

Extend node



Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Example

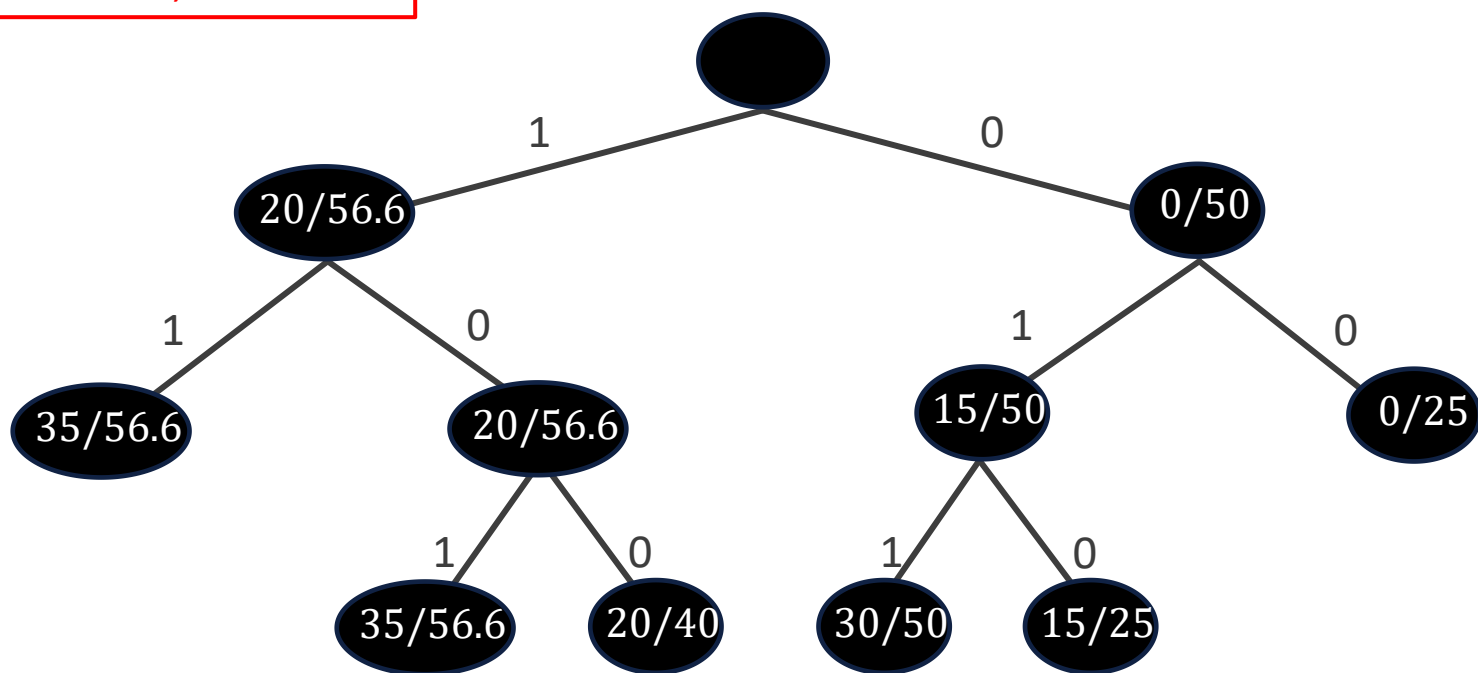
Q:

40			
----	--	--	--

$i = n + 1$, terminate

$bestv = 50$

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, w = [20, 15, 15], v = [40, 25, 25], v/w = [2, 1.67, 1.67], W = 30$



Classroom Exercise

- Draw the pruned solution space tree for the following container loading problem instance by max-profit branch-and-bound.

$$n = 3, v = [20, 40, 20], w = [2, 5, 4], W = 5$$

- Compare the result with the one solved by backtracking.



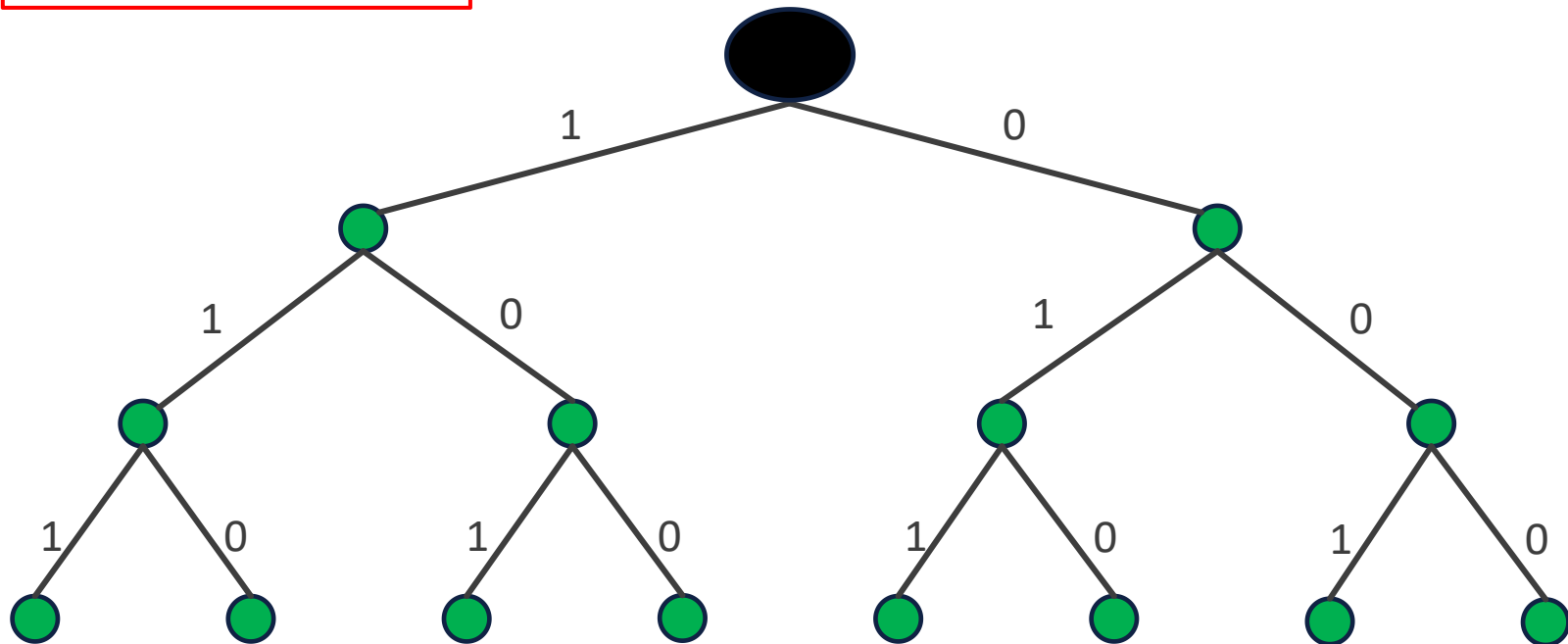
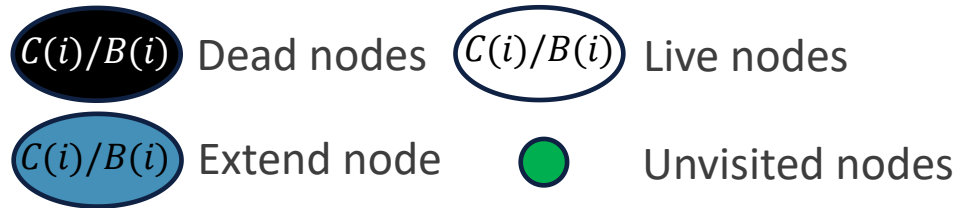
Classroom Exercise

Q:

--	--	--	--

Initialization (Line 1-2)

$bestv = 0$



$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$



Classroom Exercise

Q:

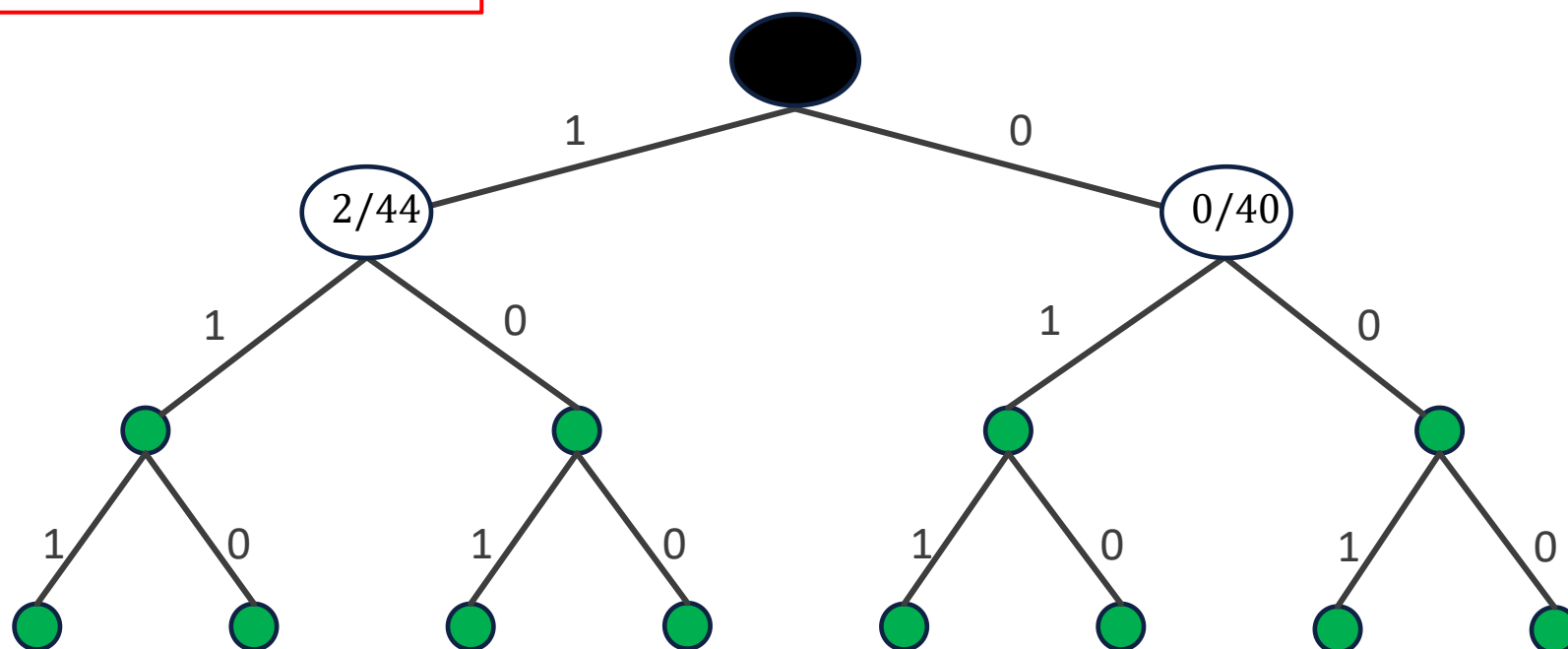
44	40		
----	----	--	--

bestv = 20

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node  Unvisited nodes



$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$



40

ExtractMax: 44

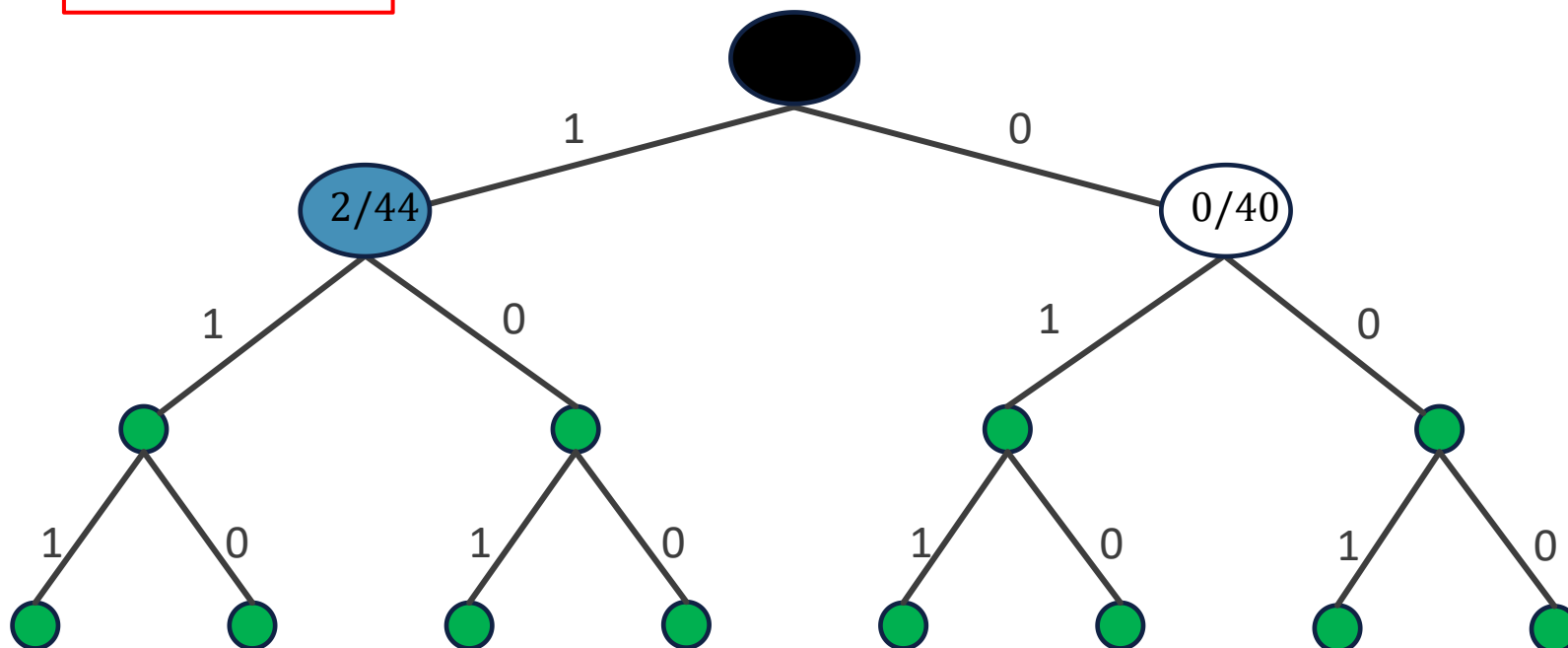
bestv = 20

Dead nodes

Live nodes

Extend node

Unvisited nodes



$$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$$



Classroom Exercise

Q:

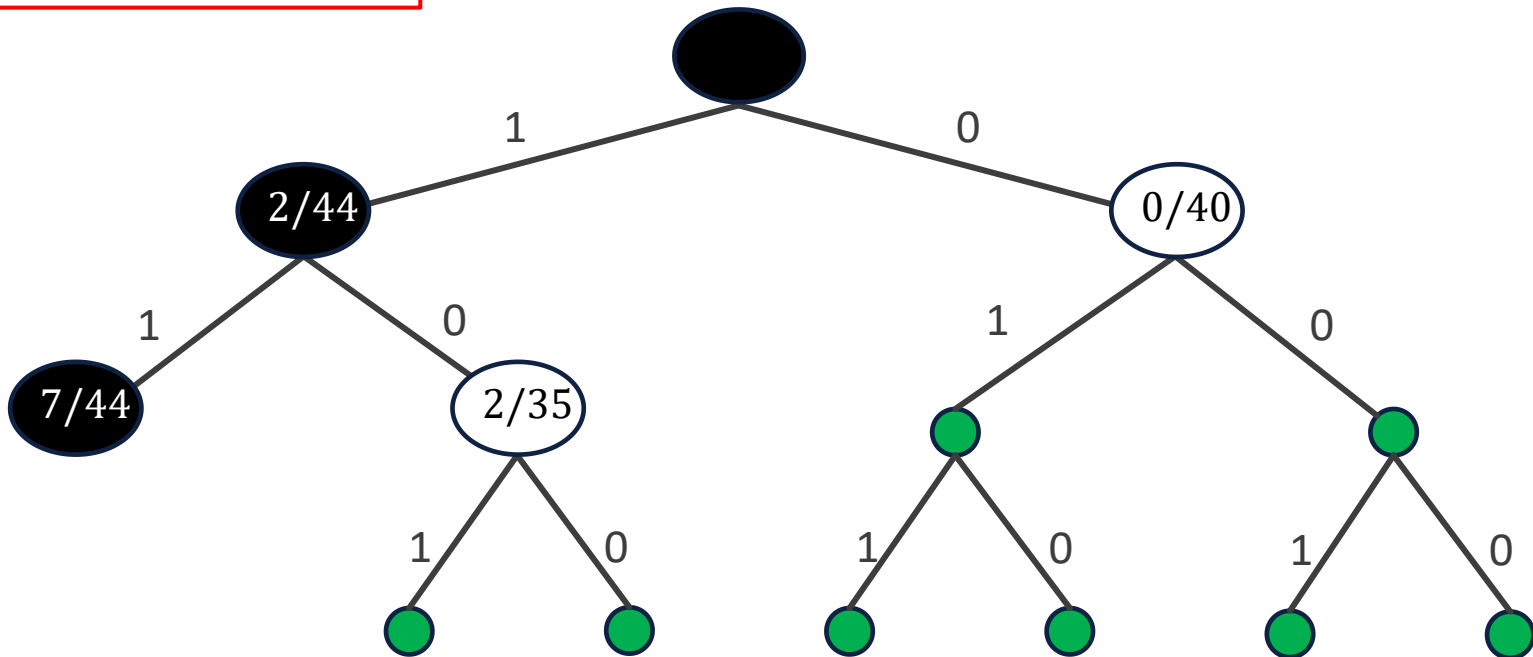
40	35		
----	----	--	--

bestv = 20

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$



Classroom Exercise

Q:

35

ExtractMax: 40

$bestv = 20$

$C(i)/B(i)$

Dead nodes

$C(i)/B(i)$

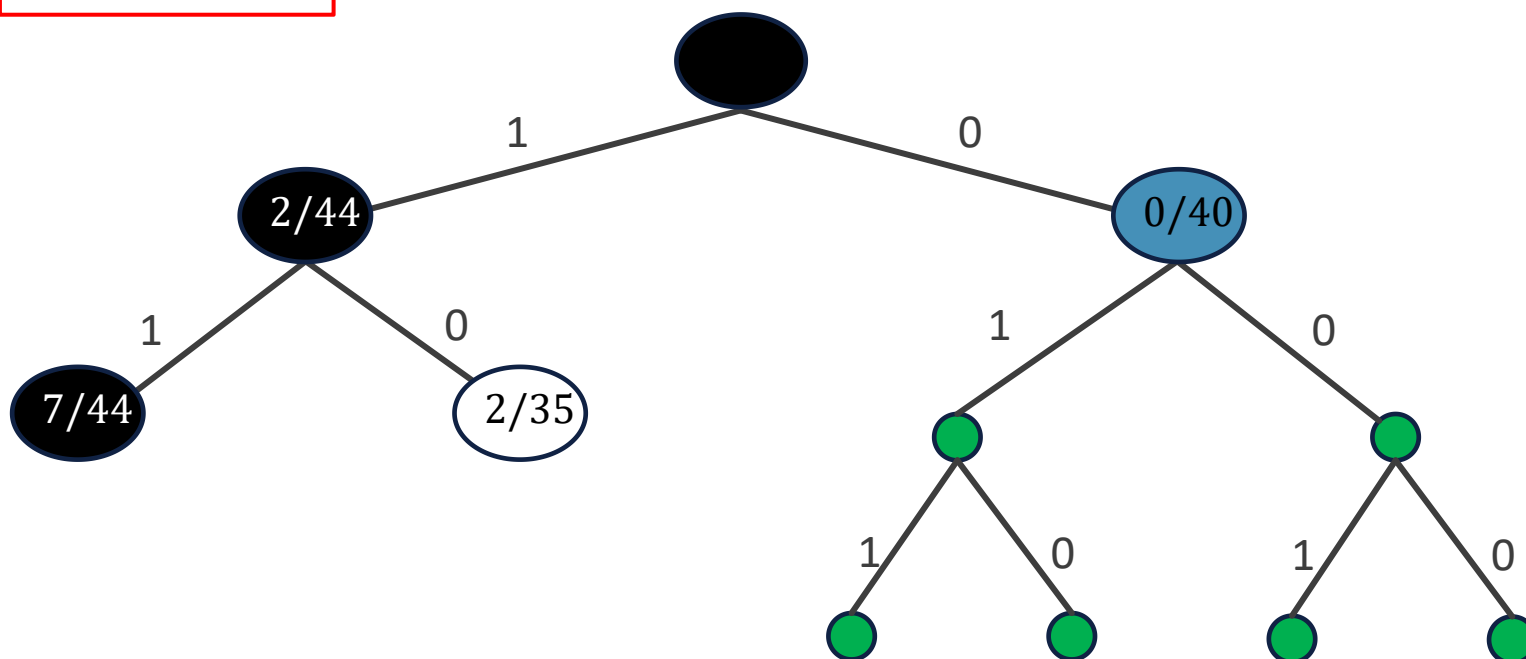
Live nodes

$C(i)/B(i)$

Extend node



Unvisited nodes



$$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$$



Classroom Exercise

Q:

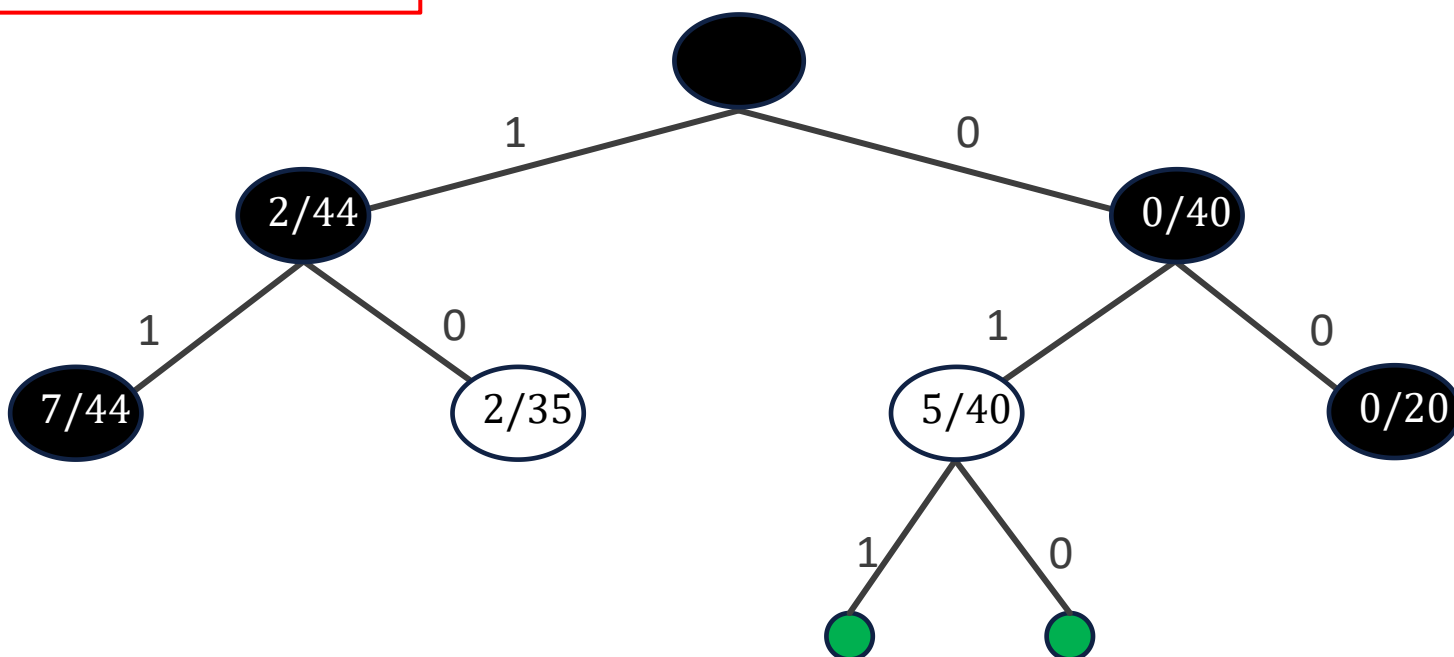
40	35	20	
----	----	----	--

bestv = 40

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node  Unvisited nodes



$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$



Classroom Exercise

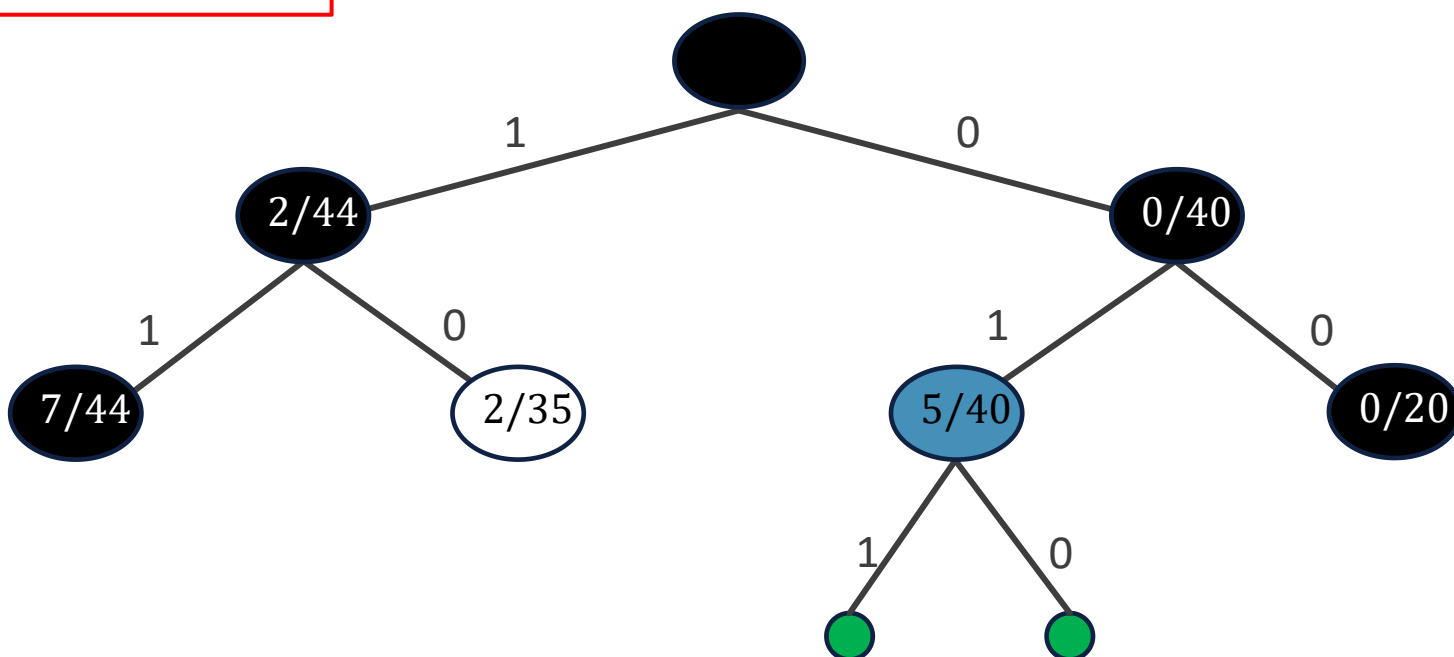
Q:

35	20		
----	----	--	--

ExtractMax: 40

bestv = 40

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes
 $C(i)/B(i)$ Extend node ● Unvisited nodes



$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$



Classroom Exercise

Q:

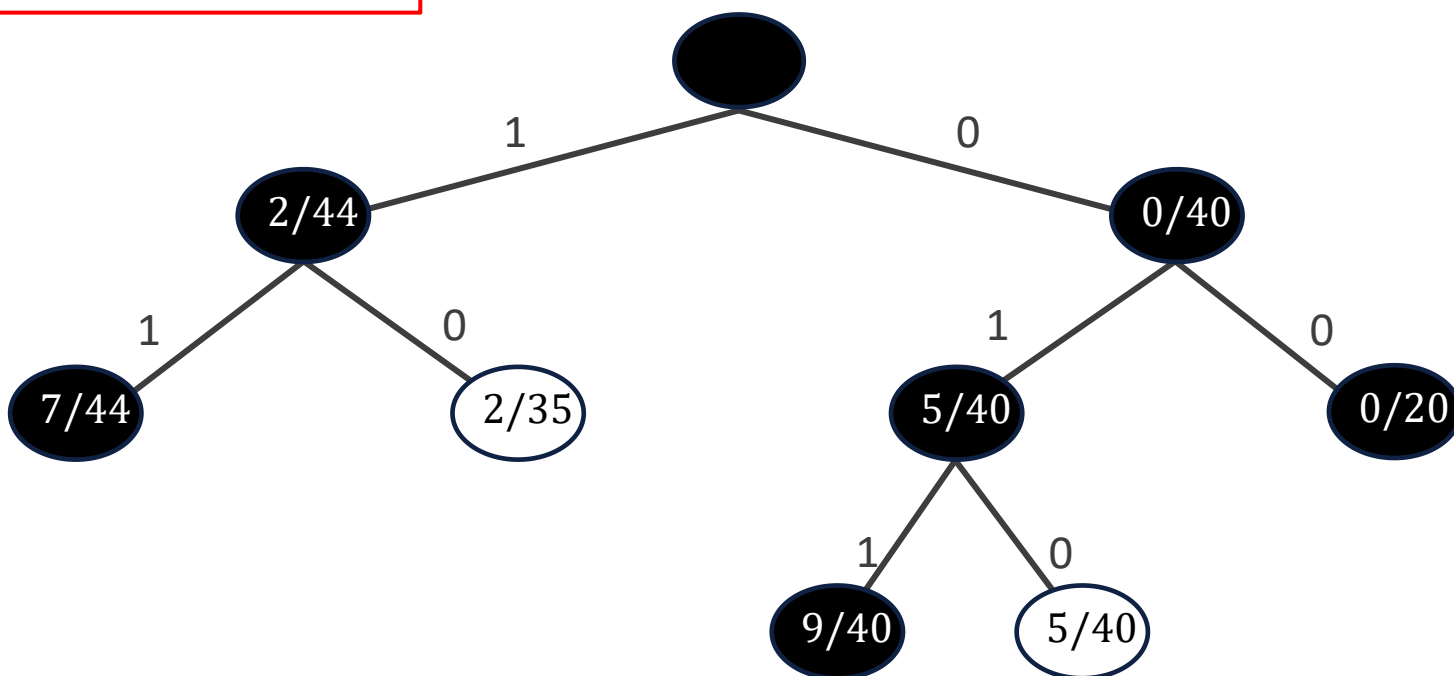
40	35	20	
----	----	----	--

bestv = 40

AddLiveNode (Line 4-9)

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node  Unvisited nodes



$$n = 3, v = [20, 40, 20], w = [2, 5, 4], v/w = [10, 8, 5], W = 5$$



Classroom Exercise

Q:

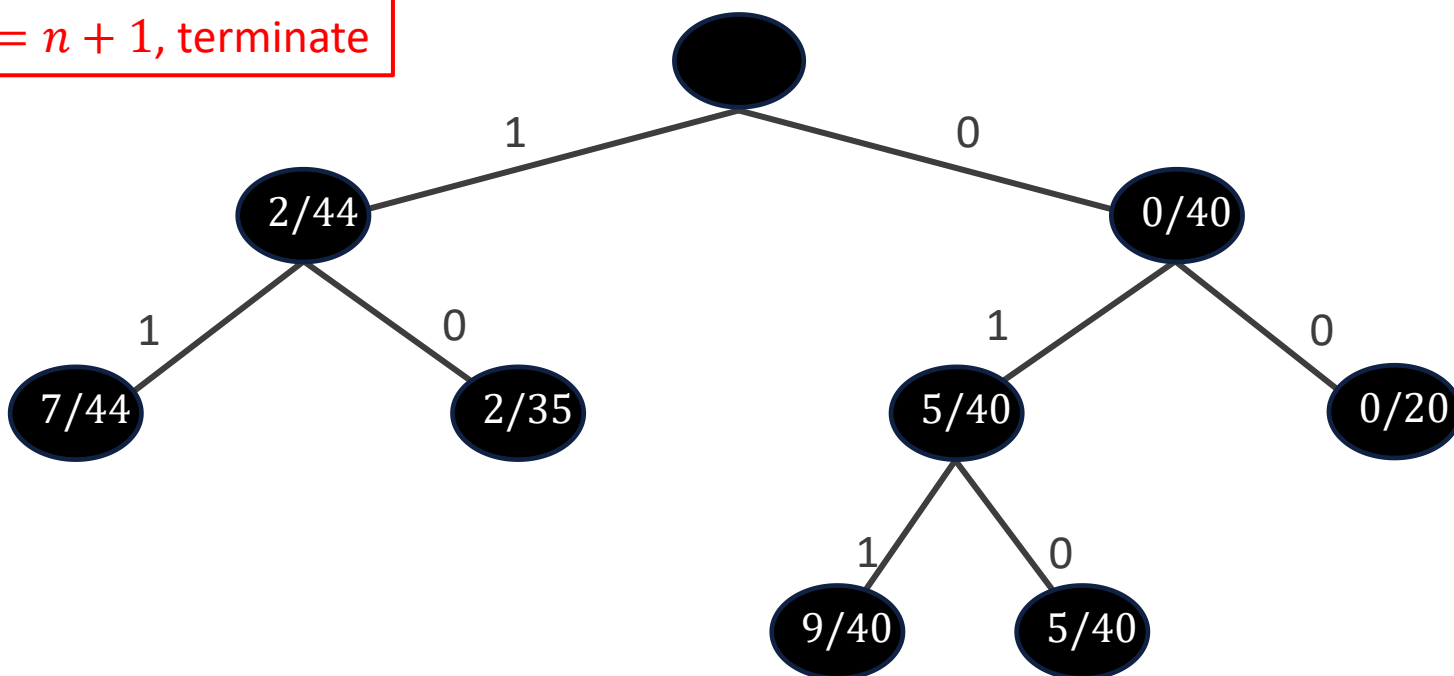
35	20		
----	----	--	--

bestv = 40

ExtractMax: 40
 $i = n + 1$, terminate

$C(i)/B(i)$ Dead nodes $C(i)/B(i)$ Live nodes

$C(i)/B(i)$ Extend node  Unvisited nodes

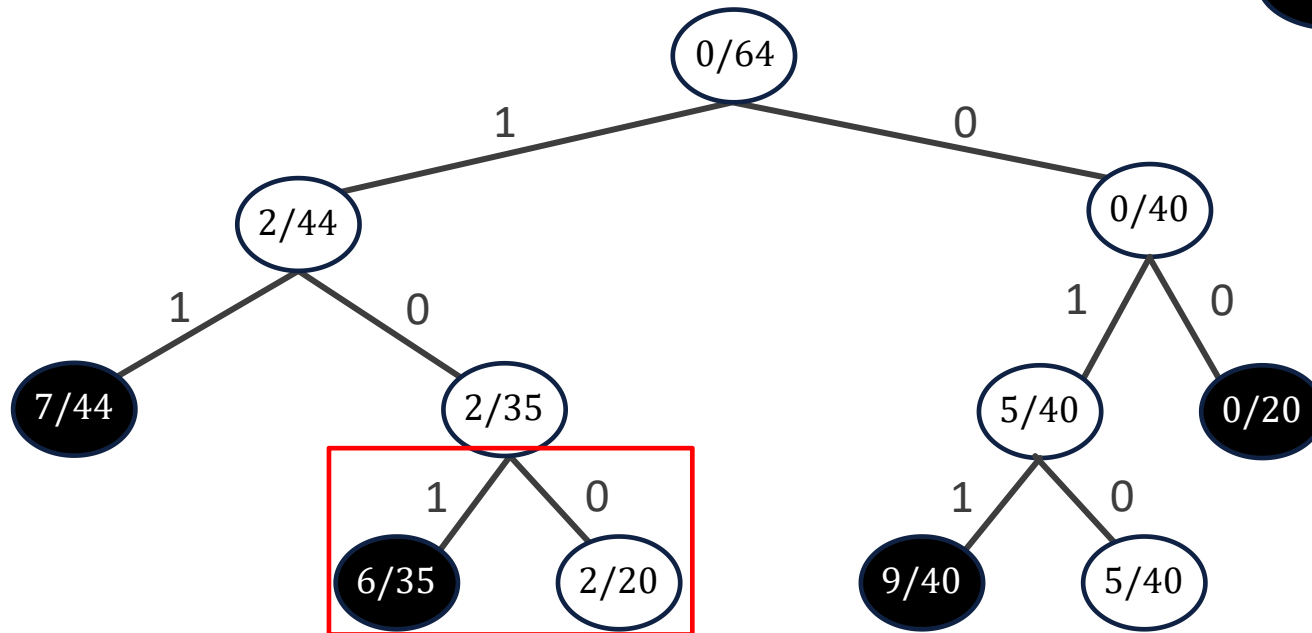


$n = 3$, $v = [20, 40, 20]$, $w = [2, 5, 4]$, $v/w = [10, 8, 5]$, $W = 5$



Classroom Exercise

$C(t)/B(t)$ Live nodes
 $C(t)/B(t)$ Dead nodes



Saved

Backtracking for $n = 3$, $v = [20, 40, 20]$, $w = [2, 5, 4]$, $v/w = [10, 8, 5]$, $W = 5$



SAT PROBLEM



SAT Problem

- We have seen the 3-CNF-SAT problem. Now we consider a more general k -CNF-SAT problem:

$$\phi = C_1 \wedge C_2 \wedge \dots \wedge C_n$$

where each C_i has the following form:

$$C_i = l_{i1} \vee l_{i2} \vee \dots \vee l_{ik}$$

and the literal l_{ij} could be one of variables in $\{x_1, x_2, \dots, x_m\}$ or its negation.

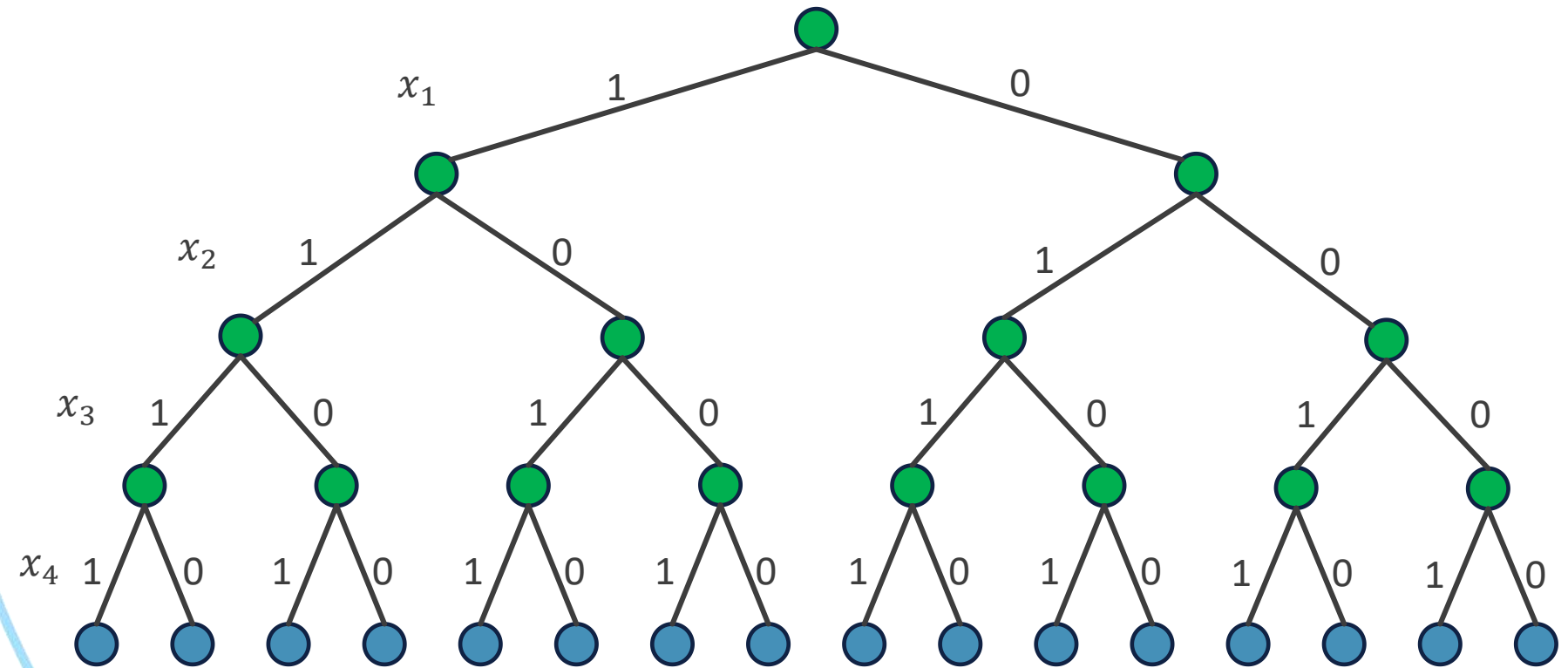
- For example, a 3-CNF-SAT with 4 variables could be:

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$$

- Notice three different parameters: n , k and m .



Solution Space Tree for SAT Problem



A k -CNF-SAT problem with 4 variable.



SAT Problem

- This is a decision problem, rather than optimization problem.
- It seems that we don't have bounding function.
- What is the constraint function?

如果某个子句有当前已经确认赋值的所有布尔变元，并且这个子句为False，那么这个节点被剪掉 ($cv > 0$)



SAT Problem

- Remember that any decision problem can also be converted to optimization problem.
- What is the optimization version for k -CNF-SAT?
- Find an assignment that satisfies the maximum number of clauses.



- Now, we can design the bounding function.
- We can calculate the lower bound cv for each node, by counting the number of satisfied clauses. cv 满足的子句数量
- For example, $x_1 = 1$:

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$$

- We get $cv = 1$.



SAT Problem

- Again, by the idea of branch-and-bound, we put cv in a max-priority queue.
- However, the different is that cv is the lower bound, rather than upper bound in 0/1 knapsack problem.
- It still works. Higher lower bound also means higher hope.



Pseudocode

MaxProfitSAT()

```
1   $i \leftarrow 1$ 
2  while  $i \neq m + 1$  do
3       $cv \leftarrow \text{ok}(i, N, 1)$ 
4      if  $cv > 0$  then
5          AddLiveNode( $cv, 1, i + 1$ )
6       $cv \leftarrow \text{ok}(i, N, 0)$ 
7      if  $cv > 0$  then
8          AddLiveNode( $cv, 0, i + 1$ )
9       $N \leftarrow \text{ExtractMax}(Q); i \leftarrow N.\text{level}$ 
10 for  $j \leftarrow m$  downto 1 do
11      $\text{bestx}[j] \leftarrow E.LChild; E \leftarrow E.\text{parent}$ 
```

check函数，判断在当前 x_i 赋值为 ch 的情况下，会不会使得某些子句变为0，如果是的话，那么就返回0，也有可能返回0/1之外的值，因为 x_i 不在那个从句里面，不改变那个句子的取值。另外一种情况是返回1。

$\text{ok}(i, N, ch)$

```
1   $cn \leftarrow 0$ 
2  for  $j \leftarrow 1$  to  $n$  do
3      if  $\text{check}(C_j, N, ch) = 0$  then
4          return 0
5      else if  $\text{check}(C_j, N, ch) = 1$  then
6           $cn \leftarrow cn + 1$ 
7  return  $cn$ 
```



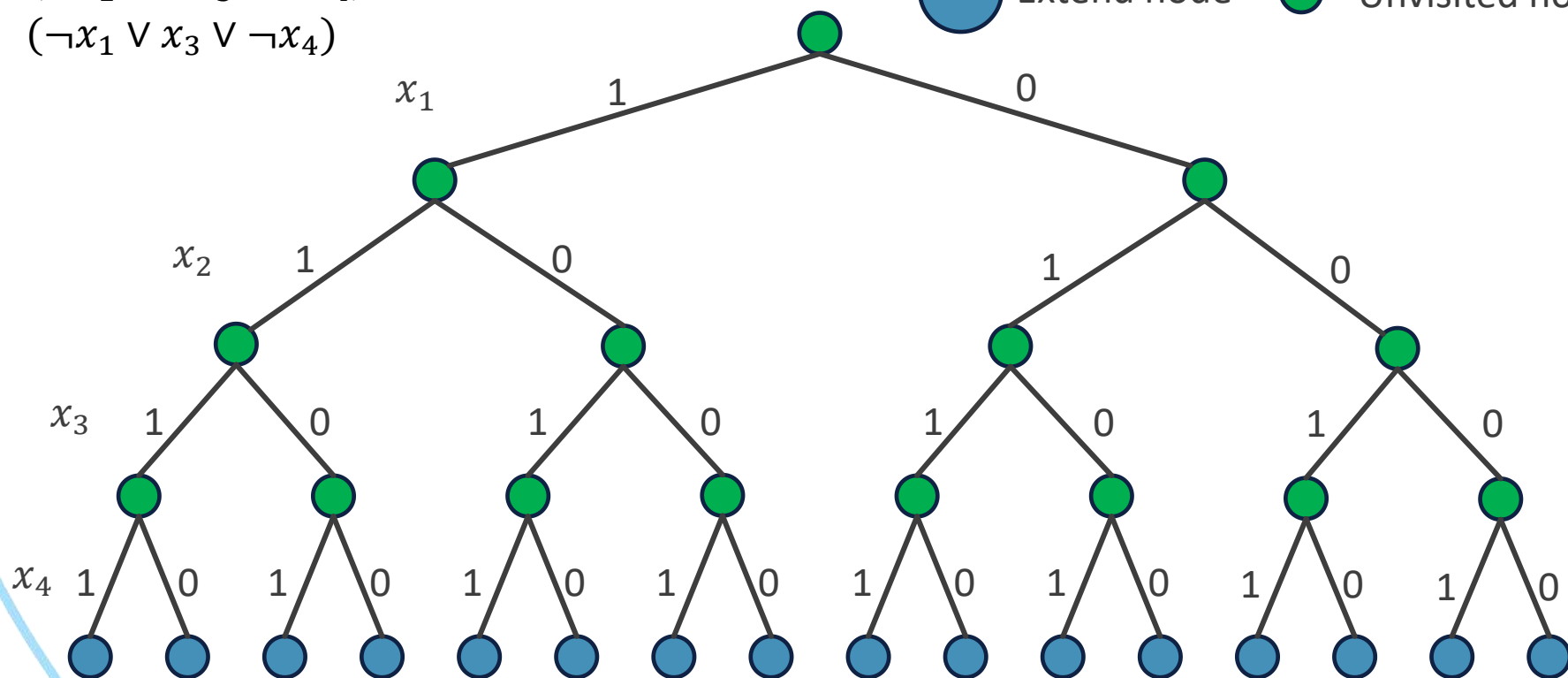
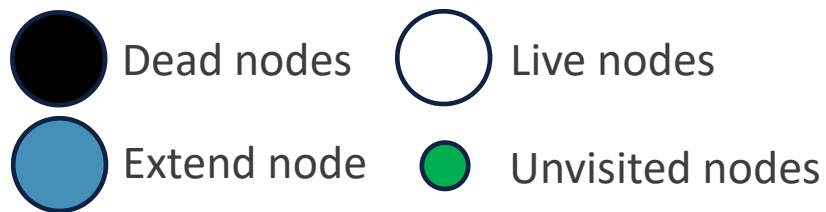
Example

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$$

$$(x_2 \vee x_3 \vee x_4) \wedge$$

$$(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$$

$$(\neg x_1 \vee x_3 \vee \neg x_4)$$



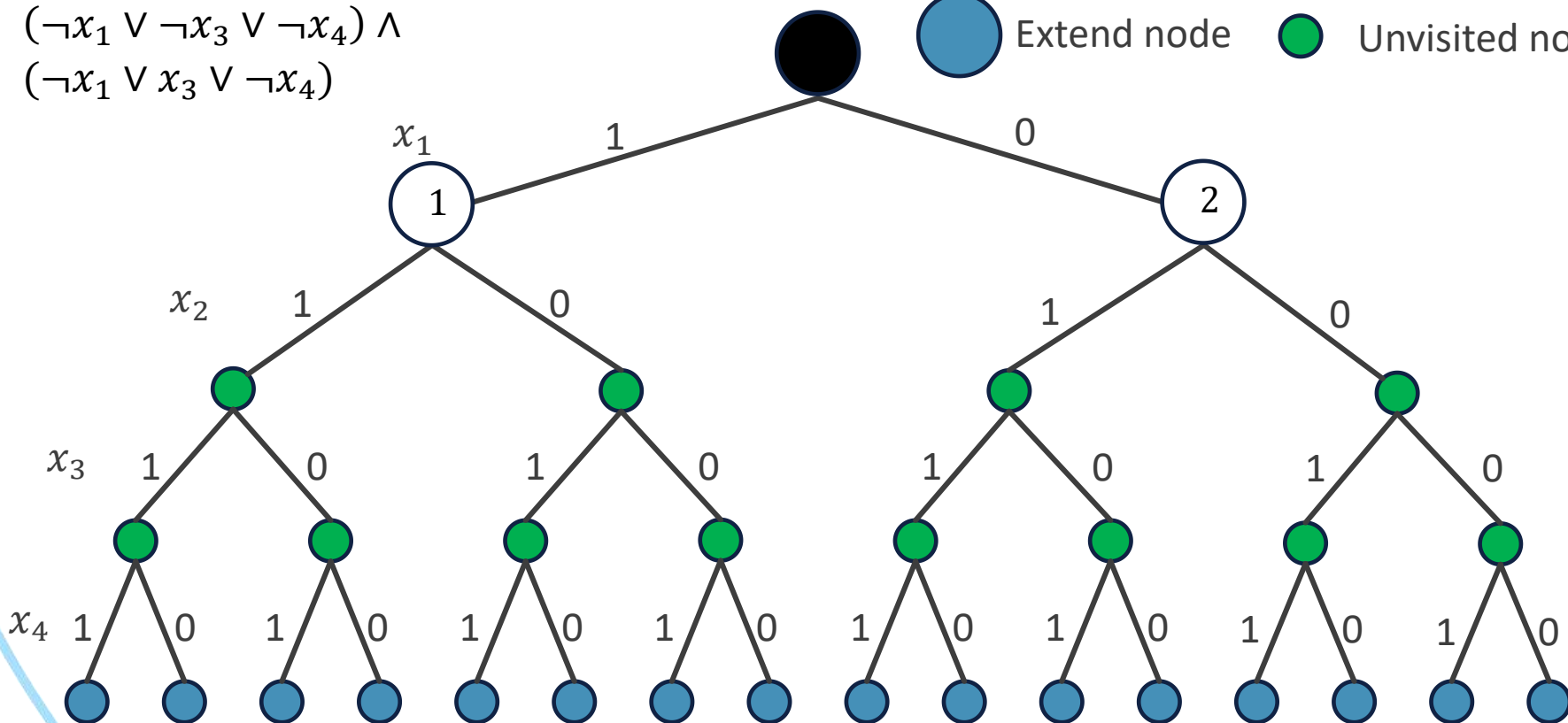


Example

$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$
 $(x_2 \vee x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4)$

$Q: \begin{bmatrix} 2 & 1 & & \end{bmatrix}$

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes



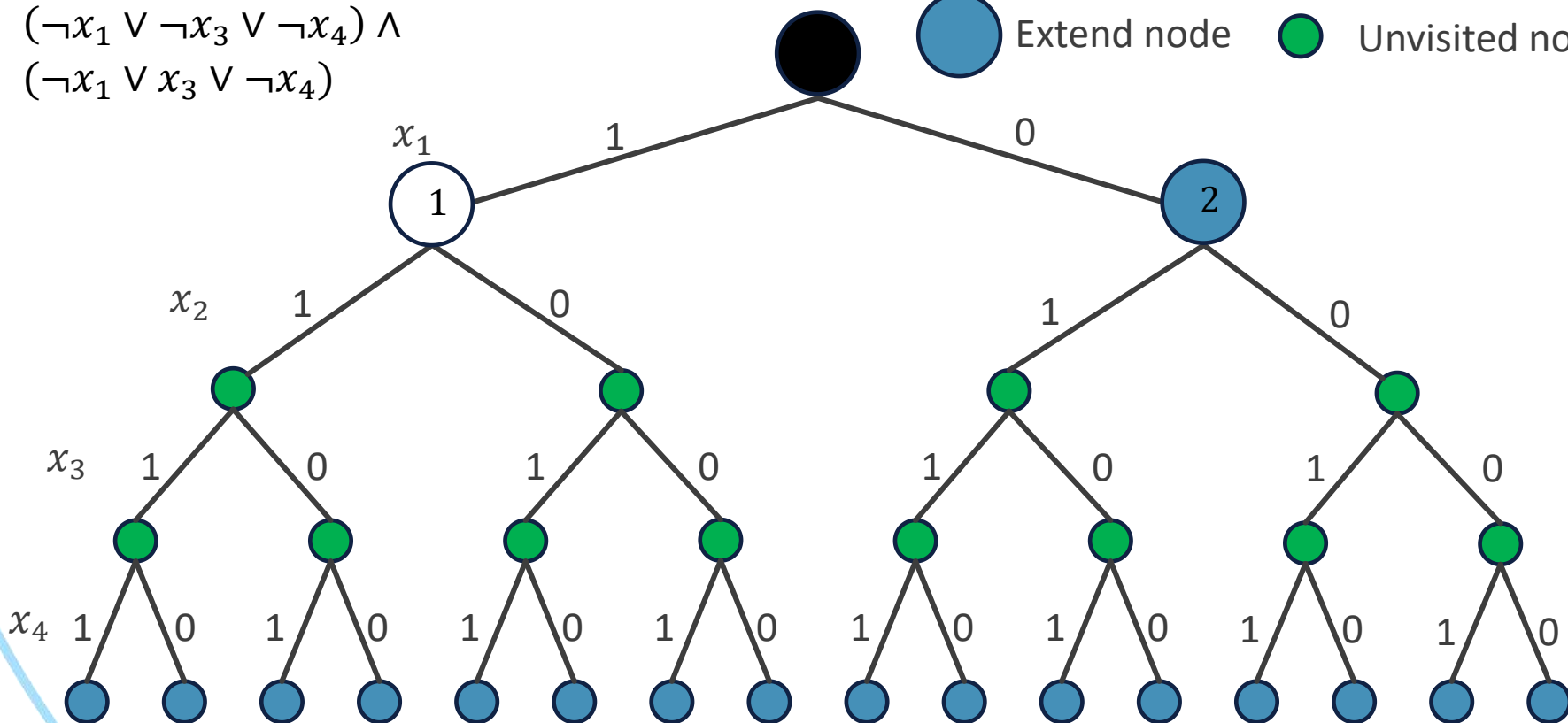
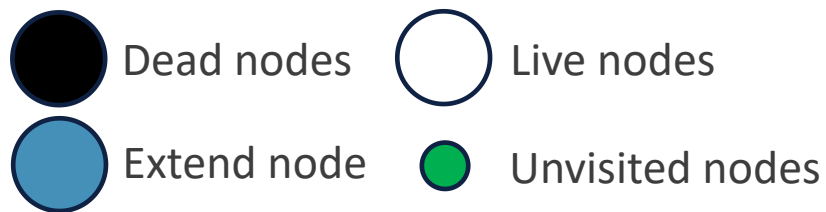


Example

$$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$$
$$(x_2 \vee x_3 \vee x_4) \wedge$$
$$(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$$
$$(\neg x_1 \vee x_3 \vee \neg x_4)$$

Q:

1			
---	--	--	--





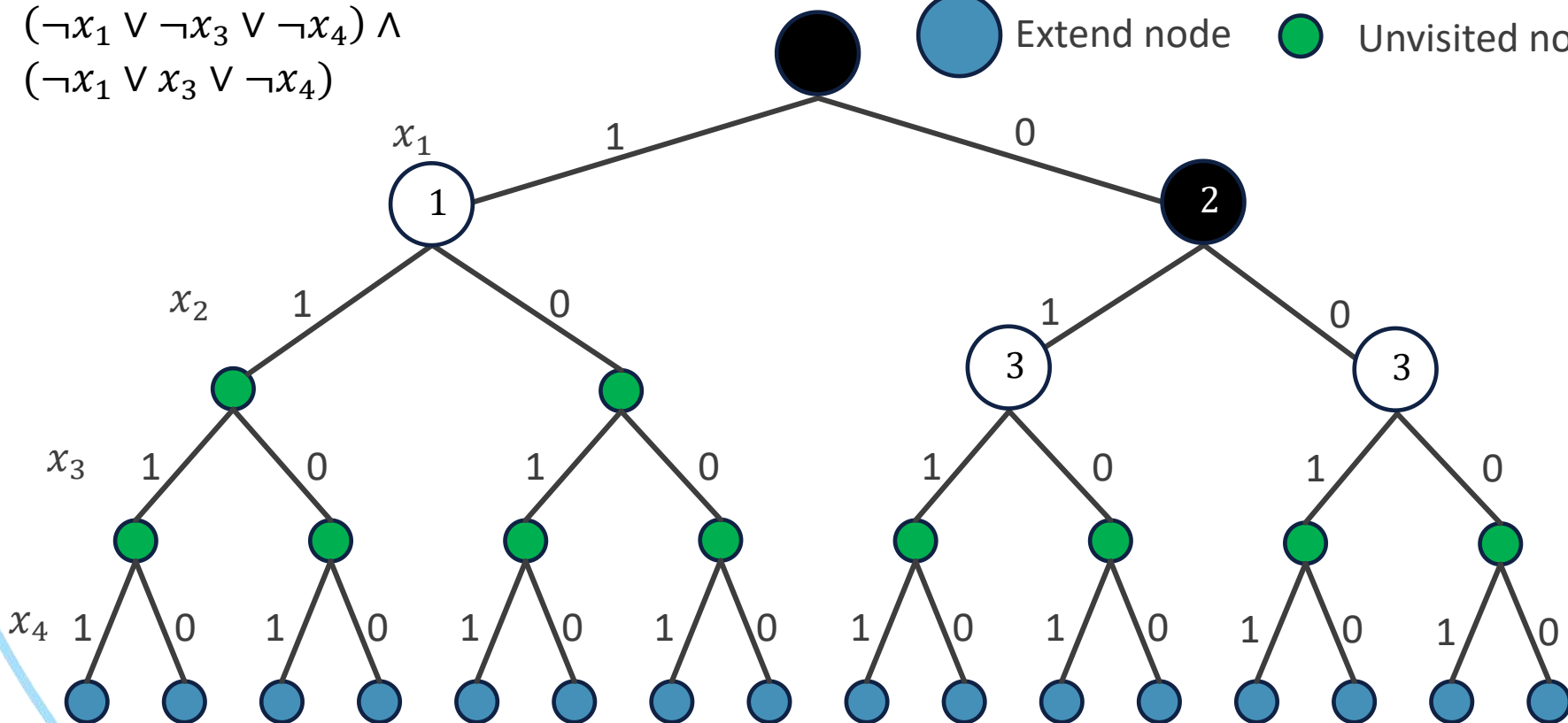
Example

$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$
 $(x_2 \vee x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4)$

Q :

3	3	1	
---	---	---	--

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes





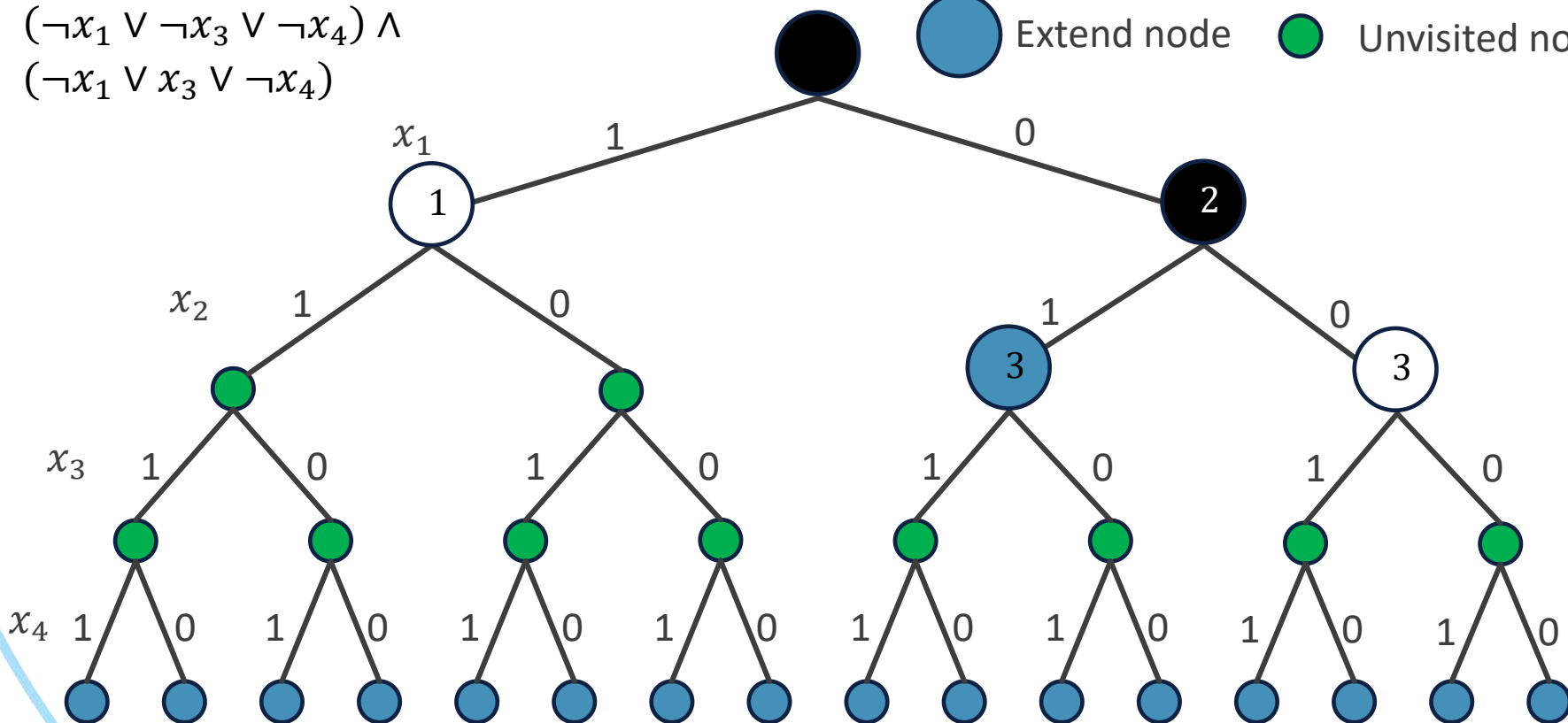
Example

$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$
 $(x_2 \vee x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4)$

Q :

3	1		
---	---	--	--

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes



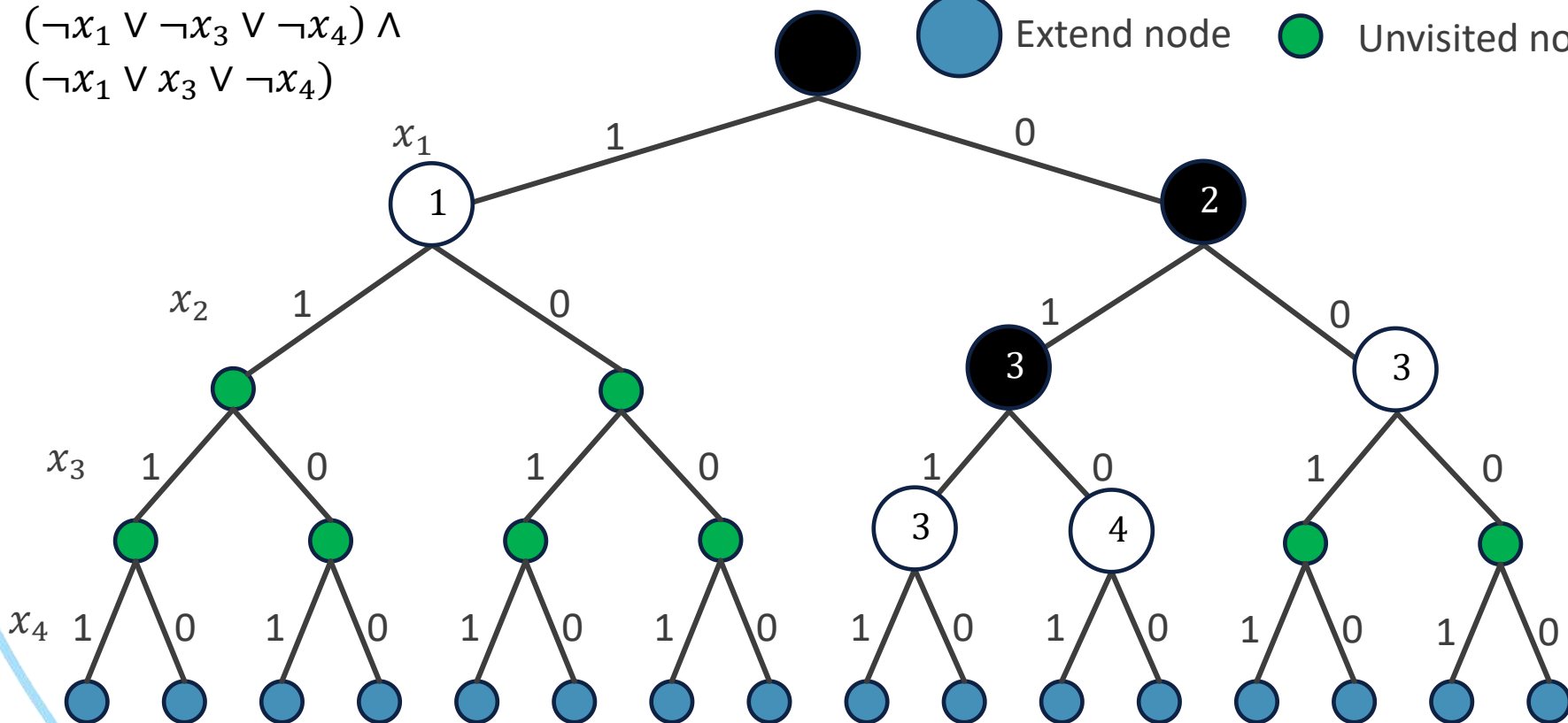


Example

$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$
 $(x_2 \vee x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4)$

$Q: \begin{bmatrix} 4 & 3 & 3 & 1 \end{bmatrix}$

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes





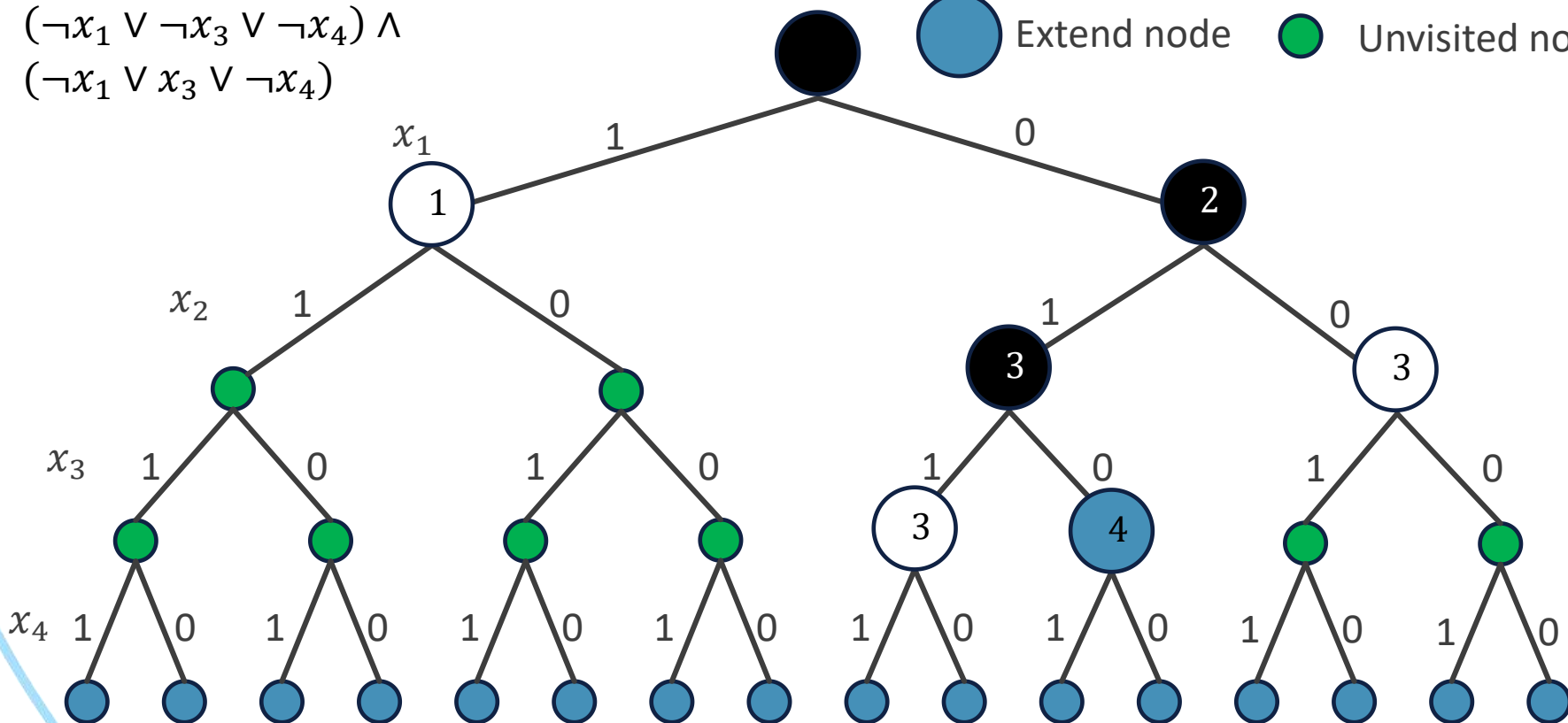
Example

$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$
 $(x_2 \vee x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4)$

Q :

3	3	1	
---	---	---	--

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes





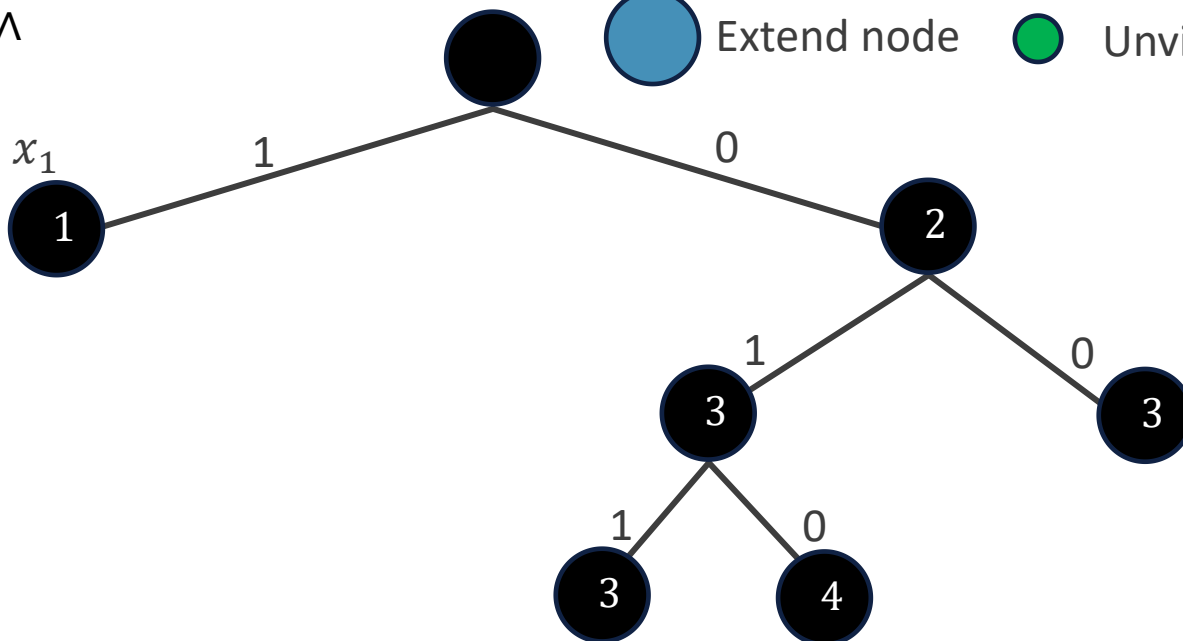
Example

$(x_1 \vee \neg x_2 \vee \neg x_3) \wedge$
 $(x_2 \vee x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4)$

Q :

3	3	1	
---	---	---	--

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes



x_2

x_3

x_4



Classroom Exercise

- Draw the pruned solution space tree for the following k -CNF-SAT problem instance by max-profit branch-and-bound.

$$\begin{aligned}\phi = & (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3) \wedge \\ & (\neg x_1 \vee \neg x_3 \vee x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4) \wedge \\ & (x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_2 \vee x_3 \vee x_4)\end{aligned}$$



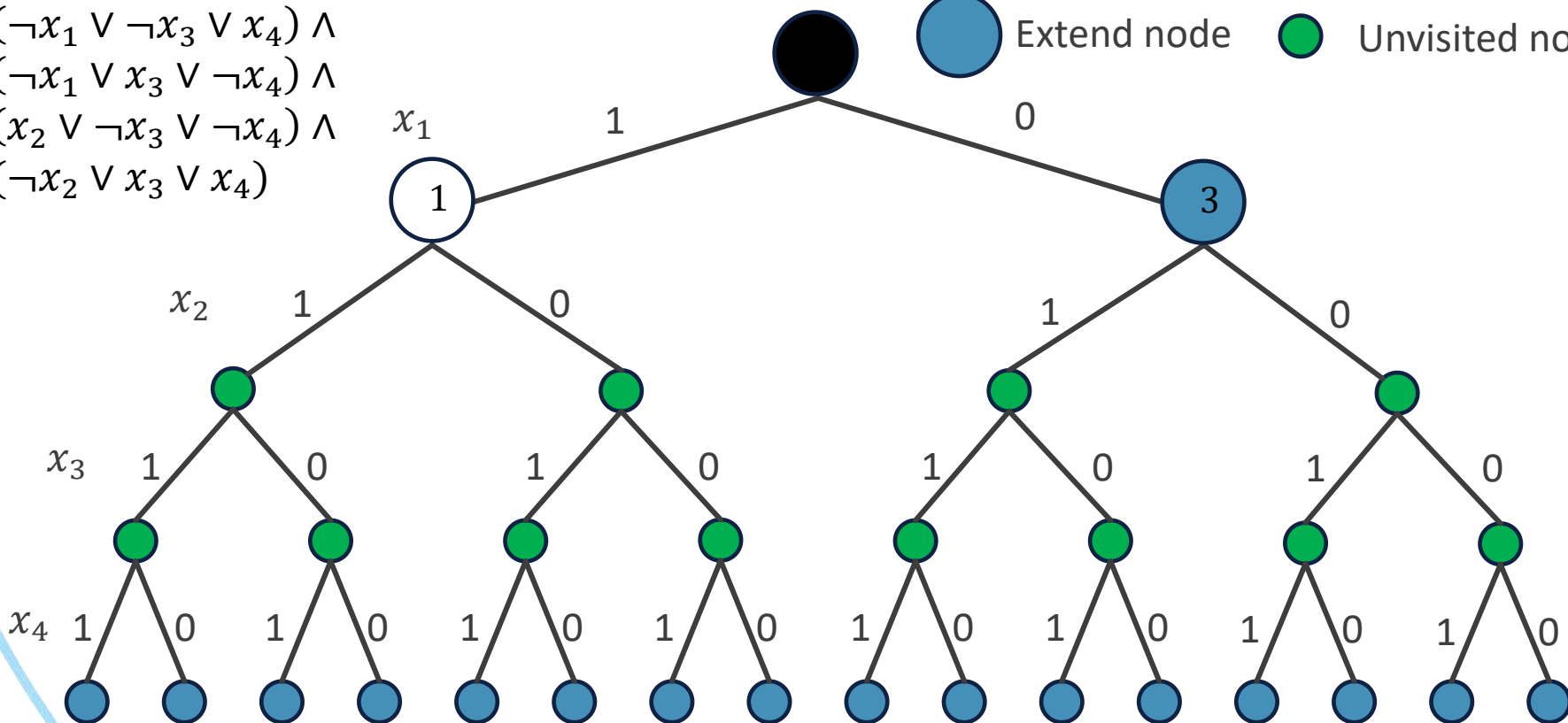
Classroom Exercise

$(\neg x_1 \vee \neg x_2 \vee x_3) \wedge$
 $(x_1 \vee x_2 \vee \neg x_3) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4) \wedge$
 $(x_2 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee x_4)$

Q:

1			
---	--	--	--

● Dead nodes ○ Live nodes
● Extend node ● Unvisited nodes





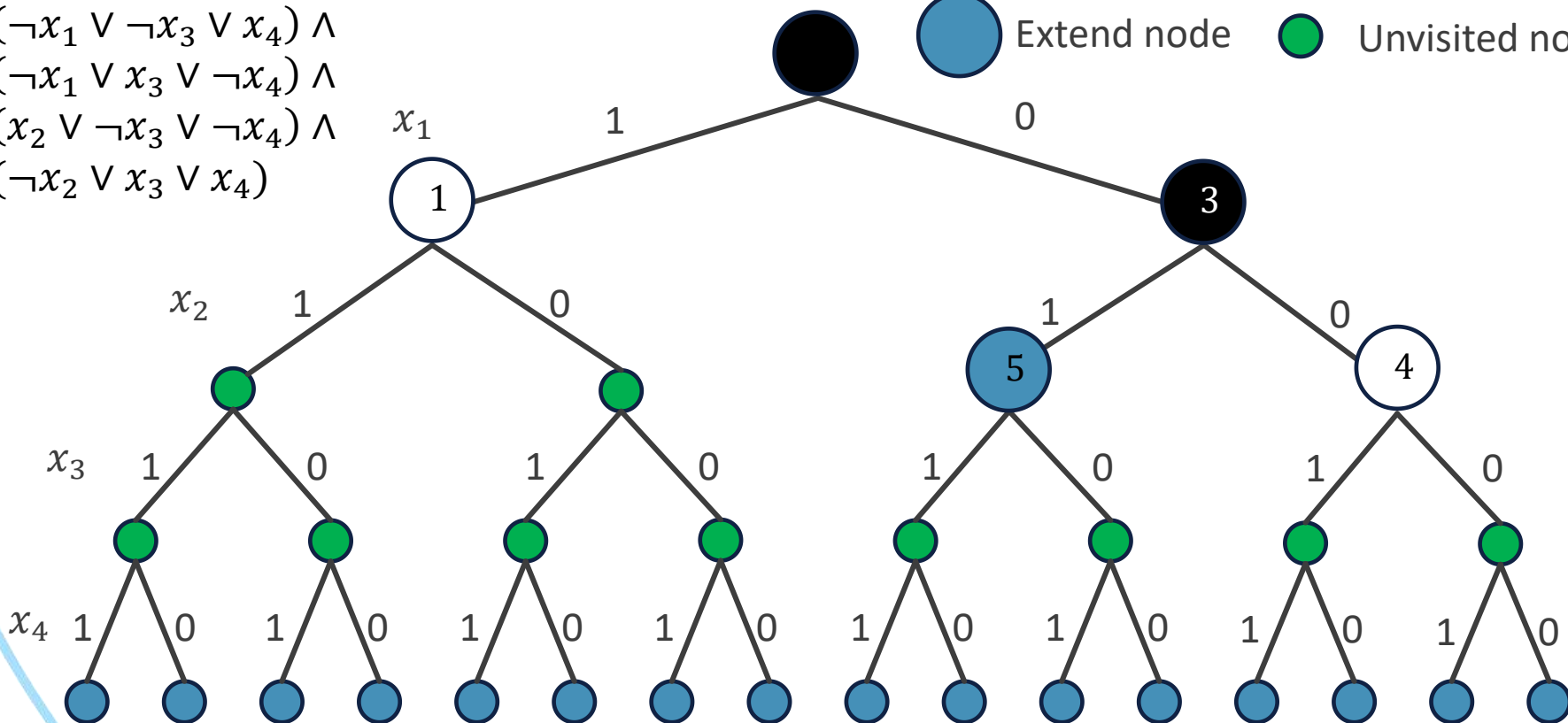
Classroom Exercise

$(\neg x_1 \vee \neg x_2 \vee x_3) \wedge$
 $(x_1 \vee x_2 \vee \neg x_3) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4) \wedge$
 $(x_2 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee x_4)$

Q :

4	1		
---	---	--	--

 Dead nodes  Live nodes
 Extend node  Unvisited nodes



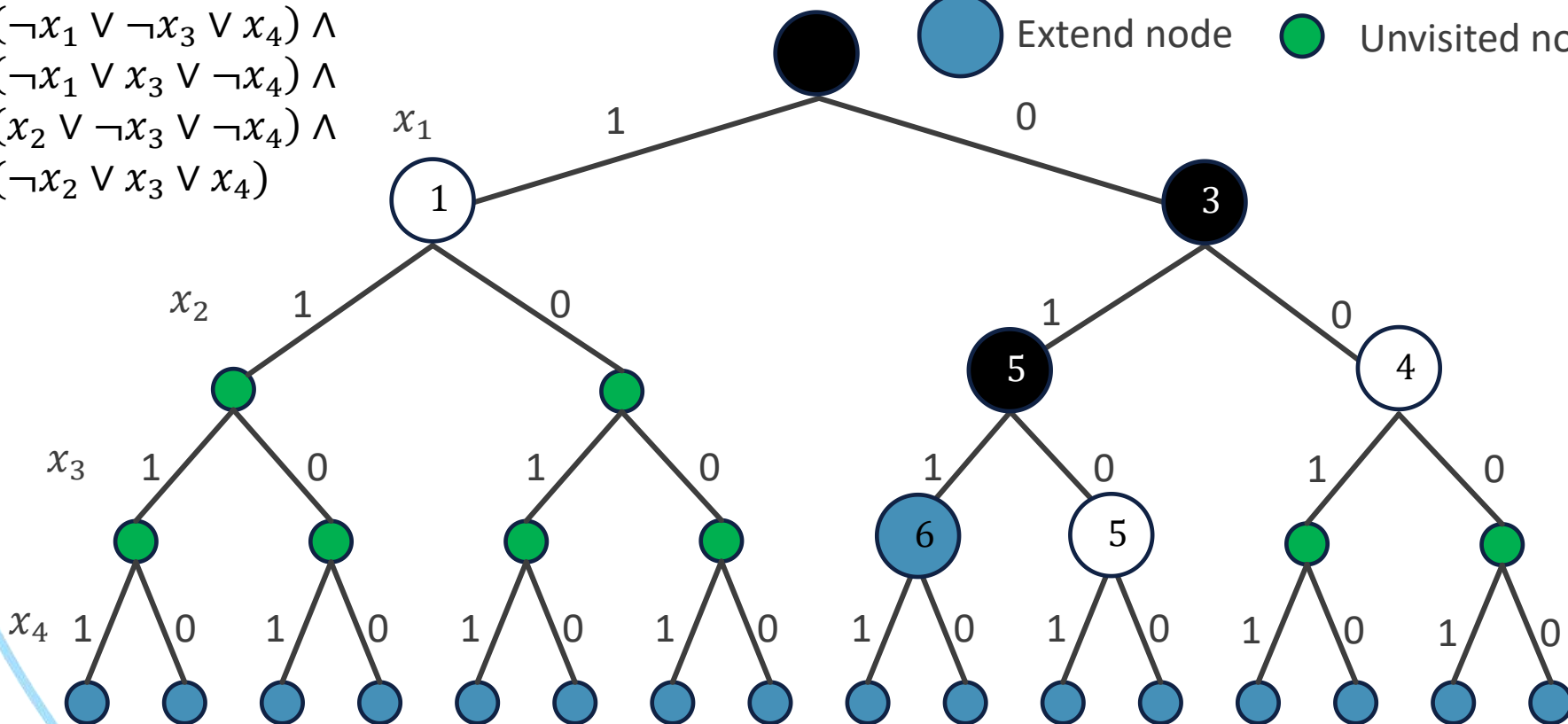
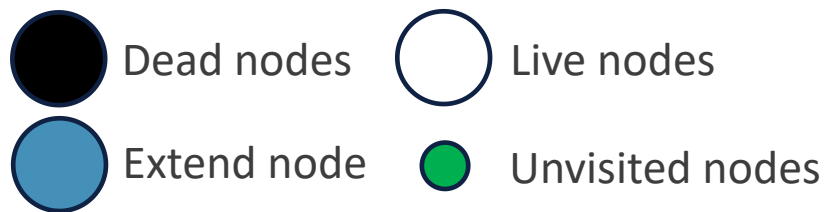


Classroom Exercise

$(\neg x_1 \vee \neg x_2 \vee x_3) \wedge$
 $(x_1 \vee x_2 \vee \neg x_3) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4) \wedge$
 $(x_2 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee x_4)$

Q :

5	4	1	
---	---	---	--





Classroom Exercise

$(\neg x_1 \vee \neg x_2 \vee x_3) \wedge$
 $(x_1 \vee x_2 \vee \neg x_3) \wedge$
 $(\neg x_1 \vee \neg x_3 \vee x_4) \wedge$
 $(\neg x_1 \vee x_3 \vee \neg x_4) \wedge$
 $(x_2 \vee \neg x_3 \vee \neg x_4) \wedge$
 $(\neg x_2 \vee x_3 \vee x_4)$

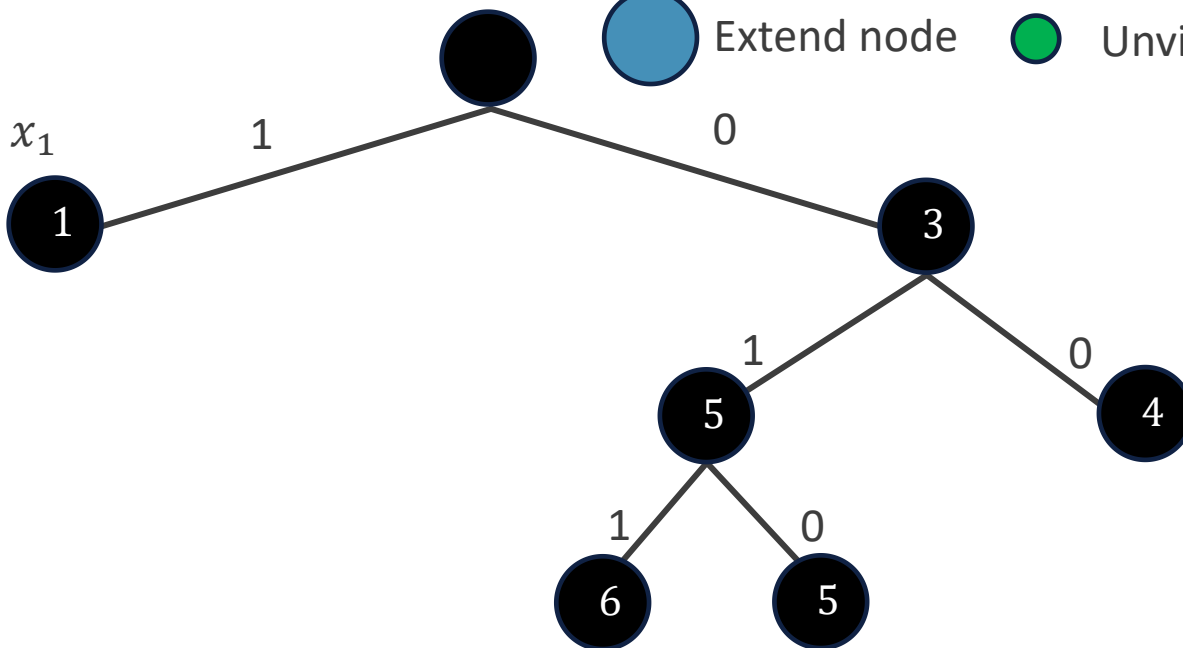
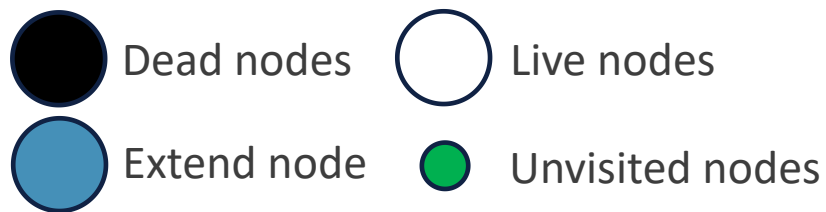
x_2

x_3

x_4

Q :

5	4	1	
---	---	---	--





TRAVELING SALESPERSON PROBLEM



Traveling Salesperson Problem

- Constraint function $C(i)$ is to simply check if the next vertex is connected to the current vertex:

$$C(i) = w[x[i], x[j]]$$

Check if $C(i) \neq \infty$.

- Bounding function $B(i)$ is the total weight if we connect $x[i]$:

$$B(i) = cw(i-1) + w[x[i-1], x[i]]$$

$$cw(i) = \sum_{j=2}^i w[x[j-1], x[j]]$$

Check if $B(i) < bestw$.



Traveling Salesperson Problem

- $B(i) \geq bestw$ is the condition to prune. If we want to prune more branches, we need to increase $B(i)$ as much as possible.
- Now, the bounding function

$$B(i) = cw(i - 1) + w[x[i - 1], x[i]]$$

only calculates the weight between $x[i - 1]$ and $x[i]$, **but ignores all the remaining paths.**



Traveling Salesperson Problem

- How to obtain the lower bound if we consider all the nodes that we haven't visited?
- Just pick the outgoing edge of each unvisited node with minimum weight, and sum them up.
 - Although it may not form a solution (a path), but it is a lower bound.
 - Just like the bound of 0/1 knapsack problem.



The Improved Bounding Function

- Let the cost of the extend node i be

$$B(i) = cw(i) + rw(i)$$

where, $cw(i)$ is as before, $rw(i)$ is the sum of costs of least-cost outgoing edges from each remaining vertices, namely

$$rw(i) = \sum_{j=i+1}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

- $rw(i)$ 从剩余每个节点出发的权值最小边的权值总和
- If $B(i) \geq bestw$, then stop search the extend node i and the following level, otherwise, continue to search.
- At the same time, we adopt min-priority queue to extract the live node with min cost.

MinWeightTSP()

```
1  MinSum  $\leftarrow$  0
2  for  $i \leftarrow 1$  to  $n$  do
3      Min  $\leftarrow \infty$ 
4      for  $j \leftarrow 1$  to  $n$  do
5          if  $w[i, j] \neq \infty$  and  $w[i, j] < Min$  then  $Min \leftarrow w[i, j]$ 
6          if  $Min = \infty$  then return  $\infty$ 
7          MinOut[i]  $\leftarrow Min$ 
8          MinSum  $\leftarrow MinSum + Min$ 
9  for  $i \leftarrow 1$  to  $n$  do  $E.x[i] \leftarrow i$ 
10  $E.s \leftarrow 1$ ;  $E.cw \leftarrow 0$ ;  $E.rw \leftarrow MinSum$ ;  $bestw \leftarrow \infty$ 
11 while  $E.s < n$  do
12     if  $E.s = n - 1$  then
13         if  $w[E.x[n - 1], E.x[n]] \neq \infty$  and  $w[E.x[n], E.x[1]] \neq \infty$  and
14              $E.cw + w[E.x[n - 1], E.x[n]] + w[E.x[n], E.x[1]] < bestw$  then
15              $bestw \leftarrow E.cw + w[E.x[n - 1], E.x[n]] + w[E.x[n], E.x[1]]$ 
16              $E.cw \leftarrow bestw$ ;  $E.lw \leftarrow bestw$ 
17              $E.s \leftarrow E.s + 1$ 
18             Insert( $Q, E$ )
```

Initialize *MinOut*

Isolated vertex

Initialize data structure

Constraint function for node $n - 1 \rightarrow n \rightarrow 1$

Increase level

Not finish here. We still put enqueue it. The algorithm terminates when a solution is dequeued ($E.s = n$).



E.s: Current node

i: Next node

Once moved,
subtract *MinOut* of
the current node.

```
18  else for  $i \leftarrow E.s + 1$  to  $n$  do
19      if  $w[E.x[E.s], E.x[i]] \neq \infty$  then
20           $cw \leftarrow E.cw + w[E.x[E.s], E.x[i]]$ 
21           $rw \leftarrow E.rw - \text{MinOut}[E.x[E.s]]$ 
22           $B(i) \leftarrow cw + rw$ 
23          if  $B(i) < \text{bestw}$  then
24              for  $j \leftarrow 1$  to  $n$  do  $N.x[j] \leftarrow E.x[j]$ 
25               $N.x[E.s + 1] \leftarrow E.x[i]$ 
26               $N.x[i] \leftarrow E.x[E.s + 1]$ 
27               $N.cw \leftarrow cw$ ;  $N.s \leftarrow E.s + 1$ 
28               $N.lw \leftarrow B(i)$ ;  $N.rw \leftarrow rw$ 
29              Insert( $Q, N$ )
30   $E \leftarrow \text{ExtractMin}(Q)$ 
31  if  $\text{bestw} = \infty$  return  $\infty$ 
32  for  $i \leftarrow 1$  to  $n$  do  $\text{bestx}[i] \leftarrow E.x[i]$ 
33  return  $\text{bestw}$ 
```

Calculate $B(i)$

Switch selected
vertex ($E.s + 1$) to i .



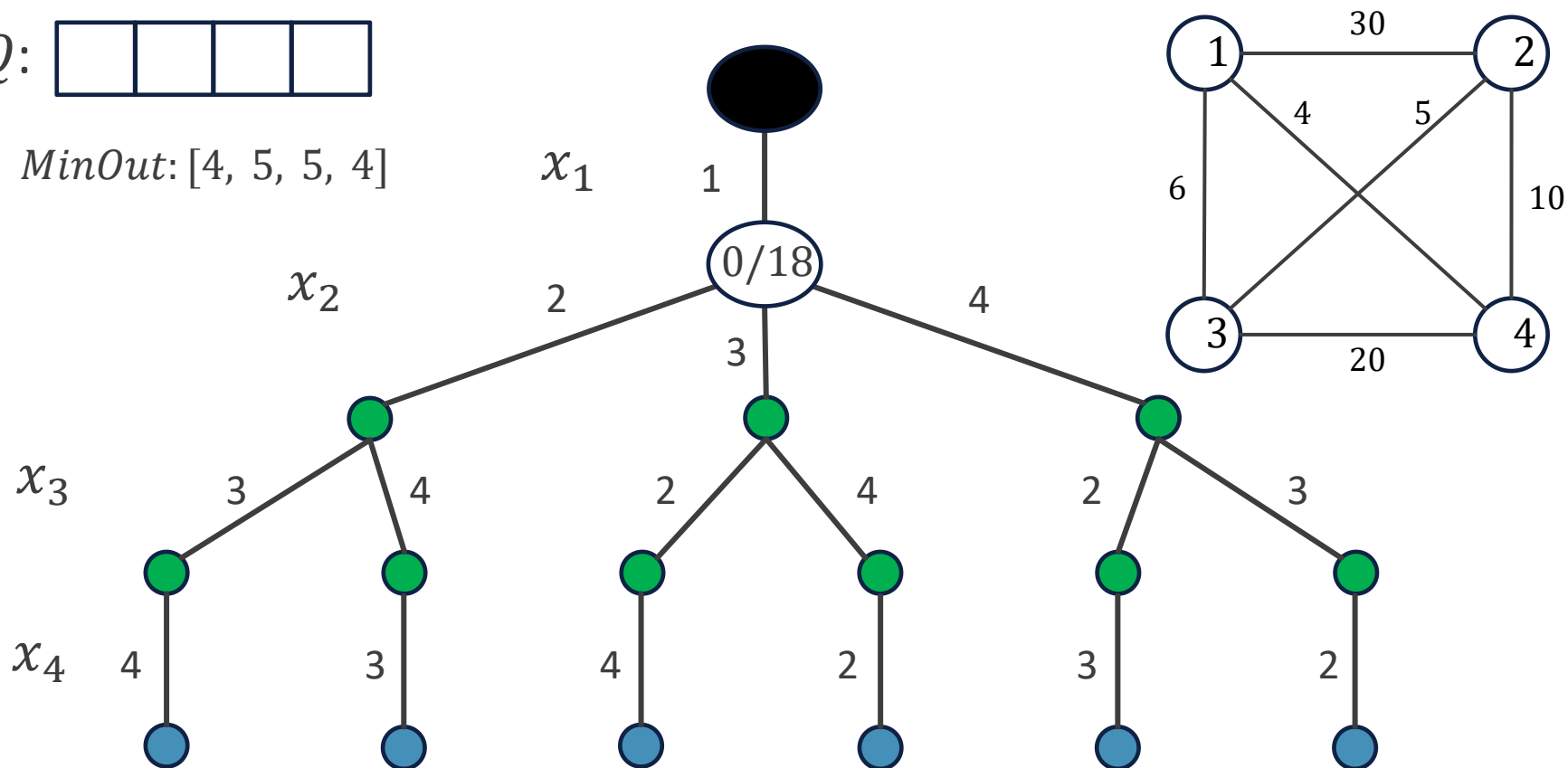
Example

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

--	--	--	--

MinOut: [4, 5, 5, 4]





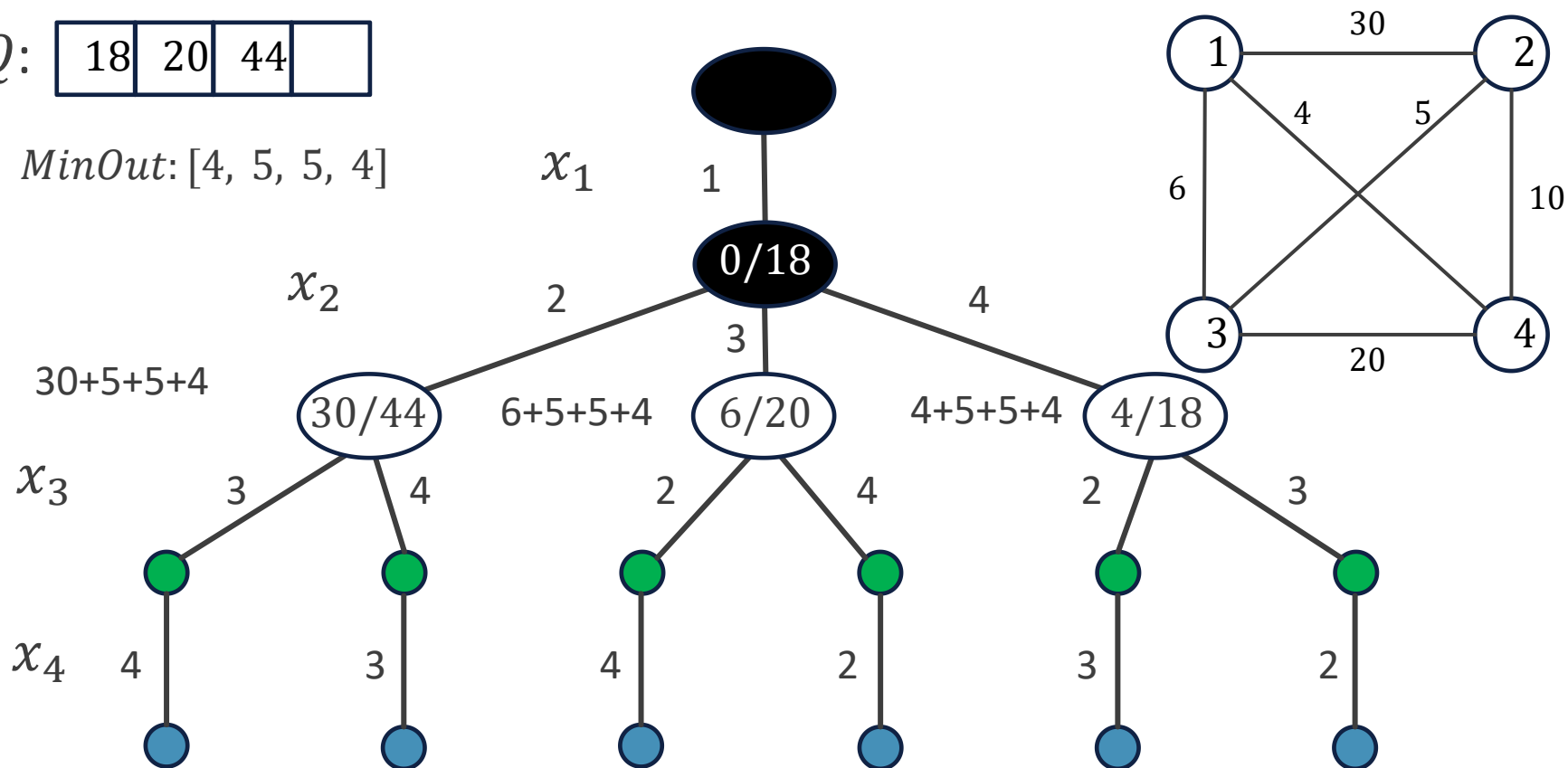
Example

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

18	20	44	
----	----	----	--

MinOut: [4, 5, 5, 4]





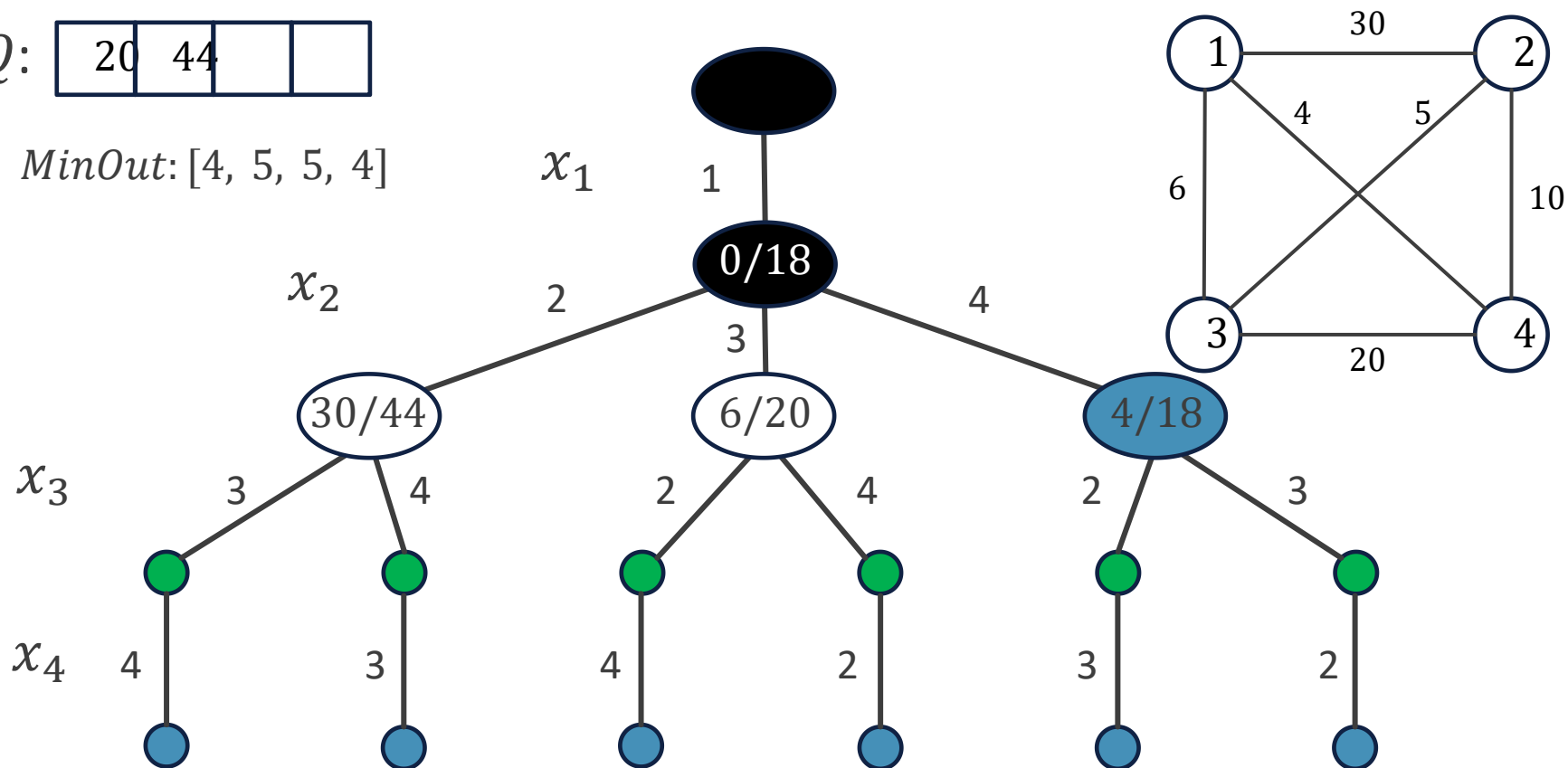
Example

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

20	44		
----	----	--	--

MinOut: [4, 5, 5, 4]





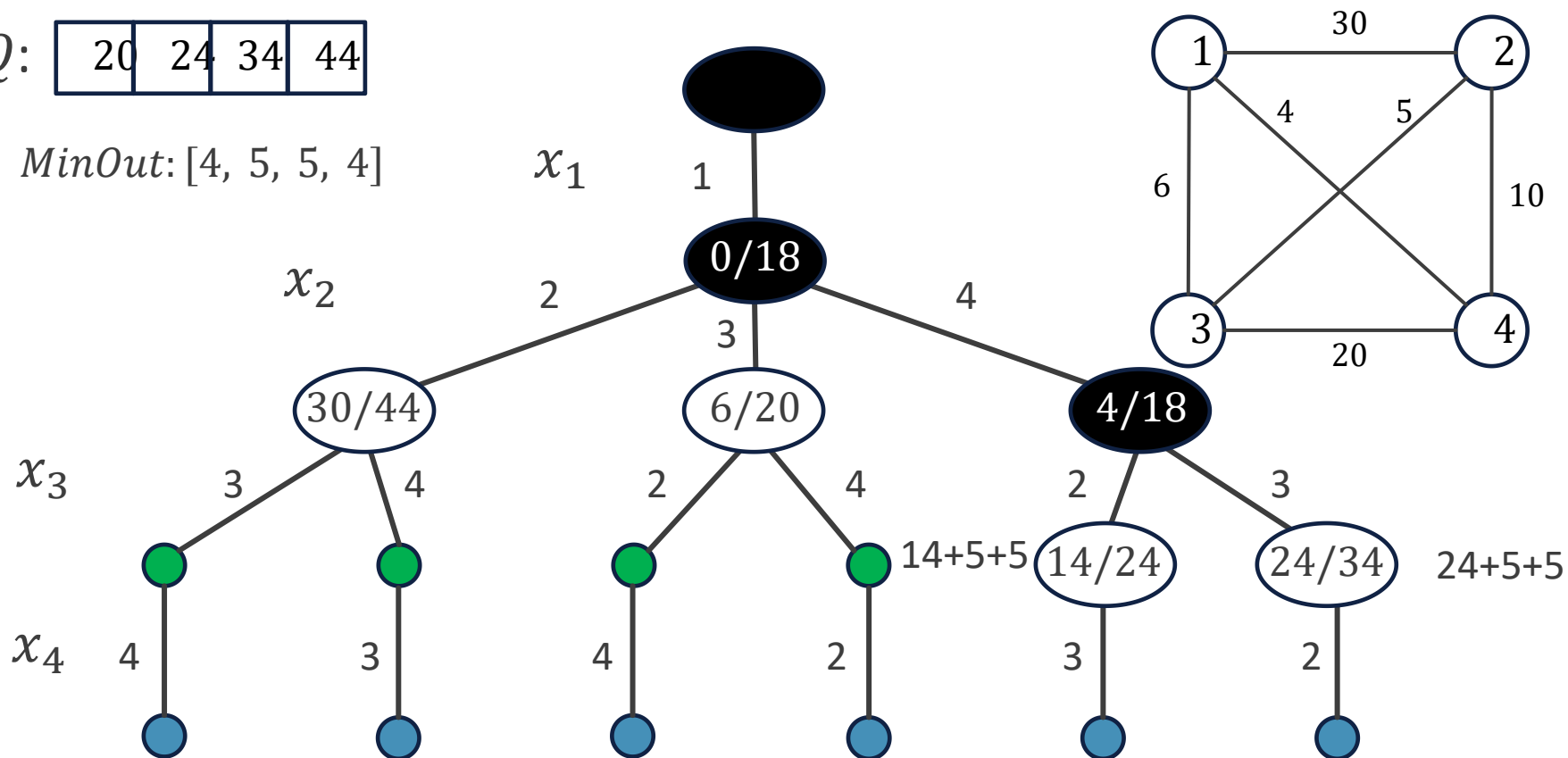
Example

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q :

20	24	34	44
----	----	----	----

$MinOut$: [4, 5, 5, 4]





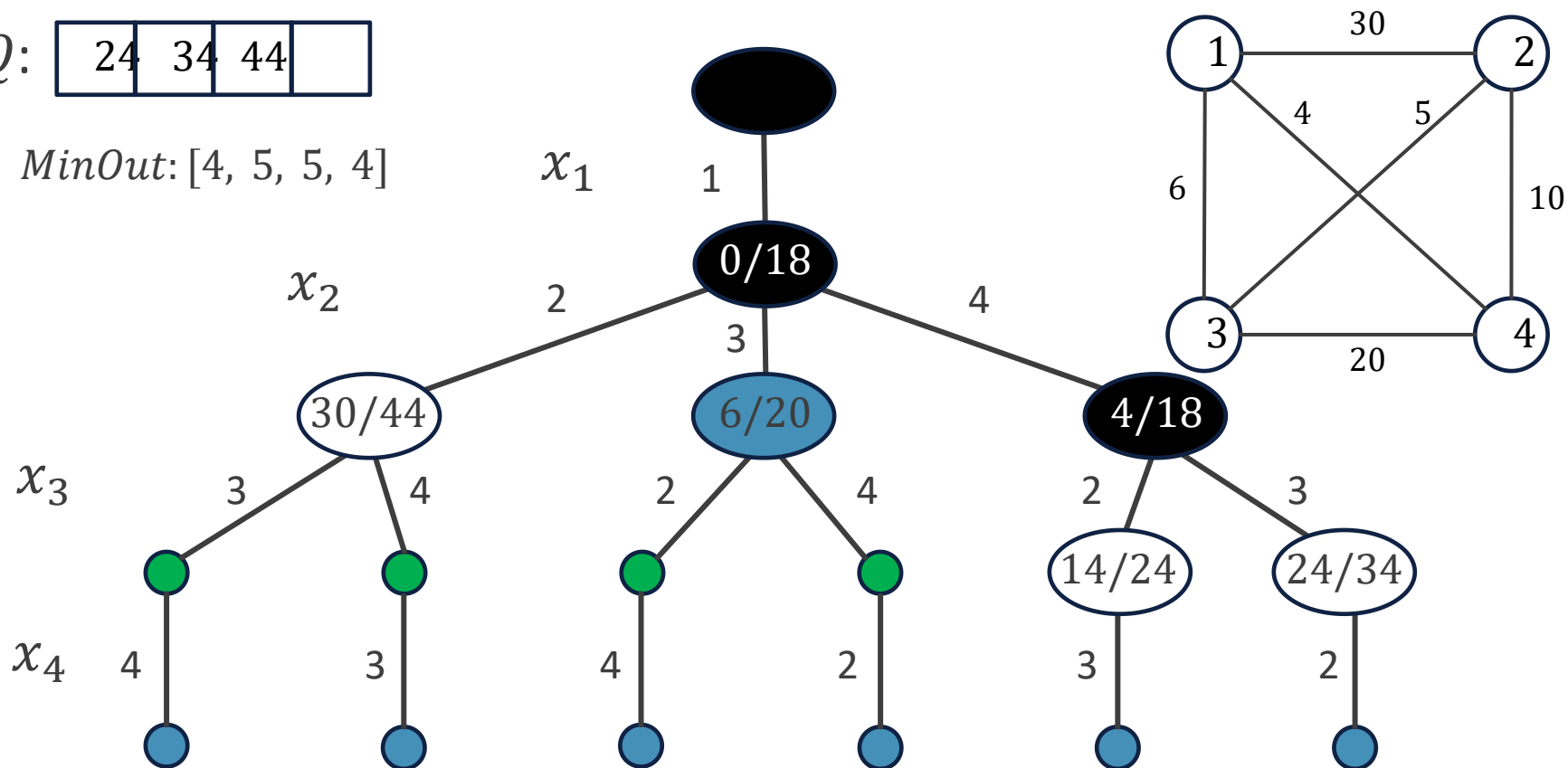
Example

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

24	34	44	
----	----	----	--

MinOut: [4, 5, 5, 4]





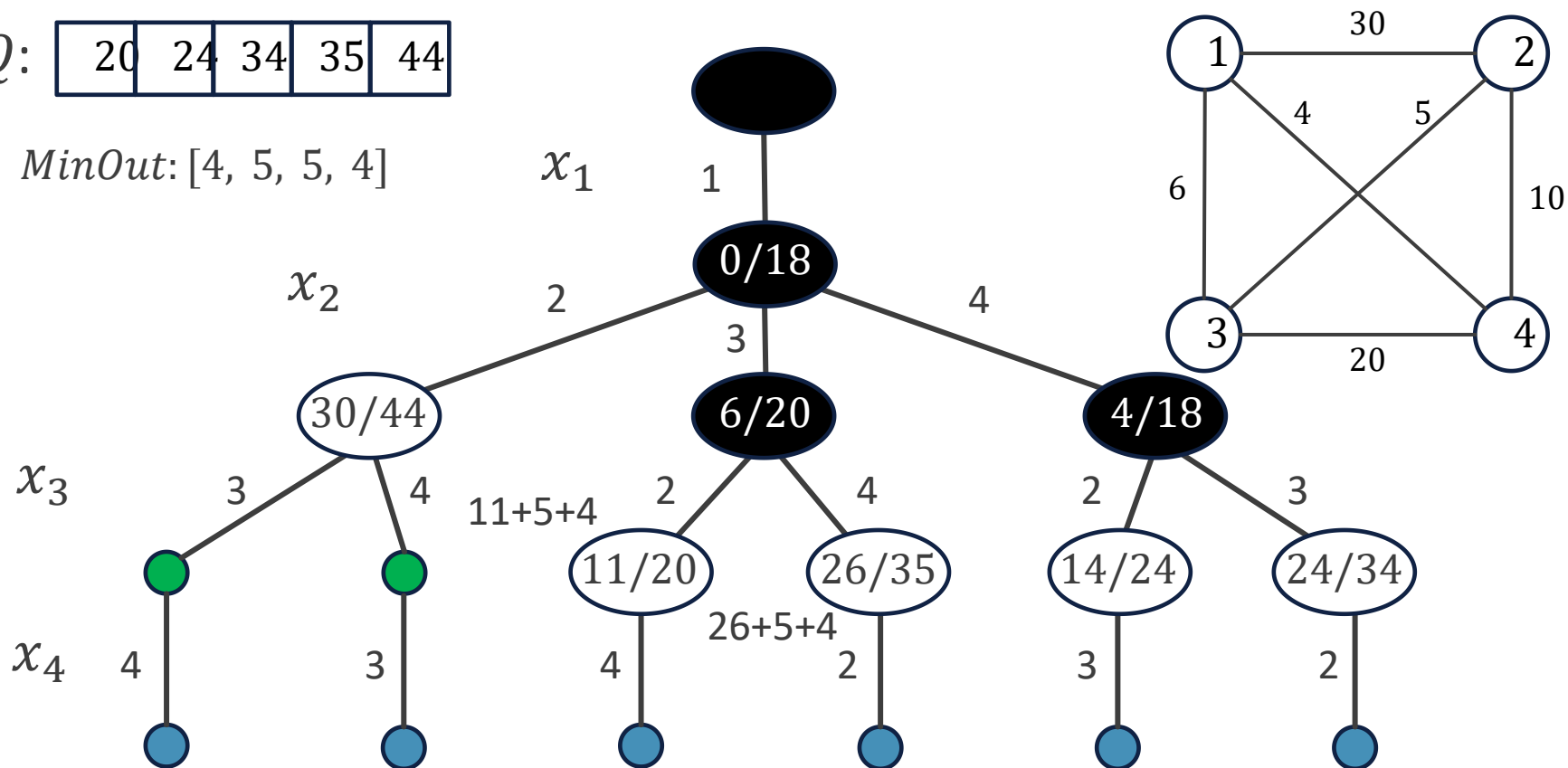
Example

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q :

20	24	34	35	44
----	----	----	----	----

$MinOut$: [4, 5, 5, 4]





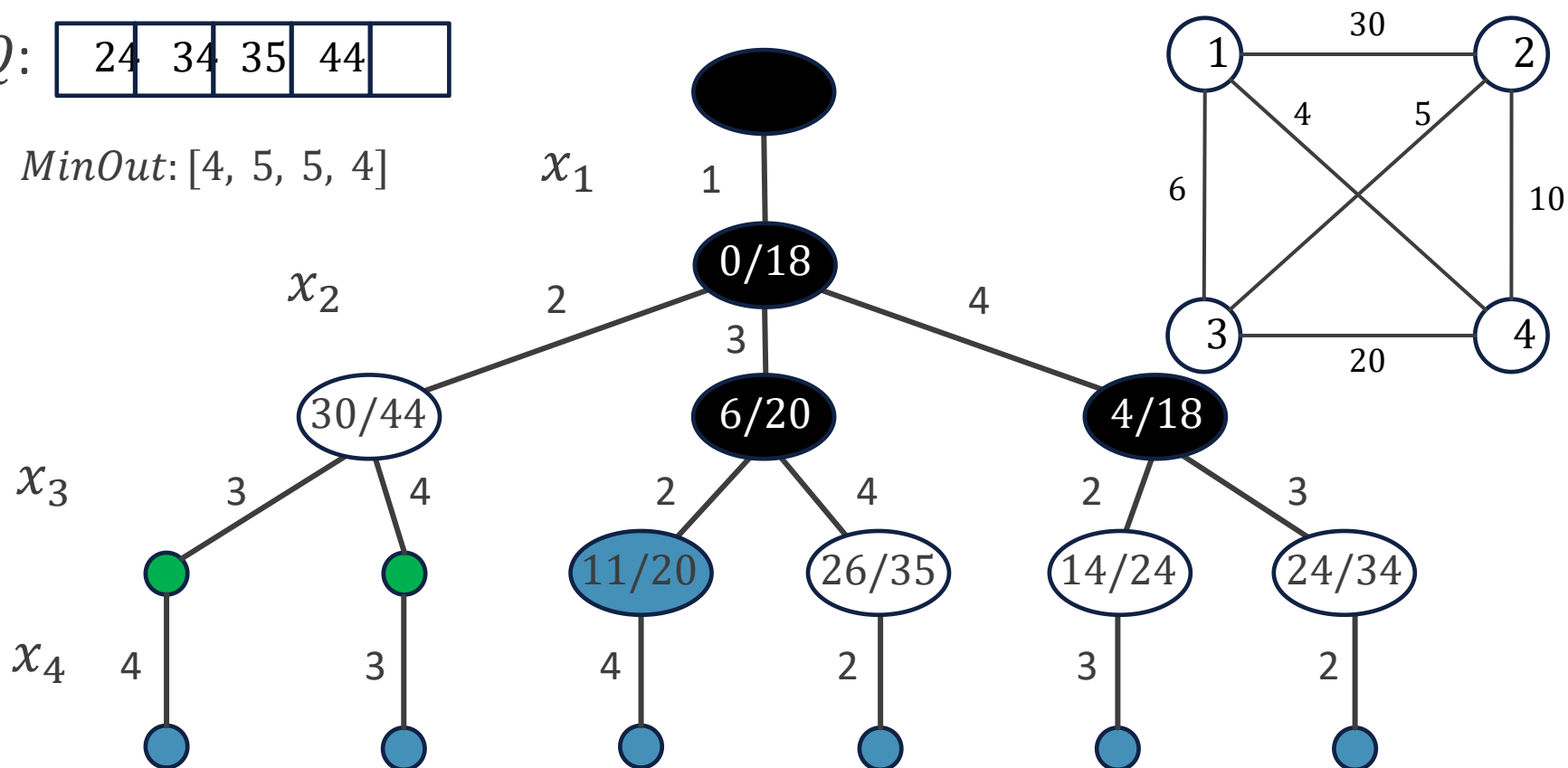
Example

$$rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

24	34	35	44	
----	----	----	----	--

MinOut: [4, 5, 5, 4]





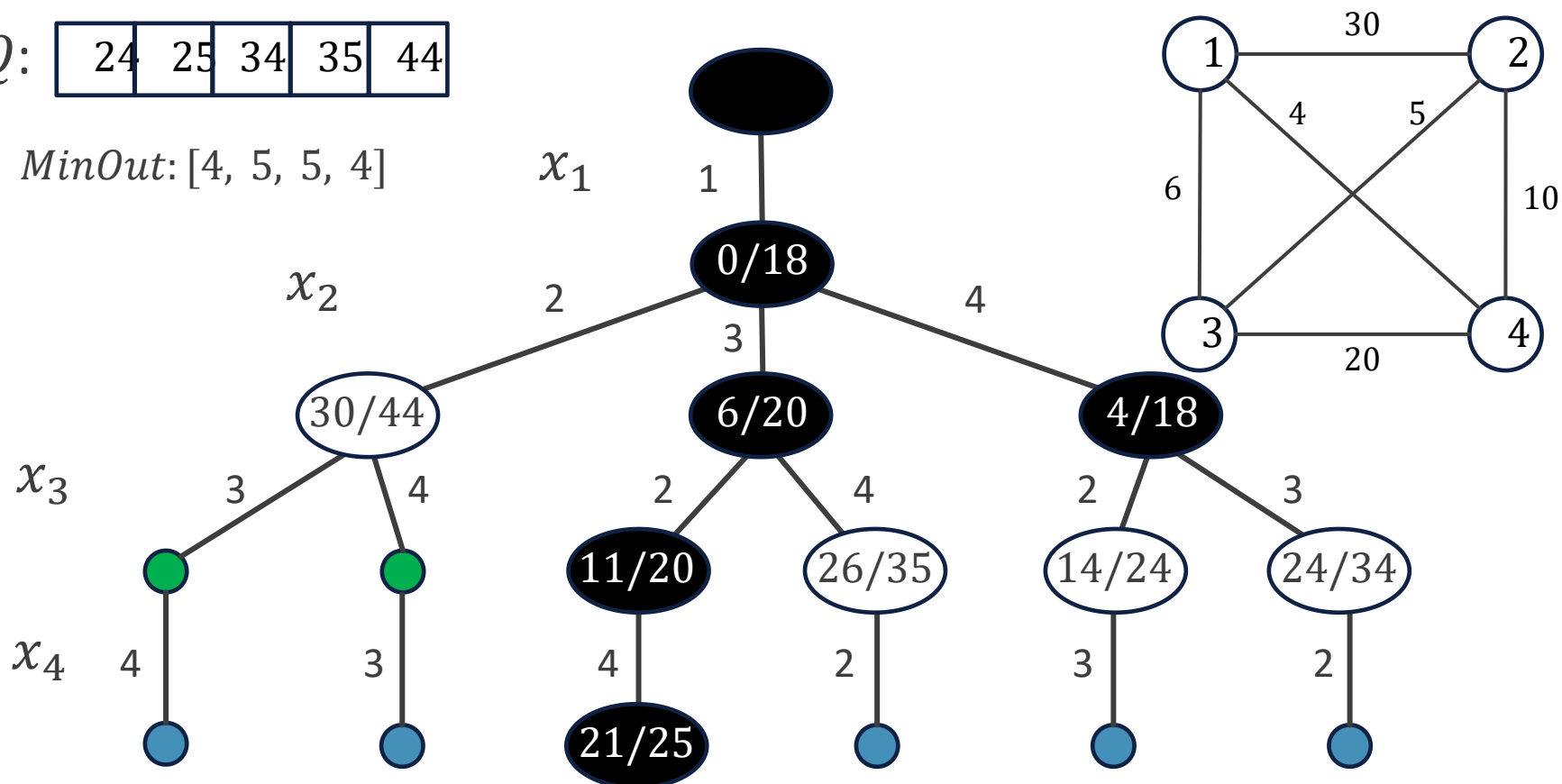
Example

$$rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q :

24	25	34	35	44
----	----	----	----	----

$MinOut$: [4, 5, 5, 4]



(1,3,2,4,1), 25



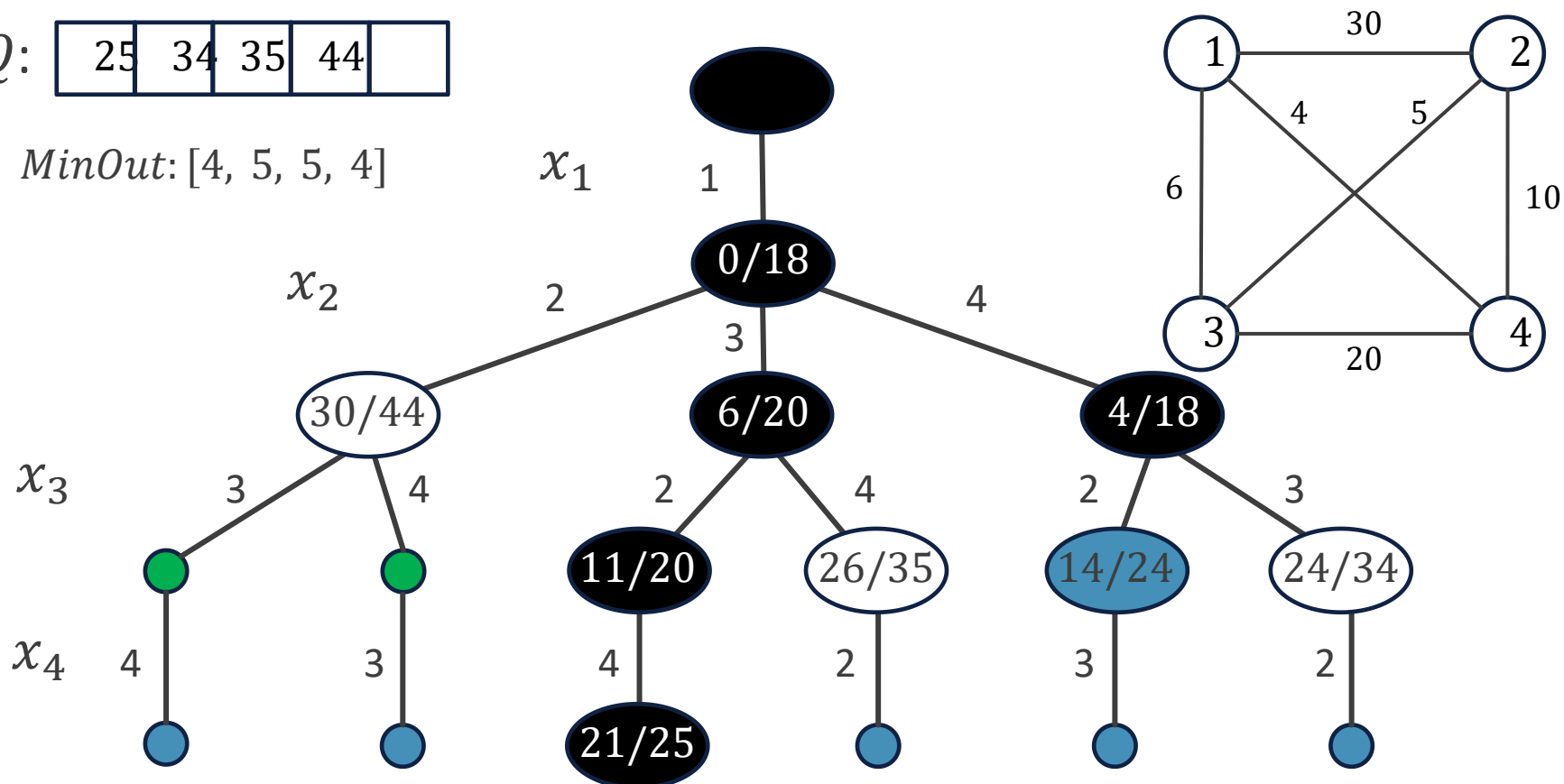
Example

$$rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

25	34	35	44	
----	----	----	----	--

MinOut: [4, 5, 5, 4]



(1,3,2,4,1), 25

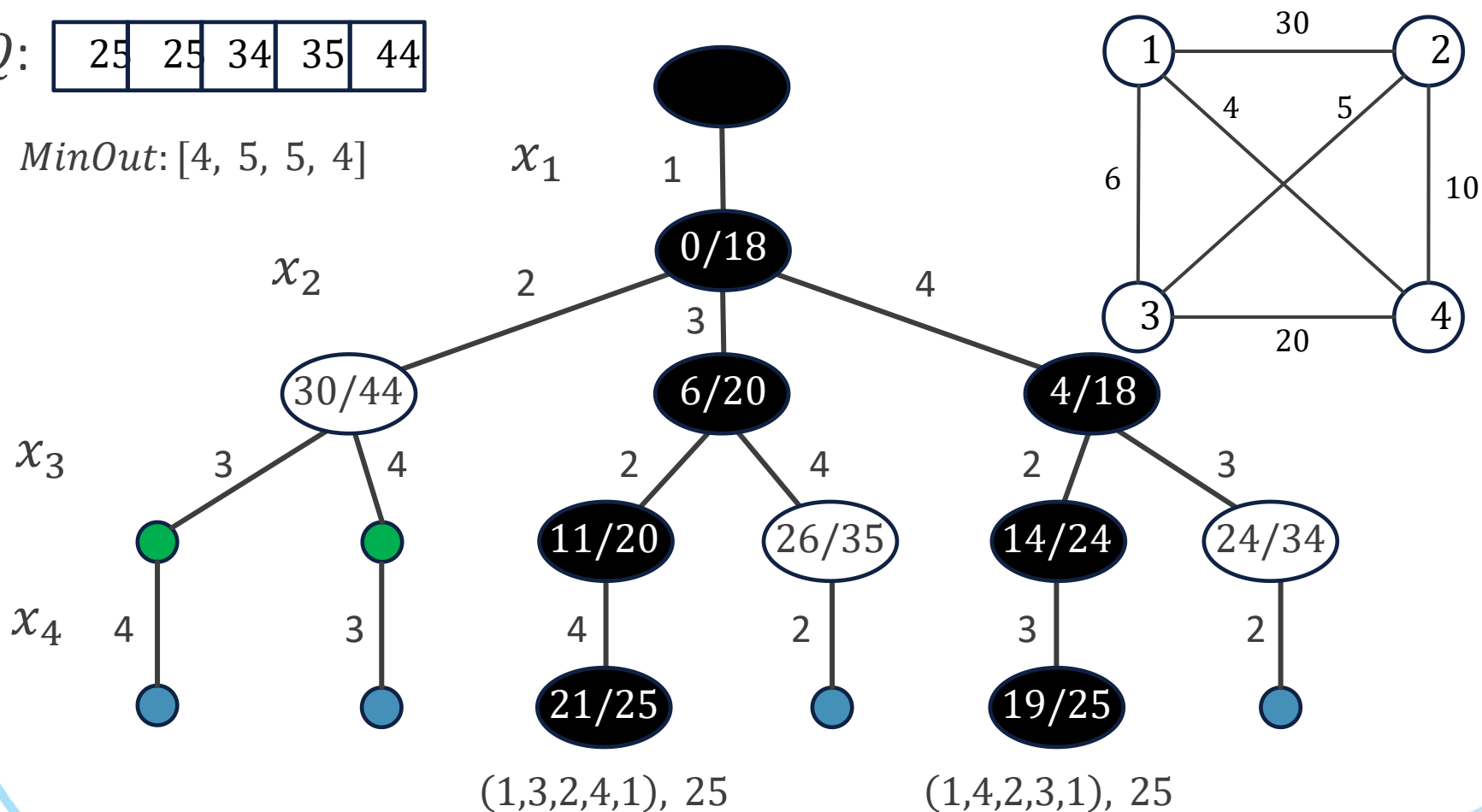


Example

Q :

25	25	34	35	44
----	----	----	----	----

$MinOut$: [4, 5, 5, 4]



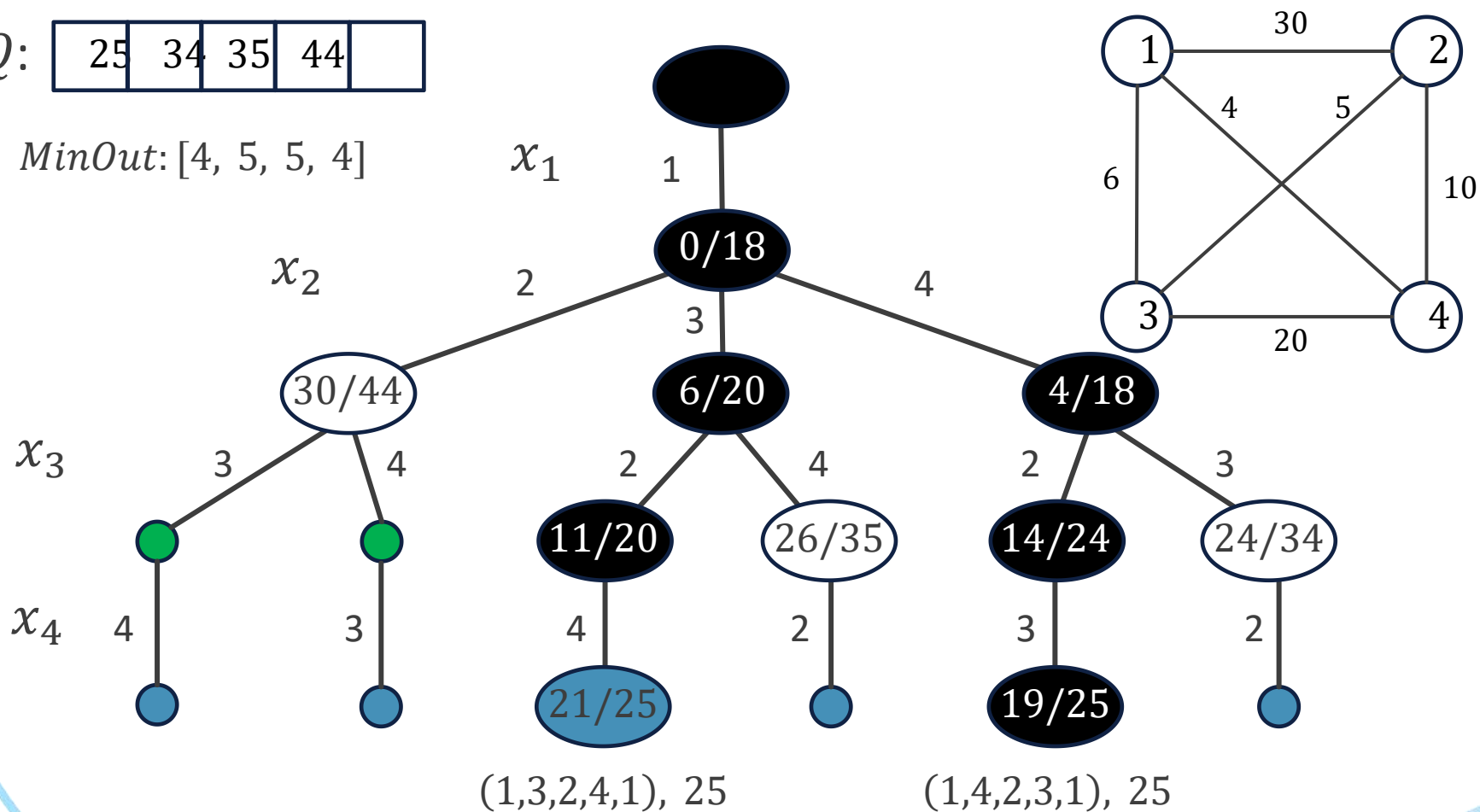


Example

Q :

25	34	35	44	
----	----	----	----	--

$MinOut$: [4, 5, 5, 4]



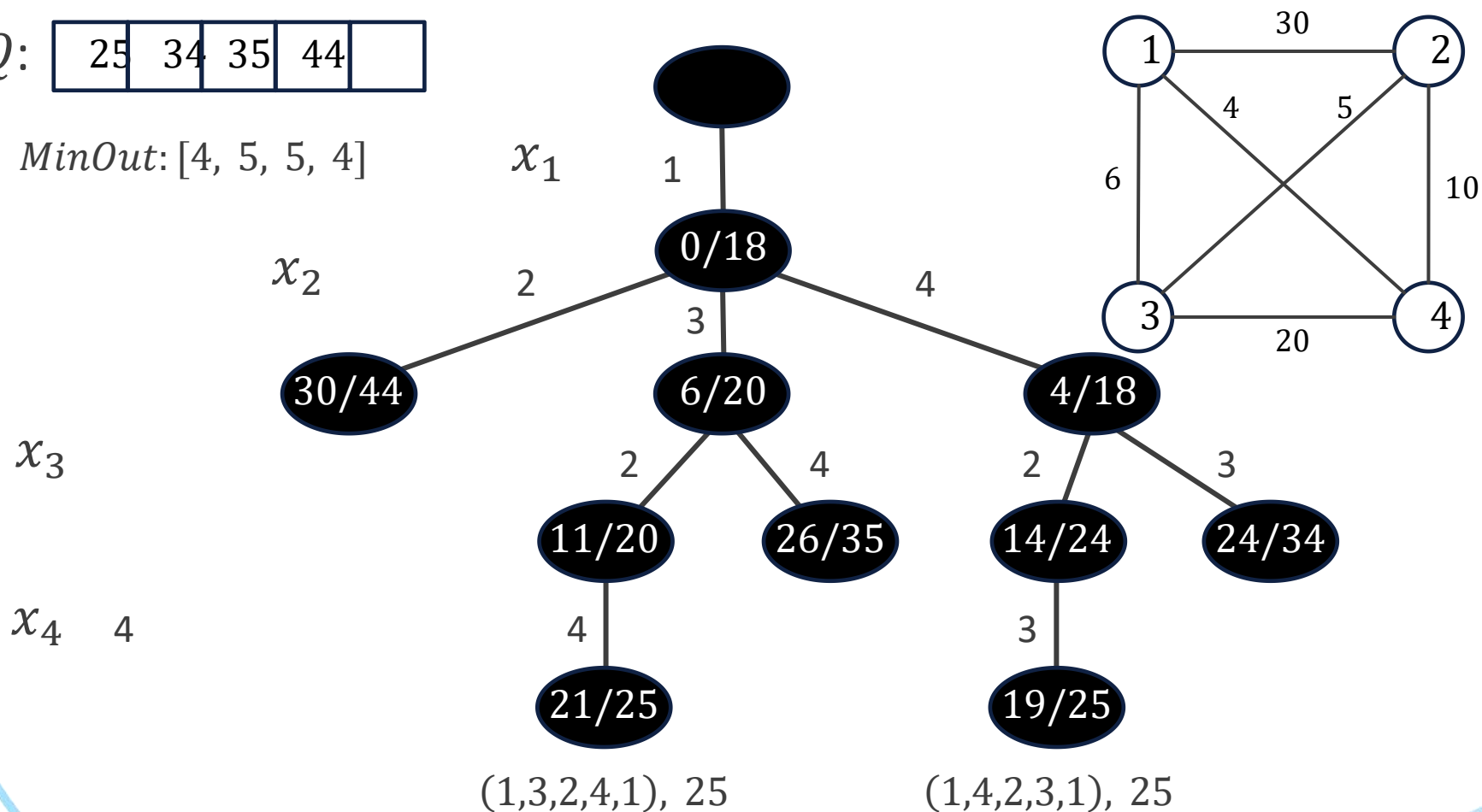


Example

Q :

25	34	35	44	
----	----	----	----	--

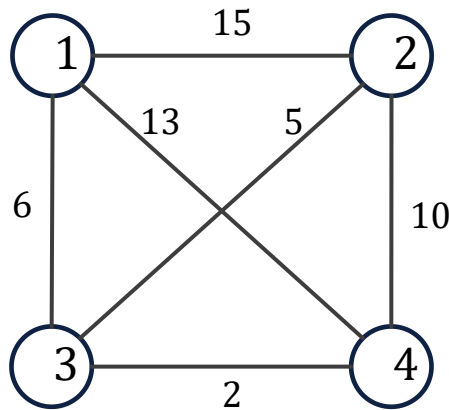
$MinOut$: [4, 5, 5, 4]





Classroom Exercise

- Draw the pruned solution space tree for the following TSP instance by max-profit branch-and-bound.





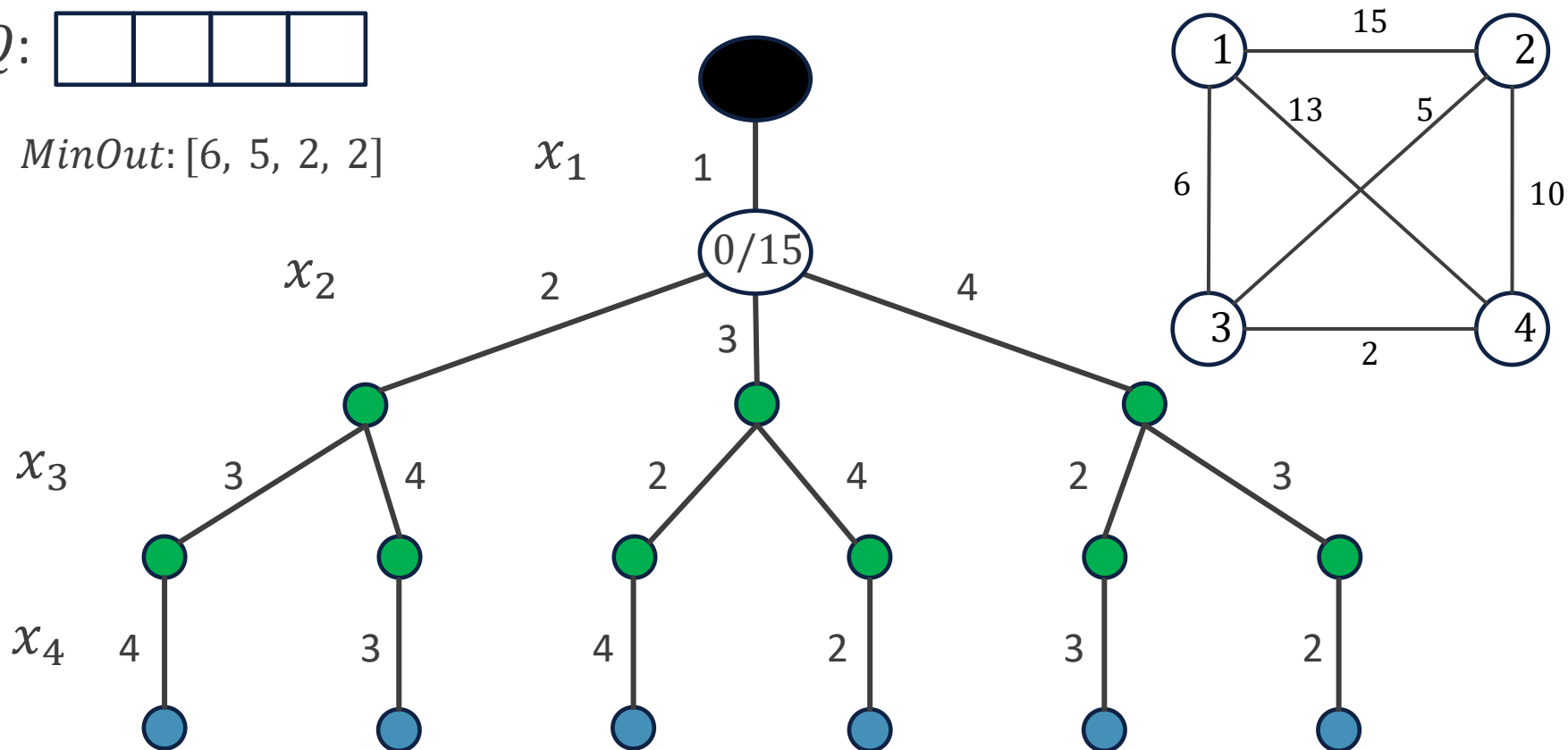
Classroom Exercise

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

--	--	--	--

MinOut: [6, 5, 2, 2]





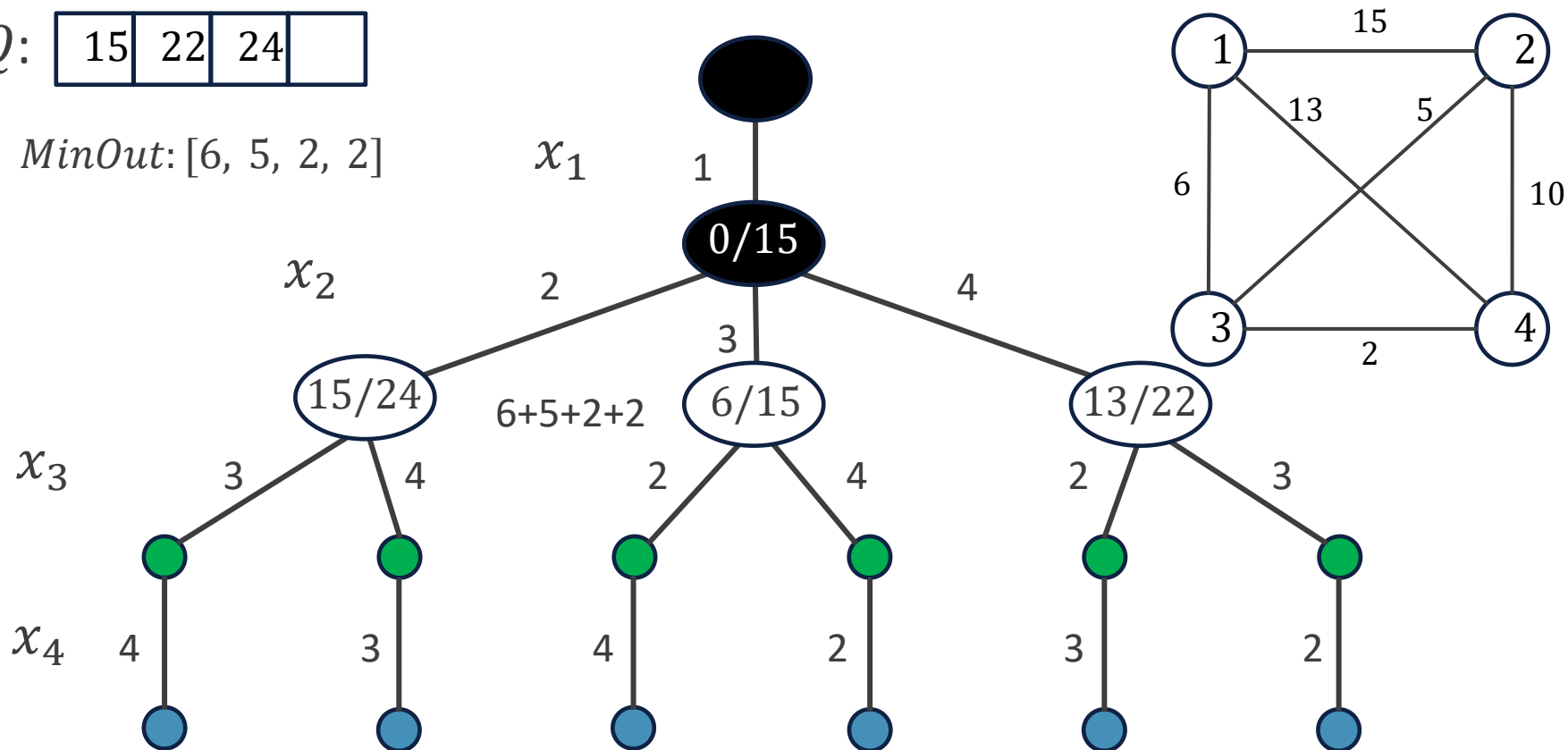
Classroom Exercise

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

15	22	24	
----	----	----	--

MinOut: [6, 5, 2, 2]





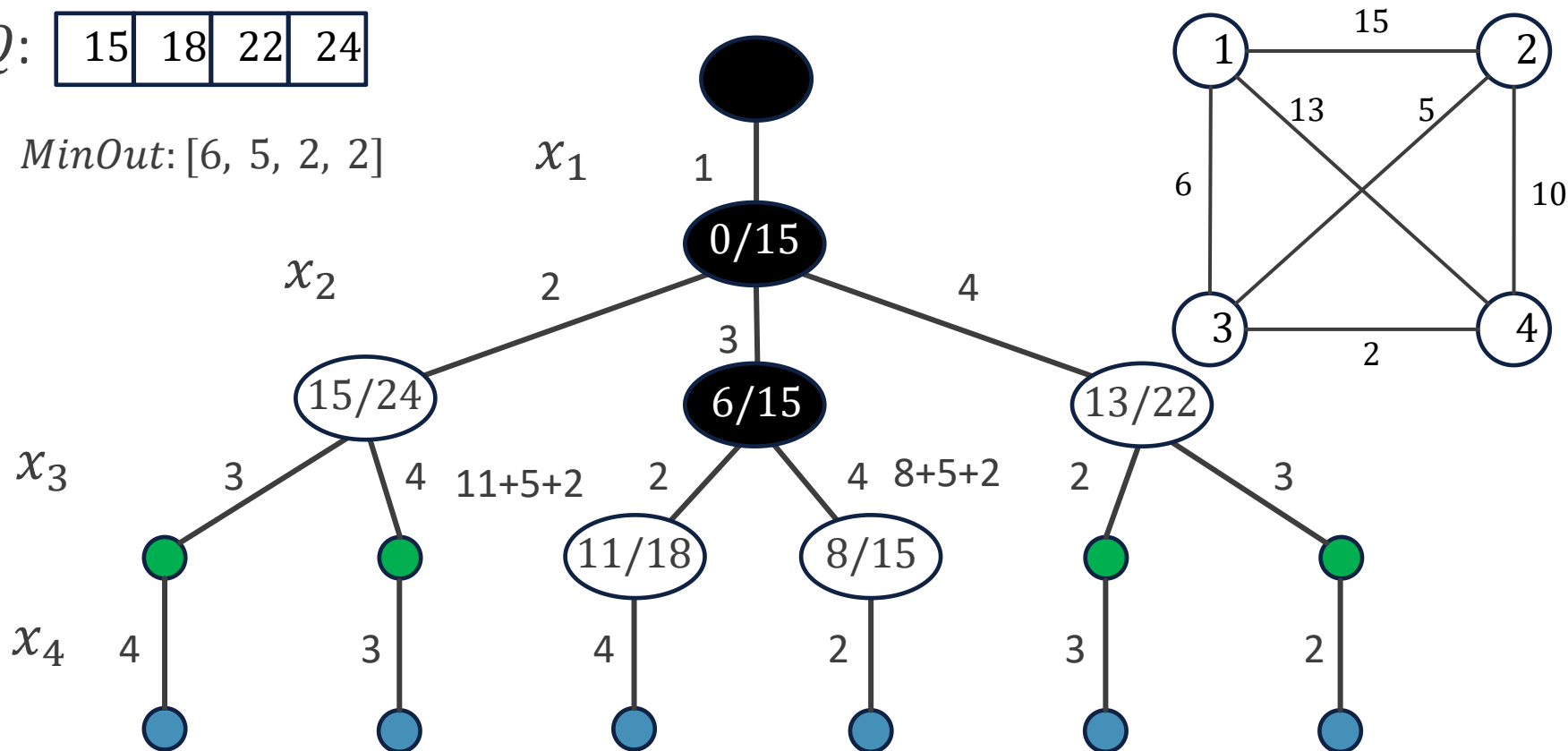
Classroom Exercise

$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

15	18	22	24
----	----	----	----

MinOut: [6, 5, 2, 2]





Classroom Exercise

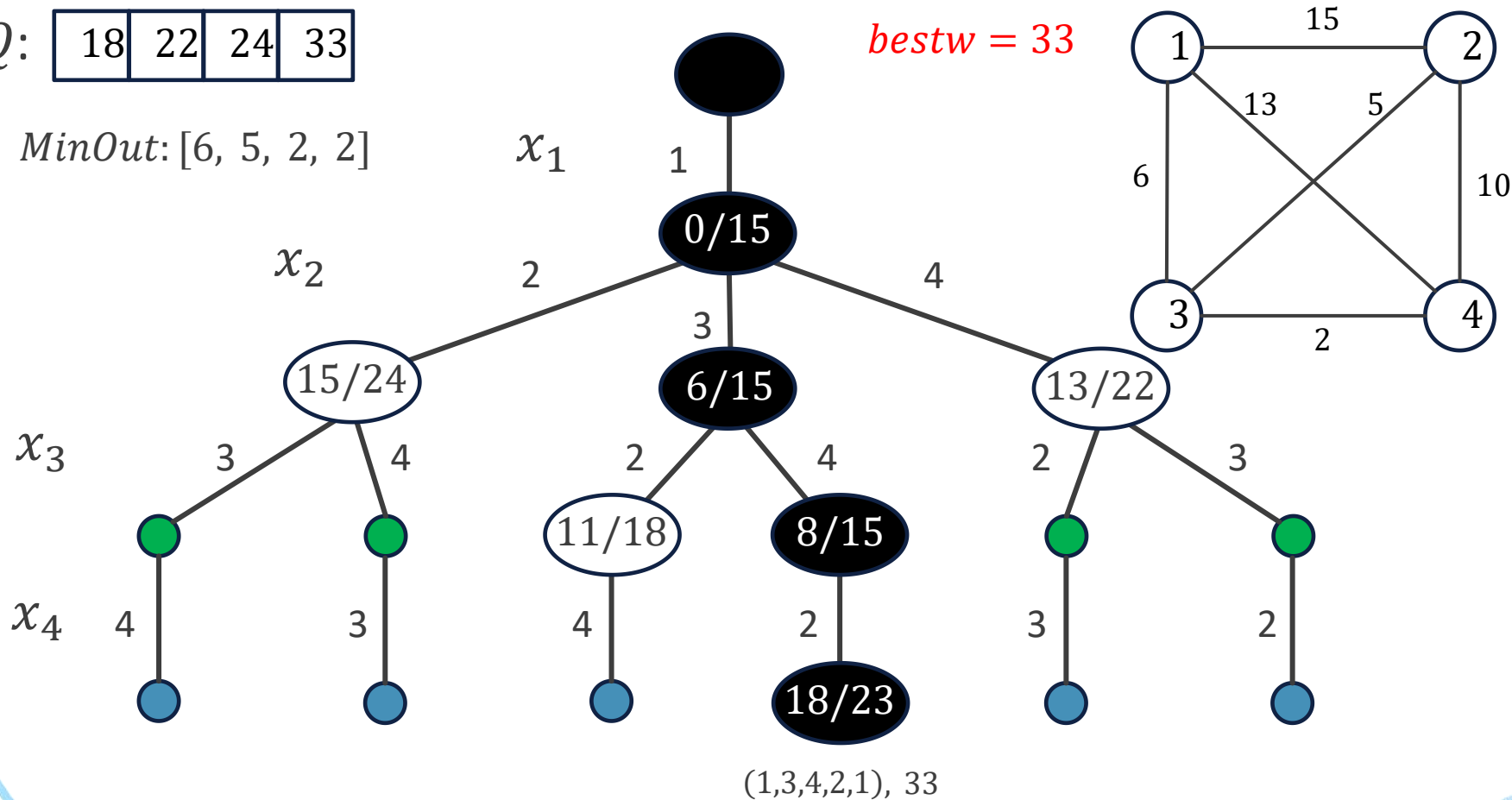
$$B(i) = cw(i) + rw(i)$$

Q:

18	22	24	33
----	----	----	----

MinOut: [6, 5, 2, 2]

bestw = 33





Classroom Exercise

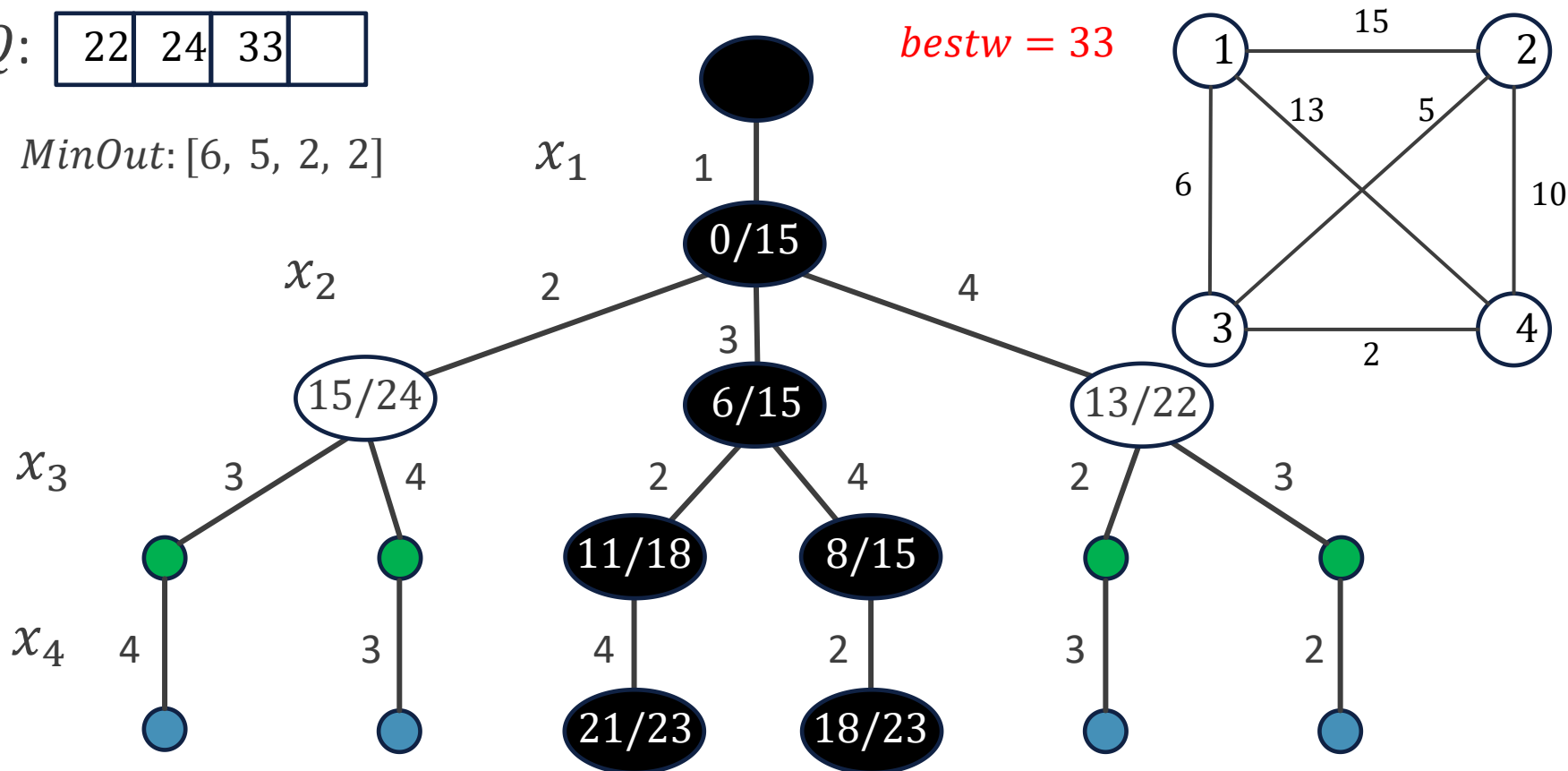
$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

22	24	33	
----	----	----	--

MinOut: [6, 5, 2, 2]

bestw = 33



34 > *bestw* (1,3,2,4,1), 34 (1,3,4,2,1), 33



Classroom Exercise

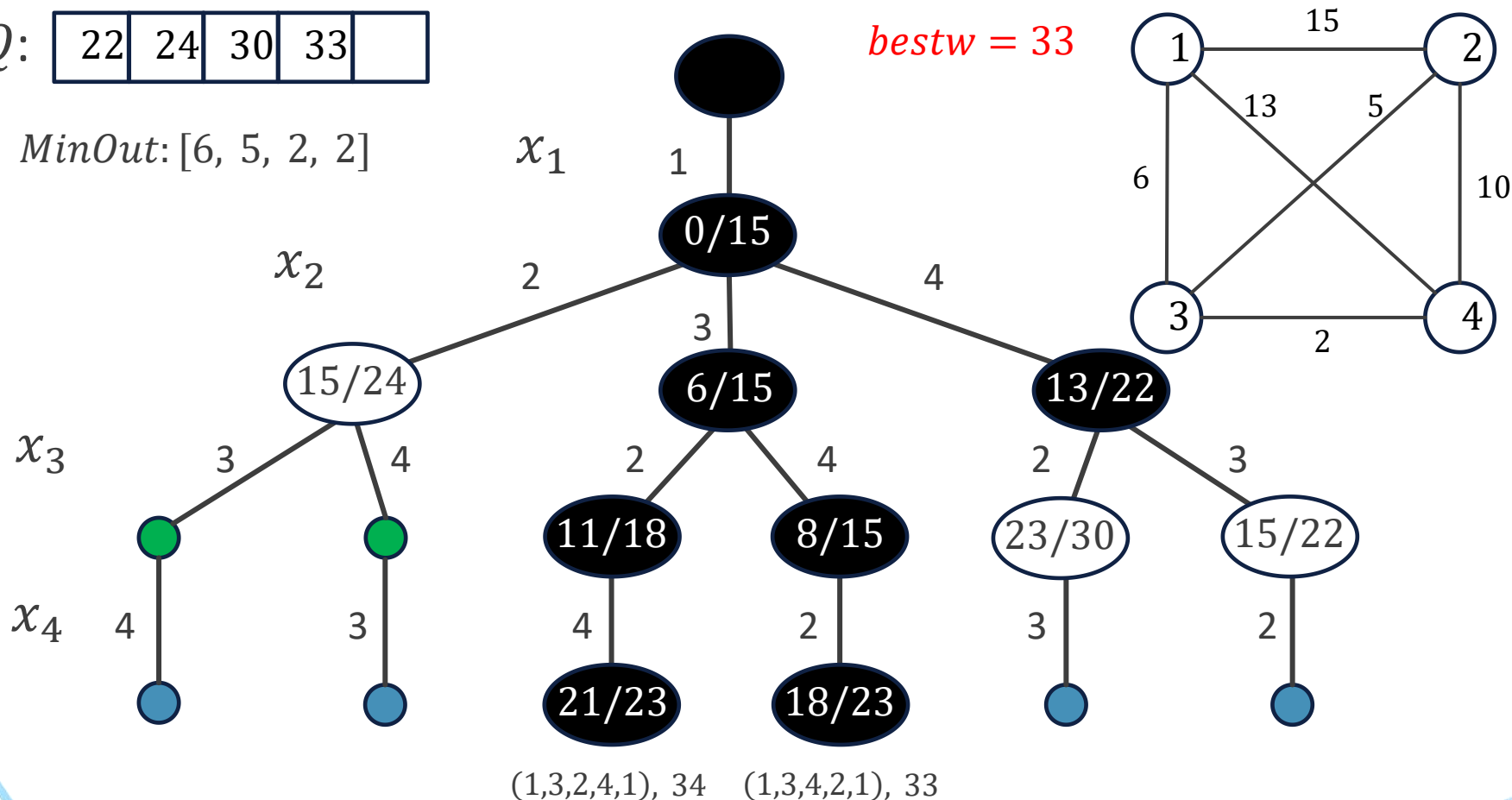
$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

22	24	30	33	
----	----	----	----	--

MinOut: [6, 5, 2, 2]

bestw = 33





Classroom Exercise

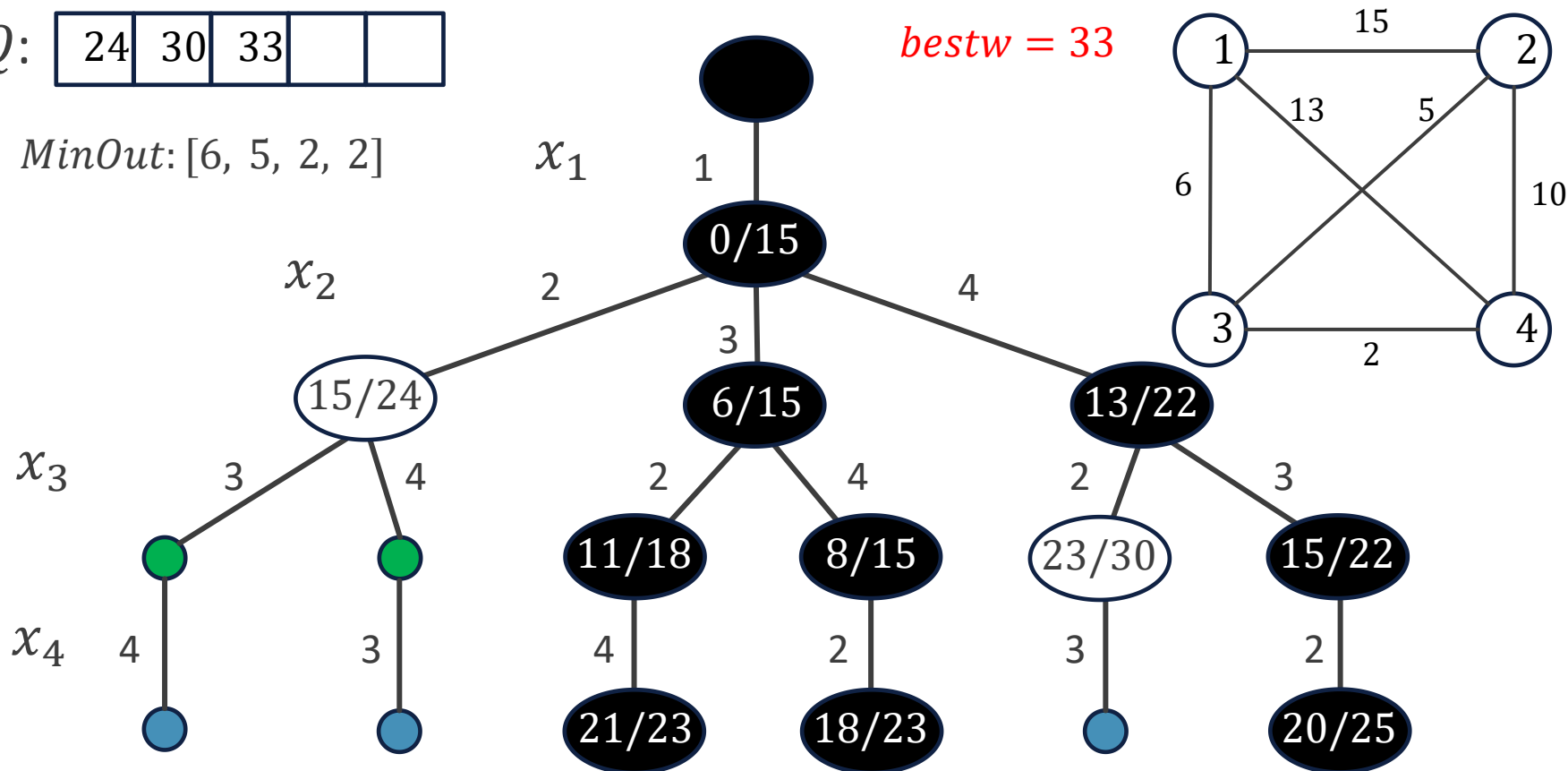
$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

24	30	33		
----	----	----	--	--

MinOut: [6, 5, 2, 2]

bestw = 33



(1,3,2,4,1), 34

(1,3,4,2,1), 33

(1,4,3,2,1), 35

35 > *bestw*



Classroom Exercise

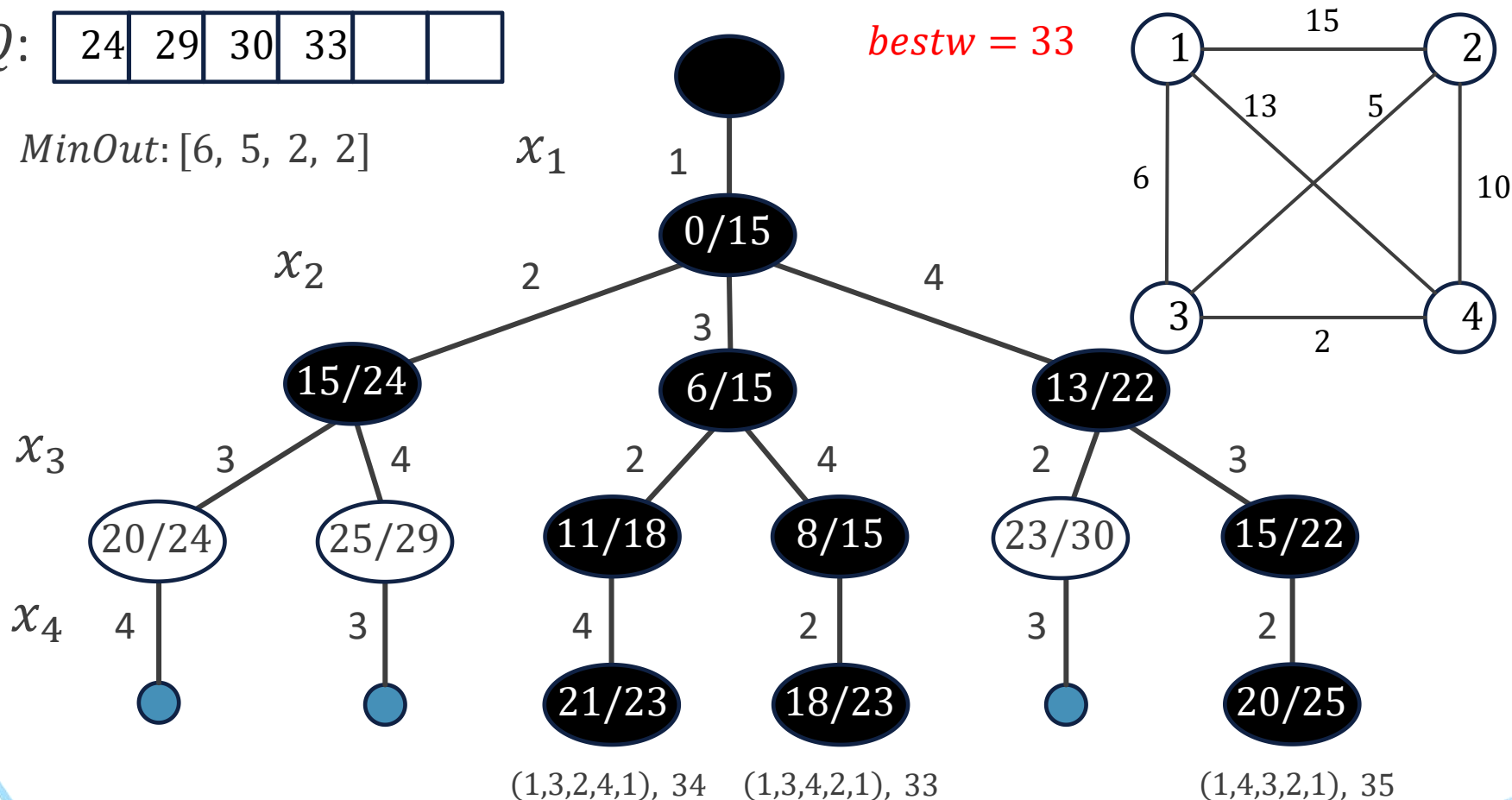
$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

24	29	30	33		
----	----	----	----	--	--

MinOut: [6, 5, 2, 2]

bestw = 33





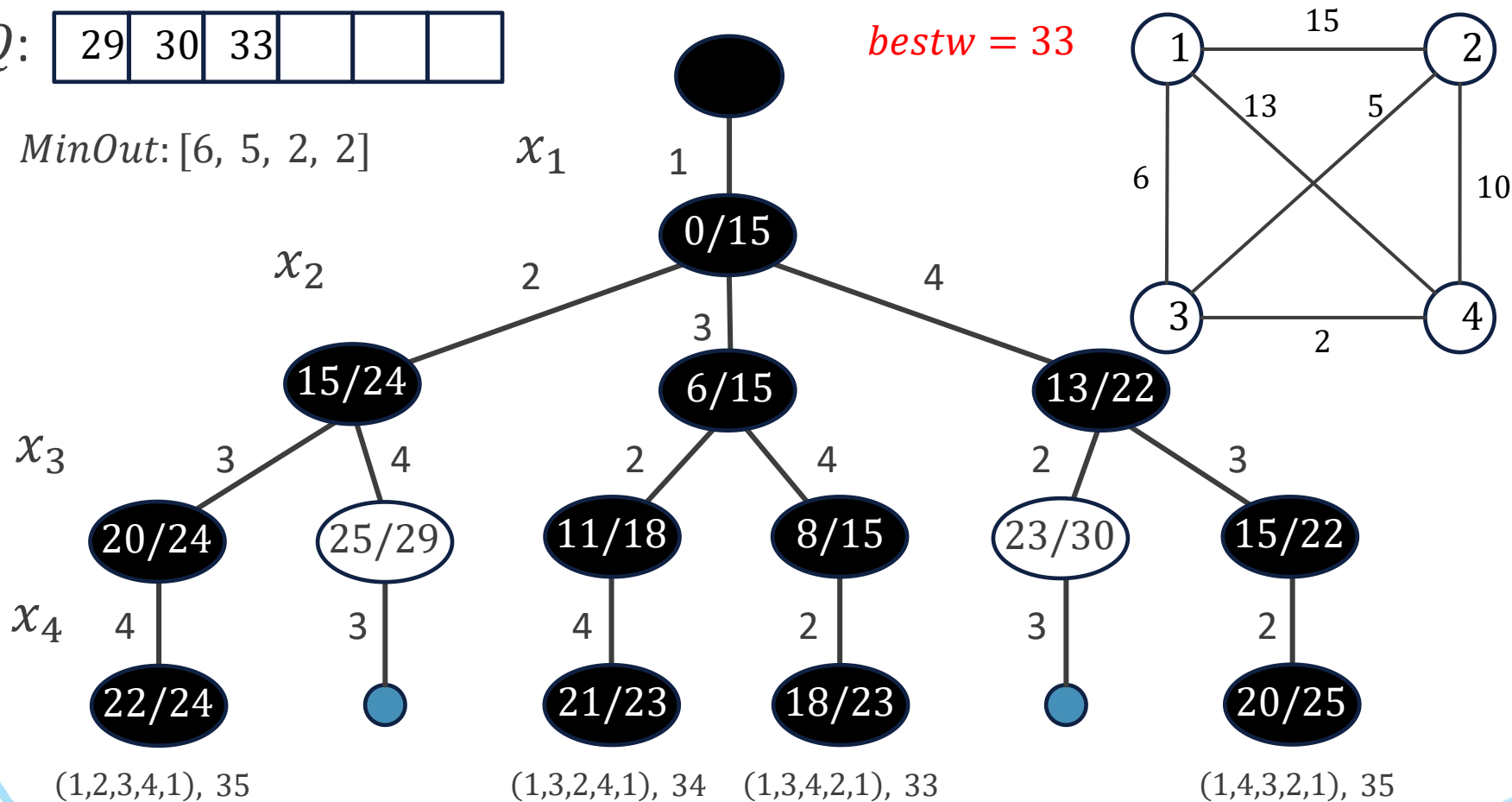
Classroom Exercise

Q:

29	30	33			
----	----	----	--	--	--

MinOut: [6, 5, 2, 2]

bestw = 33





Classroom Exercise

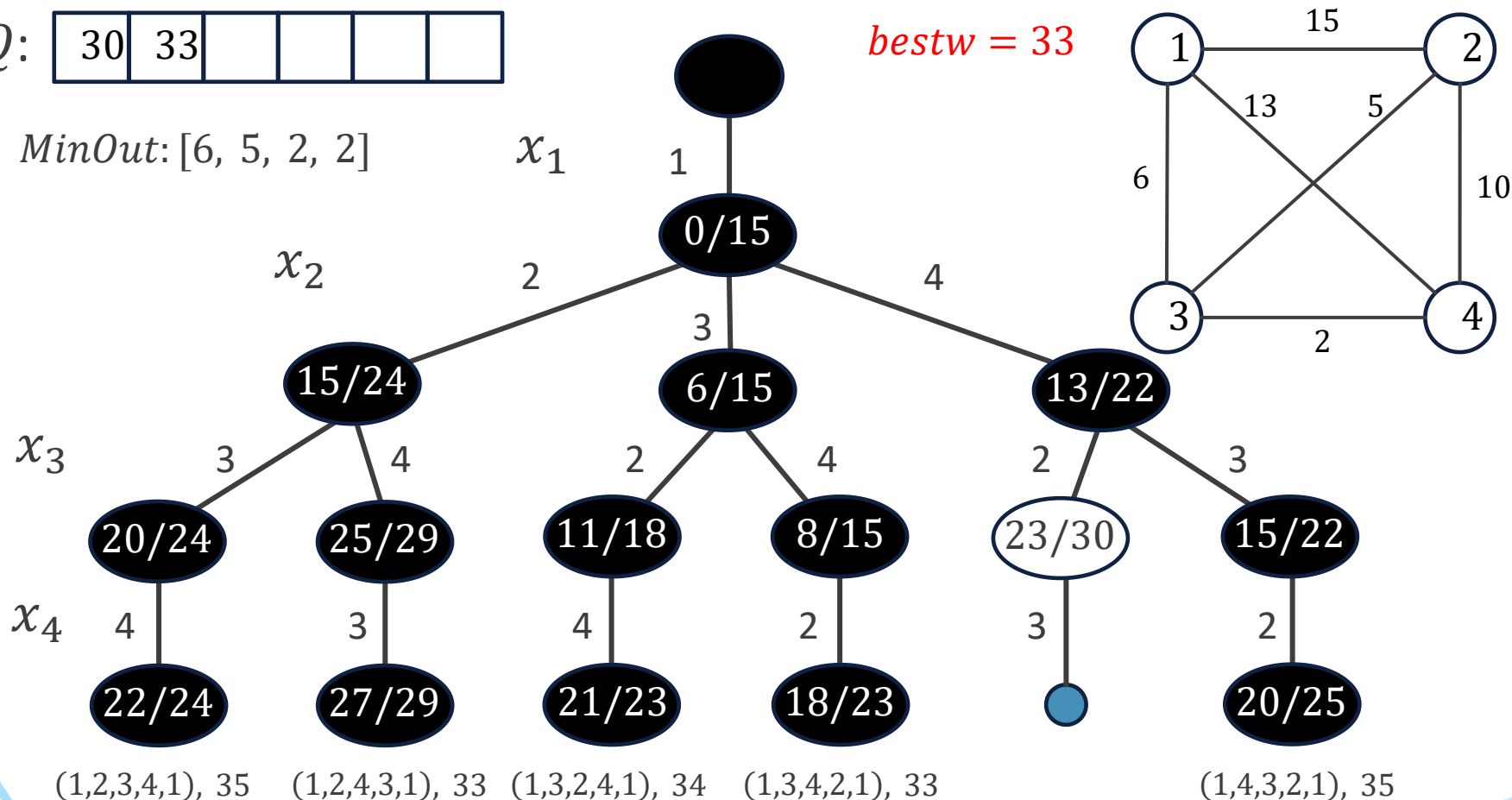
$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

30	33				
----	----	--	--	--	--

MinOut: [6, 5, 2, 2]

bestw = 33



33 = *bestw*



Classroom Exercise

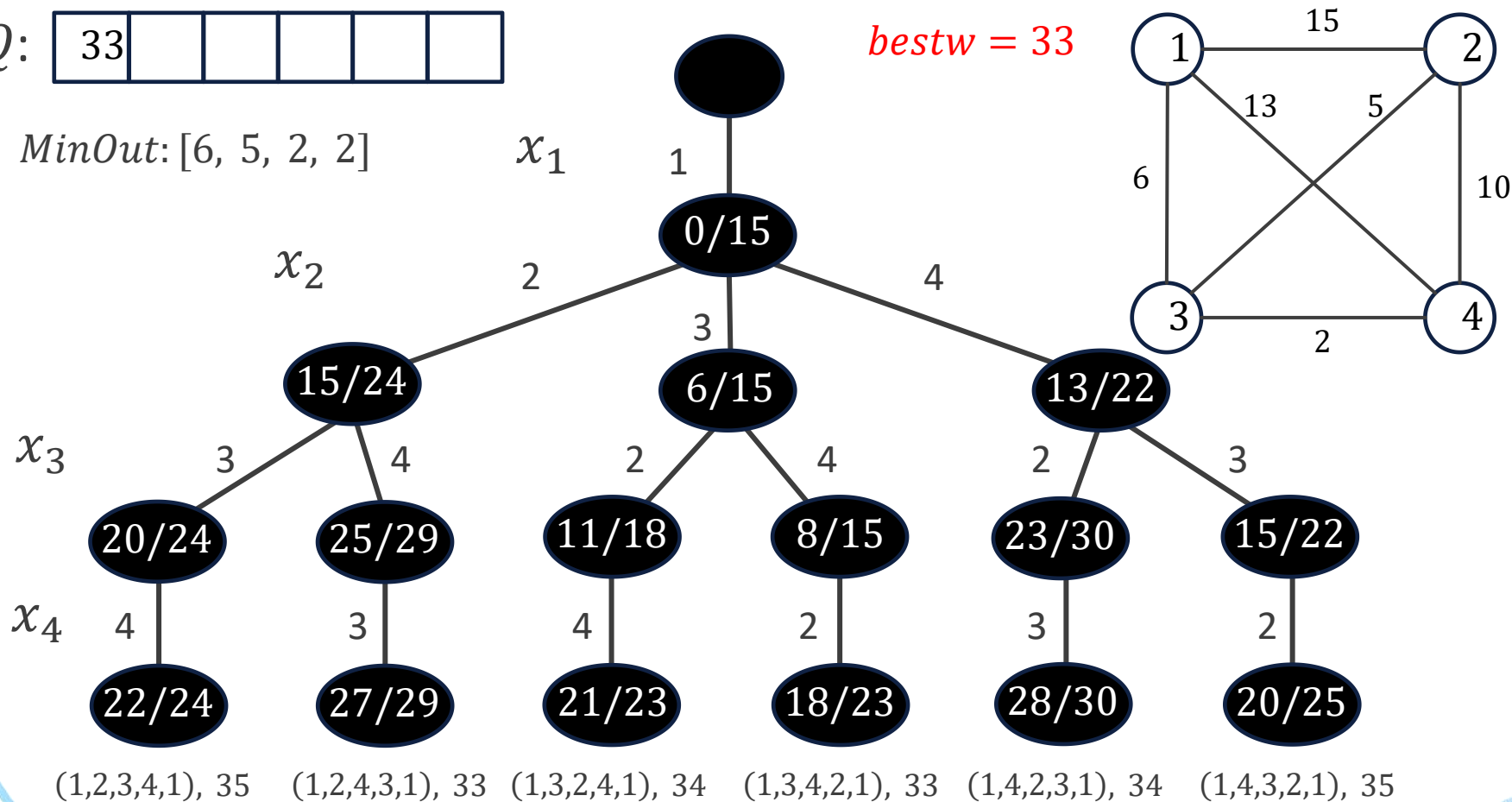
$$B(i) = cw(i) + rw(i) \quad \blacksquare \quad rw(i) = \sum_{j=i}^n \min_{i < k < n, k \neq j} \{w[x[j], x[k]]\}$$

Q:

33					
----	--	--	--	--	--

MinOut: [6, 5, 2, 2]

bestw = 33





Q :

33	34	34	35	35	
----	----	----	----	----	--

MinOut: [6, 5, 2, 2]





Better Bounding Function

- It seems this bounding function is not so efficient. Can we further improve?
- Now, *MinOut* is static no matter how we move.
- However, once we decided to go from v_1 to v_2 , v_2 should not go back to v_1 and all the other vertices should not go to v_2 .
- Therefore, we can update *MinOut* at each step.



Better Bounding Function

- Initially, $MinOut = \begin{bmatrix} \min \{30, 6, 4\} \\ \min \{30, 5, 10\} \\ \min \{5, 6, 20\} \\ \min \{4, 10, 20\} \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ 5 \\ 4 \end{bmatrix},$

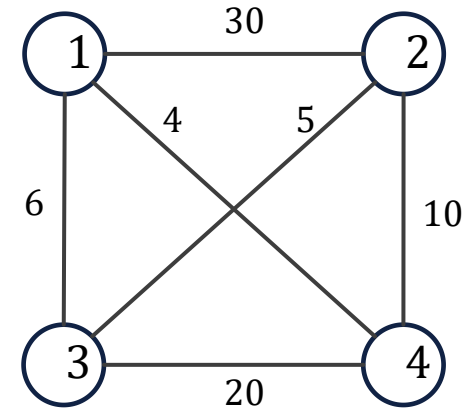
and $B(i) = 0 + 18 = 18.$

- After we choose to go from v_1 to $v_2,$

$$MinOut = \begin{bmatrix} \min \{5, 10\} \\ \min \{6, 20\} \\ \min \{4, 20\} \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \\ 4 \end{bmatrix}$$

and $B(i) = 30 + 15 = 45.$

- $B(i)$ is increased!



no path to v_1

no path to v_2



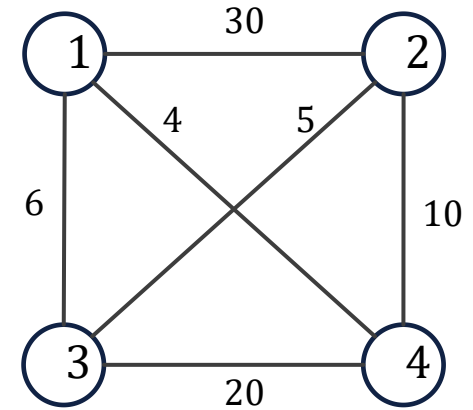
Better Bounding Function

- Then, if we choose to go from v_2 to v_3 ,

$$MinOut = \begin{bmatrix} - \\ - \\ \min\{20\} \\ \min\{4\} \end{bmatrix} = \begin{bmatrix} - \\ - \\ 20 \\ 4 \end{bmatrix}$$

and $B(i) = 35 + 24 = 59$.

- $B(i)$ is increased!



no path to v_1 and v_2

no path to v_2 and v_3

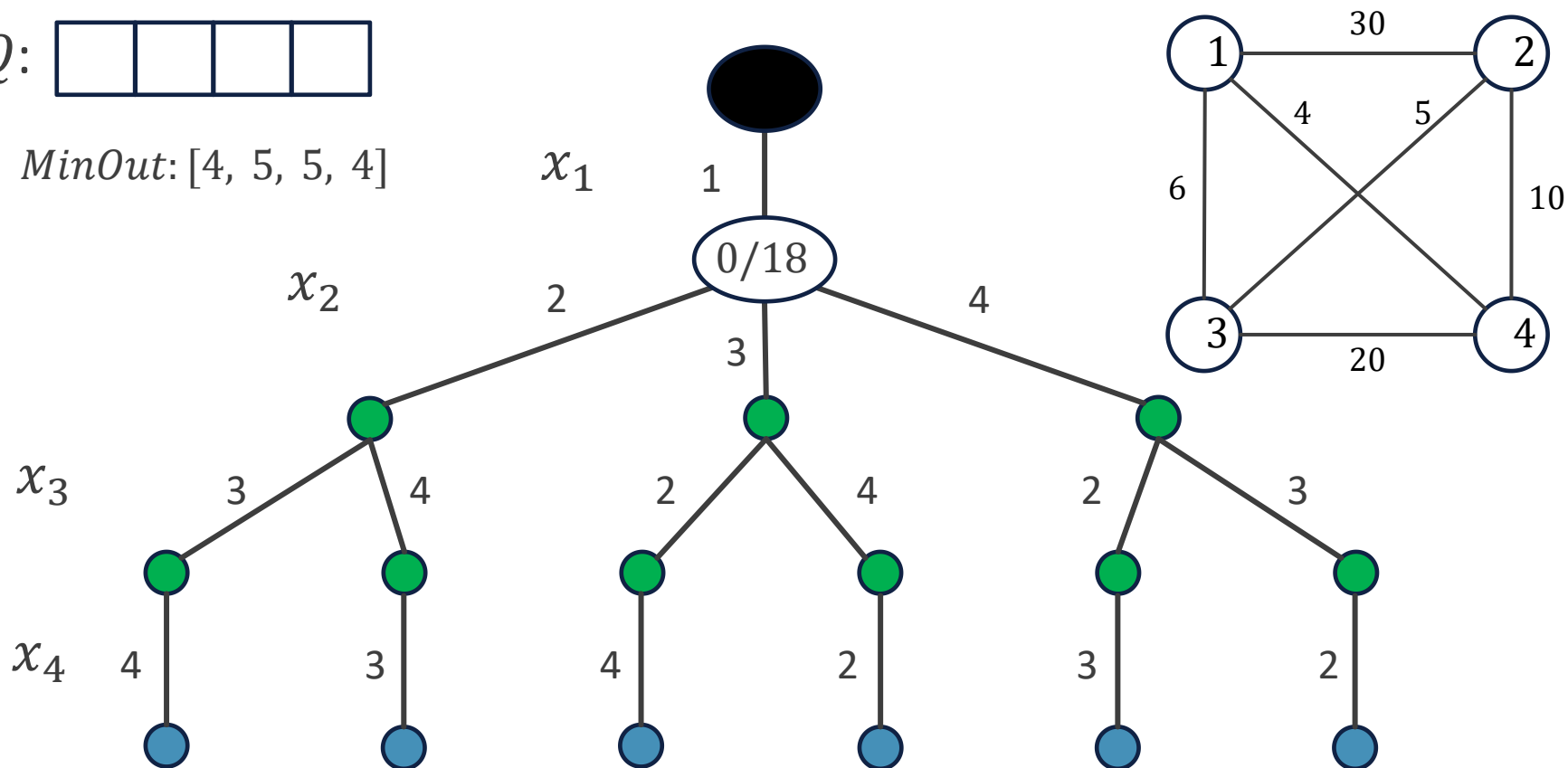


Example

Q :

--	--	--	--

$MinOut$: [4, 5, 5, 4]

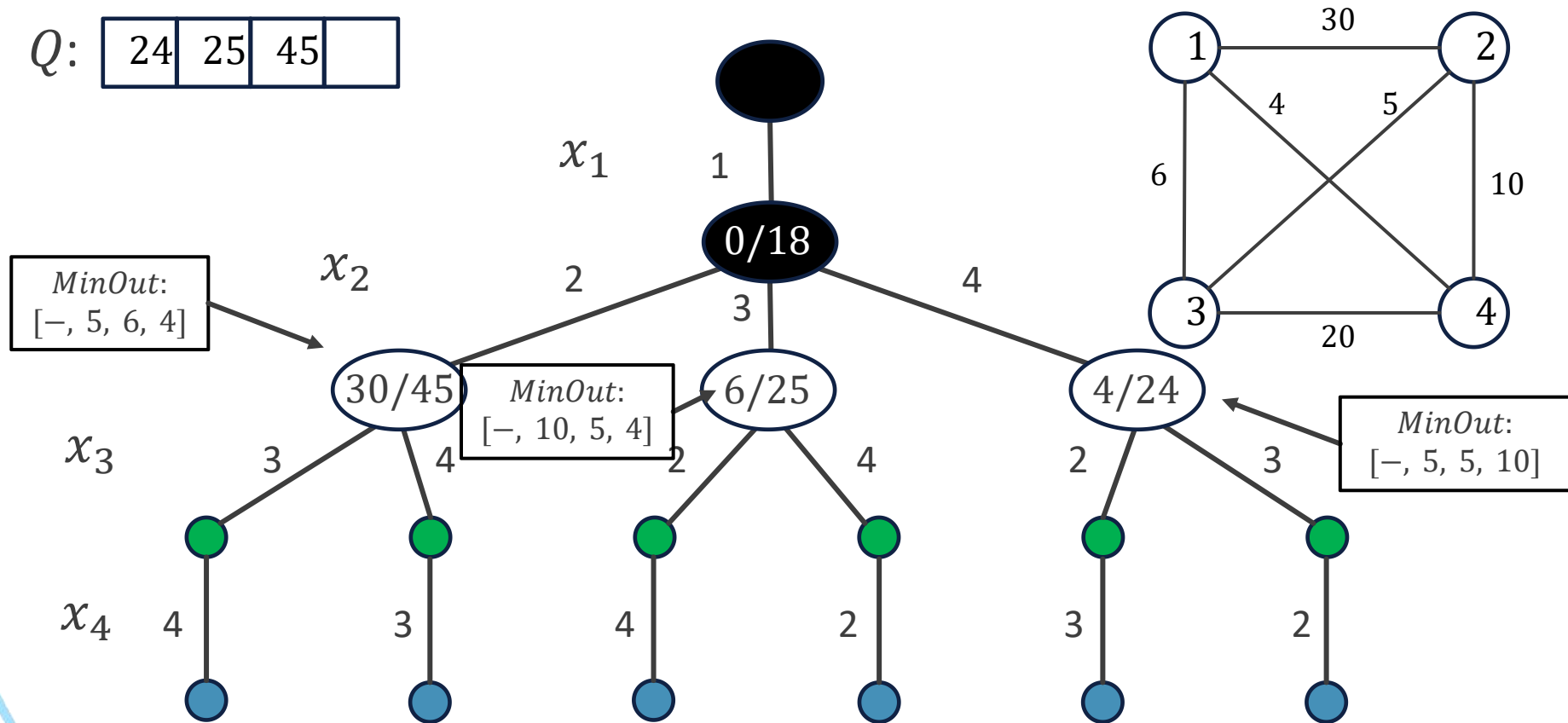




Example

Q:

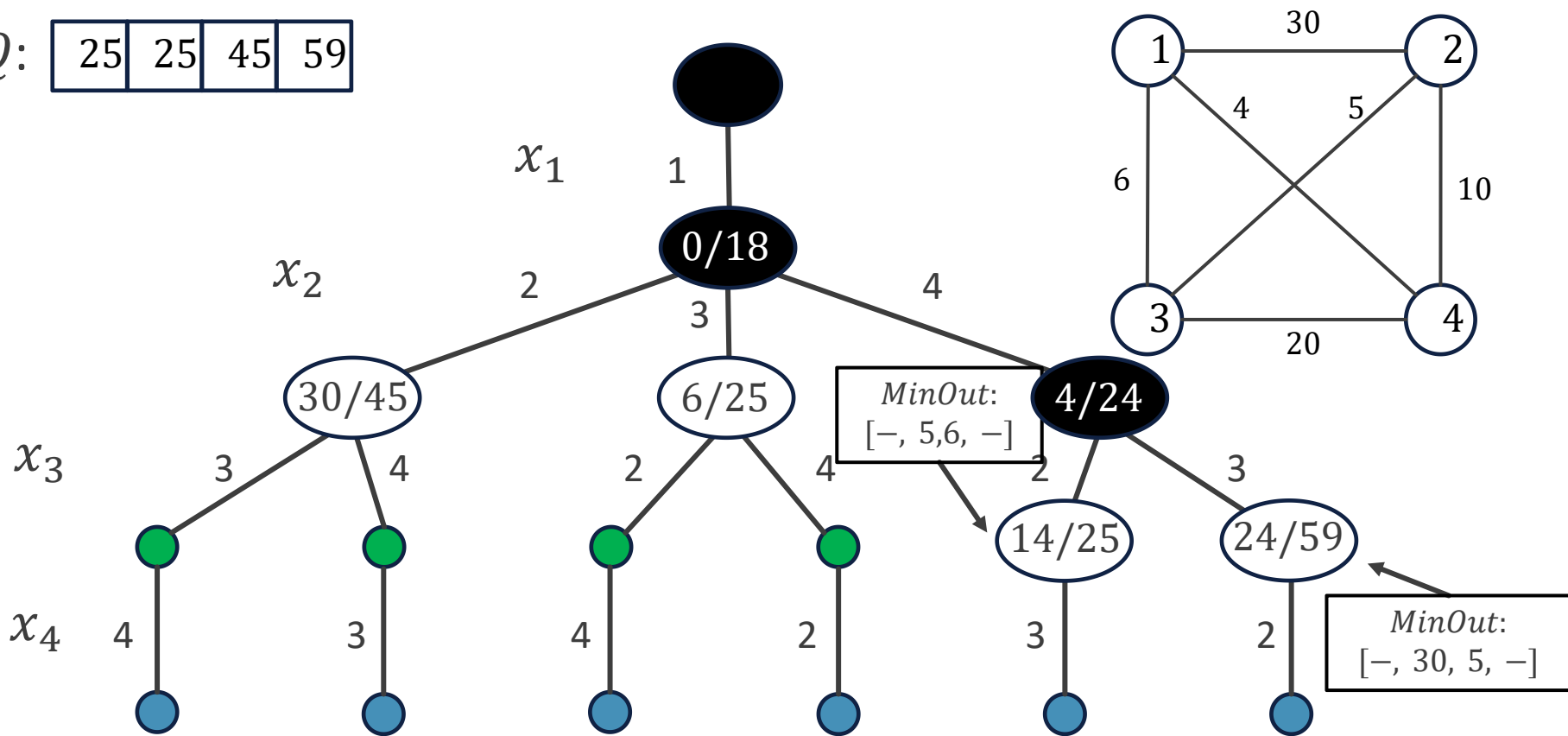
24	25	45	
----	----	----	--





Q :

25	25	45	59
----	----	----	----

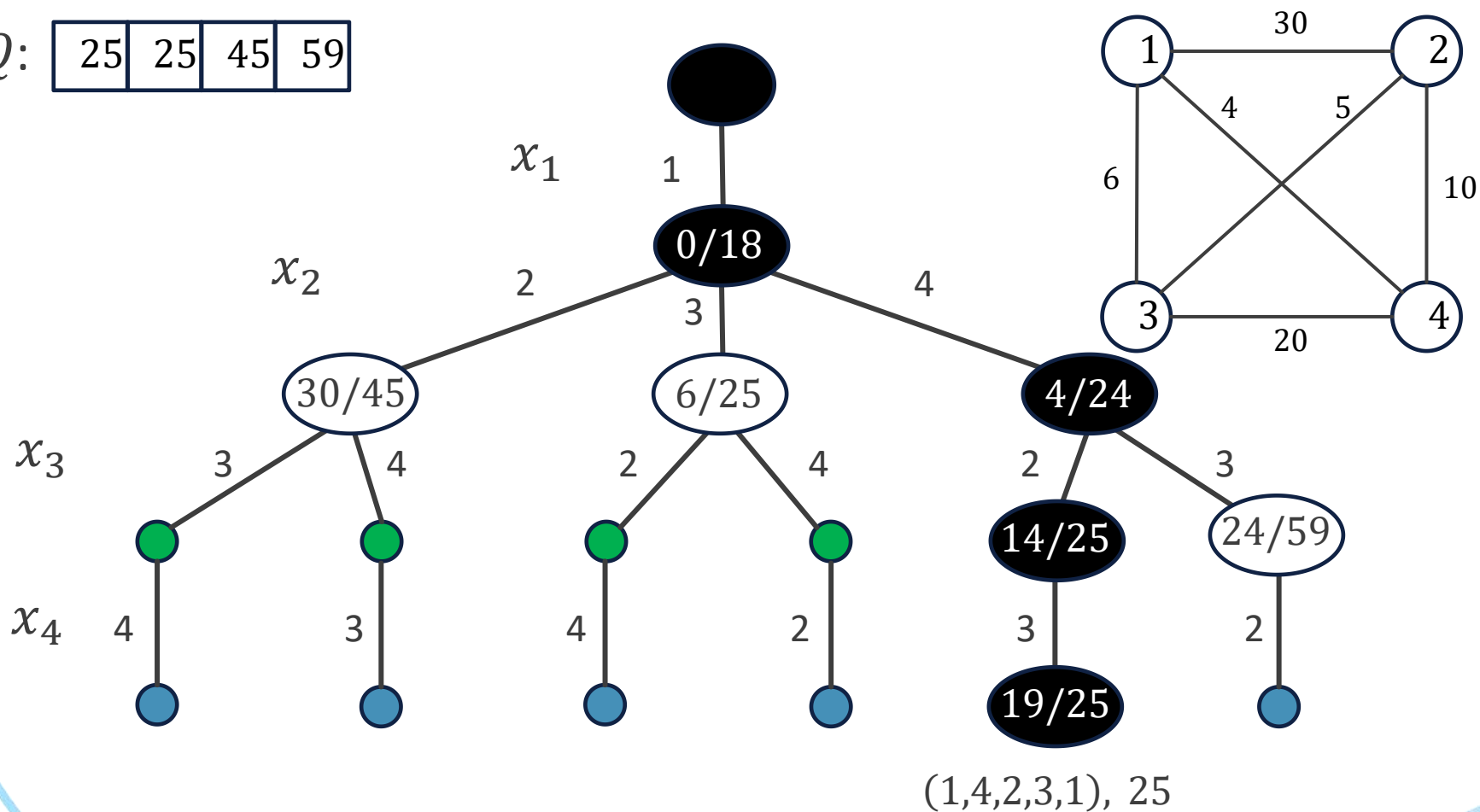




Example

Q :

25	25	45	59
----	----	----	----

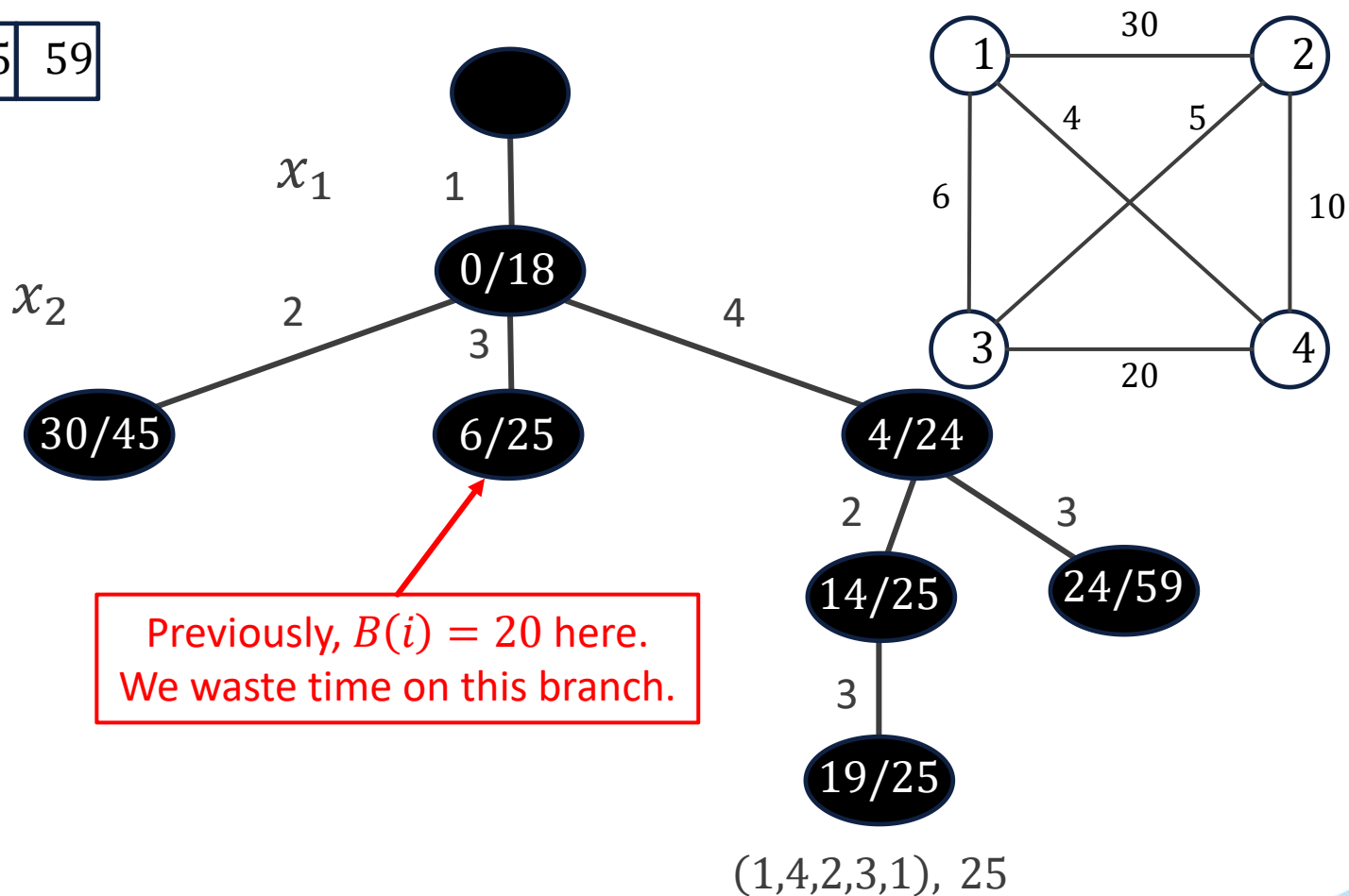




Example

Q :

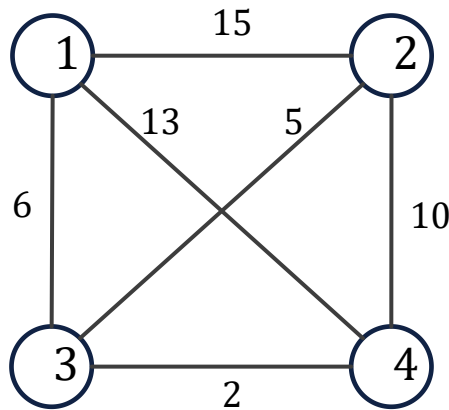
25	25	45	59
----	----	----	----





Classroom Exercise

- Draw the pruned solution space tree for the following TSP instance by improved bounding function.

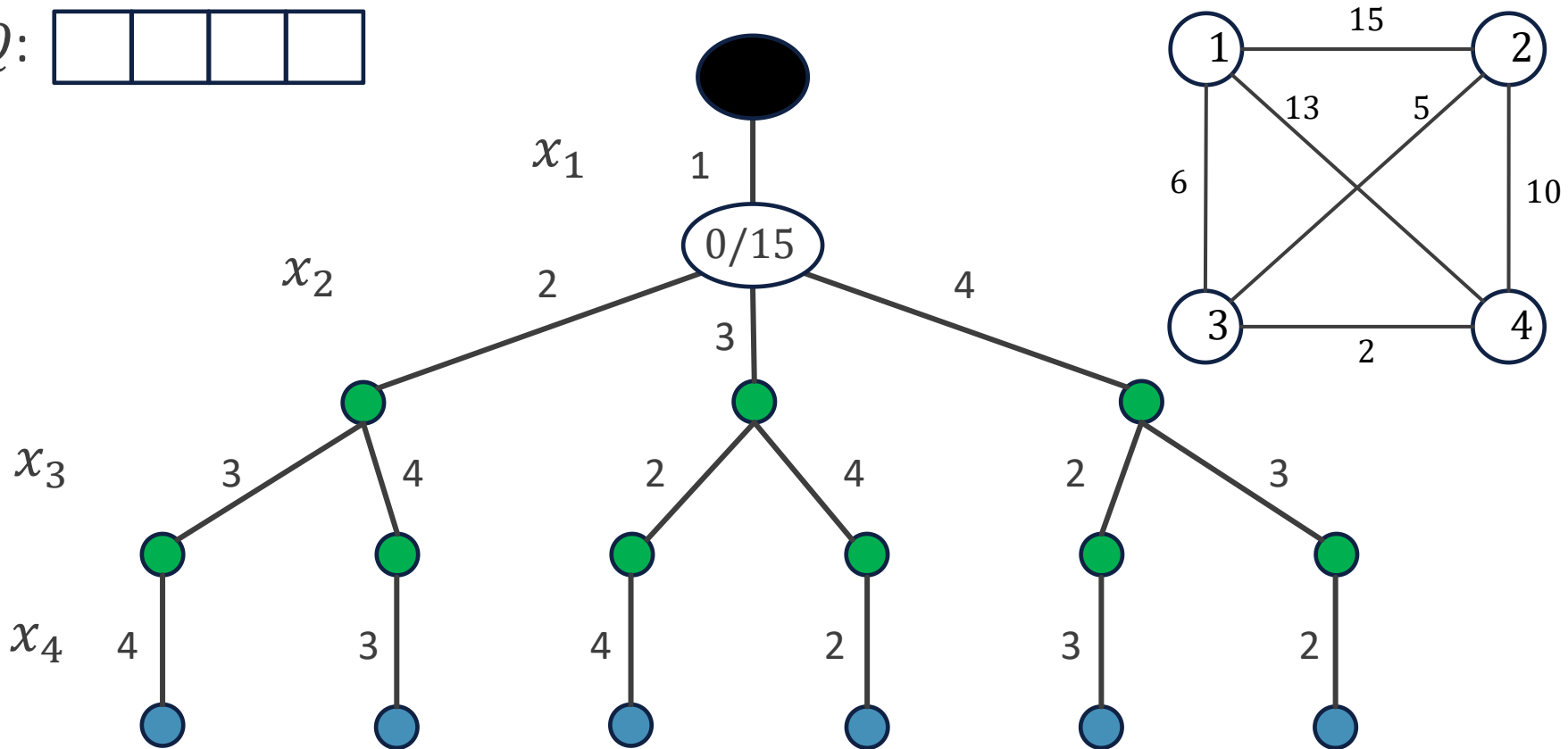




Classroom Exercise

Q:

--	--	--	--

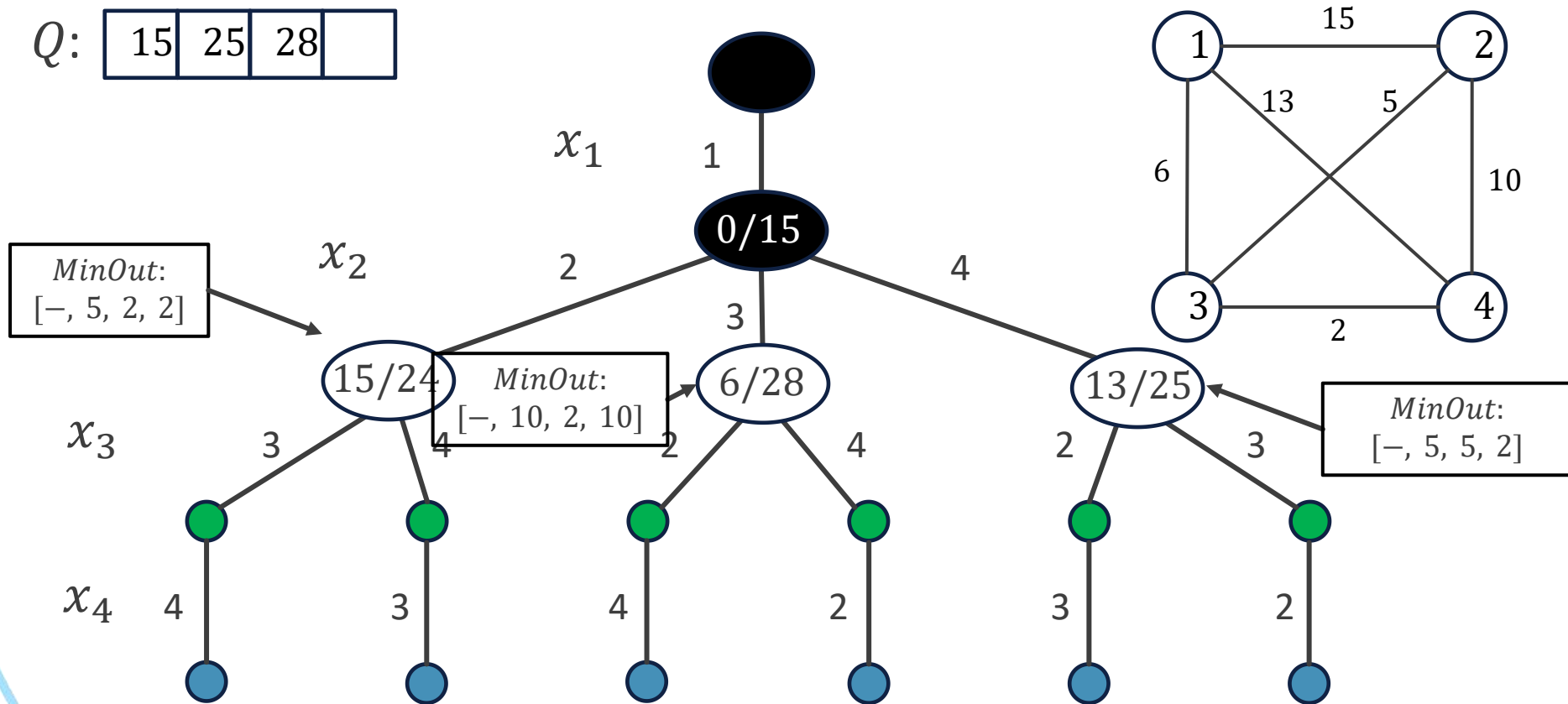




Classroom Exercise

Q:

15	25	28	
----	----	----	--

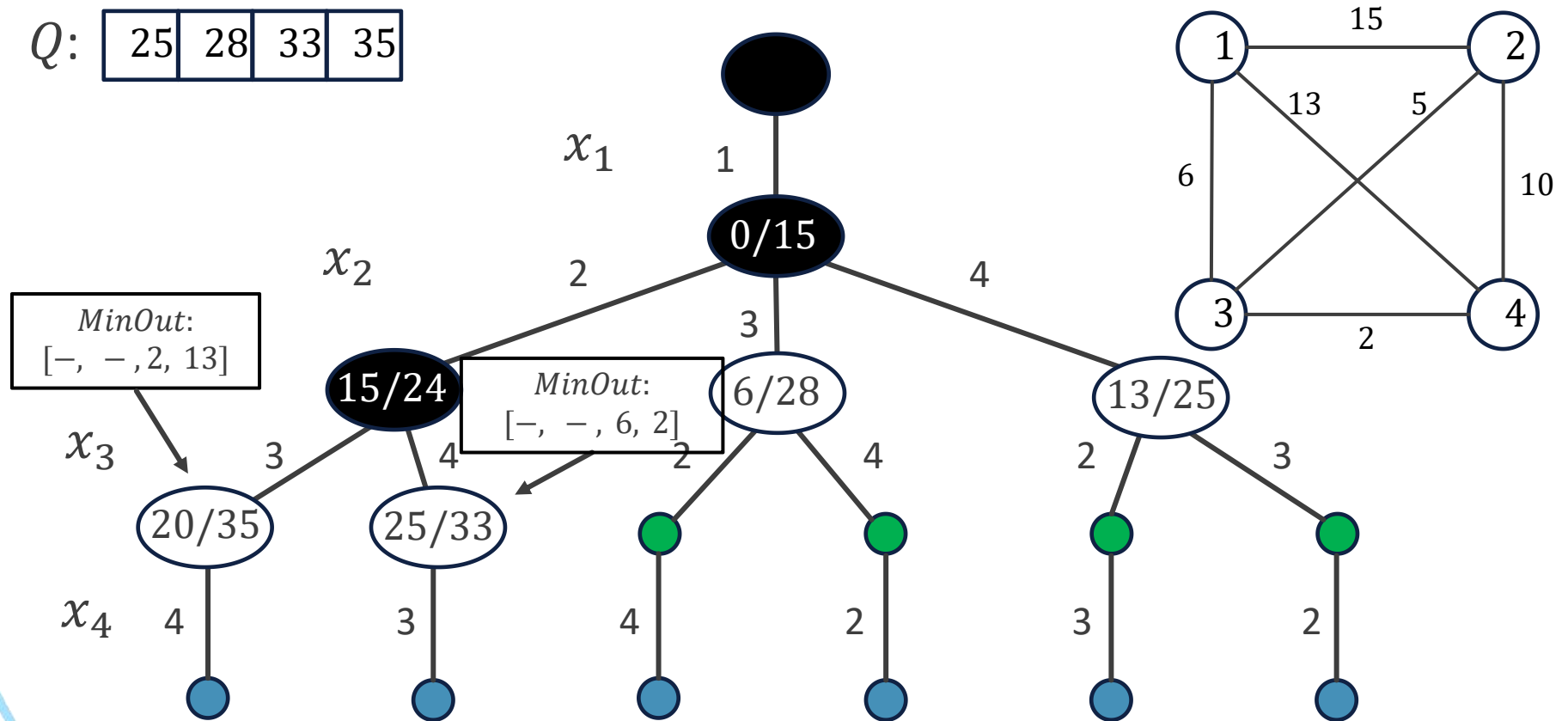




Classroom Exercise

Q:

25	28	33	35
----	----	----	----

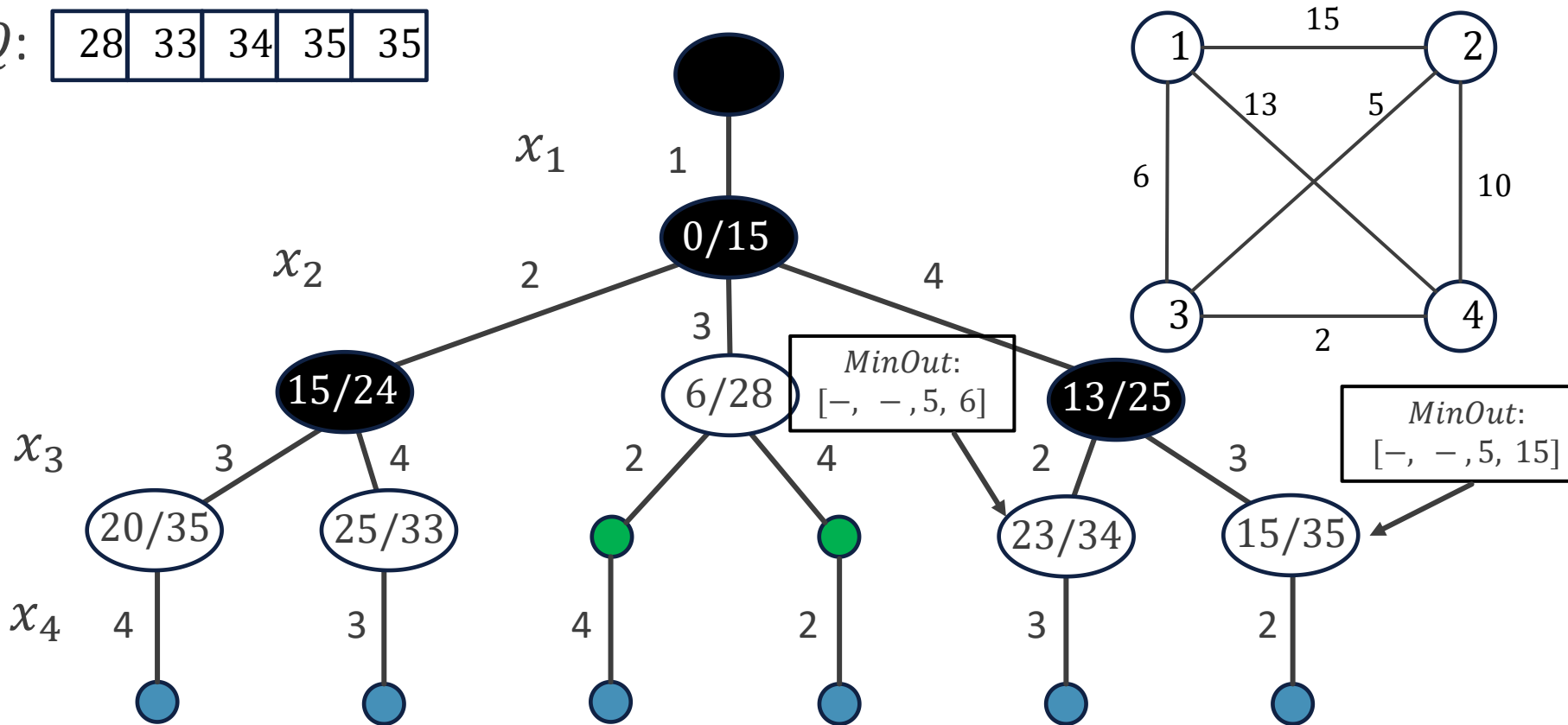




Classroom Exercise

Q:

28	33	34	35	35
----	----	----	----	----

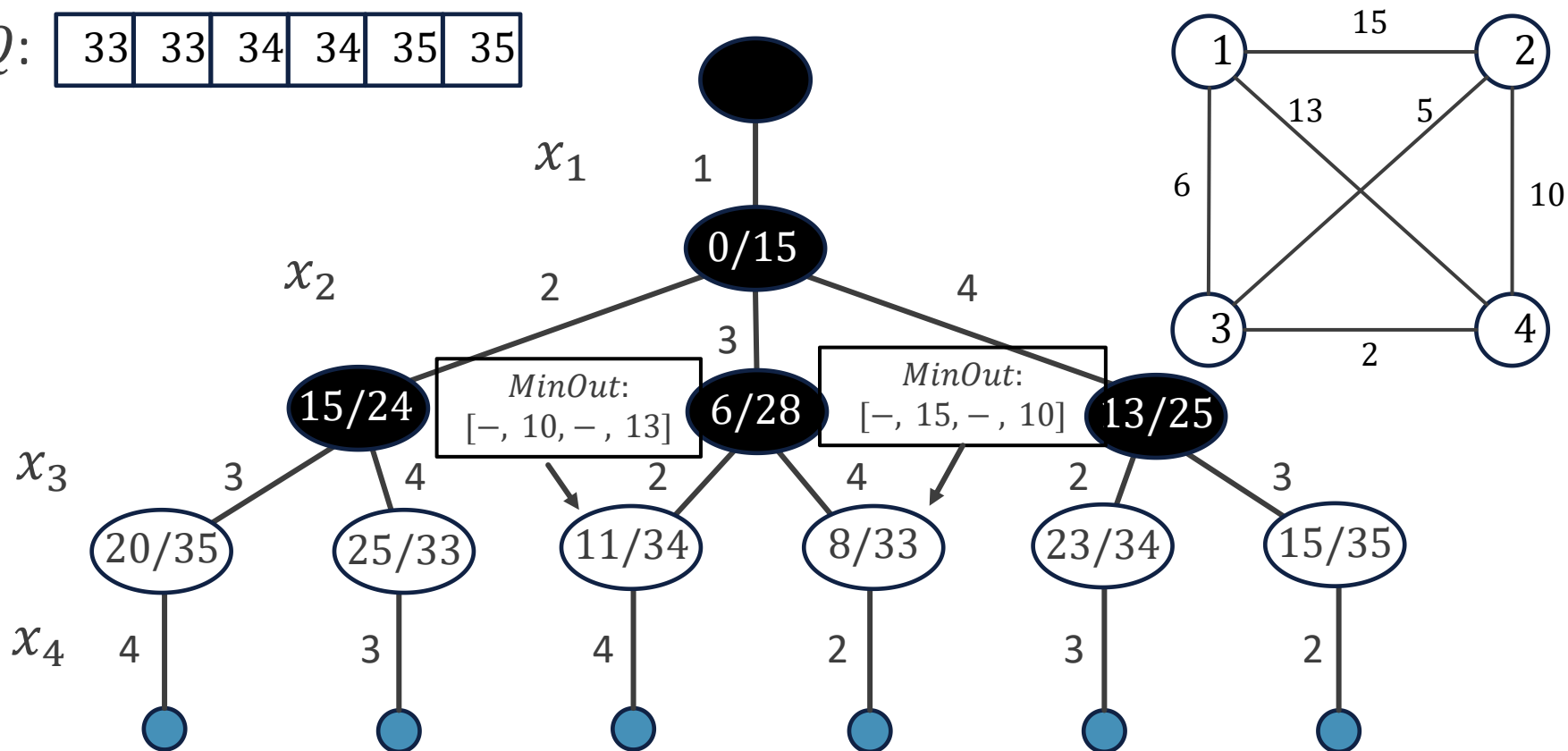




Classroom Exercise

Q:

33	33	34	34	35	35
----	----	----	----	----	----



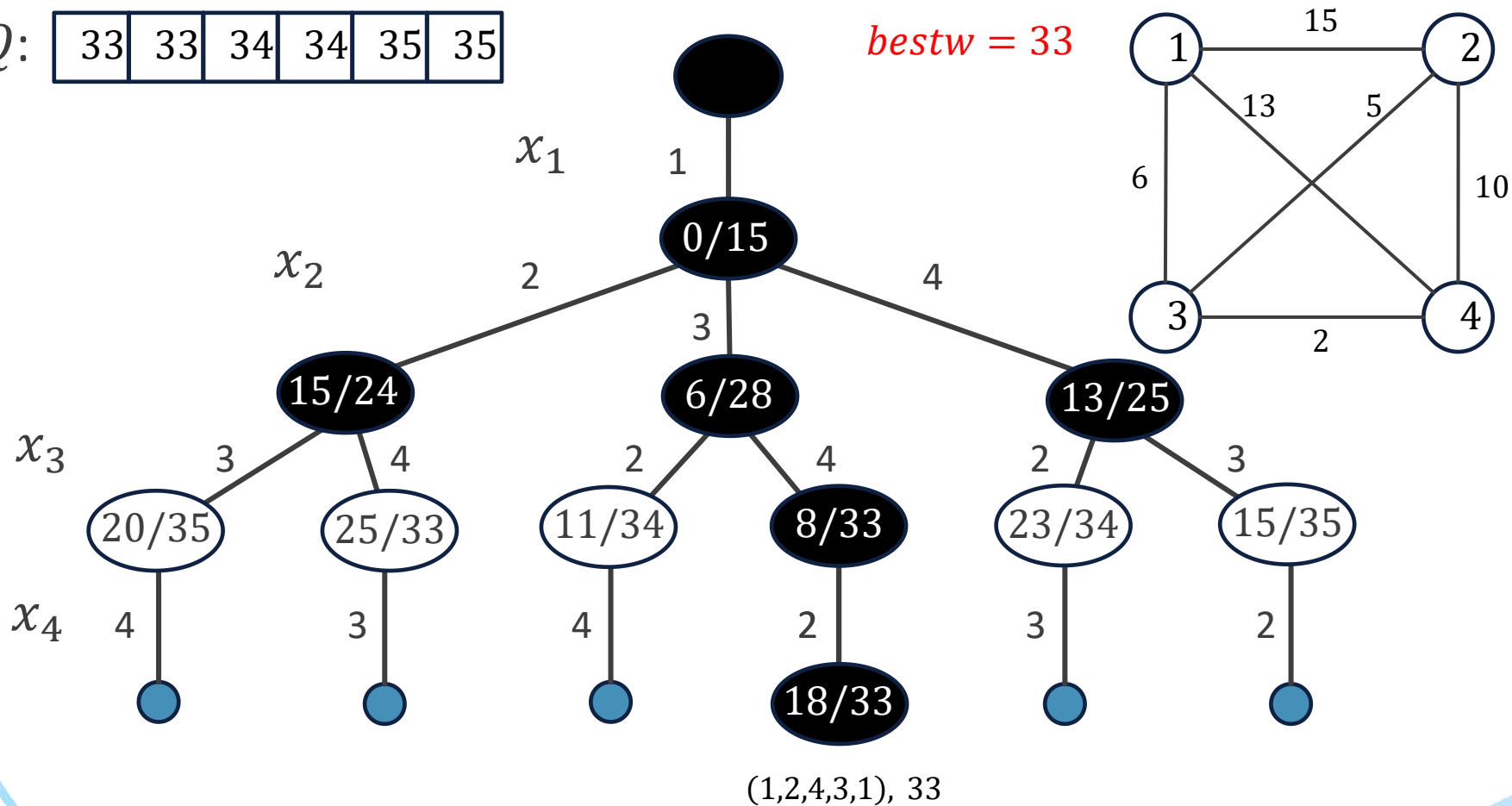


Classroom Exercise

Q:

33	33	34	34	35	35
----	----	----	----	----	----

bestw = 33



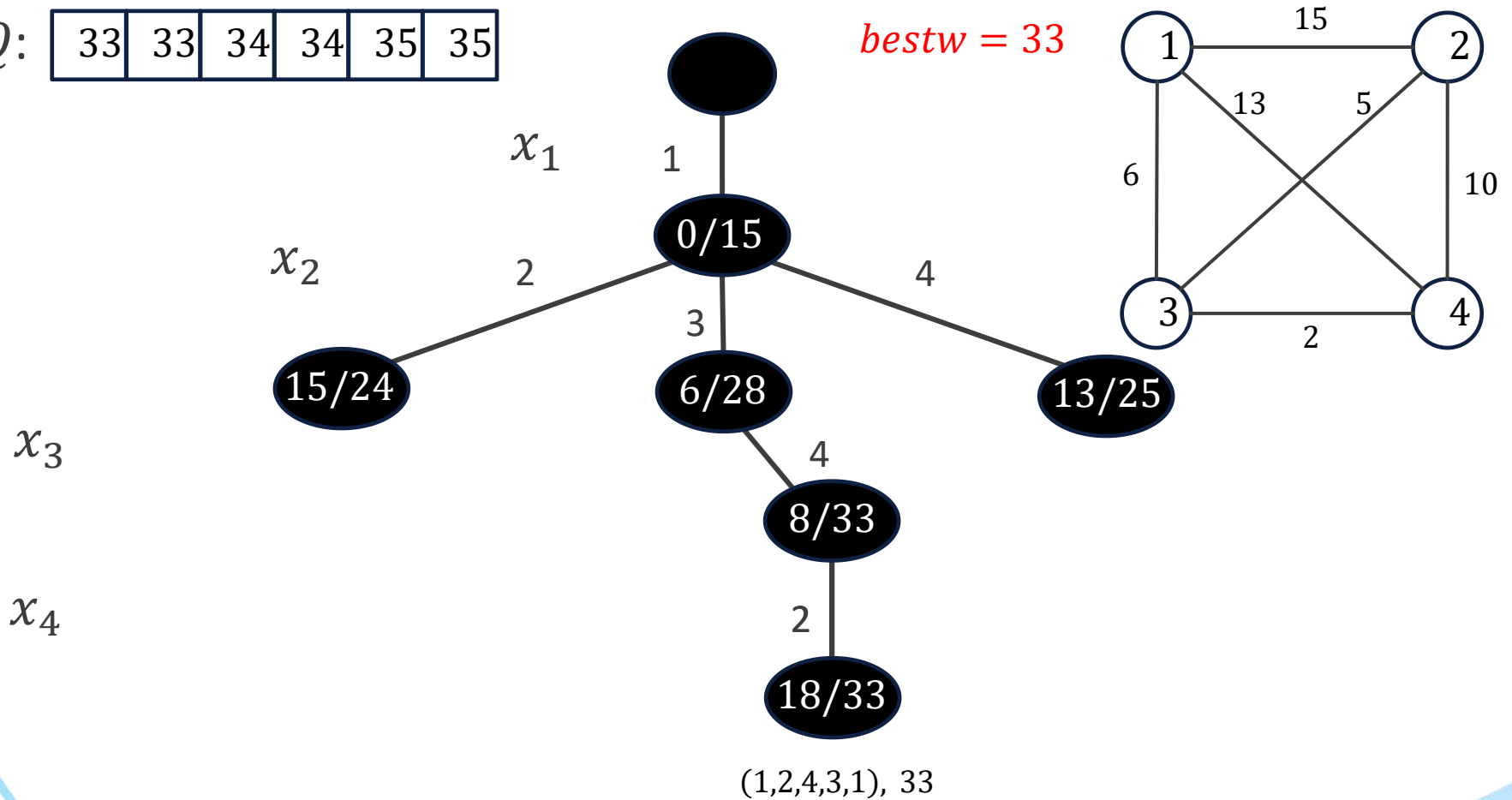


Classroom Exercise

Q:

33	33	34	34	35	35
----	----	----	----	----	----

bestw = 33





FLOW SHOP SCHEDULING PROBLEM

流水作业调度



Flow Shop Scheduling Problem

- Given n jobs $J = (j_1, j_2, \dots, j_n)$, each job has two operations processed by two machines.
- One machine can only process a single job at a time, and processing must be completed once initiated.
- Furthermore, machine 2 cannot begin processing a job until machine 1 has completed processing of the same job, namely, each job must be processed by machine 1 and machine 2 in turn.



Flow Shop Scheduling Problem

- Each job i requires a processing time of $t[i, j]$ on machine j .
- Given a scheduling solution, $F[i, j]$ denotes the finish time for job i on machine j .
- The task is to find an optimal scheduling that minimizes the total finish time:

$$f = \sum_{i=1}^n F[i, 2]$$

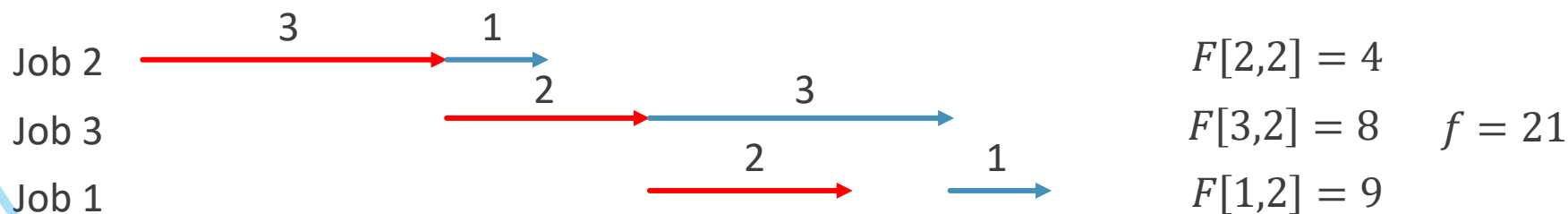
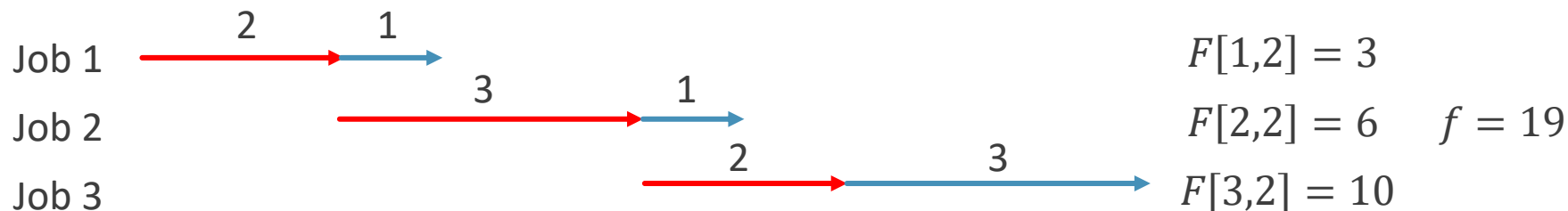


Example

$$f = \sum_{i=1}^n F[i, 2]$$

The goal is not to achieve the earliest finish time, but the earliest total finish time.

	$t[i, j]$	Machine 1	Machine 2
Machine 1 →			
Machine 2 →			
Job 1		2	1
Job 2		3	1
Job 3		2	3





- Unvisited internal nodes
- Unvisited leaf nodes





Flow Shop Scheduling Problem

- What is the constraint function for this problem?

There's no constraint function. Any permutation is a feasible solution.



Flow Shop Scheduling Problem

目标最小化 $f = \sum_{i=1}^n F[i, 2]$

- Let $x = \{x[1], x[2], \dots, x[i]\}$ be the set of jobs that has been processed up to the extend node i , then

$$f = \sum_{j=1}^i F[x[j], 2] + rf(i)$$

where

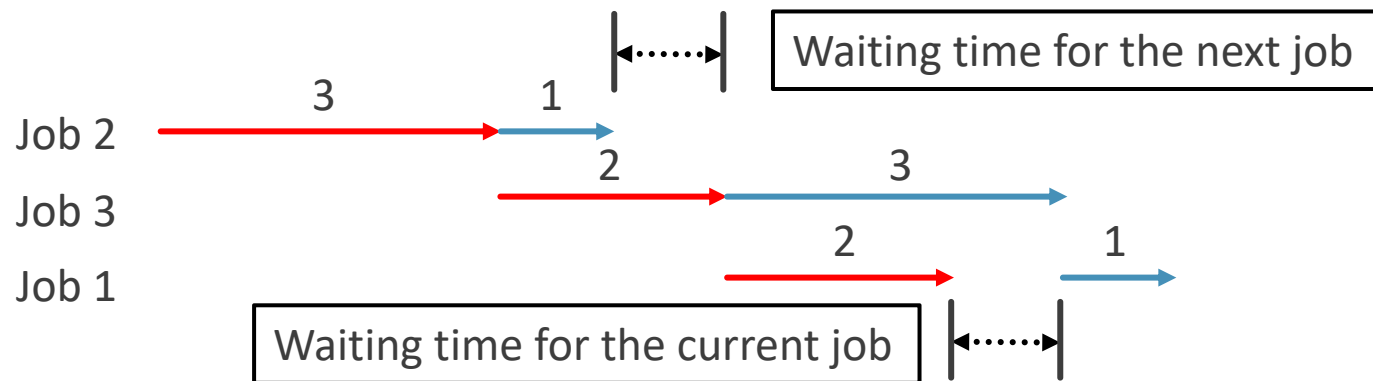
$$rf(i) = \sum_{j=i+1}^n F[x[j], 2]$$

- Computing $rf(i)$ is very difficult, we can estimate its lower bound?



Lower Bound

- Machine 1 is continuously working.
- The finish time is influenced by the waiting time of machine 2.



We can calculate the lower bound by assuming that there's no waiting time.



Lower Bound

$$f = \sum_{i=1}^n F[i, 2]$$

Lower bound 1:

- Assume each remaining job can be continuously processed in the machine 1 and 2 without waiting time. 一个作业在机器1上处理完后可以不需要等待继续在机器2上进行处理

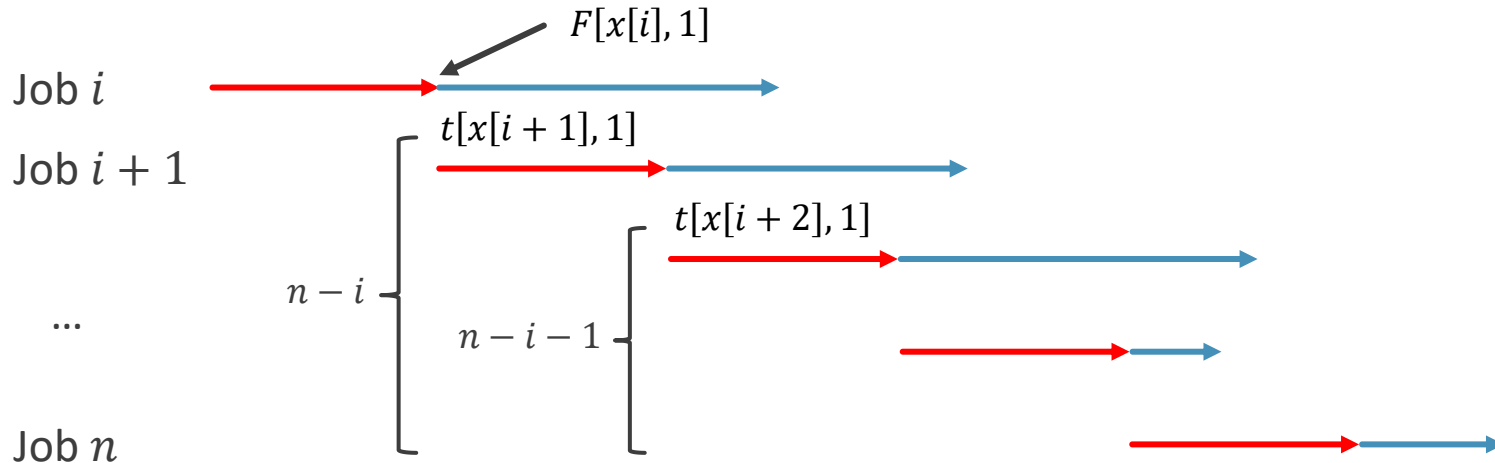
$$rf1(i) = \sum_{j=i+1}^n (F[x[i], 1] + (n - j + 1)t[x[j], 1] + t[x[j], 2])$$

- Obviously, we have $rf(i) \geq rf1(i)$.



Lower Bound

Lower bound 1:
$$rf1(i) = \sum_{j=i+1}^n (F[x[i], 1] + (n - j + 1)t[x[j], 1] + t[x[j], 2])$$



- The order from job $i + 1$ to job n matters.
- Therefore, we can sort $t[x[j], 1]$ in non-decreasing order to obtain smaller $rf1(i)' \leq rf1(i)$.



Lower bound 2:

- Assume that machine 2 has no waiting time

- 机器2在完成某个作业后，它可以不用等待，即可处理下一个作业

$$rf2(i) = \sum_{j=i+1}^n \left(\max \{F[x[i], 2], F[x[i], 1] + \min_{i \leq k \leq n} t[x[k], 1]\} + (n - j + 1)t[x[j], 2] \right)$$

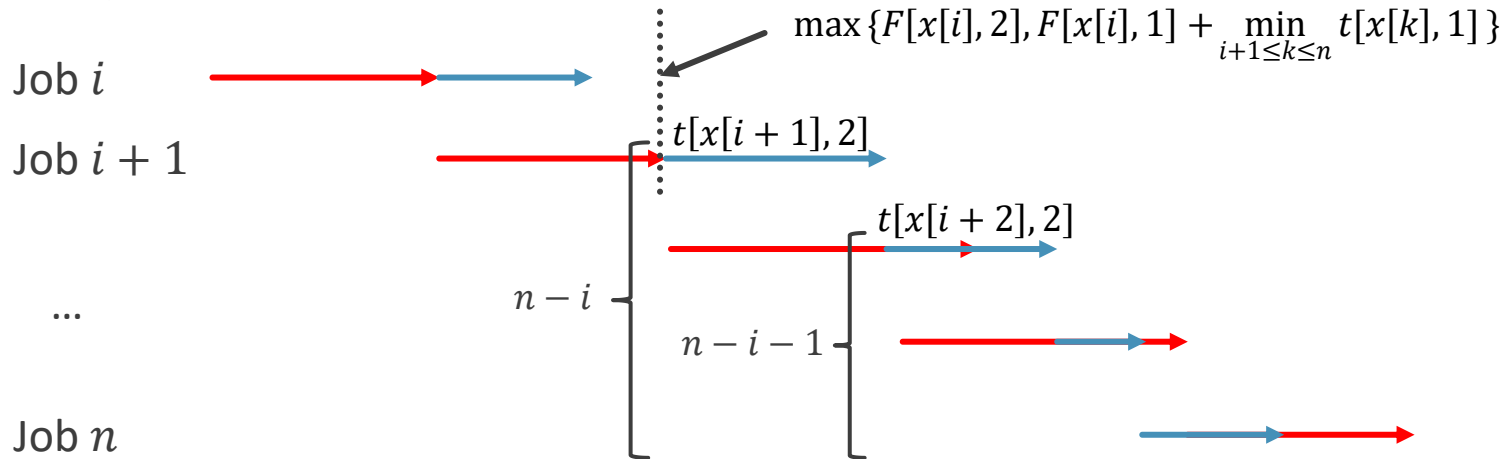
- Obviously, we have $rf(i) \geq rf2(i)$.



Lower Bound

Lower bound 2:

$$rf2(i) = \sum_{j=i+1}^n \left(\max \{F[x[i], 2], F[x[i], 1] + \min_{i+1 \leq k \leq n} t[x[k], 1]\} + (n - j + 1)t[x[j], 2] \right)$$



- The order from job $i + 1$ to job n matters.
- Therefore, we can sort $t[x[j], 2]$ in non-decreasing order to obtain smaller $rf2(i)' \leq rf2(i)$.



Lower Bound

- So, we have

$$\begin{aligned} f &= \sum_{j=1}^i F[x[j], 2] + rf(i) \\ &\geq \sum_{j=1}^i F[x[j], 2] + \max\{rf1(i), rf2(i)\} \\ &\geq \sum_{j=1}^i F[x[j], 2] + \max\{rf1(i)', rf2(i)'\} = B(i) \end{aligned}$$

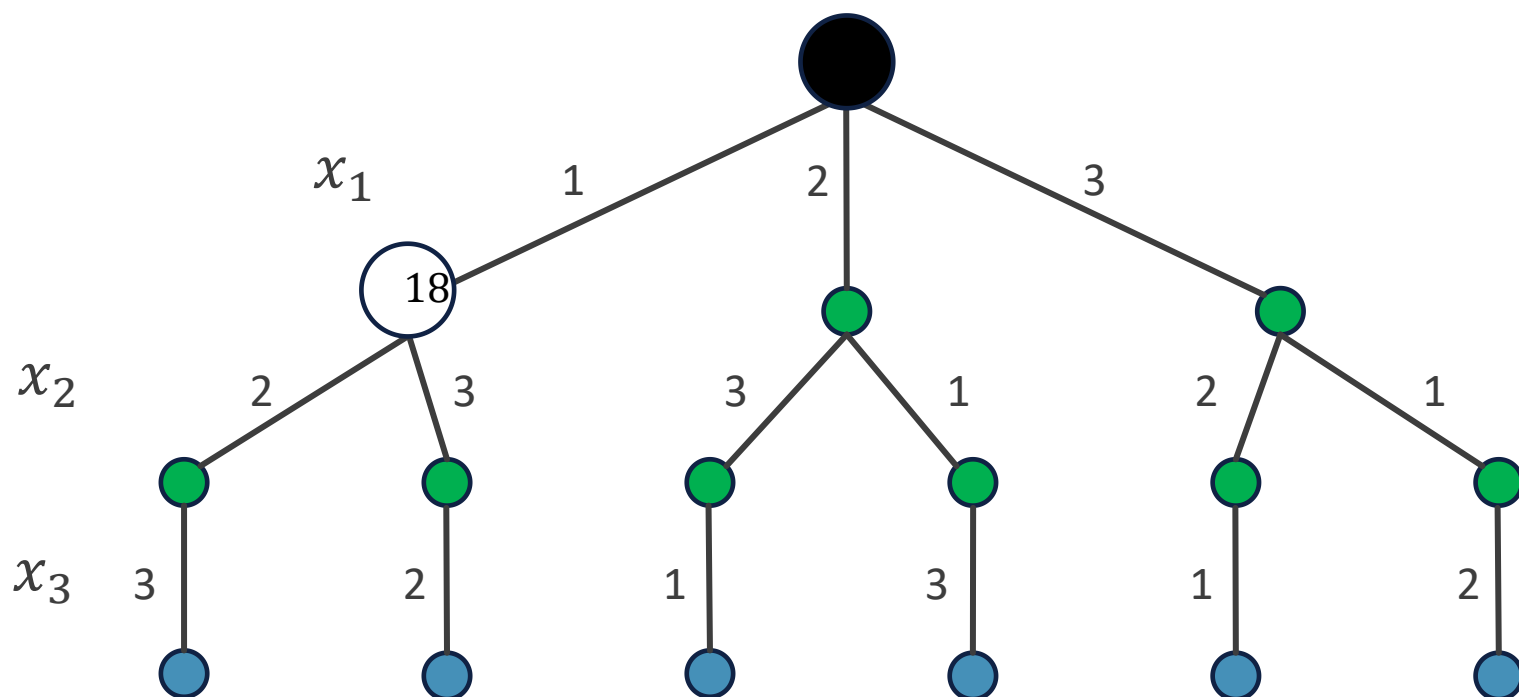
- If $B(i) \geq bestf$, then stop search the node i and the following level, otherwise, continue to search. At the same time, we use min-priority queue to extend.



Example

$t[i, j]$	Machine 1	Machine 2
Job 1	2	1
Job 2	3	1
Job 3	2	3

● Dead nodes ○ Live nodes



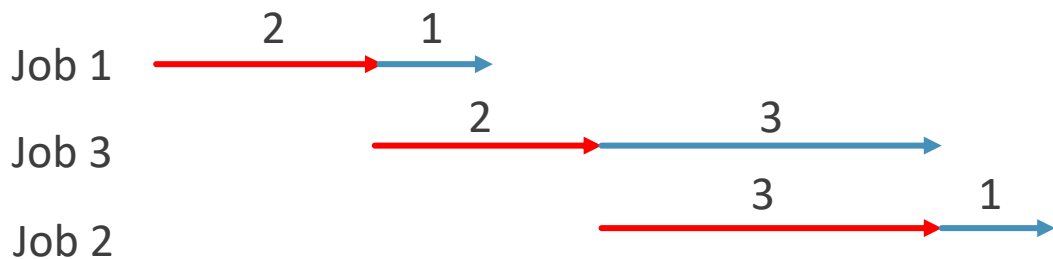


Example

$$rf1(i) = \sum_{j=i+1}^n (F[x[i], 1] + (n - j + 1)t[x[j], 1] + t[x[j], 2])$$

$rf1(1)'$:

$t[i, j]$	Machine 1	Machine 2
Job 1	2	1
Job 2	3	1
Job 3	2	3

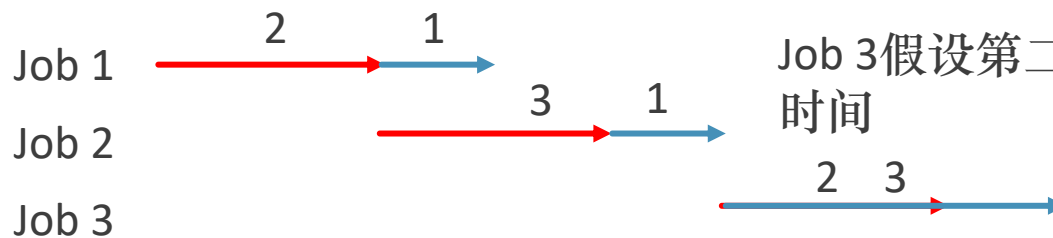


$$F[1,2] = 3$$

$$F[3,2] = 7 \quad rf1' = 18$$

$$F[2,2] = 8$$

$$rf2(1)'$$



$$F[1,2] = 3$$

$$F[2,2] = 6 \quad rf2' = 18$$

$$F[3,2] = 9$$

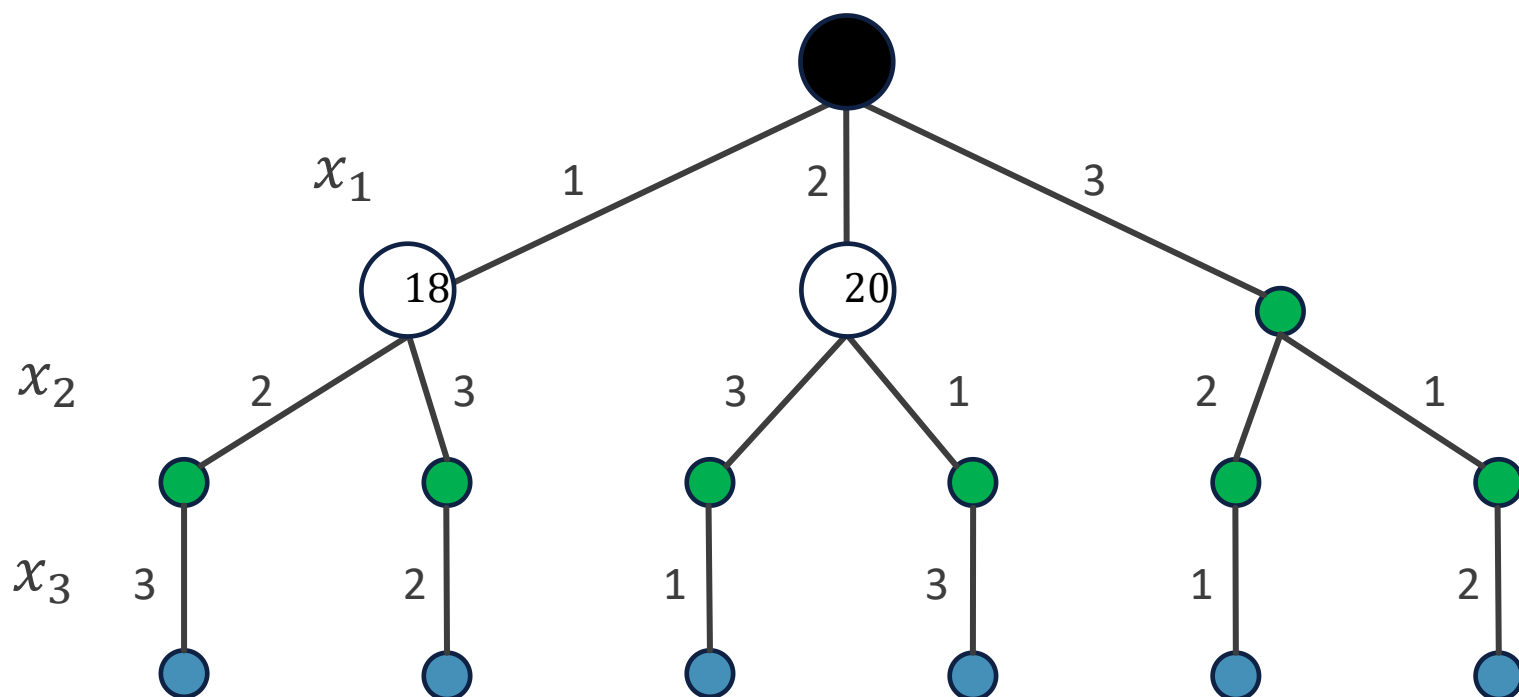
$$rf2(i) = \sum_{j=i+1}^n \left(\max \{ F[x[i], 2], F[x[i], 1] + \min_{i+1 \leq k \leq n} t[x[k], 1] \} + (n - j + 1)t[x[j], 2] \right)$$



Example

$t[i, j]$	Machine 1	Machine 2
Job 1	2	1
Job 2	3	1
Job 3	2	3

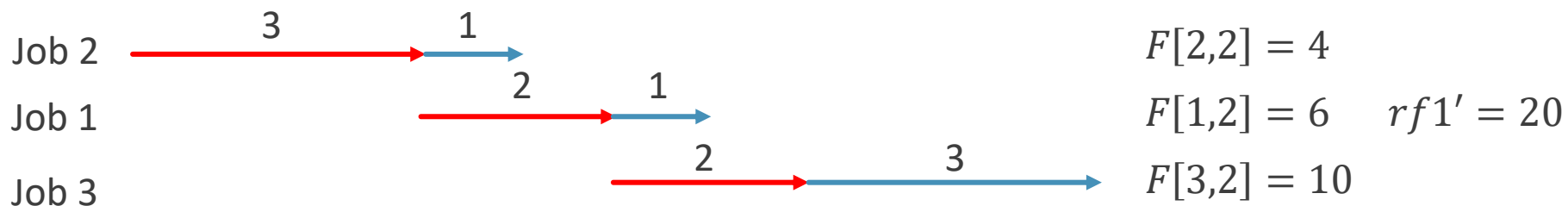
● Dead nodes ○ Live nodes





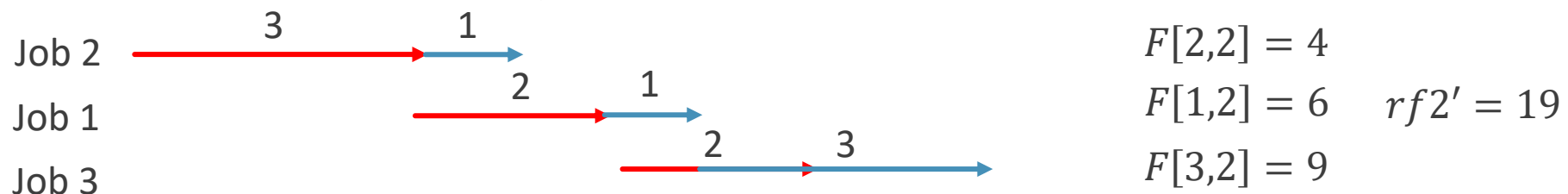
Example

$rf1(1)'$:



$rf2(1)'$:

$$rf1(i) = \sum_{j=i+1}^n (F[x[i], 1] + (n - j + 1)t[x[j], 1] + t[x[j], 2])$$



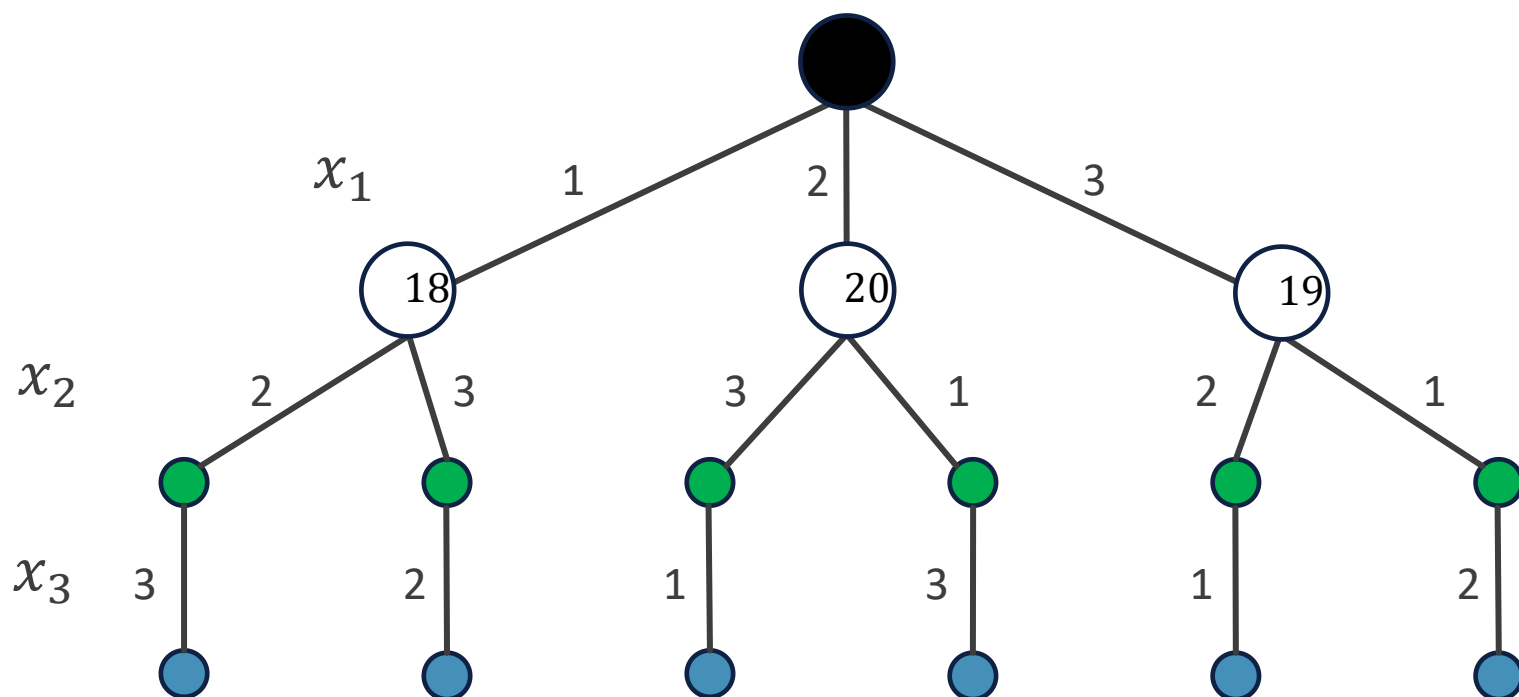
$$rf2(i) = \sum_{j=i+1}^n \left(\max \{F[x[i], 2], F[x[i], 1] + \min_{i+1 \leq k \leq n} t[x[k], 1]\} + (n - j + 1)t[x[j], 2] \right)$$



Example

$t[i, j]$	Machine 1	Machine 2
Job 1	2	1
Job 2	3	1
Job 3	2	3

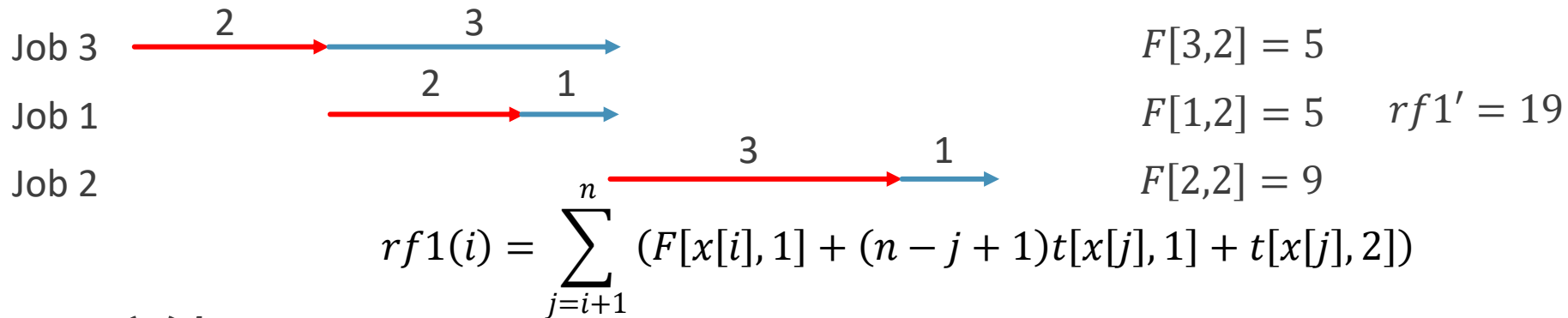
● Dead nodes ○ Live nodes



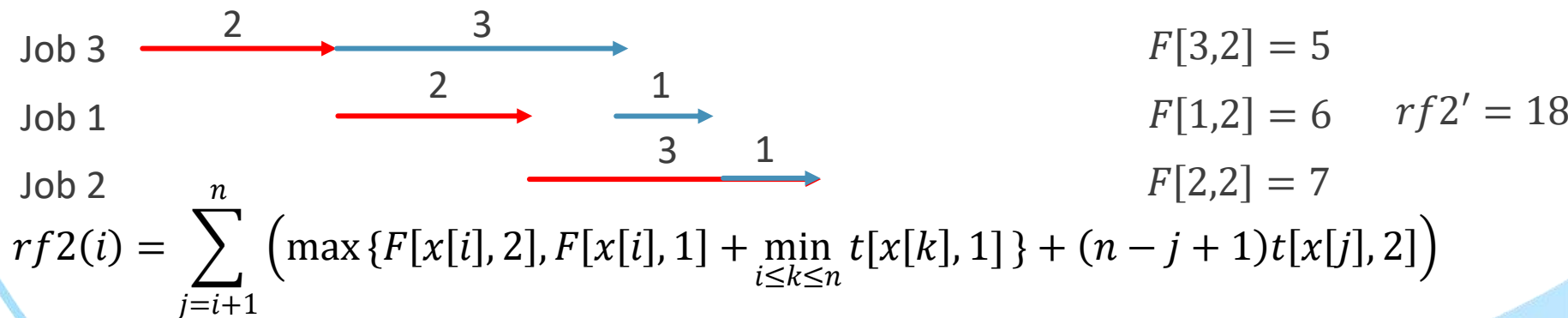


Example

$rf1(1)'$:



$rf2(1)'$:

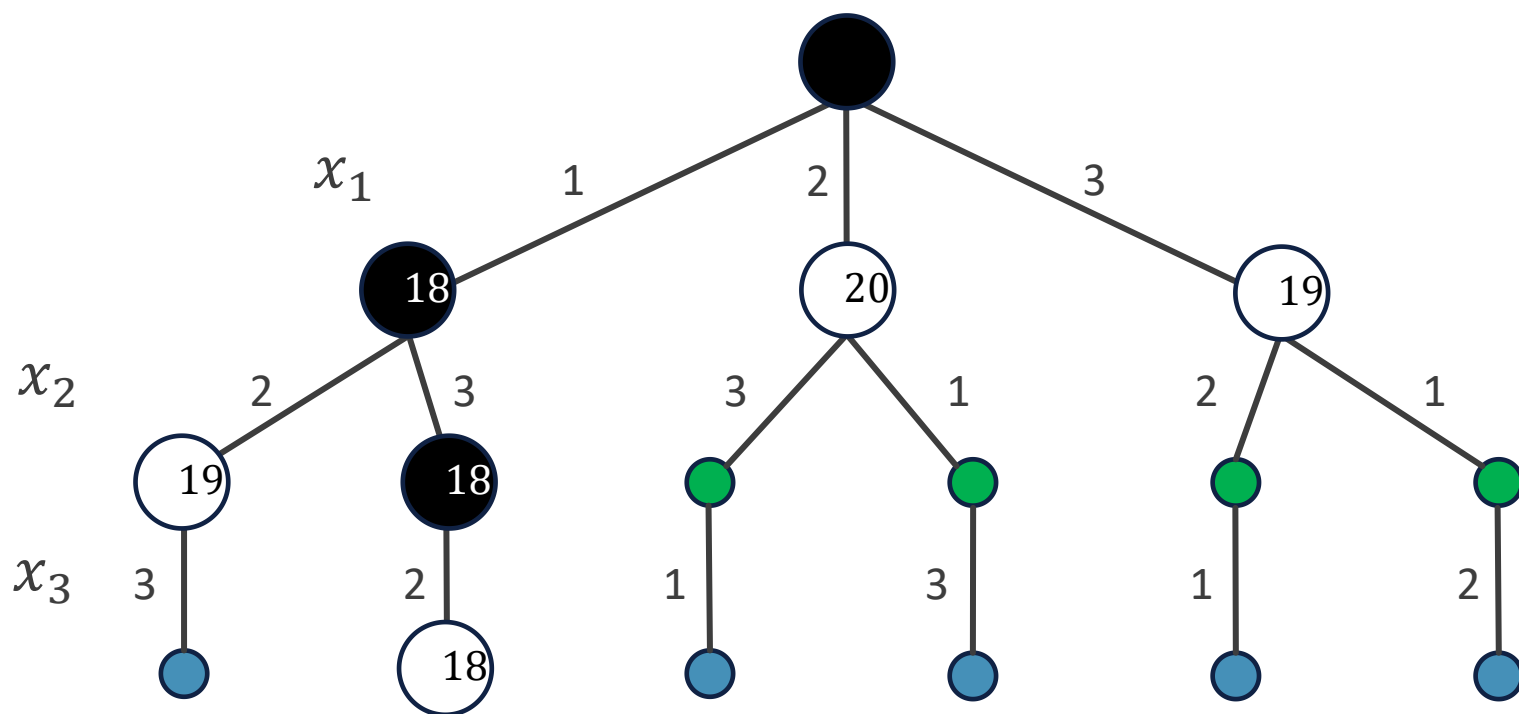




Example

$t[i, j]$	Machine 1	Machine 2
Job 1	2	1
Job 2	3	1
Job 3	2	3

● Dead nodes ○ Live nodes





Conclusion

After this lecture, you should know:

- What is the difference between backtracking and branch-and-bound.
- What kind of problem that we can use branch-and-bound.
- How can we improve the bounding function to eliminate more branches.



Homework

Page 262-263

13.1

13.2

13.4



谢谢

有问题欢迎随时跟我讨论